# Roteiro Power Apps SOx SCM-42

Versão em PDF de 03/11/2026: todas as etapas abertas, fórmulas no formato português ( ; e ;; ).

**Versão de 03/11/2026** · se o arquivo que você recebeu tiver data mais antiga, peça o novo.

Uma ação por cartão. Em cada um: **onde** você está, **o que fazer**, o que **colar** e como saber que **deu certo**. Marque a caixa e siga para o próximo. O progresso fica salvo neste navegador. As imagens são ilustrações das telas; as fotos reais estão nos links da Microsoft em cada etapa.

| A · Preparar | B · Montar o app | C · Alertas no Teams | D · Todo mês |
|---|---|---|---|
| Ajustar o Python e a lista. 40 min, uma vez. | 12 etapas curtas. Umas 3 horas, uma vez. | Chat por contrato, cobrança nos X dias antes do fechamento. 1h30, uma vez. | Dia 1 gerar e mandar e-mail; dia 16 fechar e guardar. |

**Seu Power Apps está em:** Muda as fórmulas do Power Apps para colar. Em português ele usa `;` entre itens e `;;` entre comandos. As expressões do Power Automate não mudam.

## A0. Entender: o Excel e a lista online

3 min · A · Preparar

**Objetivo:** saber a diferença entre a planilha que fica no seu computador e a lista online que o app usa. Nada para clicar aqui.

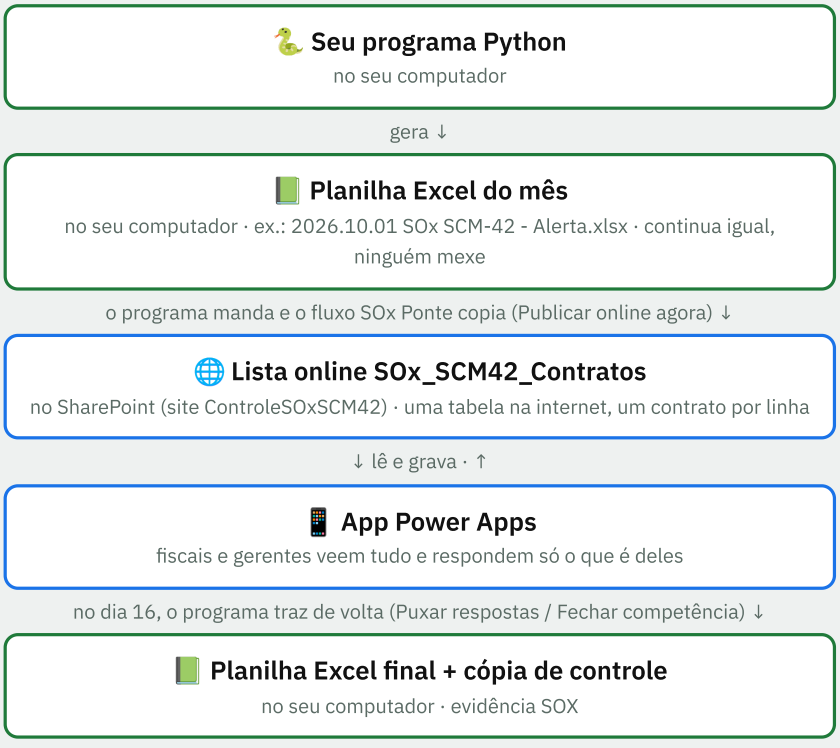



ILUSTRAÇÃO O caminho dos dados no mês. Verde: no seu computador. Azul: na internet.

1 ☑ SEU COMPUTADOR

A **planilha Excel** que o Python gera **continua no seu computador**, do jeito que é hoje. Ela não vai para a internet e ninguém edita ela.

**Deu certo se:** entendi.

2 ☑ INTERNET (SHAREPOINT)

A **lista online** é uma **cópia dos dados** dessa planilha numa tabela do SharePoint (Microsoft Lists). Parece uma planilha, mas fica na internet e várias pessoas usam ao mesmo tempo sem perder dados. O app só enxerga a lista, nunca o Excel.

**Deu certo se:** entendi.

3 ☑ QUEM COPIA

Quem manda os dados do Excel para a lista é o **seu programa**, com a ajuda de um fluxo do Power Automate (a ponte, etapas A3 a A7), pelo botão **Publicar online agora** (ou sozinho, logo depois de gerar a planilha). Você não copia nada à mão.

**Deu certo se:** entendi.

4 ☑ SE A LISTA NÃO EXISTIR

Se a lista ainda não existir no site, a ponte **cria a lista e todas as colunas** na primeira publicação. Você não precisa criar a lista.

**Deu certo se:** entendi.

5 ☑ ONDE ELA FICA

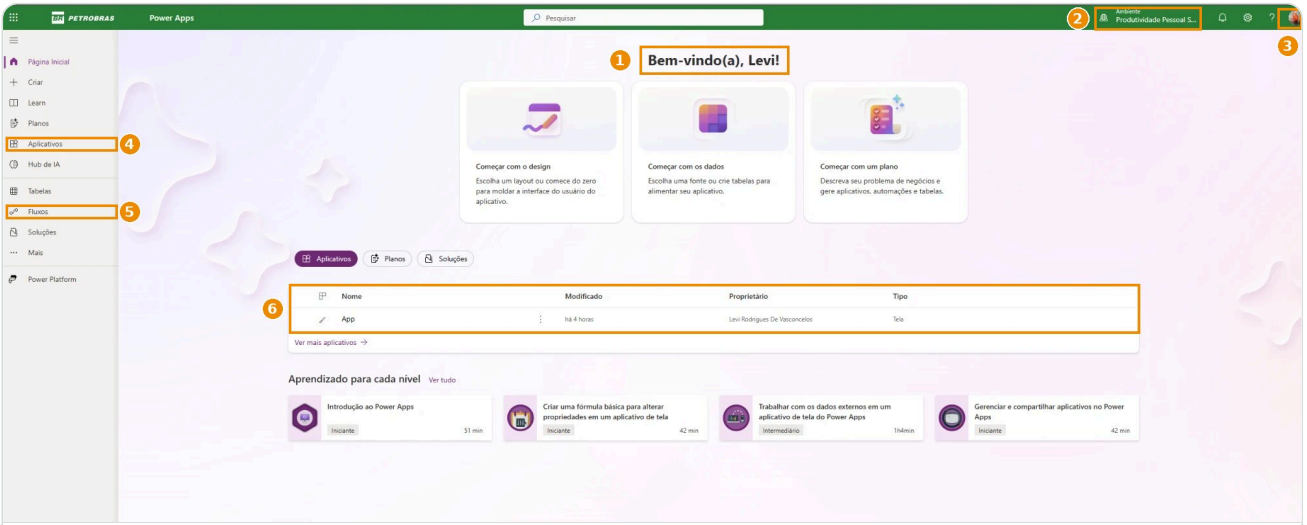
Endereço já configurado no seu programa (arquivo `sox_nuvem\sox_nuvem_config.json`, na pasta do programa): site `ControleSOxSCM42`, lista `SOx_SCM42_Contratos`. Link: abrir a lista.

**Deu certo se:** você sabe que o link leva à lista online, não ao Excel.

# A1. Conferir os acessos

10 min · A · Preparar

**Objetivo:** abrir o Power Apps, reconhecer a página inicial e anotar 2 informações (ambiente e e-mail) que serão usadas depois. Nada é criado nesta etapa.




SEU PRINT Página inicial do Power Apps (make.powerapps.com), como aparece para você. **1** boas-vindas: entrou certo · **2** Ambiente · **3** sua foto (e-mail) · **4** Aplicativos · **5** Fluxos · **6** lista dos seus apps. Clique na imagem para ver em tamanho grande.

**1** ☑ COMPUTADOR

Abra o **Edge** (ou o Chrome) no computador da empresa. Não use o celular para montar.

**2** ☑ EDGE → BARRA DE ENDEREÇO (EM CIMA)

Clique na barra de endereço, digite `make.powerapps.com` e aperte `Enter`. Se pedir login, use o seu e-mail da Petrobras e a senha de rede.

**Deu certo se:** abre a página do seu print: faixa **verde** com **PETROBRAS · Power Apps** e no meio **Bem-vindo(a), Levi!** (marca **1**).

▼ **Não deu certo?**

- **"Você precisa de uma licença" ou "Iniciar avaliação":** não clique em avaliação. Peça à TI acesso ao Power Apps do Microsoft 365 (é o que permite app ligado a lista do SharePoint).
- **Página em branco ou carregando sem parar:** aperte F5. Se continuar, abra numa janela InPrivate (Ctrl+Shift+N).

**3** ☑ CANTO DE CIMA À DIREITA (MARCA 2)

Olhe o quadro **Ambiente**. No seu aparece **Produtividade Pessoal S...**. Não mude. Anote esse nome.

**Deu certo se:** você anotou o nome do ambiente.

▼ **Não deu certo?**

**O que é ambiente:** é a "pasta" onde ficam os apps e fluxos. O app (parte B) e o fluxo do Teams (parte C) precisam ficar no **mesmo ambiente**. Quando abrir o Power Automate na parte C, confira se o canto de cima à direita mostra o mesmo nome; se não, clique nele e escolha este.

**4** ☑ CANTO DE CIMA À DIREITA (MARCA 3)

Clique na sua **foto** (último ícone da faixa verde). Abre um quadro com seu nome e, logo abaixo, o seu **e-mail**. Anote o e-mail exatamente como está.

**Deu certo se:** você tem o e-mail anotado (por exemplo nome.sobrenome@petrobras.com.br). Ele vai na etapa B3, na fórmula do administrador. Clique fora do quadro para fechar.

▼ **Não deu certo?**

Se o e-mail mostrado for diferente do que você usa no Outlook (por exemplo uma chave ou um nome@petrobras.biz), anote os dois e use o daqui na B3.

**5** ☑ MENU DA ESQUERDA (MARCA 4)

Clique em **Aplicativos**.

**Deu certo se:** abre a lista de apps. É aqui que o app **Controle SOx SCM-42** vai aparecer depois da B1, e onde você vai compartilhar na B12.

▼ **Não deu certo?**

No seu print já existe um app chamado **App** (marca **6**), criado há poucas horas. Se foi um teste seu, pode deixar ou excluir (três pontinhos ⋮ ao lado do nome → Excluir). Ele não atrapalha: na B1 você cria um novo, com outro nome.

**6** ☑ MENU DA ESQUERDA (MARCA 5)

Só para conhecer: **Fluxos** mostra os fluxos do Power Automate deste ambiente. O fluxo da parte C vai aparecer aqui também. Clique em **Página Inicial** para voltar.

**Deu certo se:** você voltou ao Bem-vindo(a).

**7** ☑ EDGE → NOVA ABA (CTRL + T)

Teste se a **lista online** já existe (não é o Excel, veja a A0): abra SOx_SCM42_Contratos. Veja qual dos 4 casos aconteceu e anote.

**Deu certo se:** você sabe em qual caso está: **1** abre uma tabela com contratos → já existe e já tem dados; **2** abre uma tabela vazia → existe, a A8 enche; **3** "Este item não existe" ou "Página não encontrada" → normal, a ponte cria na A8; **4** "Você precisa de permissão" ou "Solicitar acesso" → veja abaixo.

▼ **Não deu certo?**

- **Caso 4:** clique em Solicitar acesso, ou peça ao dono do site `ControleSOxSCM42` para te colocar como **Proprietário**. Sem isso o fluxo da ponte não consegue criar nem preencher a lista.
- **Quer saber se o site existe:** abra `https://petrobrasbr.sharepoint.com/teams/ControleSOxSCM42`. Se der "não encontrado", o site não existe: peça à TI um site do SharePoint (ou uma equipe do Teams) com esse nome, com você como proprietário, ou mande ao Claude o endereço de um site onde você já é proprietário.

**8** ☑ ESTA PÁGINA, NO TOPO

Os menus do seu Power Apps estão em **português** (Página Inicial, Criar, Aplicativos), como no seu print. Então deixe **Português** marcado no topo desta página: as fórmulas do roteiro já saem com `;` no lugar de `,`.

**Deu certo se:** o botão Português está verde.

## A2. Ajustar o Python (uma vez só)

10 min · A · Preparar

**Objetivo: Código novo (com publicação direta por cookie):** rode o `ajuste_direto.py` do mesmo jeito ( `python ajuste_direto.py Controle_SOx_SCM42.py` ) e **não** rode mais o `ajuste_app.py` , que foi feito para a versão antiga. Código antigo: rodar o `ajuste_app.py` no seu `Controle_SOx_SCM42.py` . Ele faz as 28 mudanças sozinho e guarda uma cópia do original. Você não abre nem edita o código.

**1** ☑ EXPLORADOR DE ARQUIVOS

Abra a pasta `C:\Users\GFG7\Downloads\controle_sox_scm42` . Confira que lá estão os dois arquivos: `Controle_SOx_SCM42.py` (seu programa) e `ajuste_app.py` (o ajuste).

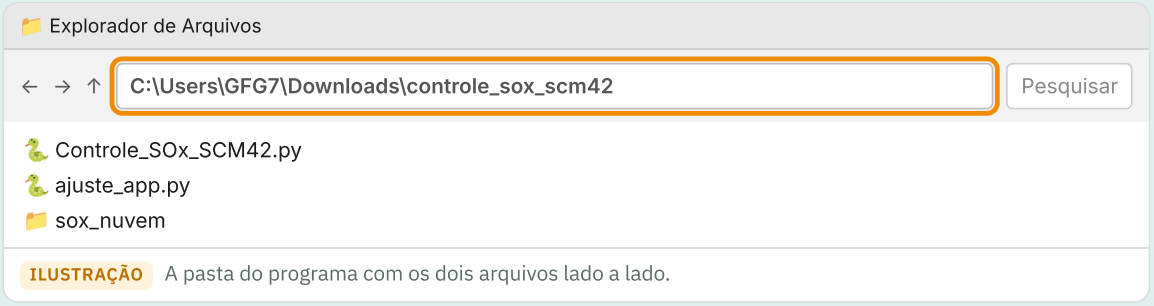



ILUSTRAÇÃO A pasta do programa com os dois arquivos lado a lado.

**Deu certo se:** os dois arquivos estão na mesma pasta.

▼ **Não deu certo?**

**O ajuste_app.py precisa ser o mais novo** (o que faz 28 ajustes, enviado por último na conversa). Na dúvida, baixe de novo e, quando o Windows perguntar, escolha **Substituir o arquivo no destino**. Se o Windows salvar como `ajuste_app (1).py` , apague o antigo e renomeie o novo para `ajuste_app.py` .

**2** ☑ SEU PROGRAMA

Se o Controle_SOx_SCM42 estiver aberto, feche a janela dele.

**Deu certo se:** o programa está fechado.

**3** ☑ WINDOWS

Aperte a tecla `Windows` , digite `cmd` e aperte `Enter` .

**Deu certo se:** abre uma **janela preta** (Prompt de Comando) escrito `C:\Users\GFG7>` .

**4** ☑ ESTA PÁGINA

Clique em **Copiar** no quadro abaixo. São 2 linhas: a primeira **entra na pasta** do programa, a segunda **roda o ajuste** no seu programa. Já estão com os seus nomes, não precisa trocar nada.

Copie as 2 linhas

```
cd /d "C:\Users\GFG7\Downloads\controle_sox_scm42"
python ajuste_app.py "Controle_SOx_SCM42.py"
```

**Deu certo se:** o botão fica escrito Copiado.

**5** ☑ JANELA PRETA

Clique dentro da janela preta e cole com `Ctrl` + `V` (ou clique com o **botão direito**, que também cola). Se a última linha ficar parada, aperte `Enter`.

C:\Windows\System32\cmd.exe

```
C:\Users\GFG7\Downloads\controle_sox_scm42> python ajuste_app.py "Controle_SOx_SCM42.py"
[feito]     Corrige o import do replace (leitura da ME33K)
[feito]     Coluna GerenteEmail na publicação
[feito]     Função que monta ;email; de fiscal e gerente
[feito]     FiscalEmail e GerenteEmail no formato ;email;
[feito]     Provisão das linhas com fórmula vai preenchida para a lista
[feito]     Fora do Prazo lido certo das fórmulas
[feito]     Republicar não apaga Houve serviço, Foi medido e Fora do prazo corrigidos no
app
[feito]     Republicar não apaga o que o pessoal respondeu no app
[feito]     Cria as colunas do OK e da aprovação na lista
[feito]     Puxar respostas também lê o OK e a aprovação
[feito]     Planilha de controle sai com o OK e a aprovação
[feito]     Fora do Prazo corrigido no app volta para a planilha
[feito]     Rascunhos do Teams continuam funcionando com ;email;
[feito]     Funções que travam o mês e guardam a cópia de controle
[feito]     Fechar competência trava a lista e guarda a cópia
[feito]     Resumo do fechamento mostra a trava e a pasta da cópia
[feito]     Reabrir competência destrava a lista
[feito]     Janela do fechamento mostra a cópia
[feito]     Publicar cria a lista online se ela não existir
[feito]     Conferir a estrutura começa pela lista de contratos
[não precisa] Ponte: Publicar espera o fluxo e mostra o andamento
[feito]     Ponte Power Automate: funções novas
[feito]     Ponte Power Automate: ligada na configuração
[feito]     Publicar usa a ponte
[feito]     Puxar respostas usa a ponte
[feito]     Fechar competência usa a ponte
[feito]     Reabrir competência usa a ponte
[feito]     Conferir estrutura não pede login na ponte
[feito]     Mensagem do Publicar explica a ponte

Pronto: 28 ajuste(s) feitos, 0 já estavam. Cópia do original: Controle_SOx_SCM42.py.bak
```

ILUSTRAÇÃO É isto que aparece quando dá certo. O texto é a saída real do script, rodado numa cópia do código que você mandou.

**Deu certo se:** aparecem as linhas **[feito]** (e uma **[não precisa]**, normal) e no fim **Pronto: 28 ajuste(s) feitos**. Se você já tinha rodado antes, aparece **[já estava]** nas que já existiam e **[feito]** só nas novas. Na pasta aparece `Controle_SOx_SCM42.py.bak` : é o original, guarde.

▼ **Não deu certo?**

- **Aparece [já estava] em algumas linhas:** normal, você já tinha rodado uma versão antiga; ele só faz o que falta.
- **[PROBLEMA]:** nada foi alterado. Tire um print da janela e mande ao Claude.
- **'python' não é reconhecido:** na segunda linha troque `python` por `py` e aperte Enter de novo.
- **Não achei o arquivo:** o nome ou a pasta estão diferentes. Confira no Explorador e mande print ao Claude.
- **Desfazer:** apague o Controle_SOx_SCM42.py e tire o `.bak` do nome da cópia.

6 ☑ PASTA DO PROGRAMA

Abra o programa normalmente (do jeito que você sempre abre) para ver se continua funcionando.

**Deu certo se:** o programa abre como antes.

Quer saber exatamente o que mudou no código? Veja a última etapa, "Referência: o que muda no código".

# A2b. Só se o script der PROBLEMA: mudar à mão

30 a 60 min · A · Preparar

**Objetivo:** fazer você mesmo as mudanças, uma por uma, no editor que vem junto com o Python (IDLE). Pule esta etapa se a A2 mostrou 28 [feito].

**1** ☑ EXPLORADOR DE ARQUIVOS

Faça uma cópia de segurança: clique no seu programa, `Ctrl` + `C` e `Ctrl` + `V`.

**Deu certo se:** aparece SEU_PROGRAMA - Copia.py. Não mexa nela.

**2** ☑ EXPLORADOR DE ARQUIVOS

Clique com o **botão direito** no seu programa → **Mostrar mais opções** (Windows 11) → **Edit with IDLE**.

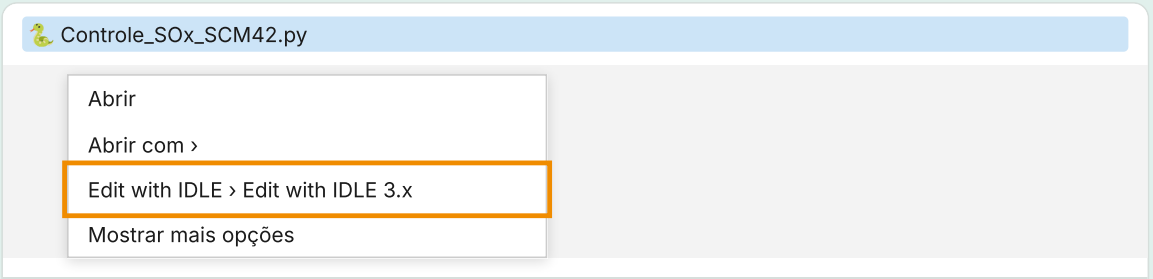



ILUSTRAÇÃO Menu do botão direito. Se não houver Edit with IDLE, use Abrir com → Bloco de Notas.

**Deu certo se:** abre uma janela com o código, colorido.

**3** ☑ ESTA PÁGINA

Vá à etapa **Referência** (última do menu). Cada mudança tem 3 partes: **Procure**, **Antes** e **Depois**. Faça uma de cada vez, na ordem.

**Deu certo se:** você está na mudança 1.

**4** ☑ IDLE

Aperte `Ctrl` + `F`, cole o texto do quadro **Procure** e clique em **Find Next**.

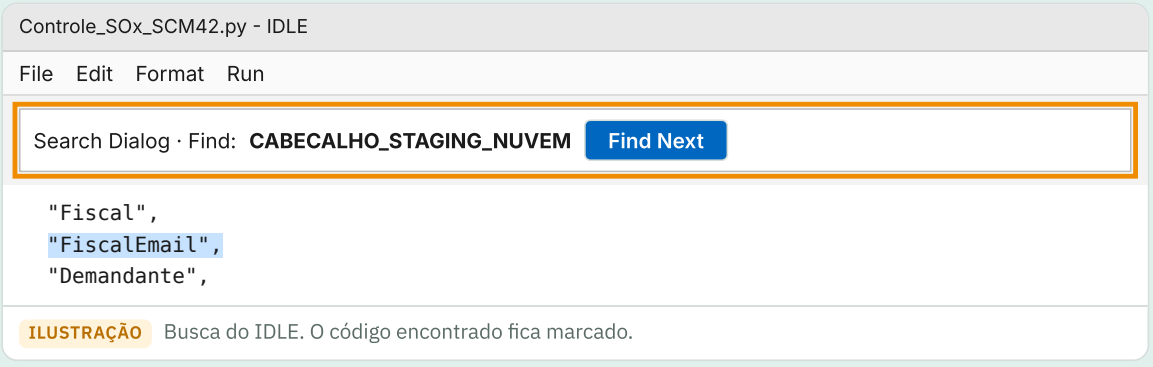



ILUSTRAÇÃO Busca do IDLE. O código encontrado fica marcado.

**Deu certo se:** o IDLE pula para o lugar certo e marca o texto.

▼ **Não deu certo?**

Não achou: confira se copiou o texto inteiro. Se mesmo assim não achar, essa parte do seu programa é diferente: mande print ao Claude e pule para a próxima.

5 ☑ IDLE

Com o mouse, selecione do começo ao fim o trecho igual ao quadro **Antes** (todas as linhas). Clique em **Copiar** no quadro **Depois** e aperte `Ctrl` + `V` no IDLE.

**Deu certo se:** o trecho virou o Depois.

▼ **Não deu certo?**

**Atenção aos espaços do começo da linha.** No Python eles fazem parte do código. Selecione desde o começo da linha (coluna 1) e não apague nem acrescente espaços. Errou? `Ctrl` + `Z` desfaz.

6 ☑ IDLE

Repita os passos 4 e 5 desta etapa para as 28 mudanças. A cada 3 ou 4, salve com `Ctrl` + `S`.

**Deu certo se:** as 28 estão feitas.

7 ☑ IDLE

Teste: menu **Run** → **Run Module** (ou `F5`).

**Deu certo se:** o seu programa abre normalmente.

▼ **Não deu certo?**

Se aparecer **SyntaxError** ou **IndentationError**, o IDLE marca a linha. Compare com o Depois daquela mudança (quase sempre são espaços). Para voltar ao começo, apague o programa e renomeie a cópia de segurança.

8 ☑ JANELA PRETA (COMO NA A2)

Para conferir, rode o script da A2. Ele deve mostrar **28 [já estava]**.

**Deu certo se:** 28 [já estava]: o programa está igual ao que o script faria.

# A3. Ponte 1: entender e criar as pastas

5 min · A · Preparar

**Objetivo: Reserva:** desde 26/10 o programa faz isto direto, sem fluxo; pode pular esta etapa. a Microsoft bloqueou o login direto do Python no SharePoint (o erro AADSTS65002 do seu print). A saída, sem depender da TI, é um fluxo do Power Automate que faz esse trabalho com o seu login. O Python e o fluxo conversam por 2 pastas do OneDrive.

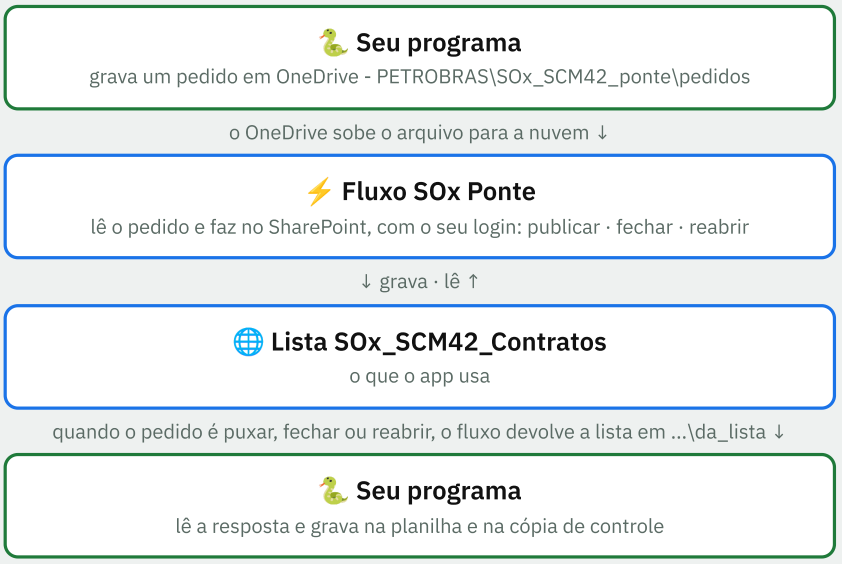



ILUSTRAÇÃO Como a ponte funciona. Verde: no seu computador. Azul: na internet.

**1** ☑ EXPLORADOR DE ARQUIVOS

Na coluna da esquerda, clique na pasta com **nuvem azul** chamada **OneDrive - PETROBRAS** (ou parecida).

**Deu certo se:** você está dentro do OneDrive da empresa.

▼ **Não deu certo?**

Sem essa pasta: clique no ícone de nuvem perto do relógio do Windows e entre com a conta da Petrobras, ou abra o app OneDrive pelo menu Iniciar. Se a pasta tiver outro nome, tudo bem: o programa acha sozinho.

**2** ☑ ONEDRIVE - PETROBRAS

Clique com o botão direito num espaço vazio → **Novo** → **Pasta**. Nome: `SOx_SCM42_ponte`. Entre nela e crie mais duas pastas: `pedidos` e `da_lista`.

**Deu certo se:** existe OneDrive - PETROBRAS\SOx_SCM42_ponte com as pastas pedidos e da_lista, cada uma com um ✓ verde ou nuvem (sincronizada).

# A4. Ponte 2: criar o fluxo e ler o pedido

15 min · A · Preparar

**Objetivo:** criar o fluxo SOx Ponte, que acorda sozinho quando o programa grava um pedido, e separar o pedido por tipo (publicar, fechar, reabrir).

1 ☑ ANTES DE COMEÇAR: DESLIGUE A TRADUÇÃO DO EDGE

Se aparecerem nomes estranhos como **"Candidate-se a cada um"** ou **se(...)**, é o **tradutor do Edge** traduzindo a página do inglês. Ele confunde os nomes das caixas. Na barra de endereço, clique no ícone de tradução **A文** → **Mostrar original** → marque **Nunca traduzir inglês** (ou nunca traduzir este site).

| No roteiro | Na tela em inglês |
|---|---|
| + Novo passo · Adicionar uma ação | + New step · Add an action |
| Renomear · Configurações · Excluir | Rename · Settings · Delete |
| Expressão · Conteúdo dinâmico | Expression · Dynamic content |
| Compor · Entradas | Compose · Inputs |
| Comutador · Ligado · Caso · Igual a · Padrão | Switch · On · Case · Equals · Default |
| Aplicar a cada · Selecionar uma saída | Apply to each · Select an output from previous steps |
| Condição · é igual a · não é igual a · Verdadeiro · Falso | Condition · is equal to · is not equal to · If yes · If no |
| Filtrar matriz · De · Editar no modo avançado | Filter array · From · Edit in advanced mode |
| Enviar uma solicitação HTTP para o SharePoint · Endereço do Site · Método · Cabeçalhos · Corpo | Send an HTTP request to SharePoint · Site Address · Method · Headers · Body |
| Criar arquivo · Caminho da Pasta · Nome do Arquivo · Conteúdo do Arquivo | Create file · Folder Path · File Name · File Content |
| Controle de simultaneidade · Grau de paralelismo | Concurrency Control · Degree of Parallelism |

**Deu certo se:** a tela está no original (inglês) e você usa a tabela para achar cada nome. Os nomes que você digita (Bruto, Pedido, Acao...) e as expressões não mudam.

2 ☑ ANTES DE COMEÇAR: OS 3 GESTOS DO EDITOR

Seu editor é o **clássico** (botões **+ Novo passo** e **Salvar** embaixo). Você vai usar só 3 gestos:

**Adicionar caixa: + Novo passo** no fim, ou o **+** entre duas caixas → **Adicionar uma ação** → busque o nome → clique.

**Renomear caixa:** três pontinhos **⋯** no canto da caixa → **Renomear** → digite o nome do roteiro → Enter.

**Colar expressão** (no roteiro aparece como **fx**): clique no campo → no painel que abre, aba **Expressão** → cole → **OK**. Não cole direto no campo: aí vira texto e a caixa fica com **!** vermelho.

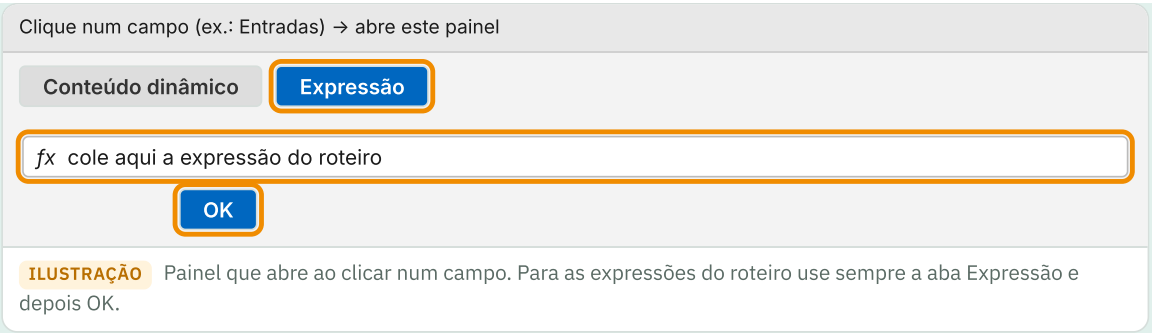



ILUSTRAÇÃO Painel que abre ao clicar num campo. Para as expressões do roteiro use sempre a aba Expressão e depois OK.

**Deu certo se:** você sabe adicionar, renomear e colar expressão. Os nomes das caixas precisam ser iguais aos do roteiro, porque uma caixa chama a outra pelo nome.

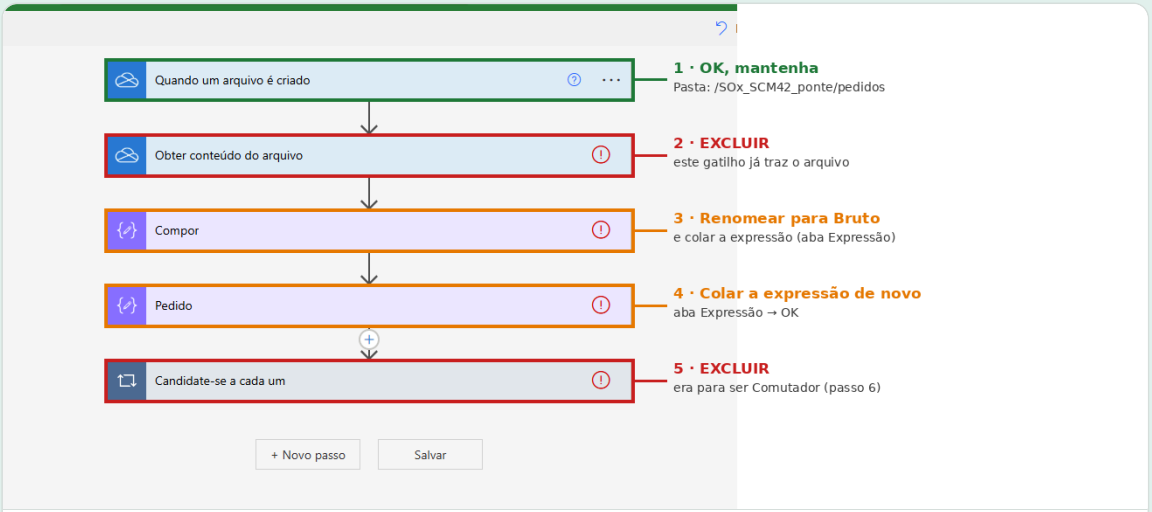
**3** ☑ NAVEGADOR

Abra make.powerautomate.com. Confira no canto de cima à direita se o Ambiente é o mesmo da A1 (Produtividade Pessoal S...).

**Deu certo se:** página inicial do Power Automate, no ambiente certo.

**4** ☑ COLUNA DA ESQUERDA

`+ Criar` → `Fluxo da nuvem automatizado`. Nome: `SOx Ponte`. Na busca do gatilho digite `arquivo é criado` e escolha **Quando um arquivo é criado** do **OneDrive for Business** (nuvem azul). Clique em `Criar`.




SEU PRINT Se o seu fluxo está assim, corrija nesta ordem: **2** e **5** excluir (três pontinhos ⋯ da caixa → Excluir); **3** renomear para Bruto e colar a expressão; **4** colar a expressão de novo. Depois siga do passo 6.

**Deu certo se:** abre o editor com a caixa do gatilho.

▼ **Não deu certo?**

Se você escolheu **Quando um arquivo é criado (somente propriedades)**, tudo bem: troque a expressão do passo 5 por `json(string(body('Conteudo')))` e, antes dela, adicione **Obter conteúdo do arquivo** renomeado `Conteudo`, com Arquivo = expressão `triggerBody()?['Id']`.

**5** ☑ CAIXA DO GATILHO

Em `Pasta`, clique no ícone de pasta → › ao lado de SOx_SCM42_ponte → escolha **pedidos**.

**Deu certo se:** a Pasta mostra /SOx_SCM42_ponte/pedidos.

**6** ☑ CAIXA DO GATILHO → ⋯

Três pontinhos do gatilho → **Configurações** → **Controle de simultaneidade** : deixe **Desligado** → **Concluído** .

**Deu certo se:** Controle de simultaneidade desligado.

▼ **Não deu certo?**

Se você ligou antes e apareceu o aviso amarelo **Trigger concurrency throttling detected**: abra o fluxo em Edit, desligue aqui e clique em Save. Ligado, o Power Automate pode pular pedidos.

**7** ☑ ABAIXO DO GATILHO

**+ Novo passo** → busque `Compor` → clique em **Compor** (Operação de Dados). Renomeie para `Bruto` . Clique em **Entradas** → aba **Expressão** → cole → **OK**:

Bruto → Entradas (aba Expressão)

```
json(string(triggerBody()))
```

**Deu certo se:** a caixa Bruto mostra um quadrinho roxo com json(...) e sem ! vermelho.

**8** ☑ ABAIXO

**+ Novo passo** → **Compor** . Renomeie para `Pedido` . **Entradas** → aba **Expressão** → cole → **OK**:

Pedido → Entradas (aba Expressão)

```
if(equals(outputs('Bruto')?['$content'], null), outputs('Bruto'),
json(base64ToString(coalesce(outputs('Bruto')?['$content'], 'e30='))))
```

**Deu certo se:** caixa Pedido sem ! vermelho. Ela entrega o pedido do programa já organizado, para as próximas caixas.

▼ **Não deu certo?**

- **! vermelho com "Bruto" no erro:** a caixa de cima não foi renomeada para Bruto (ainda se chama Compor).
- **Por que duas caixas:** o OneDrive pode entregar o arquivo como texto ou codificado. Pedido entende os dois.

**9** ☑ O QUE É O COMUTADOR (LEIA)

O **Comutador** é um desvio: ele lê a palavra que veio no pedido e manda o fluxo para o caminho certo. Cada caminho se chama **caso**. Vamos ter 3 casos: **publicar** (o programa mandou os contratos do mês), **fechar** (dia 16) e **reabrir**. Nas etapas A5 e A6 você coloca caixas dentro de cada caso.

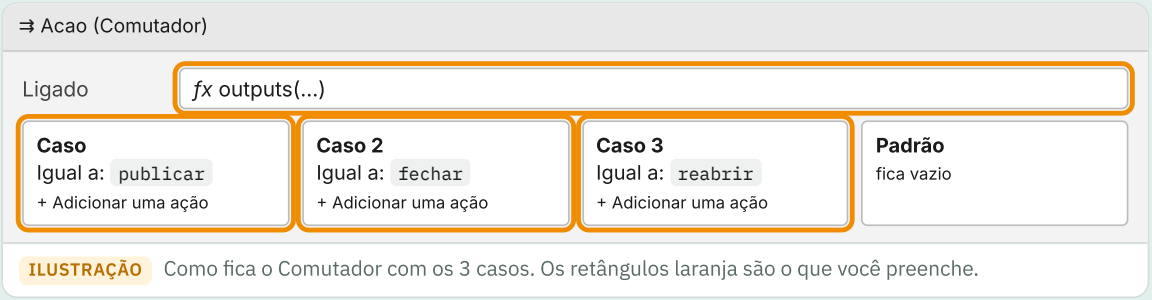



ILUSTRAÇÃO Como fica o Comutador com os 3 casos. Os retângulos laranja são o que você preenche.

**Deu certo se:** entendi o que é um caso.

**10** ☑ ABAIXO DE PEDIDO

**+ Novo passo** → busque `Comutador` → clique em **Comutador** (ícone de setas que se dividem, do grupo Controle). Renomeie para `Acao` . Clique em **Ligado** → aba **Expressão** → cole → **OK**:

Acao → Ligado (aba Expressão)

```
outputs('Pedido')?['acao']
```

**Deu certo se:** aparece a caixa Acao com um **Caso** à esquerda e o **Padrão** à direita.

▼ **Não deu certo?**

- Em inglês ela se chama **Switch** e o campo é **On**. Busque `switch`.
- **Não** serve a caixa com ícone de setas em círculo e o campo "Selecione uma saída a partir de etapas anteriores": essa é a **Aplicar a cada** (Apply to each, que o tradutor chama de "Candidate-se a cada um"). Se você renomeou essa para Acao, exclua e crie a certa.

**11** ☑ CAIXA ACAO → CASO

No **Caso**, campo **Igual a**: digite `publicar` (só digite, sem Expressão).

**Deu certo se:** o primeiro caso tem publicar.

**12** ☑ CAIXA ACAO → +

Clique no **+** que fica entre o último caso e o Padrão (**Adicionar um caso**). No novo **Caso 2**, **Igual a**: `fechar`. Repita: **Caso 3** com `reabrir`. O Padrão fica vazio. **Salvar**.

**Deu certo se:** três casos: publicar, fechar, reabrir (minúsculas, sem acento). Salvou sem ! vermelho.

▼ **Não deu certo?**

**Erro "Os casos não podem estar vazios":** é normal até a A5/A6 colocar caixas dentro deles.
Se não deixar salvar, siga para a A5 e salve no fim dela.

# A5. Ponte 3: caso publicar

30 min · A · Preparar

**Objetivo:** quando o programa manda publicar, o fluxo cria a lista (se faltar), cria as colunas (já com as opções SIM/NÃO e motivos) e grava os contratos. Tudo dentro do **Case** com `publicar`.

**1** ☑ REGRA DE OURO

Clique em **Save** a cada caixa pronta. O Power Automate só salva se todas as caixas estiverem certas; se der erro, corrija na hora, enquanto você lembra o que acabou de fazer. Para ver o que tem dentro da Acao, clique no título dela: ela abre e mostra os 3 casos.

**Deu certo se:** entendi: salvar sempre, e trabalhar dentro do Case certo.

**2** ☑ SITE ADDRESS (VALE PARA TODAS AS CAIXAS HTTP)

Em todas as caixas **Send an HTTP request to SharePoint**, o **Site Address** é o site abaixo. Se ele não aparecer na lista, escolha **Enter custom value** e cole.

Site Address

```
https://petrobrasbr.sharepoint.com/teams/ControleSOxSCM42
```

**Deu certo se:** você sabe qual endereço usar.

**3** ☑ CASE PUBLICAR → ADD AN ACTION

**Add an action** → busque `Send an HTTP request to SharePoint` → clique nela. Renomeie: ··· → **Rename** → `ListaExiste`.

**Site Address**: escolha **ControleSOxSCM42**.

**Method**: **GET**.

**Uri**: clique no campo → aba **Expression** → cole o quadro 1 → **OK**.

**Headers**: clique no ícone **T** à direita do campo (**Switch to text mode**) e cole o quadro 2.

1 · ListaExiste → Uri (aba Expression)

```
outputs('Pedido')?['uri_lista_existe']
```

2 · ListaExiste → Headers (modo texto)

```
{
  "Accept": "application/json;odata=nometadata",
  "Content-Type": "application/json;odata=nometadata"
}
```

**Deu certo se:** a caixa ListaExiste aparece sem ! vermelho.

**4** ☑ AINDA NO CASE PUBLICAR, EMBAIXO DE LISTAEXISTE

**Add an action** → busque `Condition` → clique em **Condition** (grupo Control).

Primeiro **Choose a value**: clique no campo → aba **Expression** → cole → **OK** com o quadro abaixo.

Operador do meio: **is equal to**.

Segundo **Choose a value**: clique no campo → aba **Expression** → cole → **OK** com `true`.

Condition → primeiro valor (aba Expression)

```
empty(body('ListaExiste')?['value'])
```

**Deu certo se:** aparecem duas colunas: **If yes** e **If no**, **dentro** do Case publicar, logo abaixo de ListaExiste.

▼ **Não deu certo?**

- O `true` também vai pela aba Expression. Digitado direto ele vira texto e a comparação nunca dá certo.
- **Erro ao salvar "InvalidTemplate ... 'ListaExiste' ... are not defined":** a Condition ficou fora do Case (embaixo da caixa Acao) ou a caixa HTTP não se chama exatamente `ListaExiste`. Exclua a Condition de fora, abra a Acao (clique nela), confira o nome da caixa no Case publicar e crie a Condition ali dentro, logo abaixo dela.

**5** ☑ DENTRO DO IF YES

**Add an action** → busque `Send an HTTP request to SharePoint` → clique nela. Renomeie: ··· → **Rename** → `CriarLista`.

**Site Address**: escolha **ControleSOxSCM42**.

**Method**: **POST**.

**Uri**: clique no campo → aba **Expression** → cole o quadro 1 → **OK**.

**Headers**: clique no ícone **T** à direita do campo (**Switch to text mode**) e cole o quadro 2.

**Body**: clique no campo → aba **Expression** → cole o quadro 3 → **OK**.

1 · CriarLista → Uri (aba Expression)

```
outputs('Pedido')?['uri_criar_lista']
```

2 · CriarLista → Headers (modo texto)

```
{
  "Accept": "application/json;odata=nometadata",
  "Content-Type": "application/json;odata=nometadata"
}
```

3 · CriarLista → Body (aba Expression)

```
outputs('Pedido')?['corpo_criar_lista']
```

**Deu certo se:** CriarLista pronta. O If no fica vazio.

**6** ☑ EMBAIXO DA CONDITION (FORA DELA, AINDA NO CASE PUBLICAR)

Clique no **Add an action** que fica **abaixo** da caixa Condition → busque `Apply to each` → clique. Renomeie para `CadaColuna`. Em **Select an output from previous steps**: clique no campo → aba **Expression** → cole → **OK**:

CadaColuna → Select an output (aba Expression)

```
outputs('Pedido')?['colunas']
```

**Deu certo se:** CadaColuna pronta, vazia por dentro.

**7** ☑ DENTRO DE CADACOLUNA → ADD AN ACTION

**Add an action** → busque `Send an HTTP request to SharePoint` → clique nela. Renomeie: ··· → **Rename** → `TemColuna`.

**Site Address**: escolha **ControleSOxSCM42**.

**Method**: **GET**.

**Uri** : clique no campo → aba **Expression** → cole o quadro 1 → **OK** .
**Headers** : clique no ícone **T** à direita do campo ( **Switch to text mode** ) e cole o quadro 2.

1 · TemColuna → Uri (aba Expression)

```
items('CadaColuna')?['uri_existe']
```

2 · TemColuna → Headers (modo texto)

```
{
  "Accept": "application/json;odata=nometadata",
  "Content-Type": "application/json;odata=nometadata"
}
```

**Deu certo se:** a caixa TemColuna aparece sem ! vermelho.

**8** ☑ DENTRO DE CADACOLUNA, EMBAIXO DE TEMCOLUNA

**Add an action** → **Condition** . Primeiro valor: clique no campo → aba **Expression** → cole → **OK** com o quadro abaixo. **is equal to** . Segundo valor: clique no campo → aba **Expression** → cole → **OK** com `true` .

Condition → primeiro valor (aba Expression)

```
empty(body('TemColuna')?['value'])
```

**Deu certo se:** If yes e If no dentro de CadaColuna.

**9** ☑ DENTRO DO IF YES (DE CADACOLUNA)

**Add an action** → busque `Send an HTTP request to SharePoint` → clique nela. Renomeie: ··· → **Rename** → `CriarColuna` .
**Site Address** : escolha **ControleSOxSCM42**.
**Method** : **POST**.
**Uri** : clique no campo → aba **Expression** → cole o quadro 1 → **OK** .
**Headers** : clique no ícone **T** à direita do campo ( **Switch to text mode** ) e cole o quadro 2.
**Body** : clique no campo → aba **Expression** → cole o quadro 3 → **OK** .

1 · CriarColuna → Uri (aba Expression)

```
outputs('Pedido')?['uri_criar_coluna']
```

2 · CriarColuna → Headers (modo texto)

```
{
  "Accept": "application/json;odata=nometadata",
  "Content-Type": "application/json;odata=nometadata"
}
```

3 · CriarColuna → Body (aba Expression)

```
items('CadaColuna')?['corpo']
```

**Deu certo se:** a caixa CriarColuna aparece sem ! vermelho.

**10** ☑ EMBAIXO DE CADACOLUNA (FORA DELA, AINDA NO CASE PUBLICAR)

**Add an action** → busque `Send an HTTP request to SharePoint` → clique nela. Renomeie: ··· → **Rename** → `Indexar` .

**Site Address** : escolha **ControleSOxSCM42**.
**Method** : **POST**.
**Uri** : clique no campo → aba **Expression** → cole o quadro 1 → **OK** .
**Headers** : clique no ícone **T** à direita do campo ( **Switch to text mode** ) e cole o quadro 2.
**Body** : clique no campo → aba **Expression** → cole o quadro 3 → **OK** .

1 · Indexar → Uri (aba Expression)

```
outputs('Pedido')?['uri_indexar']
```

2 · Indexar → Headers (modo texto) · com MERGE: atualiza sem apagar

```
{
  "Accept": "application/json;odata=nometadata",
  "Content-Type": "application/json;odata=nometadata",
  "IF-MATCH": "*",
  "X-HTTP-Method": "MERGE"
}
```

3 · Indexar → Body (aba Expression)

```
json('{"Indexed": true}')
```

**Deu certo se:** Indexar pronta (deixa a lista rápida quando crescer).

**11** ☑ EMBAIXO DE INDEXAR

**Add an action** → busque `Send an HTTP request to SharePoint` → clique nela. Renomeie: ··· → **Rename** → `Existentes` .
**Site Address** : escolha **ControleSOxSCM42**.
**Method** : **GET**.
**Uri** : clique no campo → aba **Expression** → cole o quadro 1 → **OK** .
**Headers** : clique no ícone **T** à direita do campo ( **Switch to text mode** ) e cole o quadro 2.

1 · Existentes → Uri (aba Expression)

```
outputs('Pedido')?['uri_existentes']
```

2 · Existentes → Headers (modo texto)

```
{
  "Accept": "application/json;odata=nometadata",
  "Content-Type": "application/json;odata=nometadata"
}
```

**Deu certo se:** a caixa Existentes aparece sem ! vermelho.

**12** ☑ EMBAIXO DE EXISTENTES

**Add an action** → `Apply to each` . Renomeie para `CadaContrato` .
**Select an output from previous steps** : clique no campo → aba **Expression** → cole → **OK** . Depois ··· da caixa → **Settings** → **Concurrency Control** : **On** → **Degree of Parallelism** : **10** → **Done** .

CadaContrato → Select an output (aba Expression)

```
outputs('Pedido')?['itens']
```

**Deu certo se:** CadaContrato pronta, com paralelismo 10 (grava 10 contratos ao mesmo tempo).

**13** ☑ DENTRO DE CADACONTRATO → ADD AN ACTION

Busque `Filter array` → clique. Renomeie para `Achado`. **From**: clique no campo → aba **Expression** → cole → **OK** com o quadro 1. Depois clique em **Edit in advanced mode** (link azul embaixo) e cole o quadro 2 na caixa que aparece.

1 · Achado → From (aba Expression)

```
body('Existentes')?['value']
```

2 · Achado → Edit in advanced mode (colar direto)

```
@equals(item()?['ChaveUnica'], items('CadaContrato')?['chave'])
```

**Deu certo se:** Achado pronta.

**14** ☑ DENTRO DE CADACONTRATO, EMBAIXO DE ACHADO

**Add an action** → **Condition**. Primeiro valor: clique no campo → aba **Expression** → cole → **OK** com o quadro. **is equal to**. Segundo: clique no campo → aba **Expression** → cole → **OK** com `true`.

Condition → primeiro valor (aba Expression)

```
empty(body('Achado'))
```

**Deu certo se:** If yes = contrato novo; If no = já existe na lista.

**15** ☑ DENTRO DO IF YES (DE CADACONTRATO)

**Add an action** → busque `Send an HTTP request to SharePoint` → clique nela. Renomeie: ··· → **Rename** → `Criar`.

**Site Address**: escolha **ControleSOxSCM42**.

**Method**: **POST**.

**Uri**: clique no campo → aba **Expression** → cole o quadro 1 → **OK**.

**Headers**: clique no ícone **T** à direita do campo (**Switch to text mode**) e cole o quadro 2.

**Body**: clique no campo → aba **Expression** → cole o quadro 3 → **OK**.

1 · Criar → Uri (aba Expression)

```
outputs('Pedido')?['uri_itens']
```

2 · Criar → Headers (modo texto)

```
{
  "Accept": "application/json;odata=nometadata",
  "Content-Type": "application/json;odata=nometadata"
}
```

3 · Criar → Body (aba Expression)

```
items('CadaContrato')?['novo']
```

**Deu certo se:** a caixa Criar aparece sem ! vermelho.

**16** ☑ DENTRO DO IF NO (DE CADACONTRATO)

**Add an action** → busque `Send an HTTP request to SharePoint` → clique nela. Renomeie: ··· → **Rename** → `Atualizar`.

**Site Address**: escolha **ControleSOxSCM42**.

`Method` : **POST**.
`Uri` : clique no campo → aba `Expression` → cole o quadro 1 → `OK`.
`Headers` : clique no ícone **T** à direita do campo ( `Switch to text mode` ) e cole o quadro 2.
`Body` : clique no campo → aba `Expression` → cole o quadro 3 → `OK`.

1 · Atualizar → Uri (aba Expression)

```
concat(outputs('Pedido')?['uri_itens'], '(', first(body('Achado'))?['Id'],
')')
```

2 · Atualizar → Headers (modo texto) · com MERGE: atualiza sem apagar

```
{
  "Accept": "application/json;odata=nometadata",
  "Content-Type": "application/json;odata=nometadata",
  "IF-MATCH": "*",
  "X-HTTP-Method": "MERGE"
}
```

3 · Atualizar → Body (aba Expression)

```
items('CadaContrato')?['atualiza']
```

**Deu certo se:** Atualizar pronta. Clique em `Save` (em cima). Ela só atualiza dados da planilha: o que o pessoal respondeu no app não é apagado.

## A6. Ponte 4: casos fechar e reabrir

20 min · A · Preparar

**Objetivo:** no dia 16, fechar marca MesFechado=SIM e deixa cada contrato do mês só leitura para o pessoal (os Proprietários do site continuam editando). Reabrir desfaz. Use o **Case 2** (fechar) e o **Case 3** (reabrir).

**1** ☑ CASE 2 (FECHAR) → ADD AN ACTION

**Add an action** → busque `Send an HTTP request to SharePoint` → clique nela. Renomeie: ··· → **Rename** → `Grupos`.

**Site Address**: escolha **ControleSOxSCM42**.

**Method**: **GET**.

**Uri**: clique no campo → aba **Expression** → cole o quadro 1 → **OK**.

**Headers**: clique no ícone **T** à direita do campo (**Switch to text mode**) e cole o quadro 2.

1 · Grupos → Uri (aba Expression)

```
outputs('Pedido')?['uri_grupos']
```

2 · Grupos → Headers (modo texto)

```
{
  "Accept": "application/json;odata=nometadata",
  "Content-Type": "application/json;odata=nometadata"
}
```

**Deu certo se:** a caixa Grupos aparece sem ! vermelho.

**2** ☑ EMBAIXO DE GRUPOS

**Add an action** → busque `Send an HTTP request to SharePoint` → clique nela. Renomeie: ··· → **Rename** → `ItensFechar`.

**Site Address**: escolha **ControleSOxSCM42**.

**Method**: **GET**.

**Uri**: clique no campo → aba **Expression** → cole o quadro 1 → **OK**.

**Headers**: clique no ícone **T** à direita do campo (**Switch to text mode**) e cole o quadro 2.

1 · ItensFechar → Uri (aba Expression)

```
outputs('Pedido')?['uri_ids']
```

2 · ItensFechar → Headers (modo texto)

```
{
  "Accept": "application/json;odata=nometadata",
  "Content-Type": "application/json;odata=nometadata"
}
```

**Deu certo se:** a caixa ItensFechar aparece sem ! vermelho.

**3** ☑ EMBAIXO DE ITENSFECHAR

**Add an action** → `Apply to each`. Renomeie para `CadaFechar`.

**Select an output from previous steps**: clique no campo → aba **Expression** → cole → **OK**. ··· →

Settings → Concurrency Control On, Degree of Parallelism 5 → Done.

CadaFechar → Select an output (aba Expression)

```
body('ItensFechar')?['value']
```

**Deu certo se:** CadaFechar pronta.

**4** ☑ DENTRO DE CADAFECHAR → ADD AN ACTION

Add an action → busque `Send an HTTP request to SharePoint` → clique nela. Renomeie: ··· → Rename → `MarcarFechado`.

Site Address: escolha **ControleSOxSCM42**.

Method: **POST**.

Uri: clique no campo → aba Expression → cole o quadro 1 → OK.

Headers: clique no ícone **T** à direita do campo (Switch to text mode) e cole o quadro 2.

Body: clique no campo → aba Expression → cole o quadro 3 → OK.

1 · MarcarFechado → Uri (aba Expression)

```
concat(outputs('Pedido')?['uri_itens'], '(', items('CadaFechar')?['Id'],
')')
```

2 · MarcarFechado → Headers (modo texto) · com MERGE: atualiza sem apagar

```
{
  "Accept": "application/json;odata=nometadata",
  "Content-Type": "application/json;odata=nometadata",
  "IF-MATCH": "*",
  "X-HTTP-Method": "MERGE"
}
```

3 · MarcarFechado → Body (aba Expression)

```
json('{"MesFechado": "SIM"}')
```

**Deu certo se:** a caixa MarcarFechado aparece sem ! vermelho.

**5** ☑ EMBAIXO DE MARCARFECHADO

Add an action → busque `Send an HTTP request to SharePoint` → clique nela. Renomeie: ··· → Rename → `Separar`.

Site Address: escolha **ControleSOxSCM42**.

Method: **POST**.

Uri: clique no campo → aba Expression → cole o quadro 1 → OK.

Headers: clique no ícone **T** à direita do campo (Switch to text mode) e cole o quadro 2.

1 · Separar → Uri (aba Expression)

```
concat(outputs('Pedido')?['uri_itens'], '(', items('CadaFechar')?['Id'],
')/breakroleinheritance(copyRoleAssignments=true,clearSubscopes=true)')
```

2 · Separar → Headers (modo texto)

```
{
  "Accept": "application/json;odata=nometadata",
  "Content-Type": "application/json;odata=nometadata"
}
```

**Deu certo se:** Separar pronta (o contrato passa a ter permissão própria).

6 ☑ EMBAIXO DE SEPARAR

**Add an action** → busque `Send an HTTP request to SharePoint` → clique nela. Renomeie: ··· → **Rename** → `TirarEdicao`.

**Site Address**: escolha **ControleSOxSCM42**.

**Method**: **POST**.

**Uri**: clique no campo → aba **Expression** → cole o quadro 1 → **OK**.

**Headers**: clique no ícone **T** à direita do campo (**Switch to text mode**) e cole o quadro 2.

1 · TirarEdicao → Uri (aba Expression)

```
concat(outputs('Pedido')?['uri_itens'], '(', items('CadaFechar')?['Id'],
')/roleassignments/getbyprincipalid(', string(body('Grupos')?
['AssociatedMemberGroup']?['Id']), ')/deleteObject()')
```

2 · TirarEdicao → Headers (modo texto)

```
{
  "Accept": "application/json;odata=nometadata",
  "Content-Type": "application/json;odata=nometadata"
}
```

**Deu certo se:** a caixa TirarEdicao aparece sem ! vermelho.

7 ☑ EMBAIXO DE TIRAREDICAO

**Add an action** → busque `Send an HTTP request to SharePoint` → clique nela. Renomeie: ··· → **Rename** → `SoLeitura`.

**Site Address**: escolha **ControleSOxSCM42**.

**Method**: **POST**.

**Uri**: clique no campo → aba **Expression** → cole o quadro 1 → **OK**.

**Headers**: clique no ícone **T** à direita do campo (**Switch to text mode**) e cole o quadro 2.

1 · SoLeitura → Uri (aba Expression)

```
concat(outputs('Pedido')?['uri_itens'], '(', items('CadaFechar')?['Id'],
')/roleassignments/addroleassignment(principalid=', string(body('Grupos')?
['AssociatedMemberGroup']?['Id']), ',roledefid=1073741826)')
```

2 · SoLeitura → Headers (modo texto)

```
{
  "Accept": "application/json;odata=nometadata",
  "Content-Type": "application/json;odata=nometadata"
}
```

**Deu certo se:** SoLeitura pronta. Os Membros do site agora só leem esse contrato.

8 ☑ CASE 3 (REABRIR) → ADD AN ACTION

**Add an action** → busque `Send an HTTP request to SharePoint` → clique nela. Renomeie: ··· → **Rename** → `ItensReabrir`.

**Site Address**: escolha **ControleSOxSCM42**.

**Method**: **GET**.

**Uri** : clique no campo → aba **Expression** → cole o quadro 1 → **OK** .
**Headers** : clique no ícone **T** à direita do campo ( **Switch to text mode** ) e cole o quadro 2.

1 · ItensReabrir → Uri (aba Expression)

```
outputs('Pedido')?['uri_ids']
```

2 · ItensReabrir → Headers (modo texto)

```
{
  "Accept": "application/json;odata=nometadata",
  "Content-Type": "application/json;odata=nometadata"
}
```

**Deu certo se:** a caixa ItensReabrir aparece sem ! vermelho.

**9** ☑ EMBAIXO DE ITENSREABRIR

**Add an action** → `Apply to each` . Renomeie para `CadaReabrir` .
**Select an output from previous steps** : clique no campo → aba **Expression** → cole → **OK** .
**Settings** : paralelismo **5**.

CadaReabrir → Select an output (aba Expression)

```
body('ItensReabrir')?['value']
```

**Deu certo se:** CadaReabrir pronta.

**10** ☑ DENTRO DE CADAREABRIR → ADD AN ACTION

**Add an action** → busque `Send an HTTP request to SharePoint` → clique nela. Renomeie: ··· → **Rename** → `Devolver` .
**Site Address** : escolha **ControleSOxSCM42**.
**Method** : **POST**.
**Uri** : clique no campo → aba **Expression** → cole o quadro 1 → **OK** .
**Headers** : clique no ícone **T** à direita do campo ( **Switch to text mode** ) e cole o quadro 2.

1 · Devolver → Uri (aba Expression)

```
concat(outputs('Pedido')?['uri_itens'], '(', items('CadaReabrir')?['Id'],
')/resetroleinheritance')
```

2 · Devolver → Headers (modo texto)

```
{
  "Accept": "application/json;odata=nometadata",
  "Content-Type": "application/json;odata=nometadata"
}
```

**Deu certo se:** a caixa Devolver aparece sem ! vermelho.

**11** ☑ EMBAIXO DE DEVOLVER

**Add an action** → busque `Send an HTTP request to SharePoint` → clique nela. Renomeie: ··· → **Rename** → `MarcarAberto` .
**Site Address** : escolha **ControleSOxSCM42**.
**Method** : **POST**.
**Uri** : clique no campo → aba **Expression** → cole o quadro 1 → **OK** .

**Headers** : clique no ícone **T** à direita do campo ( **Switch to text mode** ) e cole o quadro 2.
**Body** : clique no campo → aba **Expression** → cole o quadro 3 → **OK** .

1 · MarcarAberto → Uri (aba Expression)

```
concat(outputs('Pedido')?['uri_itens'], '(', items('CadaReabrir')?['Id'],
')')
```

2 · MarcarAberto → Headers (modo texto) · com MERGE: atualiza sem apagar

```
{
  "Accept": "application/json;odata=nometadata",
  "Content-Type": "application/json;odata=nometadata",
  "IF-MATCH": "*",
  "X-HTTP-Method": "MERGE"
}
```

3 · MarcarAberto → Body (aba Expression)

```
json('{"MesFechado": "NÃO"}')
```

**Deu certo se:** pronto. Clique em **Save** .

**Importante:** o pessoal precisa estar no grupo **Membros** do site (como manda a B12). Quem tiver acesso por outro caminho pode continuar editando depois do fechamento.

# A7. Ponte 5: devolver a lista e ligar

15 min · A · Preparar

**Objetivo:** para puxar, fechar e reabrir, o fluxo grava a lista do mês num arquivo que o programa lê. Depois você liga e testa.

**1** ☑ EMBAIXO DA CAIXA ACAO (FORA DELA, NO FIM DO FLUXO)

Clique no **+ New step** lá embaixo → **Condition**.

Primeiro valor: clique no campo → aba **Expression** → cole → **OK** com o quadro abaixo.

Operador: **is not equal to**.

Segundo valor: **digite** `nenhuma` (aqui é texto, sem Expression). Assim o fluxo **sempre** devolve a lista no fim, e o programa sabe que terminou.

Condition → primeiro valor (aba Expression)

```
outputs('Pedido')?['acao']
```

**Deu certo se:** condição pronta, fora do Switch.

▼ **Não deu certo?**

**Se você já tinha colocado `publicar` no segundo valor:** troque por `nenhuma` e salve. Com publicar, o programa fica esperando uma resposta que nunca chega.

**2** ☑ DENTRO DO IF YES

**Add an action** → busque `Send an HTTP request to SharePoint` → clique nela. Renomeie: ··· → **Rename** → `ListaMes`.

**Site Address**: escolha **ControleSOxSCM42**.

**Method**: **GET**.

**Uri**: clique no campo → aba **Expression** → cole o quadro 1 → **OK**.

**Headers**: clique no ícone **T** à direita do campo (**Switch to text mode**) e cole o quadro 2.

1 · ListaMes → Uri (aba Expression)

```
outputs('Pedido')?['uri_lista']
```

2 · ListaMes → Headers (modo texto)

```
{
  "Accept": "application/json;odata=nometadata",
  "Content-Type": "application/json;odata=nometadata"
}
```

**Deu certo se:** a caixa ListaMes aparece sem ! vermelho.

**3** ☑ EMBAIXO DE LISTAMES

**Add an action** → busque `Create file` → escolha a do **OneDrive for Business** (nuvem azul). Renomeie para `Resposta`.

**Folder Path**: ícone de pasta → › em SOx_SCM42_ponte → **da_lista**.

**File Name**: clique no campo → aba **Expression** → cole → **OK** com o quadro 1.

**File Content**: clique no campo → aba **Expression** → cole → **OK** com o quadro 2.

1 · Resposta → File Name (aba Expression)

```
concat('lista_', outputs('Pedido')?['arquivo_comp'], '__',
outputs('Pedido')?['id'], '.json')
```

2 · Resposta → File Content (aba Expression)

```
string(body('ListaMes')?['value'])
```

**Deu certo se:** Resposta pronta. Clique em `Save` : aparece "Your flow is ready" sem aviso vermelho.

**4** ☑ TOPO DO EDITOR

Clique em `Flow checker` . Se houver Errors, clique em cada um e corrija (quase sempre um nome de caixa diferente do roteiro, ou expressão colada fora da aba Expression).

**Deu certo se:** Errors (0).

**5** ☑ MY FLOWS

Clique na seta ← (canto de cima à esquerda) e depois em `My flows` . O SOx Ponte deve estar **On**. Se estiver Off: ··· → `Turn on` .

**Deu certo se:** SOx Ponte ligado. Ele roda sozinho sempre que chegar um pedido.

**6** ☑ TESTE

Faça a etapa A8 (Publicar). Depois volte aqui: clique em SOx Ponte e veja `28-day run history` .

**Deu certo se:** aparece uma execução **Succeeded** uns 1 a 5 minutos depois de publicar.

▼ **Não deu certo?**

- **Nenhuma execução:** se a janela disse **Pedido gravado na pasta sincronizada**, o OneDrive ainda não subiu o arquivo (veja o ícone de nuvem perto do relógio) ou a pasta do gatilho não é /SOx_SCM42_ponte/pedidos.
- **Failed:** clique na execução; a caixa com X vermelho mostra o erro. Tire print e mande ao Claude.
- **403 / Access denied nas caixas HTTP:** você precisa ser Proprietário do site ControleSOxSCM42.

# A8. Publicar o mês na lista

10 min · A · Preparar

**Objetivo:** o programa manda os contratos do mês para a lista, pela ponte. Na primeira vez o fluxo também cria a lista e as colunas.

**1** ☑ WINDOWS

Abra o **Outlook clássico** e deixe aberto. É por ele que o Python acha o e-mail de cada nome.

**Deu certo se:** o Outlook está aberto.

**2** ☑ SEU PROGRAMA

Abra o programa. Em **Mês** e **Ano** coloque a competência da planilha atual. Para a planilha `2026.10.01 SOx SCM-42 - Alerta` é **9** e **2026**.

**Deu certo se:** mês e ano preenchidos.

**3** ☑ SEU PROGRAMA

Clique em **Publicar online agora**. Não aparece mais código de login.

**Deu certo se:** na janela do programa aparece **Pedido enviado direto ao OneDrive online** (ou, sem cookies do Sigma-C, **Pedido gravado na pasta sincronizada**), depois **... 0 min 30 s esperando o fluxo** a cada 30 segundos, e no fim **Pronto: 231 de 231 contratos estão na lista online**. Abre a mensagem **Publicado na lista online** e o navegador mostra a lista. Normal: 1 a 5 minutos (a primeira vez, até 10).

▼ **Não deu certo?**

- **Ainda pede código de login:** o ajuste da A2 não é o mais novo. Rode de novo a A2 com o ajuste_app.py mais recente.
- **"Não achei a pasta do OneDrive":** faça a A3 e abra o OneDrive.
- **"OneDrive online não aceitou os cookies" ou "Sem cookies SSO salvos":** o programa usa os mesmos cookies do Sigma-C e das certidões ( `Downloads\automatizacao certidao` ). Abra o Sigma-C ou as certidões uma vez para renovar o login e publique de novo. Enquanto isso ele usa a pasta sincronizada, que precisa do OneDrive sincronizando.
- **Fica só esperando:** aos 2 minutos o programa lembra de conferir o OneDrive; aos 5, o histórico do fluxo. Depois de 20 minutos ele desiste e mostra o que conferir. Nada se perde: se o fluxo terminar depois, os contratos aparecem na lista.
- **"Publicado, mas com falhas":** a janela do programa lista os contratos que não entraram; mande print ao Claude.

**4** ☑ LISTA NO NAVEGADOR

Quando o programa disser **Pronto**, aperte `F5` na aba da lista. Se aparecerem só cartões com o número do contrato, clique em **Editar no modo de exibição de grade** (barra de cima): vira uma tabela. Role a tabela para a direita. Não precisa editar nada.

**Deu certo se:** aparecem os contratos numa tabela, com as colunas Contrato, Fornecedor, Situacao, FiscalEmail, GerenteEmail, OkFiscal e as outras.

▼ **Não deu certo?**

Para sair da tabela: **Sair do modo de exibição de grade**, no mesmo lugar do botão. Os cartões são a exibição feita para o pessoal; a tabela é só para conferir.

**5** ☑ LISTA NO NAVEGADOR

Na mesma tabela, escolha 3 contratos quaisquer e leia o que está escrito nas colunas `FiscalEmail` e `GerenteEmail` (role para a direita até achá-las).

**Deu certo se:** estão como `;nome.sobrenome@petrobras.com.br;` (com ; nas pontas).

▼ **Não deu certo?**

Se estiver vazio em muitos: o Outlook não achou o nome. Abra o Outlook clássico e publique de novo. Se for só um contrato, digite o e-mail na lista, no mesmo formato.

# A9. Conferir as colunas de opções

5 min · A · Preparar

**Objetivo:** no app, a pessoa escolhe SIM/NÃO ou o motivo numa lista, igual à planilha. O fluxo já cria essas 6 colunas como Escolha; aqui você só confere.

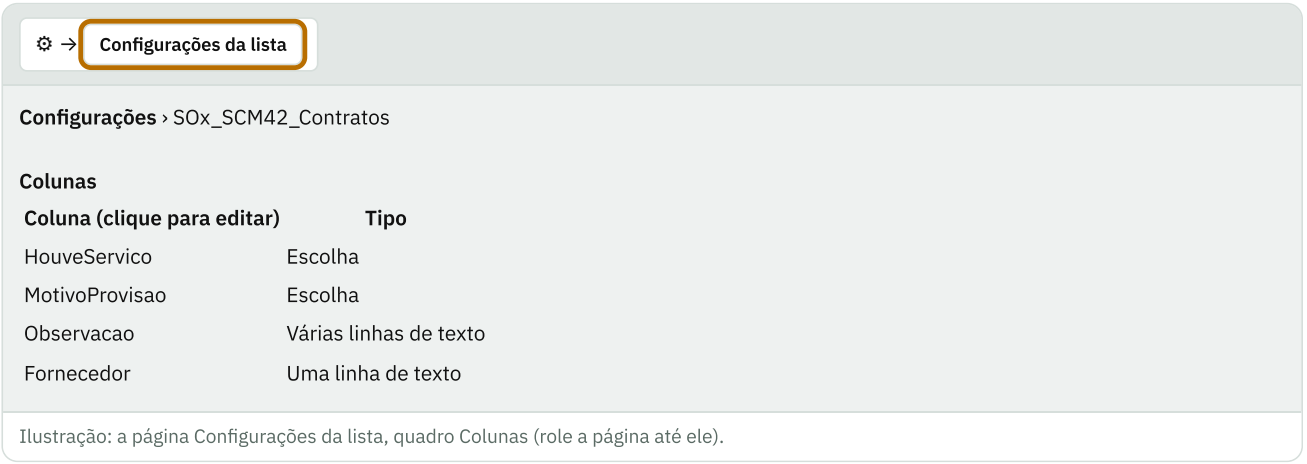



Ilustração: a página Configurações da lista, quadro Colunas (role a página até ele).

**1** ☑ LISTA → ENGRENAGEM ⚙ (CANTO DE CIMA À DIREITA, NA BARRA VERDE)

Com a lista aberta, clique na engrenagem ⚙ → **Configurações da lista**. Abre uma página branca com o título **Configurações**. Desça até o quadro **Colunas**.

**Deu certo se:** um quadro com 3 colunas de texto: **Coluna** (nome), **Tipo** e **Obrigatório**. É uma linha por coluna da lista.

▼ **Não deu certo?**

Se na engrenagem não aparecer **Configurações da lista**: clique em **Retornar para o SharePoint clássico** (embaixo, à esquerda) e use a engrenagem de novo;. Os cartões com só o número do contrato não mostram os nomes das colunas; por isso este caminho.

**2** ☑ QUADRO COLUNAS

Ache estas 6 linhas e leia o que está em **Tipo**: `HouveServico`, `FoiMedido`, `ForaDoPrazo`, `SeraSubstituido`, `DemandaContinuada`, `MotivoProvisao`.

**Deu certo se:** nas 6 o Tipo é **Opção** (ou Escolha, conforme a tela).

▼ **Não deu certo?**

Se alguma estiver como **Uma linha de texto** (acontece se a lista já existia antes da ponte): clique no nome dela → em **O tipo de informação nesta coluna é** marque **Opção (menu para escolher)** (em algumas telas: Escolha) → no quadro **Digite cada opção em uma linha separada** deixe uma por linha: `SIM` e `NÃO` (para MotivoProvisao: Atraso do Fornecedor, Atraso Fiscalização de Campo, Problema Interno GPS, Contrato Bloqueado, Outros) → **Exibir opções usando**: **Menu Suspenso** → **Permitir opções de preenchimento**: **Sim** → **Valor padrão**: caixa vazia → **OK** (fim da página). Se avisar que pode perder dados, confirme: os valores continuam.

**3** ☑ QUADRO COLUNAS

Ache as linhas `CSP`, `Justificativa` e `Observacao` e leia o **Tipo**.

**Deu certo se:** as 3 são **Várias linhas de texto**. Pronto: volte para a lista pelo nome dela, no topo da página.

▼ **Não deu certo?**

Se alguma estiver como Uma linha de texto: clique no nome → marque **Várias linhas de texto** → `Especifique o tipo de texto a ser permitido` : **Texto sem formatação** → `OK` .

# A10. Ponte 6: caso apagar

10 min · A · Preparar

**Objetivo: Reserva:** desde 26/10 o programa faz isto direto, sem fluxo; pode pular esta etapa. ao publicar, a lista do mês fica igual à planilha: o contrato que saiu da planilha é apagado da lista (também os de teste, como DEMO-PROV-001). Os que continuam são só atualizados e mantêm o que o pessoal respondeu. Rode antes a A2 com o ajuste_app.py mais novo.

**1** ☑ POWER AUTOMATE → SOX PONTE

Abra o fluxo e clique em **Edit**. Clique na caixa `Acao` (Switch) para abrir os casos.

**Deu certo se:** você vê Case, Case 2, Case 3 e Default.

**2** ☑ CAIXA ACAO → +

Clique no **+** entre o Case 3 e o Default ( **Add a case** ). No novo **Case 4**, **Equals** : `apagar` .

**Deu certo se:** Case 4 com apagar.

**3** ☑ CASE 4 → ADD AN ACTION

**Add an action** → `Apply to each` . Renomeie para `CadaApagar` .

**Select an output from previous steps** : clique no campo → aba **Expression** → cole → **OK** . ··· → **Settings** → **Concurrency Control** **On**, **Degree of Parallelism** **5** → **Done** .

CadaApagar → Select an output (aba Expression)

```
outputs('Pedido')?['ids']
```

**Deu certo se:** CadaApagar pronta.

**4** ☑ DENTRO DE CADAAPAGAR → ADD AN ACTION

**Add an action** → busque `Send an HTTP request to SharePoint` → clique nela. Renomeie: ··· → **Rename** → `Apagar` .

**Site Address** : escolha **ControleSOxSCM42**.

**Method** : **POST**.

**Uri** : clique no campo → aba **Expression** → cole o quadro 1 → **OK** .

**Headers** : clique no ícone **T** à direita do campo ( **Switch to text mode** ) e cole o quadro 2.

1 · Apagar → Uri (aba Expression)

```
concat(outputs('Pedido')?['uri_itens'], '(', string(items('CadaApagar')),
')')
```

2 · Apagar → Headers (modo texto)

```
{
  "Accept": "application/json;odata=nometadata",
  "IF-MATCH": "*",
  "X-HTTP-Method": "DELETE"
}
```

**Deu certo se:** Apagar pronta, sem ! vermelho. Clique em **Save** .

5 ☑ SEU PROGRAMA

Clique em **Publicar online agora** (mesmo mês e ano da A8).

**Deu certo se:** a janela mostra **Apagando da lista N contrato(s) que não estão mais na planilha: DEMO-PROV-001...** e depois **Apagados: N**. Na lista, os DEMO sumiram.

▼ **Não deu certo?**

**"Não consegui apagar"**: o Case 4 não está com `apagar` escrito igual, ou a caixa Apagar falhou (veja o histórico do fluxo). **DEMO continua e o programa não falou em apagar:** esse contrato está com outra competência; apague à mão na lista (marque → Excluir).

# B1. O app que já foi gerado (já feito)

5 min · B · Montar o app

**Objetivo:** você já clicou em Integrar → Power Apps → Criar um aplicativo, e o Power Apps montou a tela abaixo. **Não crie outro app.** Todo o resto da parte B só ajusta essa tela: esconde o que não serve, troca textos e acrescenta filtros, avisos e botões.

**Hoje** (o que o Power Apps gerou):

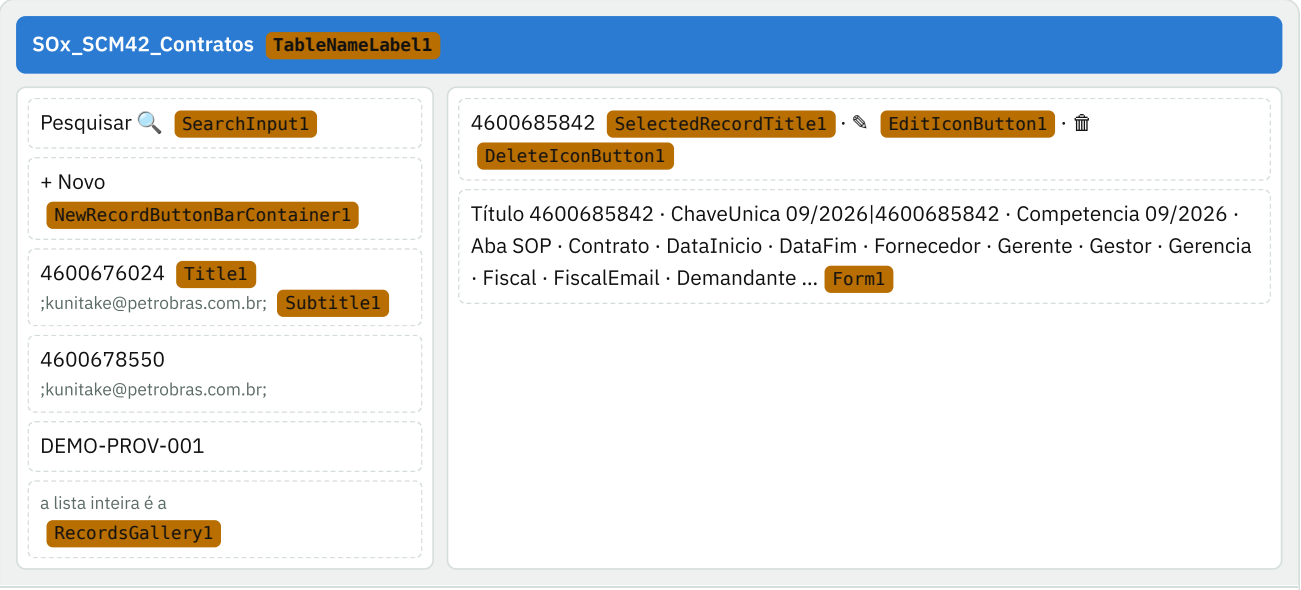



Ilustração da tela que o Power Apps já gerou (igual ao seu print). Em amarelo, o nome de cada parte na árvore da esquerda.

**Como fica** no fim da B10:

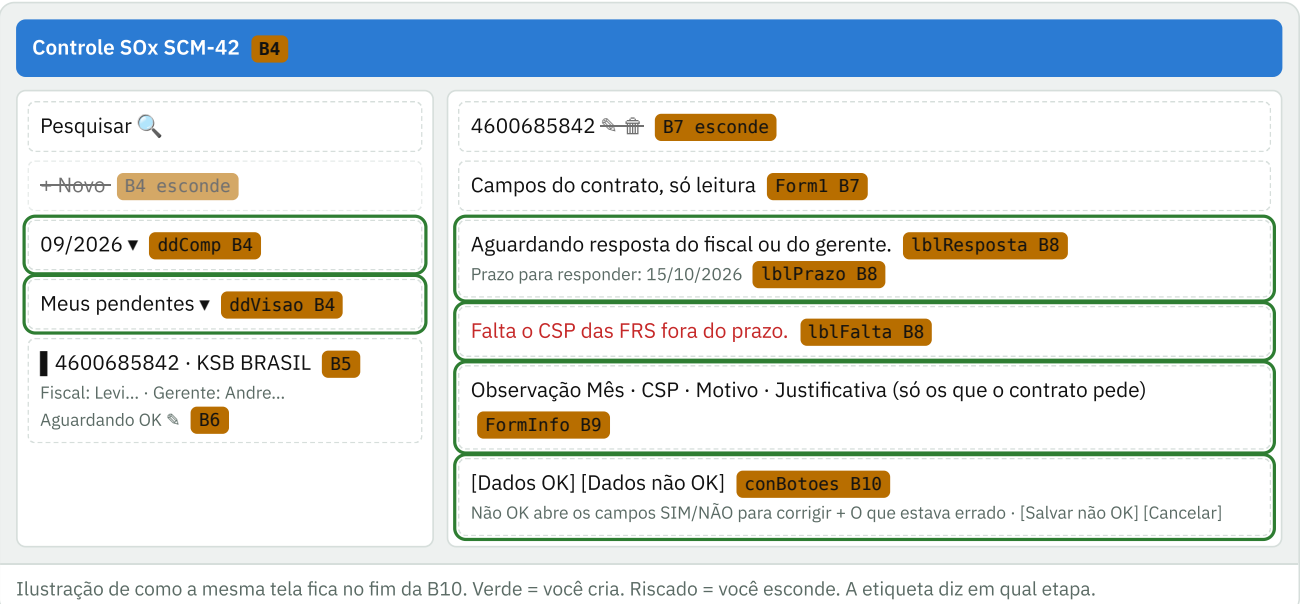



Ilustração de como a mesma tela fica no fim da B10. Verde = você cria. Riscado = você esconde. A etiqueta diz em qual etapa.

1 ☑ MAKE.POWERAPPS.COM → APLICATIVOS

Se o Studio estiver fechado: make.powerapps.com → Aplicativos → três pontinhos em **Controle SOx SCM-42** → Editar.

**Deu certo se:** abre a tela da ilustração **Hoje**: título azul SOx_SCM42_Contratos, Pesquisar, + Novo, lista com

4600676024 e o e-mail do gerente, e à direita o contrato com ✎ e 🗑.

▼ **Não deu certo?**

Se o app ainda não existir: na lista, **Integrar** → **Power Apps** → **Criar um aplicativo**, nome `Controle SOx SCM-42`, **Criar**.

2 ☑ ÁRVORE (ESQUERDA DO STUDIO)

Compare a sua tela com a ilustração **Hoje**. Clique na árvore em cada nome amarelo e veja qual parte da tela fica com borda: `TableNameLabel1`, `SearchInput1`, `NewRecordButtonBarContainer1`, `RecordsGallery1`, `EditIconButton1`, `DeleteIconButton1`, `Form1`.

**Deu certo se:** você sabe qual nome da árvore é cada parte da tela. Não mude nada ainda.

3 ☑ POWER APPS STUDIO

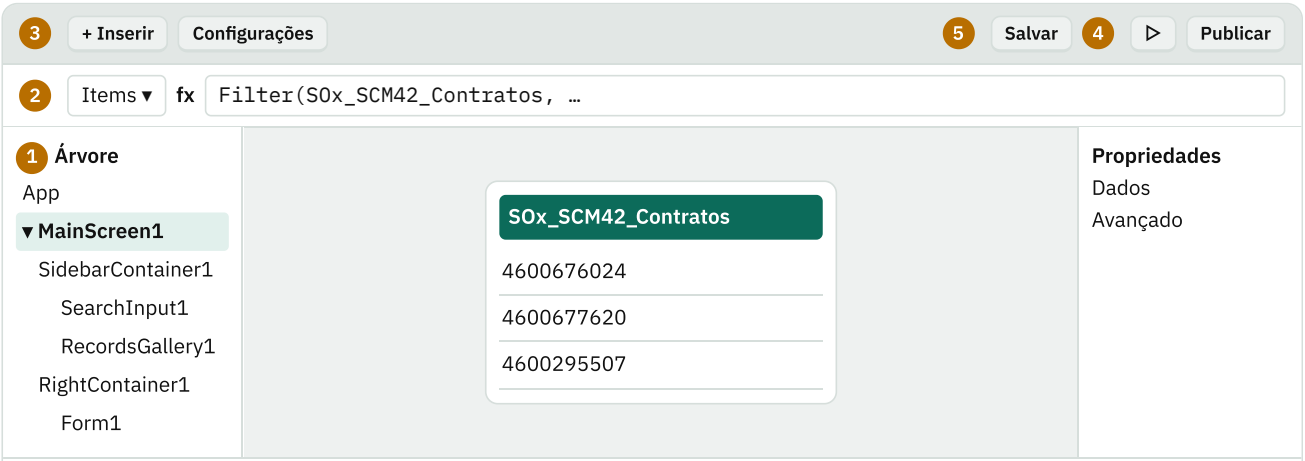
Salve: `Ctrl` + `S`. Faça isso no fim de cada etapa.

**Deu certo se:** aparece "Todas as alterações foram salvas" ou o ícone de salvar fica cinza.

# B2. Conhecer o editor (só olhar)

5 min · B · Montar o app

**Objetivo:** achar as partes do editor que você vai usar. Nada é alterado.

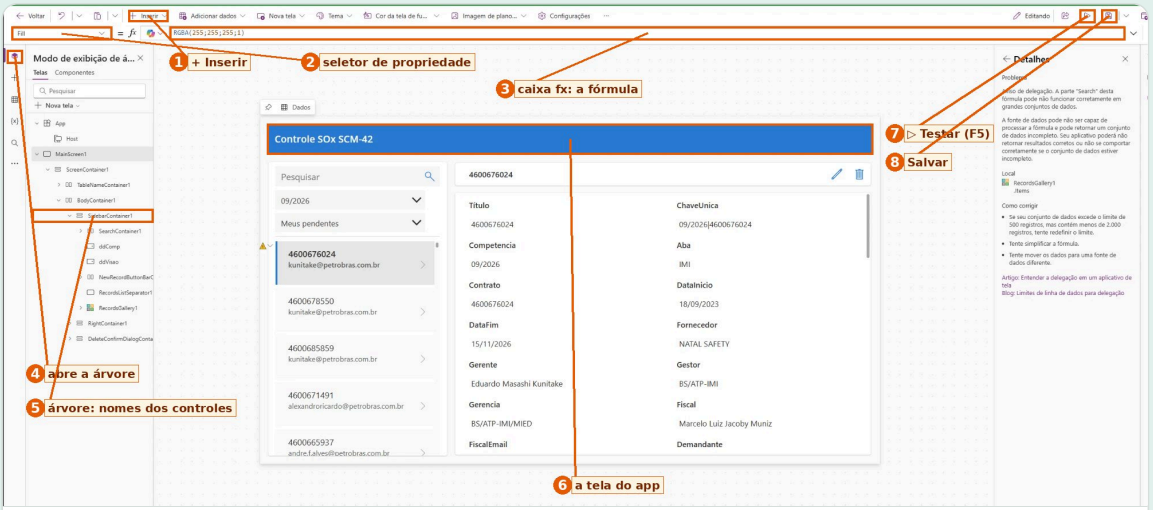



1 **Árvore**: lista de telas e controles. Clicar num nome escolhe o controle.
2 **Seletor de propriedade + caixa fx**: à esquerda escolhe a propriedade (Text, Items, OnSelect…); à direita você cola a fórmula.
3 **+ Inserir**: adiciona botão, rótulo, lista suspensa. Tem caixa de busca.
4 ▷ ou `F5`: testa o app. `Esc` volta.
5 **Salvar** ou `Ctrl` + `S`.

Ilustração do Power Apps Studio. Se a árvore estiver fechada, clique no ícone de camadas (três quadrados empilhados) na faixa da esquerda.

**1** ☑ POWER APPS STUDIO

Ache na sua tela as 5 partes numeradas da ilustração acima.




PRINT DO SEU APP Seu Studio: 1 + Inserir · 2 seletor de propriedade · 3 caixa fx (onde se cola a fórmula) · 4 ícone que abre a árvore · 5 árvore com os nomes dos controles · 6 a tela do app · 7 ▷ Testar · 8 Salvar. Clique para ampliar.

**Deu certo se:** você sabe onde estão a árvore, a caixa fx, o + Inserir, o ▷ e o Salvar.

**2** ☑ ÁRVORE

Clique em `RecordsGallery1`.

**Deu certo se:** a lista da esquerda (4600676024, 4600678550, DEMO-PROV-001…) fica com borda.

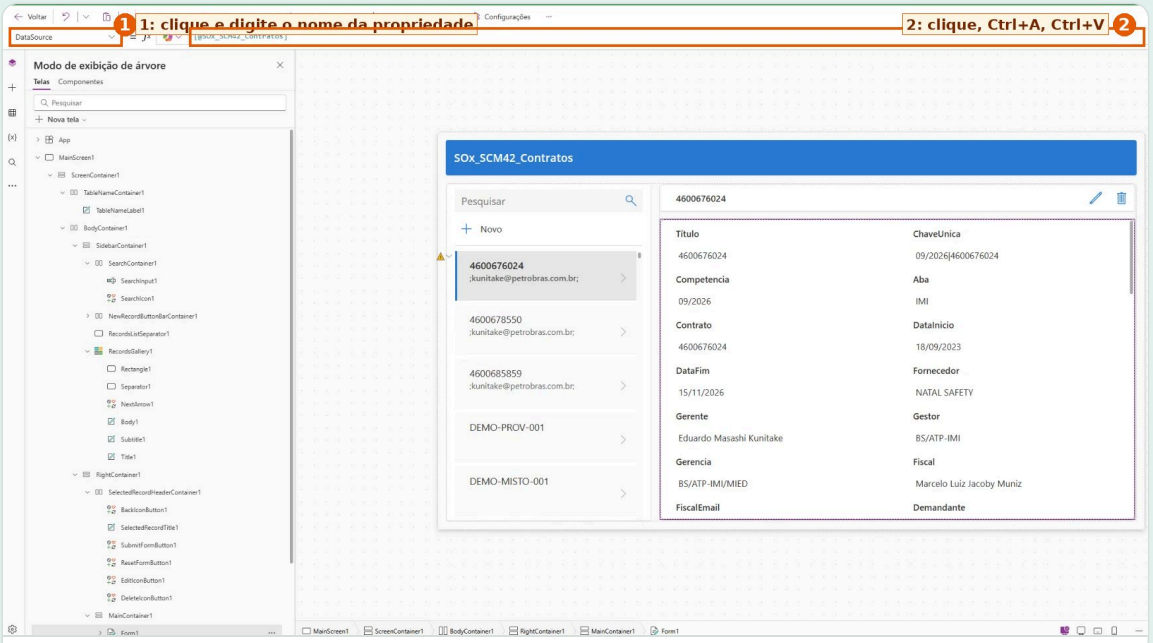
**3** ☑ SELETOR DE PROPRIEDADE

Clique no menu à esquerda do **fx** e escolha `Items`.

**Deu certo se:** a caixa fx mostra uma fórmula. Não mude nada.

## 4 ☑ COMO COLAR FÓRMULA (VALE PARA O ROTEIRO TODO)

Sempre: **clicar no controle na árvore → escolher a propriedade no menu → clicar na caixa fx → `Ctrl` + `A` → `Ctrl` + `V`.**

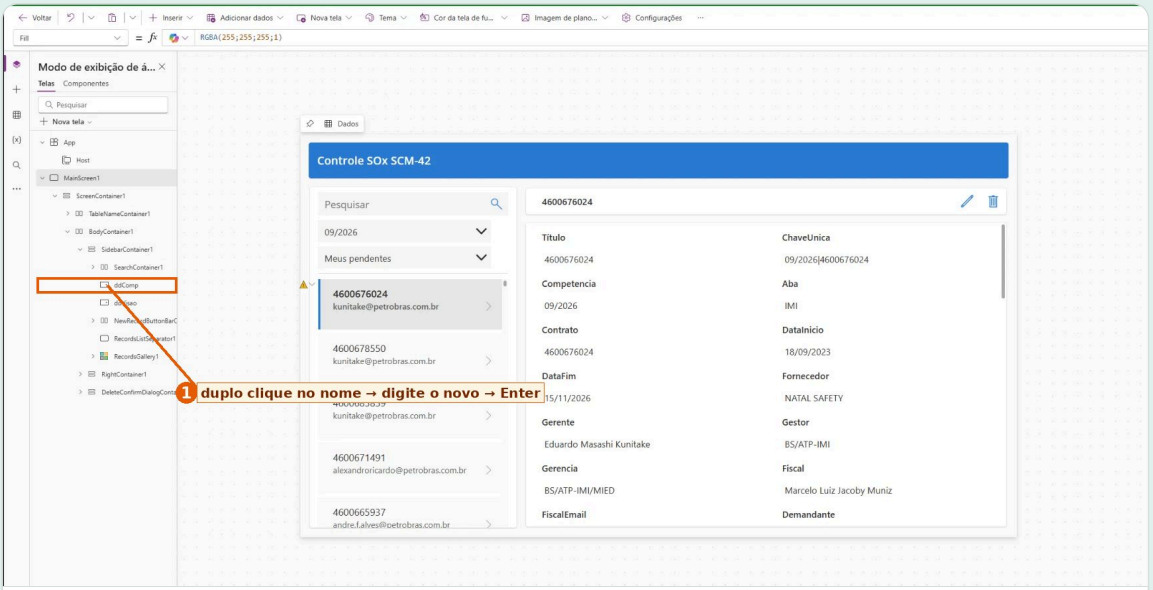



**PRINT DO SEU APP** No seu Studio: 1 é o seletor de propriedade (hoje mostra DataSource); clique nele e digite o nome da propriedade (Text, Items, Visible...). 2 é a caixa fx: clique, Ctrl+A e Ctrl+V. Clique para ampliar.

**Deu certo se:** você entendeu o gesto.

## 5 ☑ COMO RENOMEAR

Na árvore, **dois cliques** no nome do controle, digite o nome novo, `Enter`.




**PRINT DO SEU APP** Para renomear: duplo clique no nome do controle na árvore, digite o novo nome e Enter (ou ⋯ → Renomear). Clique para ampliar.

**Deu certo se:** você entendeu.

## 6 ☑ COMO INSERIR DENTRO DE UM CONTÊINER

A tela é feita de caixas (contêineres) que arrumam o que está dentro sozinhas. Para pôr algo dentro de uma caixa: **clique nela na árvore →** `+ Inserir`. O controle novo entra no fim da caixa.

Para mudar a ordem: arraste o nome na árvore, ou três pontinhos `...` → `Reordenar` → `Mover para cima`.

**Deu certo se:** você entendeu.

↗ Microsoft: Power Apps Studio (fotos reais)

# B3. Configurações de início

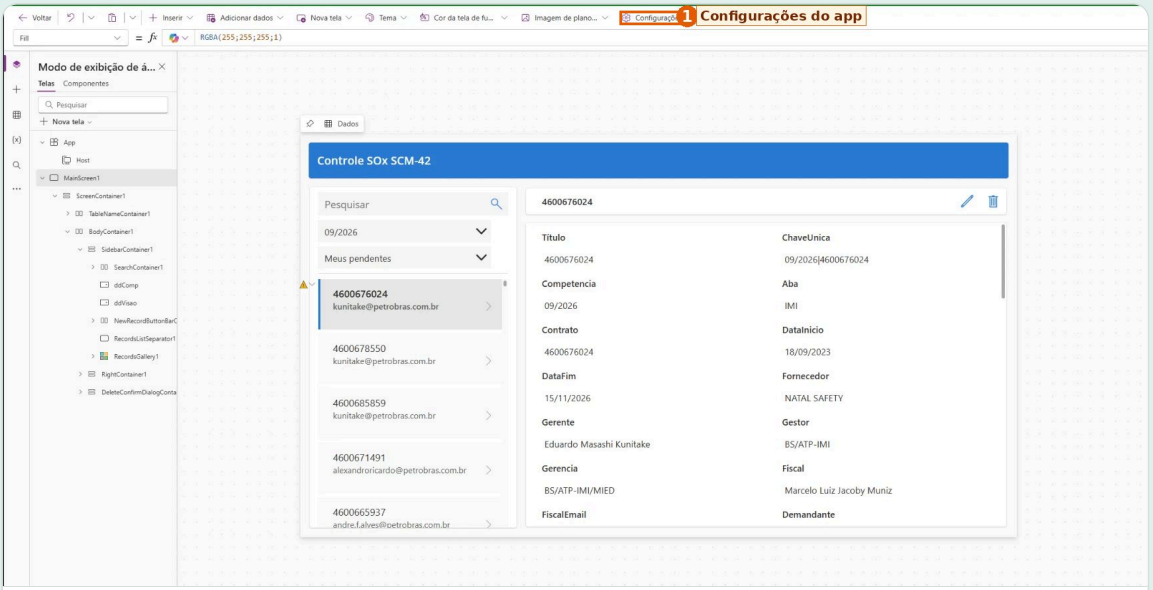
10 min · B · Montar o app

**Objetivo:** o app saber quem está usando, qual mês abrir, quem é o administrador, qual contrato está aberto e se a pessoa pode responder nele.

**Três regras que evitam os erros da B4 (valem para toda a parte B):**
1. O seletor **Seu Power Apps está em**, no topo, fica em **Português**. Todo quadro de fórmula mostra a etiqueta **formato português**: separa com `;` e encadeia com `;;`. Erro com **"Error" em que se espera BracketClose/ParenClose** = a fórmula foi colada com vírgula; copie de novo.
2. Antes de colar: clique no controle na árvore, **confira o nome da propriedade no seletor** (Items, Default, Text...), clique na caixa fx, **Ctrl+A** e **Ctrl+V**. Exceções: **App → Formulas** e **RecordsGallery1 → OnSelect**, onde você acrescenta no fim sem apagar.
3. Depois de colar, espere 2 segundos. Apareceu X vermelho: passe o mouse ou clique no X, leia os **Detalhes** (Local diz o controle e a propriedade) e compare com o roteiro. Só siga quando não houver X vermelho. Triângulo amarelo de delegação pode ficar.

**1** ☑ STUDIO → TOPO

Clique em **Configurações** (ou na engrenagem ⚙) → **Geral**. Em **Limite de linhas de dados** troque 500 por `2000`. Feche a janela.




PRINT DO SEU APP Botão Configurações na barra de cima do seu Studio. Clique para ampliar.

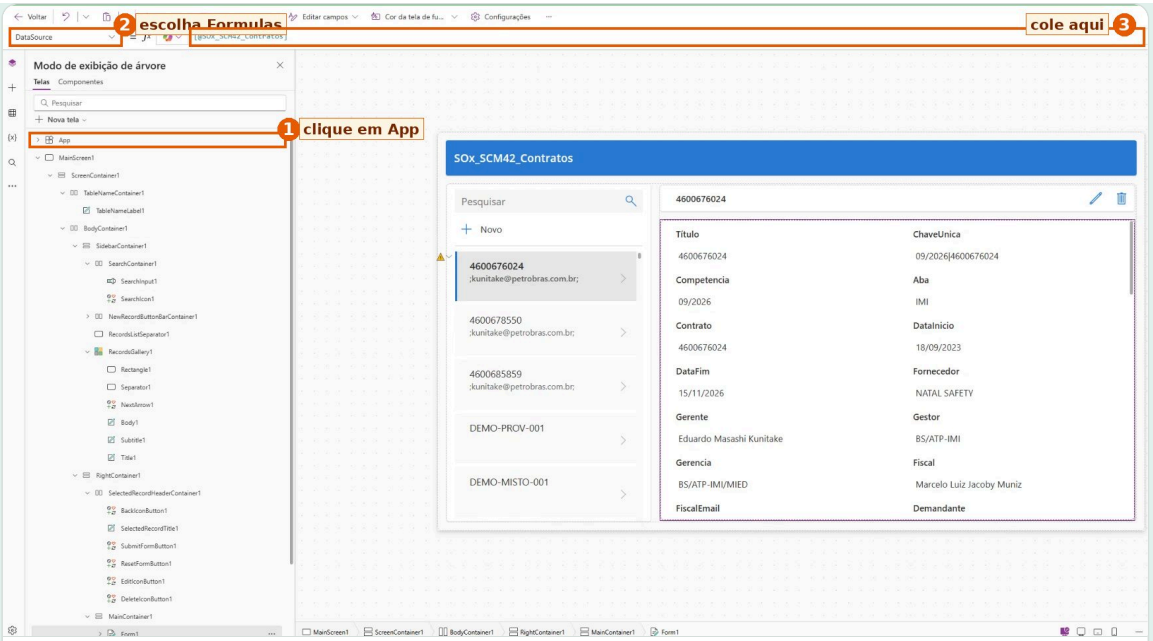
**Deu certo se:** o número ficou 2000.

**2** ☑ ÁRVORE → APP

**Hoje** o app não sabe quem é você. Clique em `App` (o primeiro item da árvore). No seletor escolha `Formulas`. **Atenção:** se a caixa fx já tiver algum texto, **não use Ctrl+A** (o app gerado pode usar o que está lá). Clique no fim do texto, aperte Enter, digite `;;` (português) ou `;` (inglês), Enter de novo e só então cole. Se estiver vazia, cole direto:

App → Formulas **formato português ( ; e ;; )**

```
varEmail = Lower(User().Email);;
varNome = User().FullName;;
varComp = Text(DateAdd(Today(); -1; TimeUnit.Months); "mm/yyyy");;
varAdmin = varEmail in ["levi.vasconcelos@petrobras.com.br"];;
varItem = RecordsGallery1.Selected;;
varMeuFG = ";" & varEmail & ";" in ";" &
Substitute(Lower(varItem.FiscalEmail & ";" & varItem.GerenteEmail); " ";
"") & ";";;
varApoioContratos = Filter(SOx_SCM42_Apoio; Email = varEmail).Title;;
varApoio = varItem.Contrato in varApoioContratos;;
varMeu = varMeuFG || varApoio;;
varPrazo = Date(Value(Right(varItem.Competencia; 4));
Value(Left(varItem.Competencia; 2)) + 1; 15);;
varFechado = varItem.MesFechado = "SIM";;
varPode = varAdmin || (varMeu && !varFechado);;
varValida = varAdmin || (varMeuFG && !varFechado);;
varFrsSemCsp = Concat(
    Filter(Split(varItem.FRSPendentes; "|") As f;
      !IsBlank(Trim(f.Value));
      CountRows(
        Filter(Split(Substitute(Substitute(varItem.CSP; Char(13); "");
Char(10); ";"); ";") As b;
          !IsBlank(Find(":"; b.Value));
          !IsBlank(Trim(Mid(b.Value; Find(":"; b.Value) + 1)));
          With({esq: Lower(Trim(Left(b.Value; Find(":"; b.Value) - 1)))};
            esq in ["*"; "todos"; "tudo"] || StartsWith(esq; "todas") ||
Trim(f.Value) in esq))) = 0);
    Trim(Value); ", ");;
varCobre = IsBlank(varFrsSemCsp);;
varPedeCSP = !IsBlank(varItem.FRSPendentes) || varItem.ForaDoPrazo.Value =
"SIM";;
varPedeMotivo = varItem.Aba = "SOP" && varItem.Situacao in
["Provisionado"; "Misto"];;
varPedeJust = varItem.Situacao in ["Provisionado"; "Fora do prazo";
"Misto"; "Incompleto"];;
varPedeServico = varItem.Situacao = "Sem medição";;
varPedeInfo = varPedeCSP || varPedeMotivo || varPedeJust ||
varPedeServico;;
varFalta = Concat(
  Filter(
    Table(
      {m: If(IsBlank(varFrsSemCsp); "Falta o CSP."; "Falta o CSP destas
FRS fora do prazo: " & varFrsSemCsp & " (preencha ao lado de cada FRS e
clique em Dados OK)."); f: varPedeCSP && (IsBlank(Trim(varItem.CSP)) ||
!varCobre)};
      {m: "Falta o motivo da provisão."; f: varPedeMotivo &&
IsBlank(varItem.MotivoProvisao.Value)};
      {m: "Falta a justificativa (mínimo 15 letras)."; f: varPedeJust &&
Len(Trim(varItem.Justificativa)) < 15});
    f);
  m; " ");;
```




**PRINT DO SEU APP** 1 clique em App na árvore · 2 no seletor escolha Formulas · 3 cole na caixa fx. Clique para ampliar.

**Deu certo se:** sem sublinhado vermelho. Em palavras: `varItem` é o contrato escolhido na lista da esquerda; `varMeu` diz se você é o fiscal **ou** o gerente dele; `varPode` diz se você ainda pode responder (até o dia 15 do mês seguinte à competência e com o mês aberto; você, admin, sempre).

▼ **Não deu certo?**

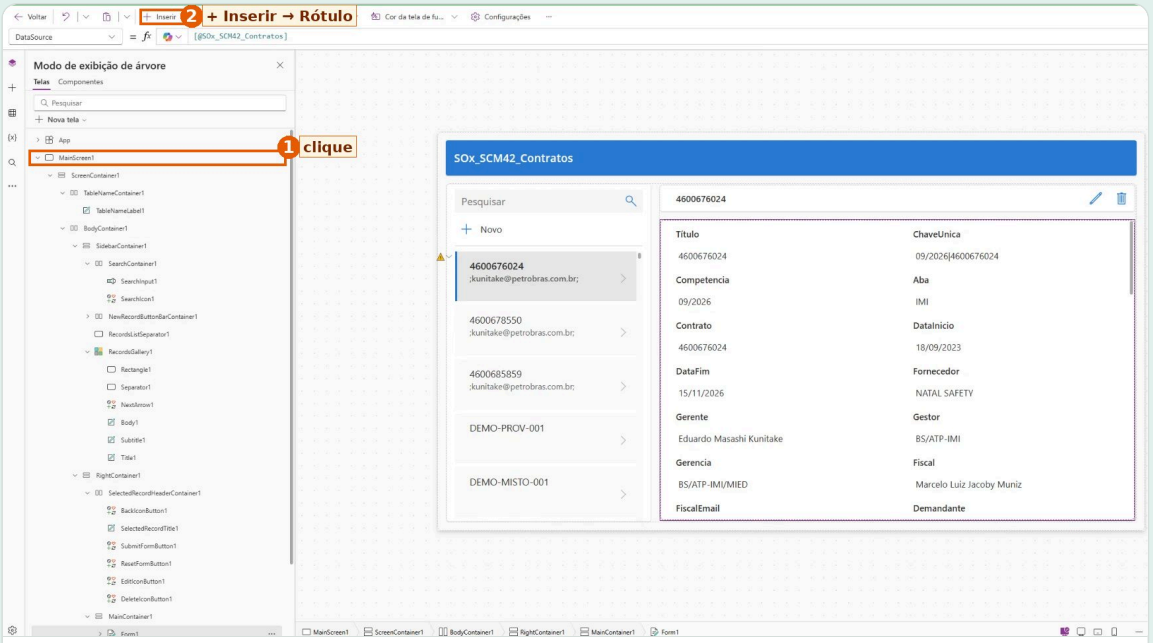
Se o seu e-mail de login não for levi.vasconcelos@petrobras.com.br, troque entre as aspas. Se o seletor não tiver `Formulas` : **Configurações** → **Atualizações** → ligue **Fórmulas nomeadas**.

## 3 ☑ ÁRVORE → MAINSCREEN1

Clique em `MainScreen1` . **+ Inserir** → busque `rótulo` → **Rótulo de texto** . Propriedade `Text` :

Rótulo de teste → Text **formato português ( ; e ;; )**

```
varEmail & " | admin: " & varAdmin
```




**PRINT DO SEU APP** 1 clique em MainScreen1 · 2 + Inserir → Rótulo de texto. Clique para ampliar.

**Deu certo se:** o rótulo mostra o seu e-mail (o mesmo da coluna FiscalEmail da lista) e `admin: true`.

▼ **Não deu certo?**

Se mostrar outro e-mail (por exemplo sua chave), avise o Claude. Se mostrar admin: false, troque o e-mail nas Formulas.

4 ☑ TELA DO MEIO

Clique no rótulo de teste e aperte `Delete`. Salve.

**Deu certo se:** o rótulo sumiu.

↗ **Microsoft: vírgula ou ponto e vírgula nas fórmulas**

# B4. Lado esquerdo: título e filtros de mês e de visão

15 min · B · Montar o app

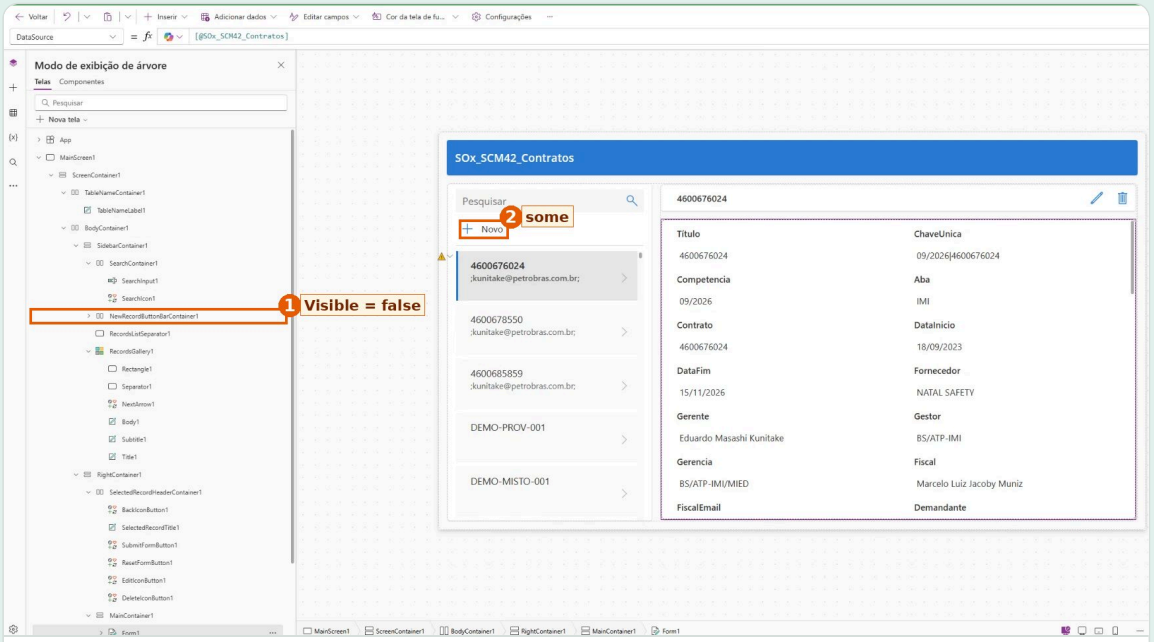
**Objetivo: Hoje** o título azul diz SOx_SCM42_Contratos, existe o **+ Novo** e a lista mistura todos os meses. **Fica:** título Controle SOx SCM-42, sem + Novo, e dois filtros embaixo do Pesquisar (mês e visão).

**1** ☑ ÁRVORE → NEWRECORDBUTTONBARCONTAINER1

**Hoje** aparece **+ Novo** embaixo do Pesquisar; ninguém cria contrato pelo app (quem cria é o Publicar). Clique em `NewRecordButtonBarContainer1` na árvore. Propriedade `Visible`:

NewRecordButtonBarContainer1 → Visible **formato português ( ; e ;; )**

```
false
```




**PRINT DO SEU APP** 1 NewRecordButtonBarContainer1 na árvore · 2 é o + Novo que vai sumir. Clique para ampliar.

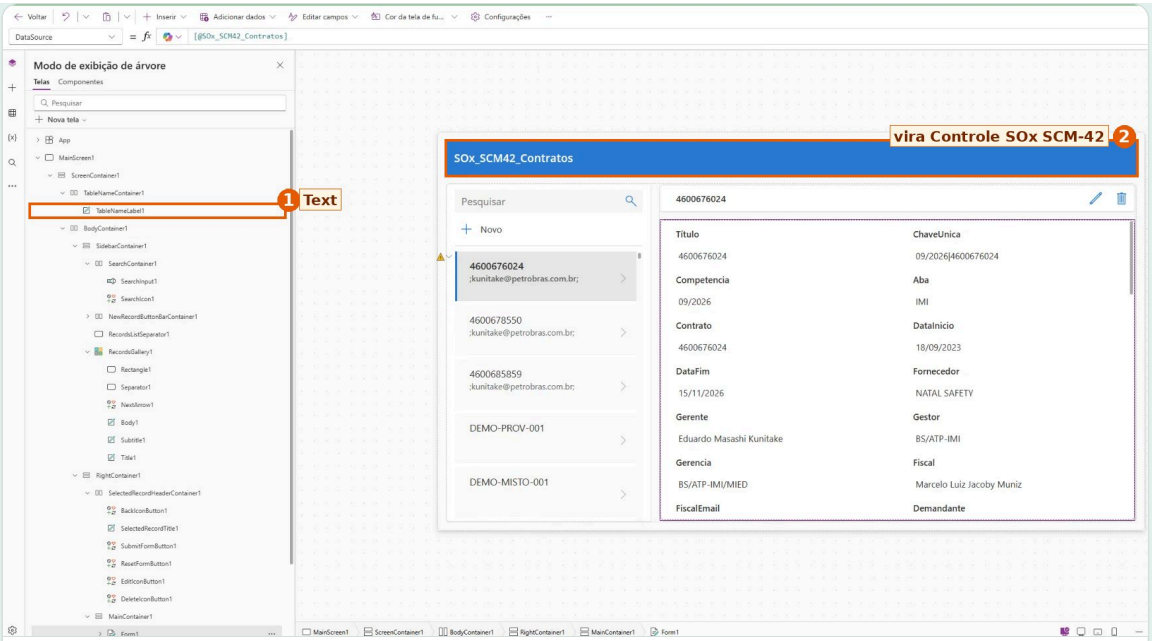
**Deu certo se:** o + Novo sumiu.

**2** ☑ ÁRVORE → TABLENAMELABEL1

**Hoje** o título azul diz SOx_SCM42_Contratos. Clique em `TableNameLabel1`. Propriedade `Text`:

TableNameLabel1 → Text **formato português ( ; e ;; )**

```
"Controle SOx SCM-42"
```

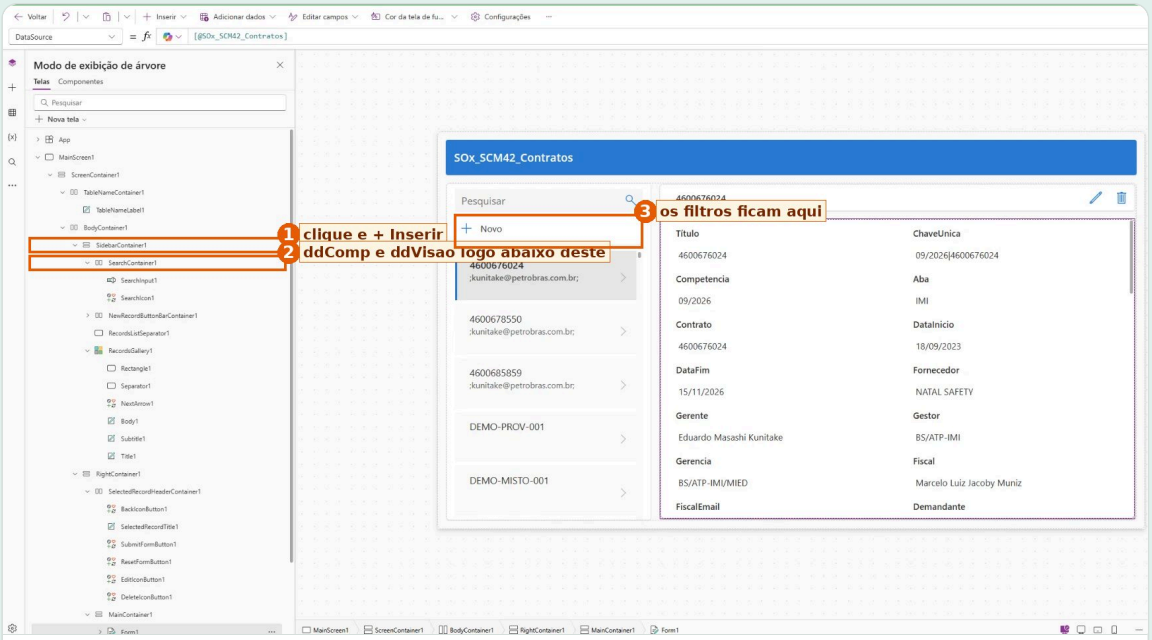



**PRINT DO SEU APP** 1 TableNameLabel1 na árvore · 2 é o título azul. Clique para ampliar.

**Deu certo se:** o título mudou.

## 3 ÁRVORE → SIDEBARCONTAINER1

Clique em `SidebarContainer1` . **+ Inserir** → busque `lista suspensa` → **Lista suspensa** (se houver duas, a de **Clássico**). Renomeie para `ddComp` . Na árvore, arraste `ddComp` para logo abaixo de `SearchContainer1` .




**PRINT DO SEU APP** 1 clique em SidebarContainer1 e insira · 2 arraste ddComp e ddVisao para logo abaixo de SearchContainer1 · 3 é onde eles aparecem (no lugar do + Novo). Clique para ampliar.

**Deu certo se:** aparece uma caixa com setinha ▾ embaixo do Pesquisar (onde estava o + Novo).

## 4 DDCOMP

Propriedade `Items` :

ddComp → Items **formato português ( ; e ;; )**

```
ForAll(Sequence(12); Text(DateAdd(Today(); -Value; TimeUnit.Months);
"mm/yyyy"))
```

**Deu certo se:** ao clicar na setinha aparecem os últimos 12 meses.

**5** ☑ DDCOMP

Propriedade `Default` :

ddComp → Default **formato português ( ; e ;; )**

```
varComp
```

**Deu certo se:** a caixa mostra o mês anterior.

▼ **Não deu certo?**

**Erro "Valor Text esperado":** você colou com colchetes. Na lista suspensa clássica (a que tem a propriedade Default) cole só `varComp` , sem colchetes. **Sem a propriedade Default**, só DefaultSelectedItems? Aí é a versão moderna: cole `[varComp]` em DefaultSelectedItems.

**6** ☑ ÁRVORE → SIDEBARCONTAINER1

Clique de novo em `SidebarContainer1` e insira outra **Lista suspensa** . Renomeie `ddVisao` e arraste para logo abaixo de `ddComp` . `Items` :

ddVisao → Items **formato português ( ; e ;; )**

```
["Meus pendentes"; "Meus contratos"; "Não OK"; "Todos"]
```

**Deu certo se:** a caixa lista as 4 opções.

**7** ☑ DDVISAO

Propriedade `Default` :

ddVisao → Default **formato português ( ; e ;; )**

```
"Meus pendentes"
```

**Deu certo se:** a caixa mostra Meus pendentes. Salve.

▼ **Não deu certo?**

Mesma regra da B4.5: com a propriedade Default, sem colchetes. Só na versão moderna (sem Default) cole `["Meus pendentes"]` em DefaultSelectedItems.

**8** ☑ ÁRVORE → SIDEBARCONTAINER1 → MÊS E GESTOR LADO A LADO

Objetivo: **mês e gestor na mesma linha**, dentro da coluna da esquerda (veja o 1º print), e não uma coluna nova ao lado de tudo. Por isso o contêiner novo vai **dentro de SidebarContainer1** (a coluna da esquerda), nunca em BodyContainer1 ou ScreenContainer1. Clique em `SidebarContainer1` → **+ Inserir** → busque `horizontal` → **Contêiner horizontal** . Renomeie `conFiltros` e, na árvore, arraste para logo abaixo de `SearchContainer1` (acima de `ddComp` ). Cole:

conFiltros → Height **formato português ( ; e ;; )**

```
44
```

conFiltros → FillPortions (desliga a altura flexível) **formato português ( ; e ;; )**

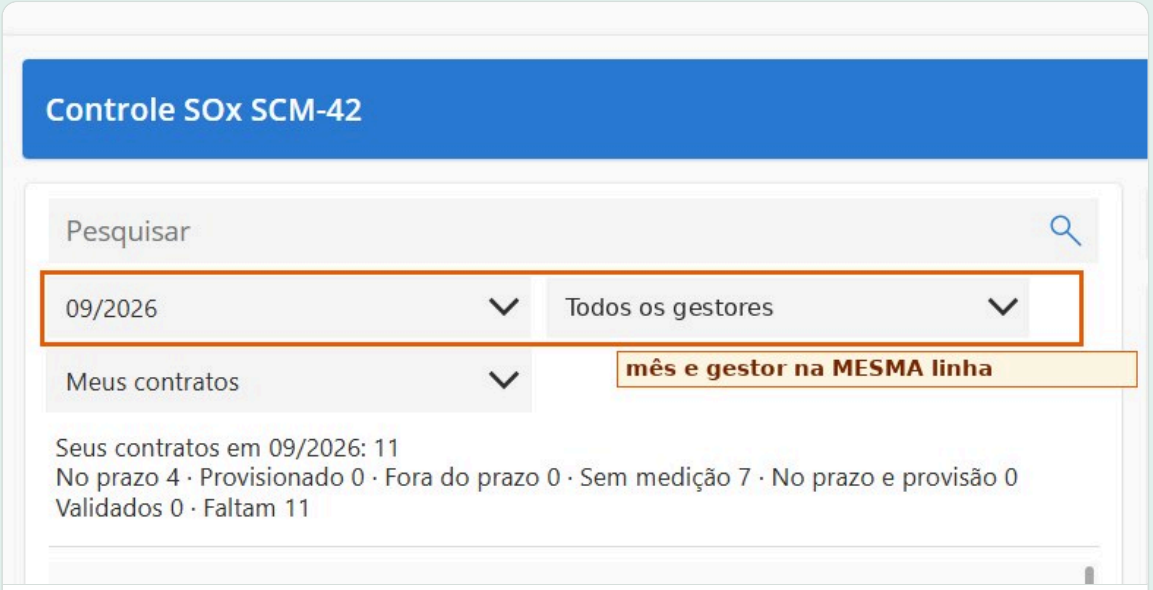
```
0
```

conFiltros → LayoutGap **formato português ( ; e ;; )**

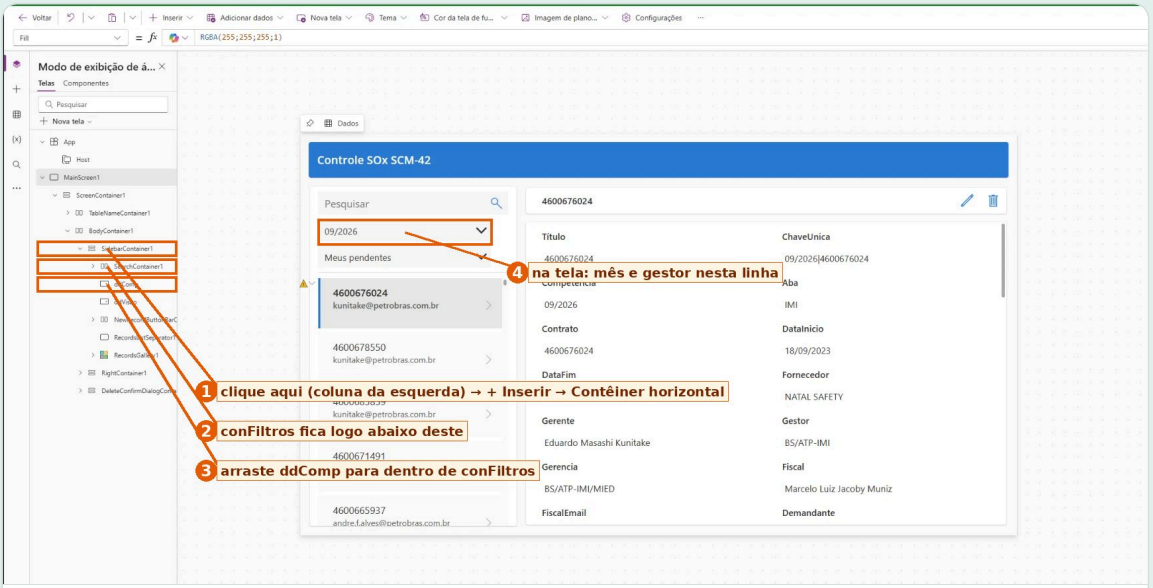
```
8
```

conFiltros → AlignInContainer **formato português ( ; e ;; )**

```
AlignInContainer.Stretch
```




**PRINT DO SEU APP** Como fica: mês e gestor lado a lado, na mesma linha, dentro da coluna da esquerda; o Meus contratos continua embaixo. Clique para ampliar.




**PRINT DO SEU APP** Seu Studio: 1 clique em SidebarContainer1 → + Inserir → Contêiner horizontal · 2 o conFiltros fica logo abaixo de SearchContainer1 · 3 arraste ddComp para dentro dele · 4 na tela, o mês e o gestor ficam nesta linha. Clique para ampliar.

**Deu certo se:** uma faixa vazia, da largura da coluna, logo abaixo do Pesquisar.

**9** ☑ ÁRVORE → DDCOMP → PARA DENTRO DE CONFILTROS

Na árvore, arraste `ddComp` para **dentro** de `conFiltros` (solte em cima do nome conFiltros). Se não arrastar: clique em `ddComp`, **Ctrl+X**, clique em `conFiltros`, **Ctrl+V**; se o nome virar `ddComp_1`, renomeie para `ddComp`.

**Deu certo se:** o filtro do mês fica dentro de conFiltros, sem X vermelho nas fórmulas.

**10** ☑ ÁRVORE → CONFILTROS

Com `conFiltros` escolhido: **+ Inserir** → **Lista suspensa** (a mesma de antes). Renomeie `ddGestor`. Cole:

ddGestor → Items **formato português ( ; e ;; )**

```
Table(
  {Value: "Todos os gestores"};
  Sort(Distinct(Filter(SOx_SCM42_Contratos; Competencia =
ddComp.Selected.Value); Gestor); Value))
```

ddGestor → Default **formato português ( ; e ;; )**

```
"Todos os gestores"
```

ddComp e ddGestor → FillPortions (nos 2: dividem a largura meio a meio) **formato português ( ; e ;; )**

```
1
```

**Deu certo se:** mês e gestor lado a lado; a lista do gestor mostra Todos os gestores e os gestores do mês (SOP, BS/ATP-IMI...). Depois cole a fórmula nova do Items da galeria (B5.1), que já usa o ddGestor.

# B5. Lado esquerdo: quais contratos aparecem

15 min · B · Montar o app

**Objetivo: Hoje** cada linha mostra o número (Title1) e o e-mail do gerente com ; antes (Subtitle1), e aparecem todos os meses e os DEMO. **Fica:** só o mês e a visão escolhidos, com 3 linhas por contrato: número · fornecedor / fiscal · gerente / resposta.

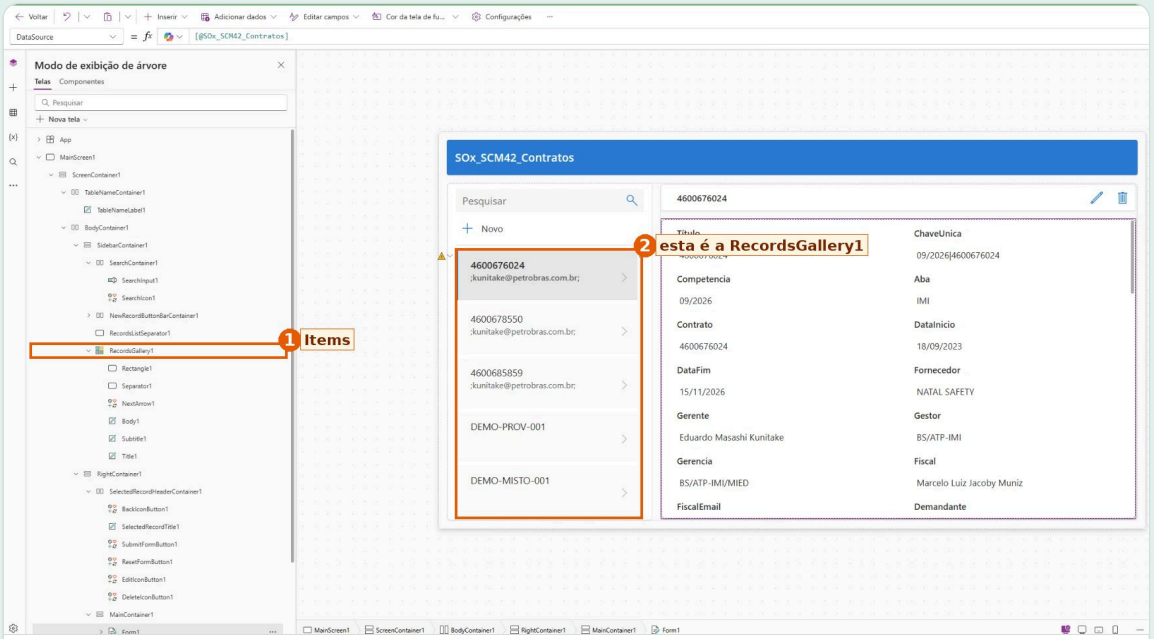
Lembre as 3 regras da B3: seletor em Português; conferir a propriedade antes de colar; sem X vermelho antes de seguir.

**1** ☑ ÁRVORE → RECORDSGALLERY1

Clique em `RecordsGallery1`, propriedade `Items`, caixa fx, `Ctrl` + `A` e cole:

RecordsGallery1 → Items **formato português ( ; e ;; )**

```
With(
  {base: Filter(SOx_SCM42_Contratos;
     Competencia = ddComp.Selected.Value
     && (ddGestor.Selected.Value = "Todos os gestores" || Gestor =
ddGestor.Selected.Value))};
  SortByColumns(
    Filter(base;
      IsBlank(SearchInput1.Text)
        || SearchInput1.Text in Contrato
        || SearchInput1.Text in Fornecedor
        || SearchInput1.Text in Fiscal;
      With({meu: ";" & varEmail & ";" in ";" &
Substitute(Lower(FiscalEmail & ";" & GerenteEmail); " "; "") & ";"
                || Contrato in varApoioContratos};
        Switch(ddVisao.Selected.Value;
          "Todos"; true;
          "Meus contratos"; meu;
          "Meus pendentes"; meu && IsBlank(OkFiscal);
          "Não OK"; OkFiscal = "NÃO OK"))));
    "Contrato";
    SortOrder.Ascending))
```




PRINT DO SEU APP 1 RecordsGallery1 na árvore · 2 é a lista de contratos. Clique para ampliar.

**Deu certo se:** sem sublinhado vermelho (um triângulo amarelo de delegação é normal).

▼ **Não deu certo?**

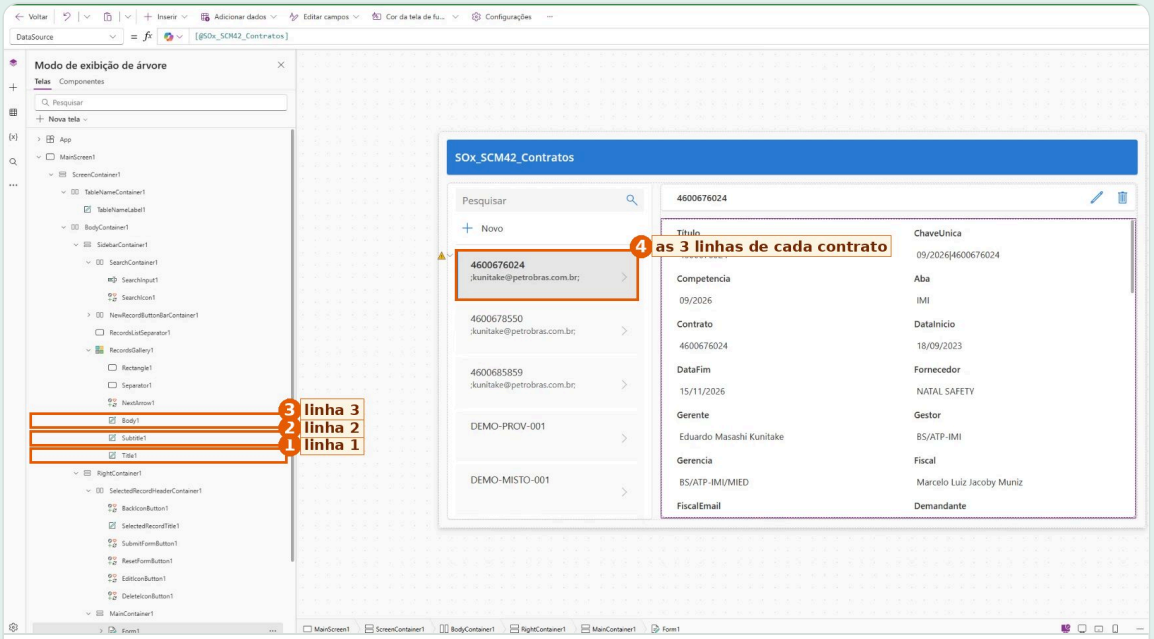
Sublinhado vermelho em `SearchInput1.Text` : a caixa de pesquisa é da versão moderna.
Troque as 4 vezes `SearchInput1.Text` por `SearchInput1.Value` .

**2** ☑ ÁRVORE → RECORDSGALLERY1 → TITLE1

**Hoje** a primeira linha é só o número (4600676024). Abra `RecordsGallery1` (▶) e clique em `Title1` . Propriedade `Text` :

Title1 → Text **formato português ( ; e ;; )**

```
ThisItem.Contrato & " · " & ThisItem.Fornecedor
```




**PRINT DO SEU APP** 1 Title1 = linha 1 · 2 Subtitle1 = linha 2 (hoje o e-mail) · 3 Body1 = linha 3 (hoje escondida) · 4 as linhas na tela. Clique para ampliar.

**Deu certo se:** cada linha mostra contrato · fornecedor.

**3** ☑ SUBTITLE1

**Hoje** a segunda linha é o e-mail do gerente (;kunitake@petrobras.com.br;). Clique em `Subtitle1` . `Text` :

Subtitle1 → Text **formato português ( ; e ;; )**

```
"Fiscal: " & ThisItem.Fiscal & " · Gerente: " & ThisItem.Gerente
```

**Deu certo se:** a segunda linha mostra Fiscal: nome · Gerente: nome.

**4** ☑ ÁRVORE → RECORDSGALLERY1 → RÓTULO ESCONDIDO

Para a lista mostrar **quem precisa preencher** (CSP/provisão, motivo da provisão, justificativa) diferente de quem só precisa validar: clique em `RecordsGallery1` , depois **uma vez** na 1ª linha → **+ Inserir** → **Rótulo de texto** . Renomeie `lblFaltaLinha` . Ele fica escondido; só calcula o que falta em cada contrato (mesma regra da faixa do alto e da planilha). Cole:

lblFaltaLinha → Text **formato português ( ; e ;; )**

```
With(
  {semCsp: Concat(
     Filter(Split(ThisItem.FRSPendentes; "|") As f;
       !IsBlank(Trim(f.Value));
       CountRows(
         Filter(Split(Substitute(Substitute(ThisItem.CSP; Char(13); "");
Char(10); ";"); ";") As b;
           !IsBlank(Find(":"; b.Value));
           !IsBlank(Trim(Mid(b.Value; Find(":"; b.Value) + 1)));
           With({esq: Lower(Trim(Left(b.Value; Find(":"; b.Value) - 1)))};
             esq in ["*"; "todos"; "tudo"] || StartsWith(esq; "todas") ||
Trim(f.Value) in esq))) = 0);
     Trim(Value); ", ")};
  If(!IsBlank(ThisItem.OkFiscal); "";
    Concat(
      Filter(
        Table(
          {m: "CSP/provisão"; f: (!IsBlank(ThisItem.FRSPendentes) ||
ThisItem.ForaDoPrazo.Value = "SIM")
                                  && (IsBlank(Trim(ThisItem.CSP)) ||
!IsBlank(semCsp))};
          {m: "motivo da provisão"; f: ThisItem.Aba = "SOP" &&
ThisItem.Situacao in ["Provisionado"; "Misto"]
                                  &&
IsBlank(ThisItem.MotivoProvisao.Value)};
          {m: "justificativa"; f: ThisItem.Situacao in ["Provisionado";
"Fora do prazo"; "Misto"; "Incompleto"]
                                  && Len(Trim(ThisItem.Justificativa)) <
15});
        f);
      m; ", ")))
```

lblFaltaLinha → Visible **formato português ( ; e ;; )**

```
false
```

**Deu certo se:** sem X vermelho (o rótulo não aparece na tela).

**5** ☑ BODY1

**Hoje** a terceira linha não aparece. Clique em `Body1` . `Text` :

Body1 → Text **formato português ( ; e ;; )**

```
If(!IsBlank(lblFaltaLinha.Text); "⚠ Preencher: " & lblFaltaLinha.Text;
   IsBlank(ThisItem.OkFiscal); "Aguardando OK";
   ThisItem.OkFiscal = "OK"; "✔ OK por " & ThisItem.OkFiscalPor;
   "✖ Não OK por " & ThisItem.OkFiscalPor)
```

Body1 → Color **formato português ( ; e ;; )**

```
If(!IsBlank(lblFaltaLinha.Text); RGBA(191; 54; 12; 1);
   IsBlank(ThisItem.OkFiscal); RGBA(96; 96; 96; 1);
   ThisItem.OkFiscal = "OK"; RGBA(46; 125; 50; 1);
   RGBA(198; 40; 40; 1))
```

**Deu certo se:** a terceira linha mostra **⚠ Preencher: CSP/provisão, justificativa** (em laranja) quando falta informação, **Aguardando OK** (cinza) quando só falta validar, ✔ OK por... (verde) ou ✖ Não OK por... (vermelho).

**▼ Não deu certo?**

Body1 escondido? Propriedade `Visible` → `true` .

6 ☑ STUDIO → TESTAR (MODO DE VISUALIZAÇÃO)

Teste a lista como o pessoal vai ver: **1** aperte `F5` (ou clique no ▷ no canto de cima à direita). A tela fica sem a árvore e sem a caixa fx: é o app rodando. **2** Na primeira caixa (mês) deixe **09/2026**, ou o mês que você publicou. **3** Clique na **segunda** caixa (hoje mostra **Meus pendentes**) e escolha **Todos**. **4** Role a lista da esquerda e clique em 2 ou 3 contratos: a direita mostra os dados de cada um. **5** Digite parte de um número de contrato ou de um fornecedor em **Pesquisar**: a lista diminui; apague o texto. **6** Aperte `Esc` (ou o **X** no canto de cima à direita) para voltar ao editor. **7** Salve com `Ctrl` + `S`.

**Deu certo se:** em **Todos** aparecem só os contratos do mês escolhido (uns 231 em 09/2026, sem os DEMO), cada um com 3 linhas: **número · fornecedor** / **Fiscal: … · Gerente: …** / **Aguardando OK**. Em **Meus pendentes** ficam só os contratos em que seu e-mail está como fiscal ou gerente (pode ser nenhum: é normal se você não for fiscal nem gerente).

▼ **Não deu certo?**

- **Lista vazia em Todos:** o mês da primeira caixa não é o publicado. Escolha 09/2026. Se continuar vazia, a coluna Competencia da lista está em outro formato; mande print de um contrato.
- **Primeira caixa vazia:** volte à B4.4 e B4.5.
- **Segunda linha ainda mostra o e-mail:** o Subtitle1 não recebeu a fórmula da B5.3; confira o seletor (Text) e cole de novo.
- **Terceira linha não aparece:** Body1 → Visible = `true` (B5.4).
- **Pesquisar não filtra:** veja a ajuda da B5.1 (.Text ou .Value).
- **F5 não abre:** use o ▷ do canto de cima à direita.

# B6. Lado esquerdo: cor e resumo

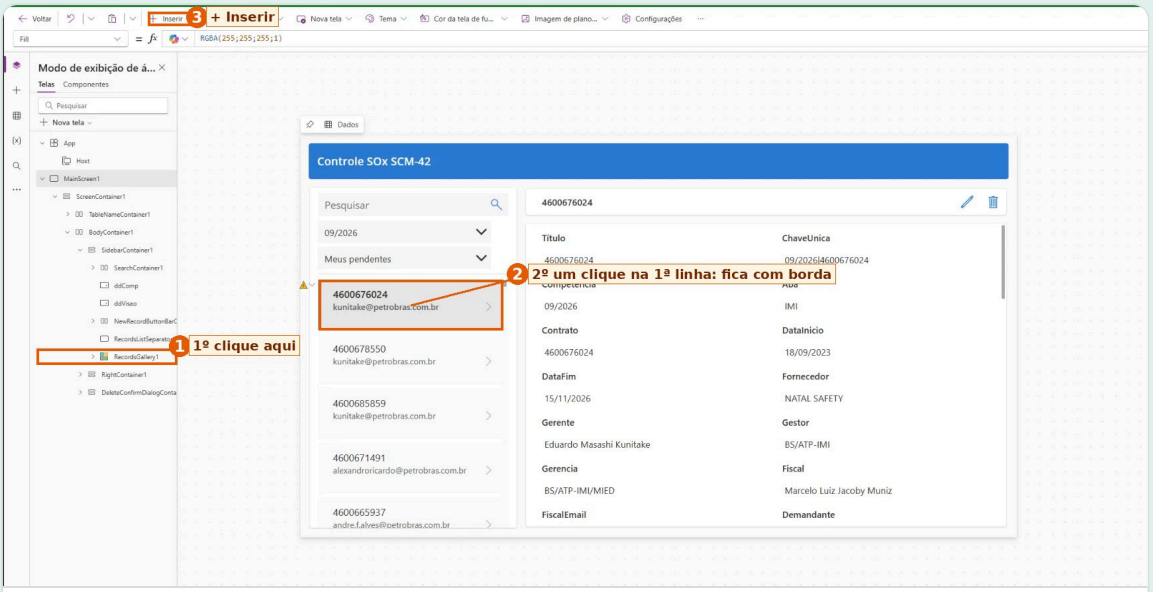
10 min · B · Montar o app

**Objetivo: Hoje** todas as linhas são cinza iguais. **Fica:** faixa à esquerda cinza (sem resposta), verde (OK) ou vermelha (não OK) e, embaixo dos filtros, o seu resumo do mês (como as abas Resumo da planilha).

Lembre as 3 regras da B3: seletor em Português; conferir a propriedade antes de colar; sem X vermelho antes de seguir.

**1** ÁRVORE → RECORDSGALLERY1

Clique em `RecordsGallery1` e depois **uma vez** na primeira linha da lista, no meio da tela.




PRINT DO SEU APP 1 um clique na primeira linha da lista · 2 + Inserir. Clique para ampliar.

**Deu certo se:** a primeira linha fica com borda: o que você inserir agora se repete em todas.

**2** ÁRVORE → RECORDSGALLERY1 → RECTANGLE1

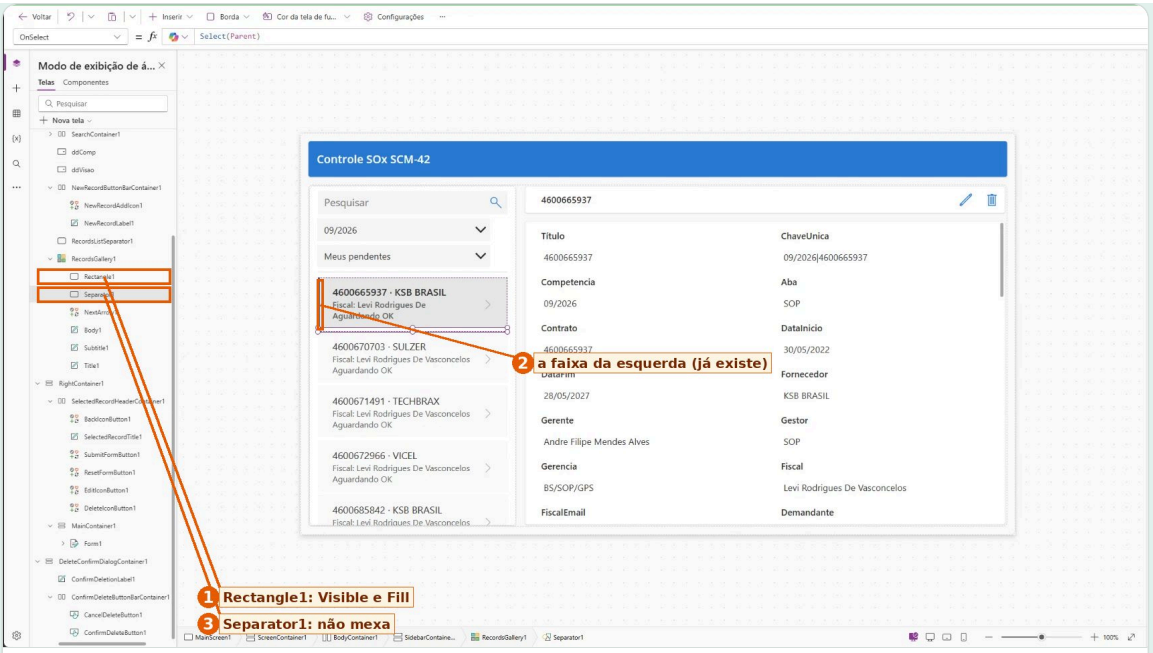
**Não insira nada:** a faixa já existe. É o `Rectangle1`, dentro de `RecordsGallery1` (hoje é a barrinha azul na borda esquerda da linha escolhida). Clique em `Rectangle1` na árvore e cole nas 2 propriedades. Não mexa no `Separator1` (é a linha fina embaixo de cada contrato).

Rectangle1 → Visible **formato português ( ; e ;; )**

```
true
```

Rectangle1 → Fill **formato português ( ; e ;; )**

```
If(!IsBlank(lblFaltaLinha.Text); RGBA(239; 108; 0; 1);
   IsBlank(ThisItem.OkFiscal); RGBA(150; 150; 150; 1);
   ThisItem.OkFiscal = "OK"; RGBA(46; 125; 50; 1);
   RGBA(198; 40; 40; 1))
```

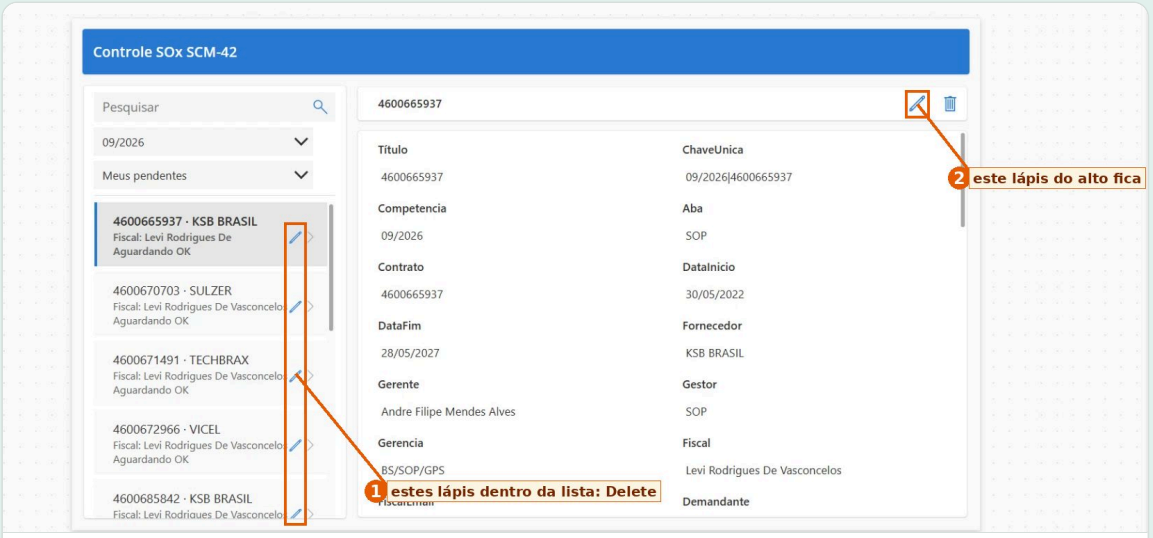



PRINT DO SEU APP Seu print: 1 Rectangle1 dentro de RecordsGallery1 · 2 é a barrinha da borda esquerda da linha, que vai virar a faixa colorida · 3 Separator1 (a linha embaixo de cada contrato): não mexa. Clique para ampliar.

**Deu certo se:** todas as linhas ganham a faixa à esquerda: laranja (falta preencher CSP/provisão, motivo ou justificativa), cinza (só falta validar), verde (OK) ou vermelha (não OK). Se você já tinha colado o Fill antes, cole de novo (agora tem o laranja; usa o lblFaltaLinha da B5).

## 3 ☑ ÁRVORE → RECORDSGALLERY1 → LÁPIS

Só se você já tinha posto o lápis dentro da lista (versão anterior deste passo): na árvore abra `RecordsGallery1` , clique no lápis ( `Icon1` ou `icoMeu` ) e aperte **Delete**. O ✎ de “é meu” fica no alto da direita, onde já está (B7.1).




PRINT DO SEU APP Seu print: 1 os lápis dentro da lista são os que saem (Delete). 2 o ✎ do alto fica. Clique para ampliar.

**Deu certo se:** as linhas da lista voltam sem o lápis e o nome do fiscal não fica cortado.

## 4 ☑ ÁRVORE → SIDEBARCONTAINER1

Resumo igual às abas **Resumo fiscal / Resumo gerente** da planilha, só da pessoa que abriu o app. É um **rótulo de texto** novo (não é separador: a linha fina entre os filtros e a lista já existe, é o `RecordsListSeparator1` ; não mexa nela). Clique em `SidebarContainer1` → **+ Inserir** →

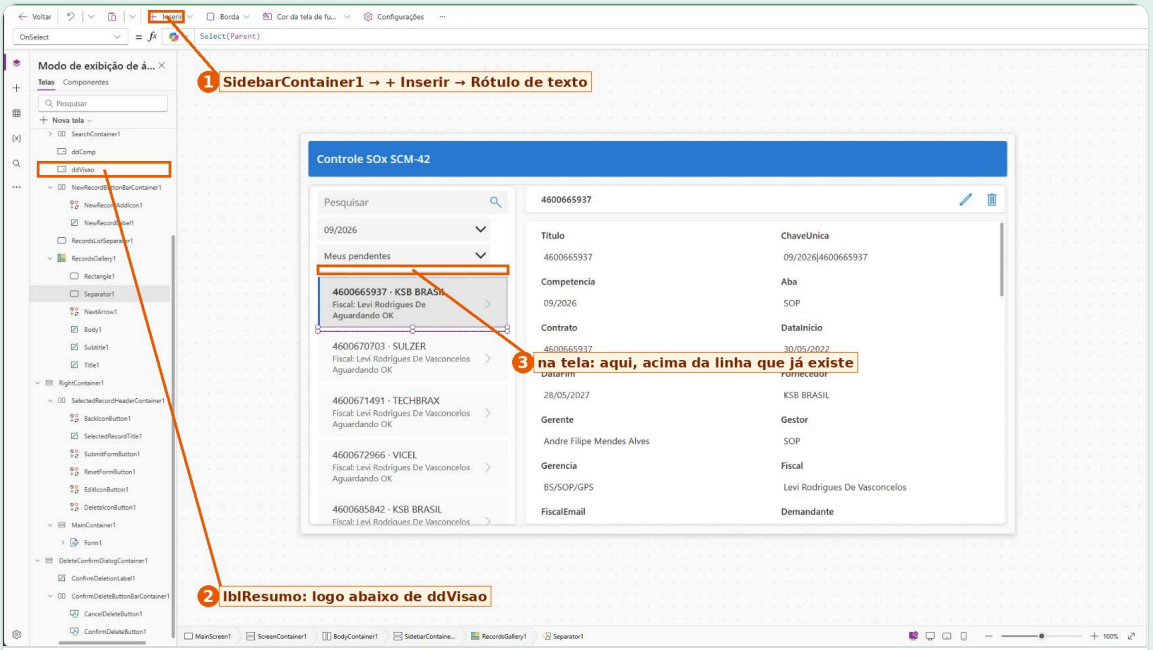
**Rótulo de texto**. Renomeie `lblResumo` e arraste na árvore para logo abaixo de `ddVisao`. Ligue **Altura automática** (`AutoHeight = true`). `Text`:

lblResumo → Text **formato português ( ; e ;; )**

```
With(
  {mes: Filter(SOx_SCM42_Contratos; Competencia = ddComp.Selected.Value)};
  With(
  {m: Filter(mes;
       ";" & varEmail & ";" in ";" & Substitute(Lower(FiscalEmail & ";" &
GerenteEmail); " "; "") & ";"
         || Contrato in varApoioContratos)};
  "Seus contratos em " & ddComp.Selected.Value & ": " & CountRows(m)
  & Char(10) & "No prazo " & CountRows(Filter(m; Situacao = "Medido no
prazo"))
  & " · Provisionado " & CountRows(Filter(m; Situacao = "Provisionado"))
  & " · Fora do prazo " & CountRows(Filter(m; Situacao = "Fora do prazo"))
  & " · Sem medição " & CountRows(Filter(m; Situacao = "Sem medição"))
  & " · No prazo e provisão " & CountRows(Filter(m; Situacao = "Misto"))
  & Char(10) & "Validados " & CountRows(Filter(m; !IsBlank(OkFiscal)))
  & " · Faltam " & CountRows(Filter(m; IsBlank(OkFiscal)))))
```

lblResumo → AlignInContainer (para ocupar a largura toda, senão o texto quebra em várias linhas) **formato português ( ; e ;; )**

```
AlignInContainer.Stretch
```




**PRINT DO SEU APP** Seu print: 1 role a árvore um pouco para cima, clique em SidebarContainer1 e depois + Inserir → Rótulo de texto · 2 na árvore, arraste o rótulo para logo abaixo de ddVisao (acima de NewRecordButtonBarContainer1) · 3 na tela ele fica entre o filtro Meus pendentes e a linha fina, que já existe (RecordsListSeparator1). Clique para ampliar.

**Deu certo se:** embaixo dos filtros aparece, por exemplo: **Seus contratos em 09/2026: 11** / No prazo 4 · Provisionado 0 · Fora do prazo 0 · Sem medição 7 · No prazo e provisão 0 / Validados 0 · Faltam 11. Os números batem com a sua linha no Resumo fiscal da planilha. Salve.

▼ **Não deu certo?**

Quem não é fiscal nem gerente de nenhum contrato vê **Seus contratos: 0**: normal.

# B7. Lado direito: todos os dados, só para leitura

20 min · B · Montar o app

**Objetivo: Hoje** no alto da direita há lápis ✎ e lixeira 🗑: ao clicar no ✎, qualquer um edita todos os campos (foi o que aconteceu no seu teste). **Fica:** sem 🗑; o ✎ continua no alto, mas só aparece nos contratos em que você é fiscal ou gerente (a marca de "é meu") e não abre mais a edição; o formulário mostra todos os dados da planilha, na mesma ordem, só para leitura, incluindo a **Observação** com o resumo das medições (No prazo / Fora do prazo / Provisionado com e sem CSP). Editar só acontece no formulário da B9, e só para o fiscal ou o gerente do contrato.

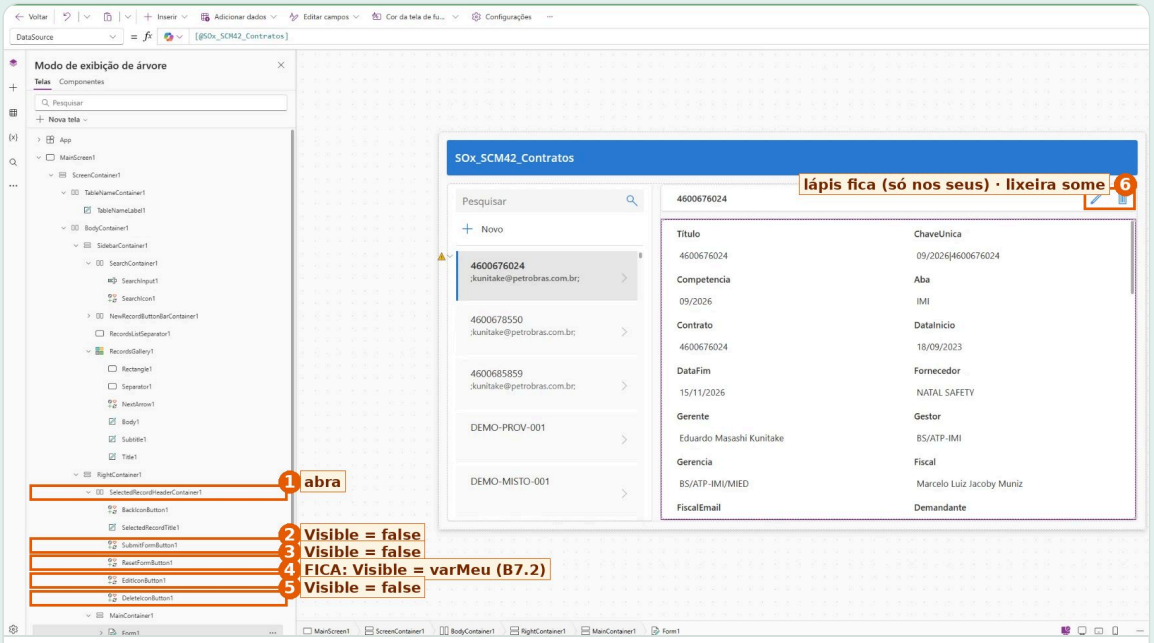
Lembre as 3 regras da B3: seletor em Português; conferir a propriedade antes de colar; sem X vermelho antes de seguir.

**1** ☑ ÁRVORE → SELECTEDRECORDHEADERCONTAINER1

**Hoje** aparecem ✎ e 🗑 ao lado do número, no alto da direita. Abra `SelectedRecordHeaderContainer1`. Nestes 3, propriedade `Visible` = `false`: `DeleteIconButton1` (a 🗑), `SubmitFormButton1` e `ResetFormButton1` (o ✓ e o X que aparecem depois do ✎).

Os 3 → Visible **formato português ( ; e ;; )**

```
false
```




**PRINT DO SEU APP** 1 abra SelectedRecordHeaderContainer1 · 2, 3 e 5 Visible = false · 4 EditIconButton1 (o ✎) fica: B7.2 · 6 a 🗑 some, o ✎ fica só nos seus contratos. *(print de antes da B4: a parte da direita e a árvore são iguais às suas hoje)* Clique para ampliar.

**Deu certo se:** a 🗑 some.

**2** ☑ ÁRVORE → EDITICONBUTTON1 (O ✎ DO ALTO)

O ✎ **fica onde está**, mas vira a marca de "é meu". Clique em `EditIconButton1` e cole nas 2 propriedades. O `OnSelect` substitui o que está lá (apague o `EditForm(Form1)`): assim ninguém mais edita todos os campos por ele.

EditIconButton1 → Visible **formato português ( ; e ;; )**

```
varMeu
```

EditIconButton1 → OnSelect **formato português ( ; e ;; )**

```
Notify("Você é fiscal ou gerente deste contrato: responda em Dados OK ou
Dados não OK, no fim do painel."; NotificationType.Information)
```

**Deu certo se:** com `F5`: o ✎ aparece no alto só quando o contrato escolhido é seu; clicando nele sai um aviso azul e nada fica editável.

**3** ☑ ÁRVORE → FORM1

Clique em `Form1`. No seletor de propriedade do seu Form1 **não existe DisplayMode**: quem deixa só leitura é o `DefaultMode`. Troque as 2 propriedades (apague o que está e cole):

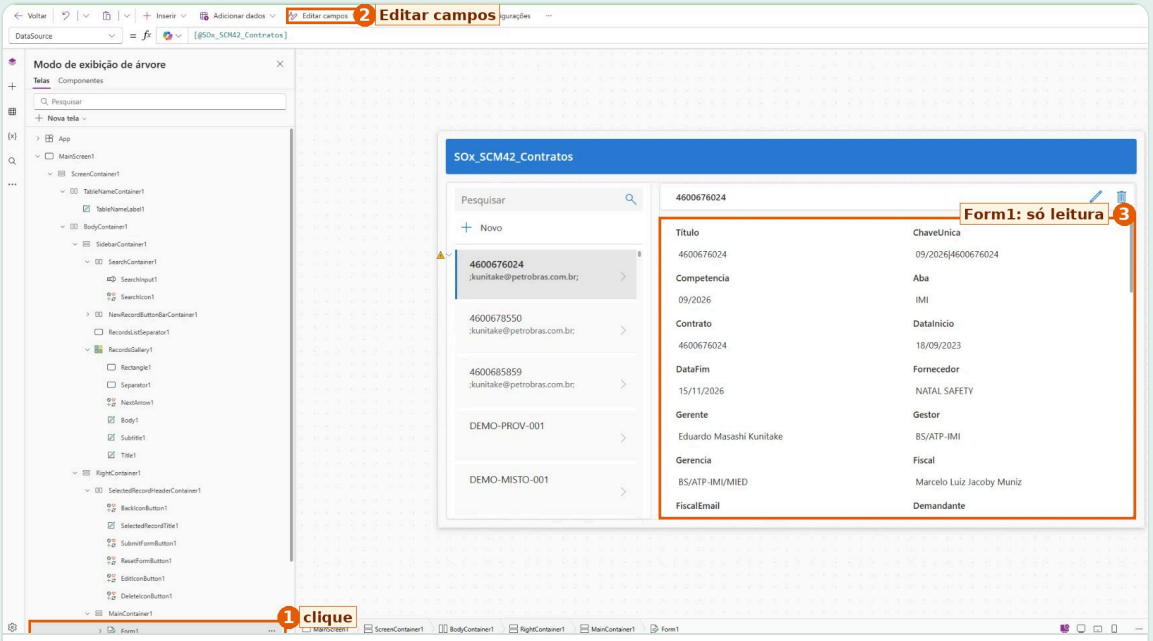
- `Item`: hoje tem algo como `RecordsGallery1.Selected`.
- `DefaultMode`: hoje tem `If(newMode; FormMode.New; editMode; FormMode.Edit; FormMode.View)`.

Form1 → Item **formato português ( ; e ;; )**

```
varItem
```

Form1 → DefaultMode **formato português ( ; e ;; )**

```
FormMode.View
```



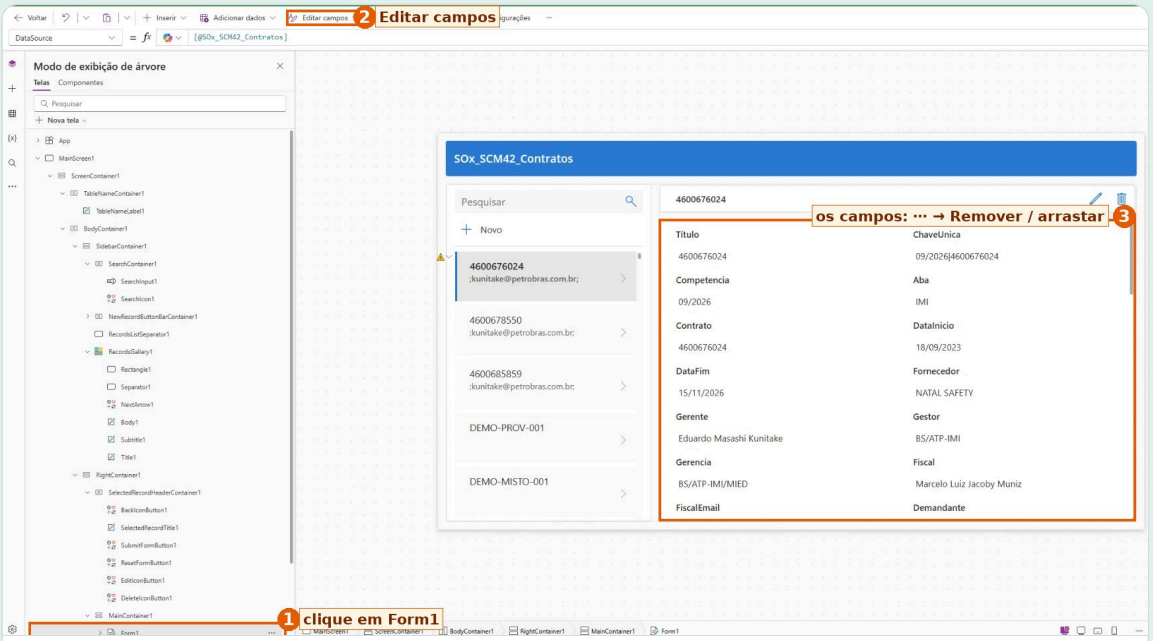

**PRINT DO SEU APP** 1 Form1 na árvore · 2 botão Editar campos (B7.3) · 3 é o formulário que fica só leitura. *(print de antes da B4: a parte da direita e a árvore são iguais às suas hoje)* Clique para ampliar.

**Deu certo se:** sem X vermelho; no painel da direita, **Modo padrão** mostra **Exibição**, e os campos ficam sem caixa de digitação (só leitura) mesmo clicando no ✎ do alto.

**4** ☑ PAINEL DA DIREITA → EDITAR CAMPOS

Com `Form1` escolhido, clique em **Editar campos** (barra de cima ou painel da direita). Remova (três pontinhos → **Remover**) os campos de controle interno, que o pessoal não precisa ver: Título, ChaveUnica, FiscalEmail, GerenteEmail, MesFechado, PrecisaIntervencao, MotivoIntervencao, OQueFazer, DadosCorretos, CampoErro, ValorCorreto, ComentarioErro,

AprovacaoGerente, ComentarioGerente, OkFiscal, OkFiscalPor, OkFiscalEm, AprovacaoPor, AprovacaoEm, Observação Mês, FRSPendentes, Anexos.




**PRINT DO SEU APP** 1 Form1 na árvore · 2 Editar campos · 3 a lista de campos abre à direita: ··· ao lado do campo → Remover; arraste para mudar a ordem. *(print de antes da B4: a parte da direita e a árvore são iguais às suas hoje)* Clique para ampliar.

**Deu certo se:** sobram os dados da planilha: Competencia, Aba, Contrato, DataInicio, DataFim, Fornecedor, Objeto, Gerente, Gestor, Gerencia, HouveServico, FoiMedido, Provisao, Justificativa, ForaDoPrazo, Fiscal, Demandante, Plataforma, SeraSubstituido, DemandaContinuada, CSP, MotivoProvisao, Situacao, Observacao.

▼ **Não deu certo?**

Para que servem os campos removidos: **PrecisaIntervencao, MotivoIntervencao, OQueFazer** = o que o programa calculou que falta (vira o aviso vermelho da B8 e o lembrete do Teams). **OkFiscal, OkFiscalPor, OkFiscalEm, DadosCorretos, ComentarioErro** = o registro da validação (quem deu OK ou não OK, quando e por quê); aparece no texto da B8. **Observação Mês** = a observação que a pessoa escreve (B9). **AprovacaoGerente, AprovacaoPor, AprovacaoEm, ComentarioGerente, CampoErro, ValorCorreto** = do desenho antigo, não são mais usados.

**5** ☑ EDITAR CAMPOS → ORDEM

No mesmo painel, arraste os campos para a ordem da planilha: Contrato, DataInicio, DataFim, Fornecedor, Objeto, Gerente, Gestor, Gerencia, HouveServico, FoiMedido, Provisao, Justificativa, ForaDoPrazo, Fiscal, Demandante, Plataforma, SeraSubstituido, DemandaContinuada, CSP, MotivoProvisao, Situacao, Observacao. Deixe Competencia e Aba no começo.

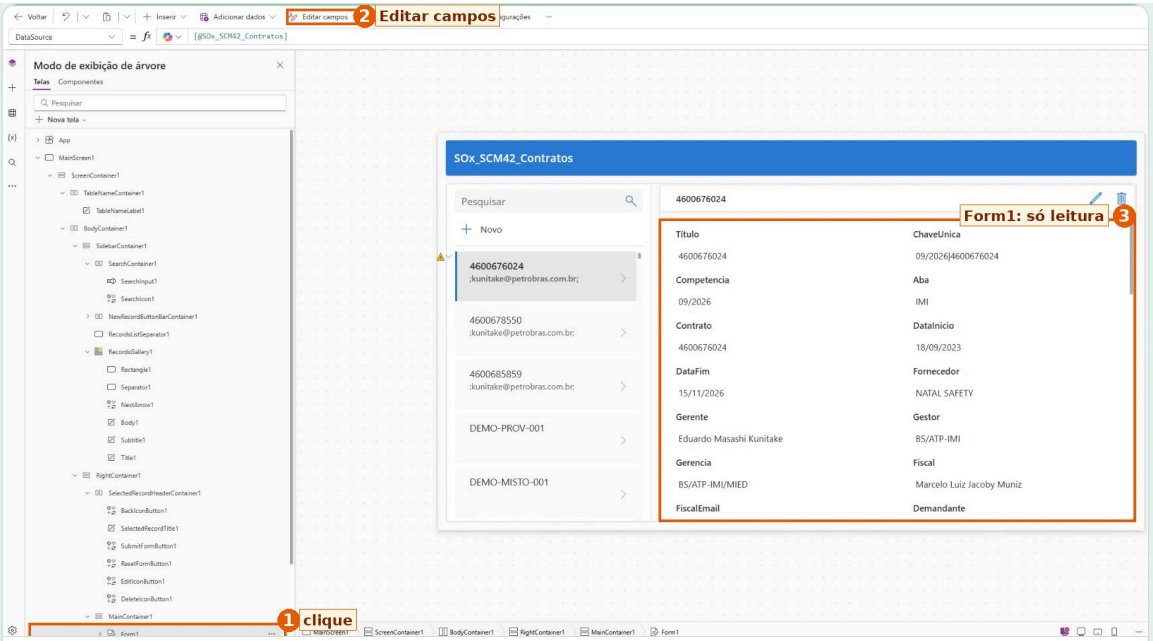
**Deu certo se:** o formulário segue a ordem das colunas da planilha.

**6** ☑ ÁRVORE → FORM1 → CARTÃO OBSERVACAO

Abra `Form1` e clique no cartão `Observacao_DataCard1` (o cartão, não o que está dentro). Propriedade `WidthFit` :

Observacao_DataCard1 → WidthFit **formato português ( ; e ;; )**

```
true
```




**PRINT DO SEU APP** 1 Form1 na árvore: clique na setinha › ao lado para abrir e achar Observacao_DataCard1. *(print de antes da B4: a parte da direita e a árvore são iguais às suas hoje)* Clique para ampliar.

**Deu certo se:** a Observação ocupa a largura toda e mostra o resumo das medições como na planilha.

### ▼ Não deu certo?

Não deixa alterar: painel da direita → **Avançado** → **Desbloquear para alterar propriedades**.
Texto cortado: clique no que está dentro do cartão (DataCardValue...) e ligue `AutoHeight` = `true`.

# B8. Lado direito: faixa de situação no alto

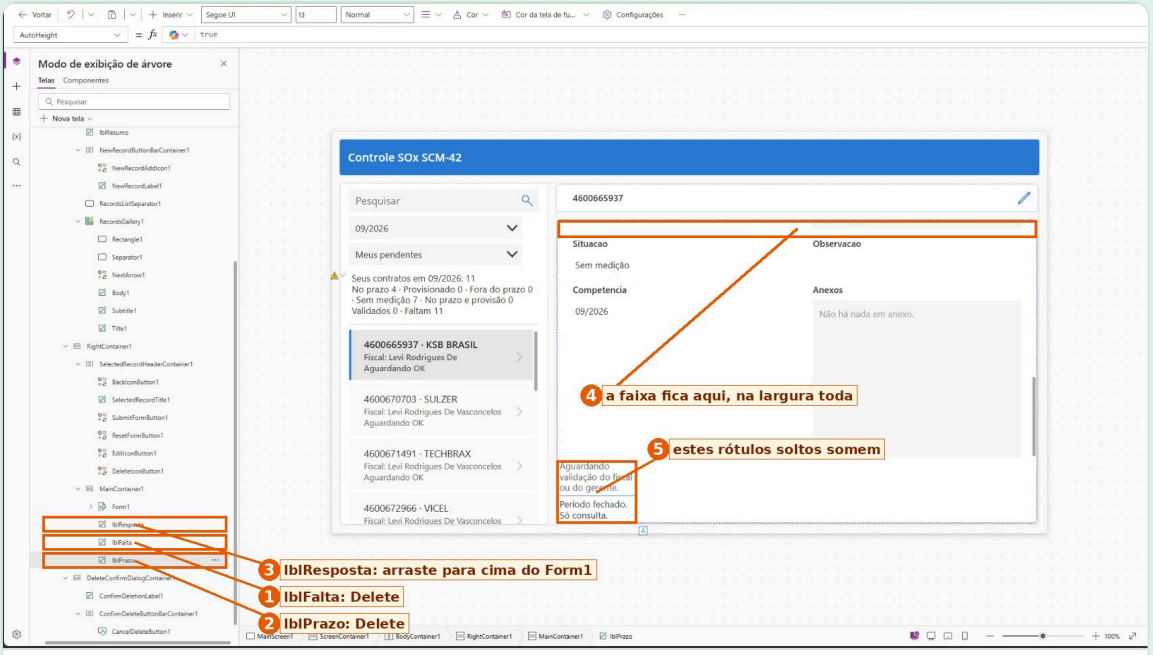
10 min · B · Montar o app

**Objetivo: Hoje** (se você já fez a versão anterior) há 3 rótulos soltos embaixo do formulário, estreitos e quebrados. **Fica: uma faixa só**, logo abaixo do número do contrato e acima dos dados, na largura toda, com fundo colorido: cinza = aguardando; laranja = aguardando e falta informação (CSP, motivo, justificativa, escrito na própria faixa); verde = Dados OK (por quem e quando); vermelha = Dados não OK (e o que estava errado). Na última linha, o prazo (dia 15 do mês seguinte) ou "Período fechado".

Lembre as 3 regras da B3: seletor em Português; conferir a propriedade antes de colar; sem X vermelho antes de seguir.

**1** ☑ ÁRVORE → MAINCONTAINER1 → LBLFALTA E LBLPRAZO

Se você fez a versão anterior desta etapa: na árvore, clique em `lblFalta` → **Delete**; clique em `lblPrazo` → **Delete**. O `lblResposta` fica: ele vira a faixa. (Se ainda não tem nenhum dos 3: clique em `MainContainer1` → **+ Inserir** → **Rótulo de texto** e renomeie `lblResposta`.)




PRINT DO SEU APP Seu print: 1 e 2 apague lblFalta e lblPrazo · 3 arraste lblResposta para cima do Form1 (dentro de MainContainer1) · 4 a faixa fica no alto, na largura toda · 5 os rótulos soltos de baixo somem. Clique para ampliar.

**Deu certo se:** em `MainContainer1` sobram `Form1` e `lblResposta`.

**2** ☑ ÁRVORE → LBLRESPOSTA → SUBIR

Na árvore, arraste `lblResposta` para **cima** do `Form1` (ou: três pontinhos do lblResposta → **Reordenar** → **Trazer para a frente** até ficar acima). Assim a faixa fica logo abaixo do número do contrato.

**Deu certo se:** na tela, o texto fica no alto do painel da direita, acima de Título/Competência.

**3** ☑ LBLRESPOSTA

Com `lblResposta` escolhido, cole em cada propriedade (troque o que estiver):

lblResposta → Text **formato português ( ; e ;; )**

```
If(IsBlank(varItem.OkFiscal);
   "Aguardando validação do fiscal ou do gerente";
   varItem.OkFiscal = "OK";
   "✔ Dados OK · " & varItem.OkFiscalPor & " · " & varItem.OkFiscalEm;
   "✖ Dados não OK · " & varItem.OkFiscalPor & " · " & varItem.OkFiscalEm
     & Char(10) & "O que estava errado: " & varItem.ComentarioErro)
& If(IsBlank(varItem.OkFiscal) && !IsBlank(varFalta); Char(10) & "⚠ " &
varFalta; "")
& If(IsBlank(varItem.'Observação Mês'); ""; Char(10) & "Observação do mês:
" & varItem.'Observação Mês')
& Char(10) & If(varFechado;
   "Período fechado: só consulta.";
   Today() > varPrazo;
   "Prazo vencido em " & Text(varPrazo; "dd/mm/yyyy") & ": responda o
quanto antes.";
   "Prazo para responder: " & Text(varPrazo; "dd/mm/yyyy"))
```

lblResposta → Color **formato português ( ; e ;; )**

```
If(IsBlank(varItem.OkFiscal) && !IsBlank(varFalta); RGBA(138; 80; 0; 1);
   IsBlank(varItem.OkFiscal); RGBA(60; 60; 60; 1);
   varItem.OkFiscal = "OK"; RGBA(27; 94; 32; 1);
   RGBA(183; 28; 28; 1))
```

lblResposta → Fill **formato português ( ; e ;; )**

```
If(IsBlank(varItem.OkFiscal) && !IsBlank(varFalta); RGBA(255; 243; 224;
1);
   IsBlank(varItem.OkFiscal); RGBA(243; 243; 243; 1);
   varItem.OkFiscal = "OK"; RGBA(232; 245; 233; 1);
   RGBA(253; 236; 234; 1))
```

lblResposta → AlignInContainer **formato português ( ; e ;; )**

```
AlignInContainer.Stretch
```

lblResposta → AutoHeight **formato português ( ; e ;; )**

```
true
```

lblResposta → PaddingLeft, PaddingRight (os 2) **formato português ( ; e ;; )**

```
16
```

lblResposta → PaddingTop, PaddingBottom (os 2) **formato português ( ; e ;; )**

```
10
```

lblResposta → BorderThickness **formato português ( ; e ;; )**

```
0
```

lblResposta → Size **formato português ( ; e ;; )**

```
13
```

**Deu certo se:** a faixa ocupa a largura toda, com fundo cinza/laranja/verde/vermelho conforme o contrato, e o texto inteiro aparece em poucas linhas.

## 4 ☑ APP → FORMULAS

Uma linha das Fórmulas mudou (o prazo vencido não trava mais; só o fechamento pelo programa trava). Clique em `App` → propriedade `Formulas`, ache a linha que começa com `varFechado` e troque a linha inteira por:

App → Formulas, linha varFechado **formato português ( ; e ;; )**

```
varFechado = varItem.MesFechado = "SIM";;
```

**Deu certo se:** o contrato de 09/2026 deixa de mostrar “Período fechado” e mostra “Prazo vencido em 15/10/2026: responda o quanto antes.” enquanto você não fechar o mês pelo programa.

# B9. Lado direito: formulário de validação

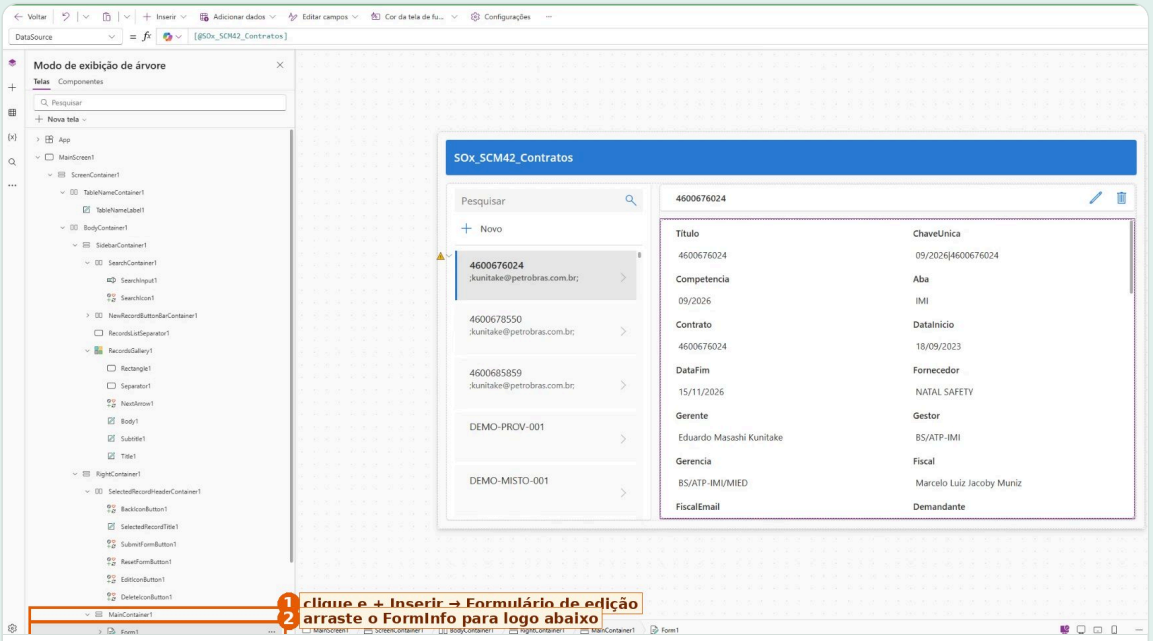
25 min · B · Montar o app

**Objetivo: Hoje** não existe. **Fica:** embaixo dos avisos, um formulário pequeno que só o **fiscal ou o gerente** do contrato vê, enquanto o contrato não foi validado. Os mesmos campos que são destravados na planilha:
• **Observação Mês**: sempre aparece (opcional).
• **CSP, motivo da provisão, justificativa**: aparecem quando o contrato pede (fora do prazo, provisionado, misto, incompleto) e são obrigatórios para dar OK. A **Provisão** se monta sozinha pelo CSP, como na planilha.
• **Será substituído** e **Demanda continuada**: aparecem quando estão em branco na planilha, para a pessoa responder.
• **Houve serviço, Foi medido, Fora do prazo** (e os dois acima, se já preenchidos) e **O que estava errado**: só aparecem depois de clicar em **Dados não OK**. A pessoa corrige o valor; no Puxar respostas a correção vai para a planilha, que pinta de vermelho a diferença em relação ao que o programa leu.

Lembre as 3 regras da B3: seletor em Português; conferir a propriedade antes de colar; sem X vermelho antes de seguir.

1 ☑ ÁRVORE → MAINCONTAINER1

Clique em `MainContainer1` → **+ Inserir** → busque `formulário` → **Formulário de edição**. Renomeie para `FormInfo`. Se pedir fonte de dados, escolha **SOx_SCM42_Contratos**. Na árvore, arraste `FormInfo` para logo abaixo de `lblPrazo`.




PRINT DO SEU APP 1 clique em MainContainer1 e insira o Formulário de edição · 2 arraste o FormInfo para logo abaixo de Form1. *(print de antes da B4: a parte da direita e a árvore são iguais às suas hoje)* Clique para ampliar.

**Deu certo se:** aparece um formulário novo embaixo dos avisos.

▼ **Não deu certo?**

Antes desta etapa rode uma vez o **Publicar online agora** com o ajuste_direto.py novo: ele cria a coluna **Observação Mês** na lista (se ela já existia com o nome antigo ObservacaoMes, ele só troca o nome). Depois, no Studio: painel **Dados** (cilindro) → três pontinhos da lista → **Atualizar**.

2 FORMINFO

Cole nas 5 propriedades:

FormInfo → DataSource **formato português ( ; e ;; )**

```
SOx_SCM42_Contratos
```

FormInfo → Item **formato português ( ; e ;; )**

```
varItem
```

FormInfo → DefaultMode **formato português ( ; e ;; )**

```
FormMode.Edit
```

FormInfo → Visible **formato português ( ; e ;; )**

```
varPode && IsBlank(varItem.OkFiscal)
```

FormInfo → OnSuccess **formato português ( ; e ;; )**

```
With({it: FormInfo.LastSubmit; idx: FormInfo.LastSubmit.ID};
  With({cobre: IsBlank(Concat(
             Filter(Split(it.FRSPendentes; "|") As f;
               !IsBlank(Trim(f.Value));
               CountRows(
                 Filter(Split(Substitute(Substitute(it.CSP; Char(13); "");
Char(10); ";"); ";") As b;
                   !IsBlank(Find(":"; b.Value));
                   !IsBlank(Trim(Mid(b.Value; Find(":"; b.Value) + 1)));
                   With({esq: Lower(Trim(Left(b.Value; Find(":"; b.Value)
- 1)))};
                     esq in ["*"; "todos"; "tudo"] || StartsWith(esq;
"todas") || Trim(f.Value) in esq))) = 0);
             Trim(Value); ", "))};
    With({completo:
          !((!IsBlank(it.FRSPendentes) || it.ForaDoPrazo.Value = "SIM")
&& (IsBlank(Trim(it.CSP)) || !cobre))
          && !(it.Aba = "SOP" && it.Situacao in ["Provisionado"; "Misto"]
&& IsBlank(it.MotivoProvisao.Value))
          && !(it.Situacao in ["Provisionado"; "Fora do prazo"; "Misto";
"Incompleto"] && Len(Trim(it.Justificativa)) < 15);
         quando: Text(Now(); "dd/mm/yyyy hh:mm")};
      If(!IsBlank(it.FRSPendentes);
        Patch(SOx_SCM42_Contratos; LookUp(SOx_SCM42_Contratos; ID = idx);
{ForaDoPrazo: {Value: If(cobre; "NÃO"; "SIM")}}));;
      If(varNaoOk;
        If(IsBlank(Trim(it.ComentarioErro));
          Notify("Salvo. Para registrar Não OK, escreva o que estava
errado."; NotificationType.Warning);
          Patch(SOx_SCM42_Contratos; LookUp(SOx_SCM42_Contratos; ID =
idx);
            {OkFiscal: "NÃO OK"; DadosCorretos: "NÃO"; OkFiscalPor:
varNome; OkFiscalEm: quando});;
          Set(varNaoOk; false);;
          Notify("Não OK registrado. A correção vai para a planilha no
Puxar respostas."; NotificationType.Warning));
        If(!varValida;
          Notify("Salvo. Agora o fiscal ou o gerente confere e dá o OK.";
NotificationType.Success);
          completo;
          Patch(SOx_SCM42_Contratos; LookUp(SOx_SCM42_Contratos; ID =
idx);
            {OkFiscal: "OK"; DadosCorretos: "SIM"; OkFiscalPor: varNome;
OkFiscalEm: quando});;
          Notify("Dados confirmados. Obrigado!";
NotificationType.Success);
          Notify("Salvo, mas ainda falta informação obrigatória. Veja o
aviso em vermelho."; NotificationType.Warning))
      )
    )
  )
)
```

**Deu certo se:** sem X vermelho. Some no editor quando o contrato escolhido não é seu: normal.

▼ **Não deu certo?**

Para vê-lo enquanto monta, troque Visible por `true` e, no fim da B9, volte a fórmula.

## 3 ☑ PAINEL DA DIREITA → EDITAR CAMPOS

Com `FormInfo` escolhido: **Editar campos** → **remova todos** e deixe só estes 10, nesta ordem: Observação Mês, CSP, MotivoProvisao, Justificativa, HouveServico, FoiMedido, ForaDoPrazo, SeraSubstituido, DemandaContinuada, ComentarioErro. Falta algum? **+ Adicionar campo**.

**Deu certo se:** FormInfo tem só esses 10 campos.

## 4 ÁRVORE → FORMINFO → CARTÕES

**Antes de colar, destranque o cartão.** Os cartões do formulário vêm trancados (cadeado ao lado do nome na tela; se tentar colar aparece *Não é possível editar em editor somente leitura*). Para cada cartão em que for mexer, nesta etapa e na próxima: clique no cartão na árvore → painel da direita → aba **Avançado** → botão **Desbloquear para alterar propriedades** (cadeado). Só precisa uma vez por cartão.

Depois, abra `FormInfo`. Em cada cartão (o cartão, não o que está dentro; o número no fim do nome é 2 ou outro), propriedade `Visible`:

CSP_DataCard... → Visible **formato português ( ; e ;; )**

```
varPedeCSP || varNaoOk
```

MotivoProvisao_DataCard... → Visible **formato português ( ; e ;; )**

```
varPedeMotivo || (varNaoOk && varItem.Aba = "SOP")
```

Justificativa_DataCard... → Visible **formato português ( ; e ;; )**

```
varPedeJust || varNaoOk
```

SeraSubstituido_DataCard... → Visible **formato português ( ; e ;; )**

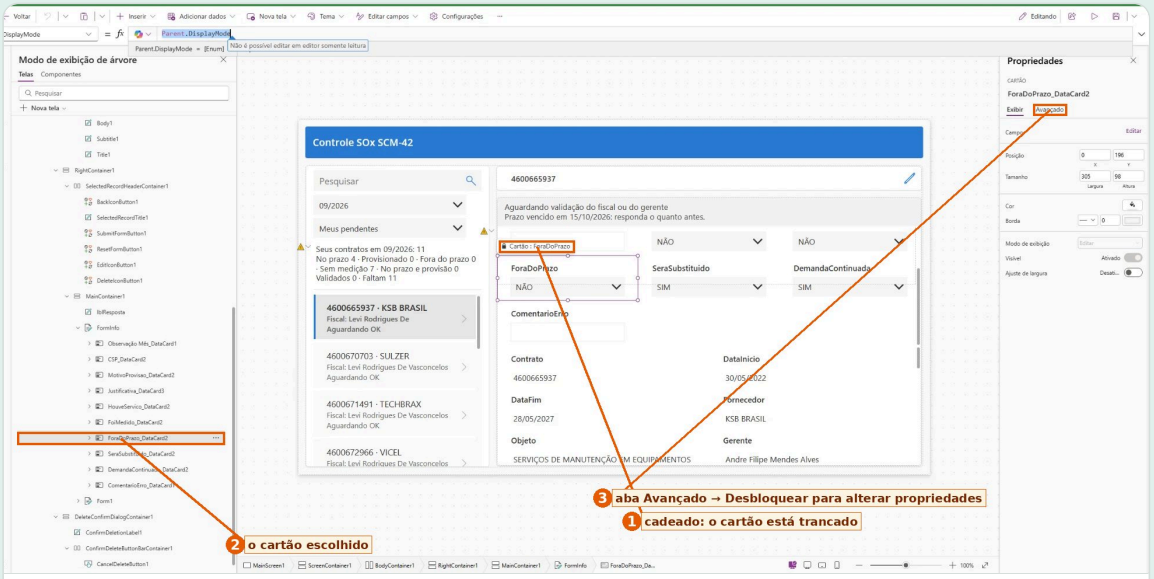
```
IsBlank(varItem.SeraSubstituido.Value) || varNaoOk
```

DemandaContinuada_DataCard... → Visible **formato português ( ; e ;; )**

```
IsBlank(varItem.DemandaContinuada.Value) || varNaoOk
```

HouveServico, FoiMedido, ForaDoPrazo e ComentarioErro (os 4 cartões) → Visible **formato português ( ; e ;; )**

```
varNaoOk
```




**PRINT DO SEU APP** Seu print: 1 o cadeado mostra que o cartão está trancado · 2 o cartão escolhido na árvore (ForaDoPrazo_DataCard2) · 3 aba Avançado → Desbloquear para alterar propriedades. Depois disso a caixa fx aceita a fórmula. Clique para ampliar.

**Deu certo se:** sem erro. Observação Mês fica sempre visível (não mexa).

▼ **Não deu certo?**

Não deixa alterar: painel da direita → **Avançado** → **Desbloquear para alterar propriedades** (cadeado).

**5** ☑ CARTÕES FORADOPRAZO E COMENTARIOERRO

Mais dois ajustes:

ForaDoPrazo_DataCard... → DisplayMode **formato português ( ; e ;; )**

```
If(IsBlank(varItem.FRSPendentes); Parent.DisplayMode; DisplayMode.View)
```

ComentarioErro_DataCard... → DisplayName **formato português ( ; e ;; )**

```
"O que estava errado (obrigatório no Não OK)"
```

Observação Mês_DataCard... → DisplayName **formato português ( ; e ;; )**

```
"Observação Mês (opcional)"
```

**Deu certo se:** sem erro. Salve. Com FRS pendente, Fora do prazo é calculado pelo CSP (igual à fórmula da planilha).

**6** ☑ CARTÕES DE SIM/NÃO E MOTIVO → CAIXA DE DENTRO

Nas listas de SIM/NÃO aparecem **Item 1, Item 2...**: a caixa de dentro está com itens de exemplo. Em cada um destes 5 cartões, abra o cartão na árvore (setinha ›) e clique na caixa de dentro ( `DataCardValue...` , a de ícone de lista): **HouveServico, FoiMedido, ForaDoPrazo, SeraSubstituido, DemandaContinuada**. Cole nas 3 propriedades. Depois faça o mesmo no **MotivoProvisao**, com a lista de motivos da planilha.

Caixa de dentro dos 5 cartões SIM/NÃO → Items **formato português ( ; e ;; )**

```
["SIM"; "NÃO"]
```

→ IsSearchable **formato português ( ; e ;; )**

```
false
```

→ SelectMultiple **formato português ( ; e ;; )**

```
false
```

Caixa de dentro do MotivoProvisao → Items **formato português ( ; e ;; )**

```
["Atraso do Fornecedor"; "Atraso Fiscalização de Campo"; "Problema Interno GPS"; "Contrato Bloqueado"; "Outros"]
```

**Deu certo se:** ao abrir a lista aparecem só SIM e NÃO (e, no motivo, os 5 motivos), sem a caixa “Localizar itens”.

**7** ☑ CSP POR FRS: UMA LISTA PRÓPRIA

O CSP fica numa **lista com uma linha por FRS**: o número da FRS é só leitura (não dá para apagar) e ao lado vai a caixa do CSP. **1.** Clique em `MainContainer1` → **+ Inserir** → busque `galeria` → **Galeria vertical em branco** . Renomeie `galCsp` e, na árvore, arraste para logo **acima** de `FormInfo` . Cole:

galCsp → Items **formato português ( ; e ;; )**

```
ForAll(
  Filter(Split(varItem.FRSPendentes; "|"); !IsBlank(Trim(Value))) As f;
  {FRS: Trim(f.Value);
   CSP: With(
     {linha: LookUp(
        Split(Substitute(Substitute(varItem.CSP; Char(13); ""); Char(10);
";"); ";") As b;
        !IsBlank(Find(":"; b.Value)) && Trim(f.Value) in Left(b.Value;
Find(":"; b.Value) - 1))};
     If(IsBlank(linha.Value); ""; Trim(Mid(linha.Value; Find(":";
linha.Value) + 1))))})
```

galCsp → Height **formato português ( ; e ;; )**

```
CountRows(Filter(Split(varItem.FRSPendentes; "|"); !IsBlank(Trim(Value))))
* 52 + 8
```

galCsp → Visible **formato português ( ; e ;; )**

```
FormInfo.Visible && !IsBlank(varItem.FRSPendentes)
```

galCsp → TemplateSize **formato português ( ; e ;; )**

```
52
```

galCsp → FillPortions **formato português ( ; e ;; )**

```
0
```

galCsp → AlignInContainer **formato português ( ; e ;; )**

```
AlignInContainer.Stretch
```

galCsp → Fill **formato português ( ; e ;; )**

```
RGBA(245; 249; 255; 1)
```

**Deu certo se:** (no editor a galeria pode ficar vazia: ela depende do contrato escolhido ter FRS fora do prazo).

**8** ☐ DENTRO DE GALCSP: CRIAR LBLFRS E TXTCSP

**2.** `lblFrs` e `txtCsp` **não existem ainda: você cria agora**, dentro da galeria `galCsp` do passo anterior. **a)** Na árvore, clique em `galCsp`. Na tela, a galeria mostra uma moldura; clique no **lápis** que aparece no canto de cima à esquerda da 1ª linha (ou dentro da 1ª linha) para entrar no modelo. **b)** **+ Inserir** → busque `rótulo` → em **Clássico**, **Rótulo de texto**. Na árvore ele aparece **dentro** de `galCsp` (setinha ›). Renomeie para `lblFrs` (dois cliques no nome ou ⋯ → Renomear). **c)** Clique de novo em `galCsp` → lápis → **+ Inserir** → busque `entrada` → em **Clássico**, **Entrada de texto**. Renomeie para `txtCsp`. **Use os controles da seção Clássico:** o moderno não tem as propriedades `Default` e `HintText` (tem Value e Placeholder) e a fórmula do passo 3 usa `txtCsp.Text`. **d)** Com cada um escolhido, ache a propriedade no seletor (ao lado de fx) e cole:

lblFrs → Text **formato português ( ; e ;; )**

```
"FRS " & ThisItem.FRS
```

lblFrs → X **formato português ( ; e ;; )**

```
16
```

lblFrs → Y **formato português ( ; e ;; )**

```
6
```

lblFrs → Width **formato português ( ; e ;; )**

```
200
```

lblFrs → Height **formato português ( ; e ;; )**

```
40
```

txtCsp → Default **formato português ( ; e ;; )**

```
ThisItem.CSP
```

txtCsp → HintText **formato português ( ; e ;; )**

```
"número do CSP"
```

txtCsp → X **formato português ( ; e ;; )**

```
220
```

txtCsp → Y **formato português ( ; e ;; )**

```
6
```

txtCsp → Width **formato português ( ; e ;; )**

```
260
```

txtCsp → Height **formato português ( ; e ;; )**

```
40
```

**Deu certo se:** na árvore: galCsp › lblFrs e txtCsp. Escolhendo na lista da esquerda um contrato com FRS fora do prazo, cada linha mostra “FRS 1020278950” e a caixa do CSP ao lado. Contrato sem FRS pendente: a galeria some (normal).

▼ **Não deu certo?**

**Inseriu fora da galeria** (aparece solto na árvore, fora de galCsp): apague, clique em galCsp, entre no modelo pelo lápis e insira de novo. **Só achou Default/HintText faltando**: foi inserido o moderno; apague e insira o da seção Clássico. **X vermelho em ThisItem.FRS**: o Items de galCsp não foi colado (passo 1). **Não aparece a seção Clássico** no + Inserir: role a lista até o fim (Clássico fica por último) ou use o moderno assim: `txtCsp → Value` = `ThisItem.CSP`, `txtCsp → Placeholder` = `"número do CSP"`, e no passo 3 troque `txtCsp.Text` por `txtCsp.Value`.

**9** ☐ CARTÃO CSP (DESTRANCADO)

**3.** O que for digitado na lista vai para a coluna CSP no formato da planilha ( `FRS: CSP` , uma por linha) quando clicar em Dados OK. Abra `CSP_DataCard2` e veja o nome da caixa de dentro (ex.: `DataCardValue12` ). Cole as 2 propriedades no **cartão**, trocando `DataCardValueX` por esse nome:

CSP_DataCard2 → Update **formato português ( ; e ;; )**

```
If(IsBlank(varItem.FRSPendentes);
  DataCardValueX.Text;
  Concat(galCsp.AllItems; FRS & ": " & Trim(txtCsp.Text); Char(10)))
```

CSP_DataCard2 → Visible **formato português ( ; e ;; )**

```
IsBlank(varItem.FRSPendentes) && (varPedeCSP || varNaoOk)
```

**Deu certo se:** sem X vermelho. Com FRS fora do prazo o cartão CSP some e fica só a lista; sem FRS (Fora do prazo marcado à mão) aparece a caixa simples de CSP.

**10** ☐ FORMINFO → APARÊNCIA

Para o formulário não ficar espremido em 3 colunas nem cortar linhas: com `FormInfo` escolhido, cole nas propriedades abaixo. A altura acompanha quantos cartões estão visíveis (os nomes dos cartões são os do seu print; se algum tiver outro número, troque).

FormInfo → Columns **formato português ( ; e ;; )**

```
2
```

FormInfo → FillPortions (desliga a altura flexível) **formato português ( ; e ;; )**

```
0
```

FormInfo → Height **formato português ( ; e ;; )**

```
24 + 100 * RoundUp(CountRows(Filter(Table(
  {v: 'Observação Mês_DataCard1'.Visible}; {v: CSP_DataCard2.Visible};
  {v: MotivoProvisao_DataCard2.Visible}; {v:
Justificativa_DataCard3.Visible};
  {v: HouveServico_DataCard2.Visible}; {v: FoiMedido_DataCard2.Visible};
  {v: ForaDoPrazo_DataCard2.Visible}; {v:
SeraSubstituido_DataCard2.Visible};
  {v: DemandaContinuada_DataCard2.Visible}; {v:
ComentarioErro_DataCard1.Visible}); v)) / 2; 0)
```

FormInfo → Fill **formato português ( ; e ;; )**

```
RGBA(245; 249; 255; 1)
```

FormInfo → BorderColor **formato português ( ; e ;; )**

```
RGBA(200; 215; 235; 1)
```

FormInfo → BorderThickness **formato português ( ; e ;; )**

```
1
```

**Deu certo se:** o formulário de validação fica em 2 colunas, com fundo azul bem claro, sem cortar nada; no contrato sem pendência aparece só Observação Mês (e Será substituído/Demanda continuada se estiverem em branco). Os dados da planilha continuam embaixo (Form1).

▼ **Não deu certo?**

Se aparecerem todos os 10 campos em qualquer contrato: o Visible dos cartões não pegou porque eles estavam trancados. Destranque cada um (B9.4) e cole o Visible de novo.

**11** ☐ APP → FORMULAS (ATUALIZAR)

A regra do CSP mudou: antes, o modelo `1020297563:` que o programa grava (sem o número do CSP depois dos dois-pontos) contava como preenchido. Agora o app faz igual à planilha: cada FRS fora do prazo precisa de uma linha **FRS: CSP** com o CSP preenchido (ou **todas: CSP**). Clique em `App` → `Formulas` → clique na caixa fx → **Ctrl+A** → cole o bloco inteiro abaixo (ele substitui tudo, já com a linha nova do varFechado).

App → Formulas (bloco inteiro) **formato português ( ; e ;; )**

```
varEmail = Lower(User().Email);;
varNome = User().FullName;;
varComp = Text(DateAdd(Today(); -1; TimeUnit.Months); "mm/yyyy");;
varAdmin = varEmail in ["levi.vasconcelos@petrobras.com.br"];;
varItem = RecordsGallery1.Selected;;
varMeuFG = ";" & varEmail & ";" in ";" &
Substitute(Lower(varItem.FiscalEmail & ";" & varItem.GerenteEmail); " ";
"") & ";";;
varApoioContratos = Filter(SOx_SCM42_Apoio; Email = varEmail).Title;;
varApoio = varItem.Contrato in varApoioContratos;;
varMeu = varMeuFG || varApoio;;
varPrazo = Date(Value(Right(varItem.Competencia; 4));
Value(Left(varItem.Competencia; 2)) + 1; 15);;
varFechado = varItem.MesFechado = "SIM";;
varPode = varAdmin || (varMeu && !varFechado);;
varValida = varAdmin || (varMeuFG && !varFechado);;
varFrsSemCsp = Concat(
    Filter(Split(varItem.FRSPendentes; "|") As f;
      !IsBlank(Trim(f.Value));
      CountRows(
        Filter(Split(Substitute(Substitute(varItem.CSP; Char(13); "");
Char(10); ";"); ";") As b;
          !IsBlank(Find(":"; b.Value));
          !IsBlank(Trim(Mid(b.Value; Find(":"; b.Value) + 1)));
          With({esq: Lower(Trim(Left(b.Value; Find(":"; b.Value) - 1)))};
            esq in ["*"; "todos"; "tudo"] || StartsWith(esq; "todas") ||
Trim(f.Value) in esq))) = 0);
    Trim(Value); ", ");;
varCobre = IsBlank(varFrsSemCsp);;
varPedeCSP = !IsBlank(varItem.FRSPendentes) || varItem.ForaDoPrazo.Value =
"SIM";;
varPedeMotivo = varItem.Aba = "SOP" && varItem.Situacao in
["Provisionado"; "Misto"];;
varPedeJust = varItem.Situacao in ["Provisionado"; "Fora do prazo";
"Misto"; "Incompleto"];;
varPedeServico = varItem.Situacao = "Sem medição";;
varPedeInfo = varPedeCSP || varPedeMotivo || varPedeJust ||
varPedeServico;;
varFalta = Concat(
  Filter(
    Table(
      {m: If(IsBlank(varFrsSemCsp); "Falta o CSP."; "Falta o CSP destas
FRS fora do prazo: " & varFrsSemCsp & " (preencha ao lado de cada FRS e
clique em Dados OK)."); f: varPedeCSP && (IsBlank(Trim(varItem.CSP)) ||
!varCobre)};
      {m: "Falta o motivo da provisão."; f: varPedeMotivo &&
IsBlank(varItem.MotivoProvisao.Value)};
      {m: "Falta a justificativa (mínimo 15 letras)."; f: varPedeJust &&
Len(Trim(varItem.Justificativa)) < 15});
    f);
  m; " ");;
```

E também: FormInfo → OnSuccess (Ctrl+A e cole; usa a mesma regra do CSP) formato português ( ; e ;; )

```
With({it: FormInfo.LastSubmit; idx: FormInfo.LastSubmit.ID};
  With({cobre: IsBlank(Concat(
             Filter(Split(it.FRSPendentes; "|") As f;
               !IsBlank(Trim(f.Value));
               CountRows(
                 Filter(Split(Substitute(Substitute(it.CSP; Char(13); "");
Char(10); ";"); ";") As b;
                   !IsBlank(Find(":"; b.Value));
                   !IsBlank(Trim(Mid(b.Value; Find(":"; b.Value) + 1)));
                   With({esq: Lower(Trim(Left(b.Value; Find(":"; b.Value)
- 1)))};
                     esq in ["*"; "todos"; "tudo"] || StartsWith(esq;
"todas") || Trim(f.Value) in esq))) = 0);
             Trim(Value); ", "))};
    With({completo:
          !((!IsBlank(it.FRSPendentes) || it.ForaDoPrazo.Value = "SIM")
&& (IsBlank(Trim(it.CSP)) || !cobre))
          && !(it.Aba = "SOP" && it.Situacao in ["Provisionado"; "Misto"]
&& IsBlank(it.MotivoProvisao.Value))
          && !(it.Situacao in ["Provisionado"; "Fora do prazo"; "Misto";
"Incompleto"] && Len(Trim(it.Justificativa)) < 15);
         quando: Text(Now(); "dd/mm/yyyy hh:mm")};
      If(!IsBlank(it.FRSPendentes);
        Patch(SOx_SCM42_Contratos; LookUp(SOx_SCM42_Contratos; ID = idx);
{ForaDoPrazo: {Value: If(cobre; "NÃO"; "SIM")}}));;
      If(varNaoOk;
        If(IsBlank(Trim(it.ComentarioErro));
          Notify("Salvo. Para registrar Não OK, escreva o que estava
errado."; NotificationType.Warning);
          Patch(SOx_SCM42_Contratos; LookUp(SOx_SCM42_Contratos; ID =
idx);
            {OkFiscal: "NÃO OK"; DadosCorretos: "NÃO"; OkFiscalPor:
varNome; OkFiscalEm: quando});;
          Set(varNaoOk; false);;
          Notify("Não OK registrado. A correção vai para a planilha no
Puxar respostas."; NotificationType.Warning));
        If(!varValida;
          Notify("Salvo. Agora o fiscal ou o gerente confere e dá o OK.";
NotificationType.Success);
          completo;
          Patch(SOx_SCM42_Contratos; LookUp(SOx_SCM42_Contratos; ID =
idx);
            {OkFiscal: "OK"; DadosCorretos: "SIM"; OkFiscalPor: varNome;
OkFiscalEm: quando});;
          Notify("Dados confirmados. Obrigado!";
NotificationType.Success);
          Notify("Salvo, mas ainda falta informação obrigatória. Veja o
aviso em vermelho."; NotificationType.Warning))
      )
    )
  )
)
```

**Deu certo se:** sem X vermelho. Num contrato com FRS fora do prazo, a faixa laranja diz **Falta o CSP destas FRS fora do prazo: 1020297563 ...** até o número do CSP ser escrito depois dos dois-pontos.

| Campo | Planilha | App (fiscal ou gerente) |
|---|---|---|
| CSP, Justificativa, Motivo da provisão (SOP) | Destravados | Aparecem quando o contrato pede; obrigatórios para o OK |
| Provisão | Fórmula pelo CSP | Só leitura; se monta pelo CSP no Puxar |

| | | |
|---|---|---|
| Será substituído, Demanda continuada | Destravados | Aparecem quando em branco; no Dados não OK, sempre |
| Houve serviço, Foi medido, Fora do prazo | Destravados | Só no Dados não OK; a diferença fica vermelha na planilha |
| Observação Mês | (cópia de controle) | Sempre, opcional |
| O resto | Travado | Só leitura (Form1) |

# B10. Lado direito: botões Dados OK e Dados não OK

15 min · B · Montar o app

**Objetivo: Hoje** não existe. **Fica:** embaixo do formulário de validação, os botões. O fiscal **ou** o gerente clica em **Dados OK**: o app grava o que foi preenchido e registra o OK se o obrigatório estiver completo. Basta um dos dois. Se algo estiver errado, clica em **Dados não OK**: aparecem os campos para corrigir e **O que estava errado**; ao clicar em **Salvar não OK**, fica registrado. Quem não é fiscal nem gerente do contrato não vê botão nenhum (só consulta).

Lembre as 3 regras da B3: seletor em Português; conferir a propriedade antes de colar; sem X vermelho antes de seguir.

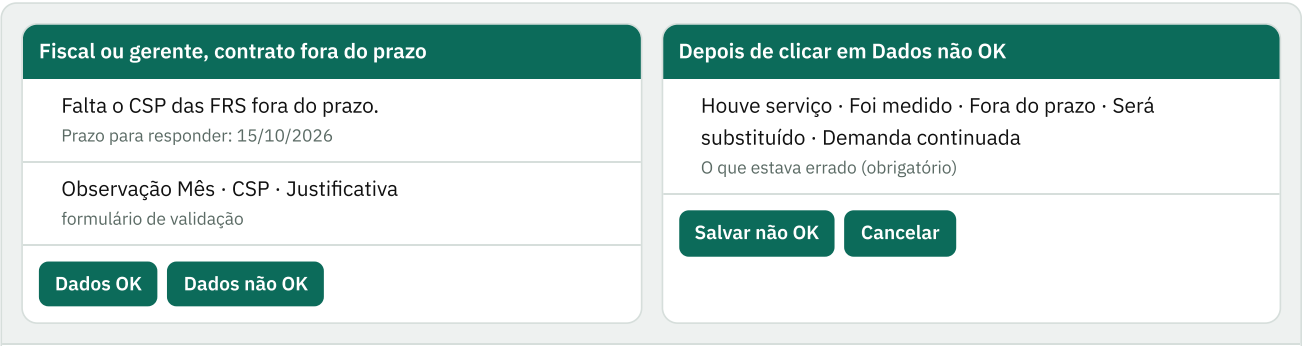



Ilustração do lado direito para quem é fiscal ou gerente do contrato. Para os outros, só os dados (sem formulário e sem botões). Depois de validado, os botões somem e aparece quem respondeu.

1 ☐ ÁRVORE → MAINCONTAINER1

Clique em `MainContainer1` → **+ Inserir** → busque `horizontal` → **Contêiner horizontal**. Renomeie `conBotoes`. Na árvore, deixe `conBotoes` logo abaixo de `FormInfo`. Cole (se você já tinha feito, troque pelos valores abaixo: é isso que tira o espaço grande e faz a área sumir depois do OK):

conBotoes → Height **formato português ( ; e ;; )**

```
44
```

conBotoes → FillPortions (desliga a altura flexível; ou desligue o botão Altura flexível no painel da direita) **formato português ( ; e ;; )**

```
0
```

conBotoes → Visible **formato português ( ; e ;; )**

```
varPode && (IsBlank(varItem.OkFiscal) || varNaoOk)
```

conBotoes → LayoutJustifyContent (botões no canto direito) **formato português ( ; e ;; )**

```
LayoutJustifyContent.End
```

conBotoes → LayoutGap **formato português ( ; e ;; )**

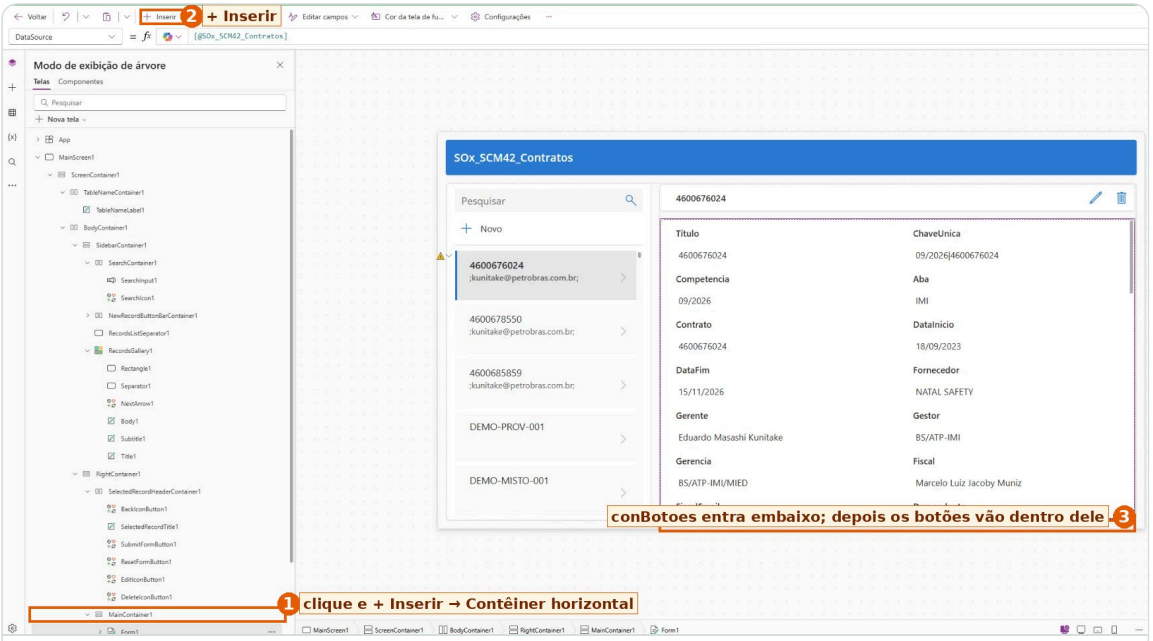
```
8
```

conBotoes → PaddingRight **formato português ( ; e ;; )**

```
16
```

conBotoes → LayoutAlignItems **formato português ( ; e ;; )**

```
LayoutAlignItems.Center
```




**PRINT DO SEU APP** 1 MainContainer1 na árvore · 2 + Inserir → Contêiner horizontal · 3 ele entra embaixo do Form1. Depois, para cada botão: clique em conBotoes na árvore → + Inserir → Botão. *(print de antes da B4: a parte da direita e a árvore são iguais às suas hoje)* Clique para ampliar.

**Deu certo se:** uma faixa baixa (44) embaixo do formulário de validação, com os botões pequenos no canto direito; depois do OK ela some junto com o formulário.

## 2 ☐ ÁRVORE → CONBOTOES

Clique em `conBotoes` → **+ Inserir** → **Botão**. Renomeie `btnOk`. Cole nas 3 propriedades:

btnOk → Text **formato português ( ; e ;; )**

```
"Dados OK"
```

btnOk, btnNaoOk, btnSalvarNaoOk, btnCancelar → Width (nos 4) **formato português ( ; e ;; )**

```
120
```

os 4 → Height **formato português ( ; e ;; )**

```
32
```

os 4 → Size **formato português ( ; e ;; )**

```
11
```

btnOk → Fill **formato português ( ; e ;; )**

```
RGBA(46; 125; 50; 1)
```

btnOk → Visible **formato português ( ; e ;; )**

```
varValida && IsBlank(varItem.OkFiscal) && !varNaoOk
```

btnOk → OnSelect **formato português ( ; e ;; )**

```
SubmitForm(FormInfo)
```

**Deu certo se:** as 3 sem erro.

**3** ☐ CONBOTOES

De novo em `conBotoes` : outro **Botão** , renomeie `btnNaoOk` :

btnNaoOk → Text **formato português ( ; e ;; )**

```
"Dados não OK"
```

btnNaoOk → Fill **formato português ( ; e ;; )**

```
RGBA(198; 40; 40; 1)
```

btnNaoOk → Visible **formato português ( ; e ;; )**

```
varValida && IsBlank(varItem.OkFiscal) && !varNaoOk
```

btnNaoOk → OnSelect **formato português ( ; e ;; )**

```
Set(varNaoOk; true)
```

**Deu certo se:** as 3 sem erro.

**4** ☐ CONBOTOES

Outro **Botão** em `conBotoes` , renomeie `btnSalvarNaoOk` :

btnSalvarNaoOk → Text **formato português ( ; e ;; )**

```
"Salvar não OK"
```

btnSalvarNaoOk → Visible **formato português ( ; e ;; )**

```
varNaoOk
```

btnSalvarNaoOk → OnSelect **formato português ( ; e ;; )**

```
SubmitForm(FormInfo)
```

**Deu certo se:** as 3 sem erro.

**5** ☐ CONBOTOES

Último **Botão** em `conBotoes` , renomeie `btnCancelar` :

btnCancelar → Text **formato português ( ; e ;; )**

```
"Cancelar"
```

btnCancelar → Visible **formato português ( ; e ;; )**

```
varNaoOk
```

btnCancelar → OnSelect **formato português ( ; e ;; )**

```
ResetForm(FormInfo);;
Set(varNaoOk; false)
```

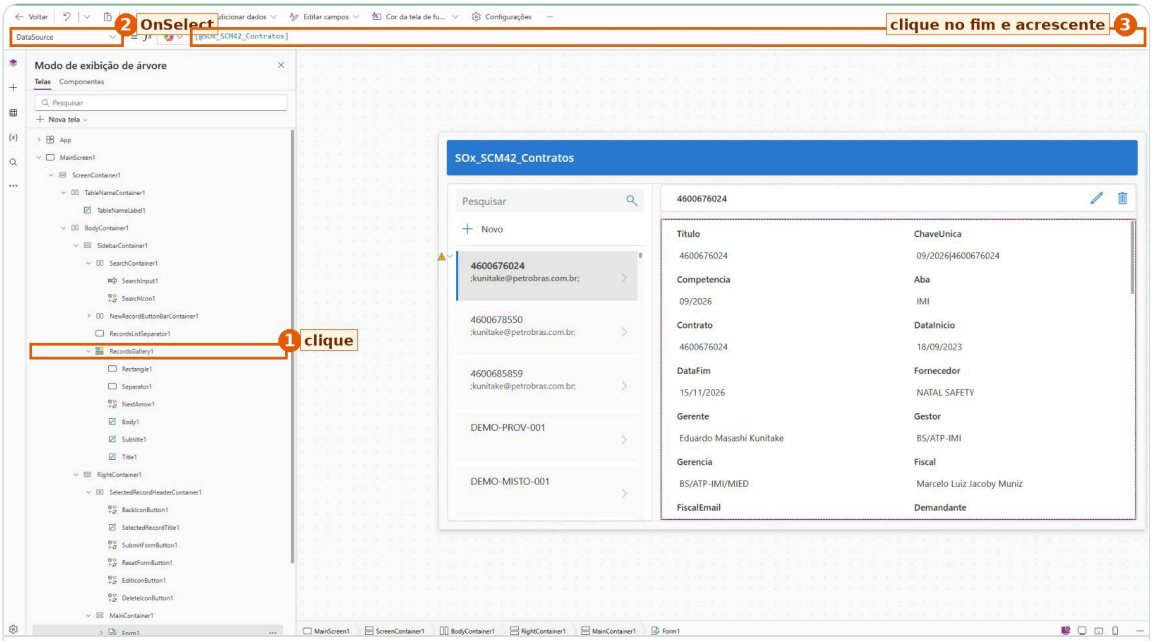
**Deu certo se:** as 3 sem erro.

**6** ☐ ÁRVORE → RECORDSGALLERY1

Para fechar o Não OK ao trocar de contrato: clique em `RecordsGallery1`, propriedade `OnSelect`. Clique no **fim** do que já está escrito na caixa fx (não apague nada) e acrescente:

RecordsGallery1 → OnSelect (acrescentar no fim) **formato português ( ; e ;; )**

```
;;
Set(varNaoOk; false)
```




**PRINT DO SEU APP** 1 RecordsGallery1 · 2 escolha OnSelect · 3 clique no fim da fórmula que já existe e acrescente. *(print de antes da B4: a parte da direita e a árvore são iguais às suas hoje)* Clique para ampliar.

**Deu certo se:** sem erro. Se a caixa estava vazia, cole só `Set(varNaoOk; false)` (em português), sem o `;;` do começo. Salve. No editor os botões podem parecer sumidos: eles dependem do contrato escolhido.

# B10b. Pessoas de apoio: tela só sua, por link

30 min · B · Montar o app

**Objetivo:** Em alguns contratos, outras pessoas (que não são o fiscal nem o gerente) ajudam a preencher. **Fica:** uma **tela separada, "Apoio por contrato"**, que só você abre, por um **link próprio** (o link normal do app + `&tela=apoio`). Nela: escolhe o contrato, digita o nome da pessoa, clica em **Adicionar apoio**; embaixo, a lista de todos os apoios cadastrados, com ✕ para tirar. **Na tela dos contratos não aparece nada a mais** para ninguém. Se alguém sem ser você abrir esse link, volta direto para a tela normal. Quem é apoio, só naquele contrato: vê o contrato em **Meus contratos / Meus pendentes**, preenche CSP, motivo, justificativa e observação e clica em **Salvar**. O **Dados OK / Dados não OK** continua só do fiscal ou do gerente.

Lembre as 3 regras da B3: seletor em Português; conferir a propriedade antes de colar; sem X vermelho antes de seguir.

**1** ☐ PROGRAMA → PUBLICAR ONLINE AGORA

A lista onde o app guarda o apoio (**SOx_SCM42_Apoio**: contrato, e-mail, nome, quem incluiu) é criada **sozinha pelo programa**. Rode o `ajuste_direto.py` novo (2 ajustes novos de apoio) e depois **Publicar online agora**.

**Deu certo se:** na janela do programa aparece **Lista criada no site: SOx_SCM42_Apoio** (só na primeira vez).

**2** ☐ STUDIO → DADOS → ADICIONAR DADOS (2 VEZES)

No Studio: ícone **Dados** (cilindro) → **Adicionar dados**. **1.** Busque `SharePoint` → site ControleSOxSCM42 → marque **SOx_SCM42_Apoio** → **Conectar**. **2.** **Adicionar dados** de novo → busque `Usuários do Office 365` (Office 365 Users) → **Conectar**. É ele que deixa procurar a pessoa pelo nome.

**Deu certo se:** no painel Dados aparecem SOx_SCM42_Contratos, SOx_SCM42_Apoio e Usuários do Office 365.

**3** ☐ APP → FORMULAS

Clique em `App` → `Formulas` → caixa fx → **Ctrl+A** → cole o bloco inteiro (agora tem `varMeuFG`, `varApoio` e `varValida`).

App → Formulas (bloco inteiro) **formato português ( ; e ;; )**

```
varEmail = Lower(User().Email);;
varNome = User().FullName;;
varComp = Text(DateAdd(Today(); -1; TimeUnit.Months); "mm/yyyy");;
varAdmin = varEmail in ["levi.vasconcelos@petrobras.com.br"];;
varItem = RecordsGallery1.Selected;;
varMeuFG = ";" & varEmail & ";" in ";" &
Substitute(Lower(varItem.FiscalEmail & ";" & varItem.GerenteEmail); " ";
"") & ";";;
varApoioContratos = Filter(SOx_SCM42_Apoio; Email = varEmail).Title;;
varApoio = varItem.Contrato in varApoioContratos;;
varMeu = varMeuFG || varApoio;;
varPrazo = Date(Value(Right(varItem.Competencia; 4));
Value(Left(varItem.Competencia; 2)) + 1; 15);;
varFechado = varItem.MesFechado = "SIM";;
varPode = varAdmin || (varMeu && !varFechado);;
varValida = varAdmin || (varMeuFG && !varFechado);;
varFrsSemCsp = Concat(
    Filter(Split(varItem.FRSPendentes; "|") As f;
      !IsBlank(Trim(f.Value));
      CountRows(
        Filter(Split(Substitute(Substitute(varItem.CSP; Char(13); "");
Char(10); ";"); ";") As b;
          !IsBlank(Find(":"; b.Value));
          !IsBlank(Trim(Mid(b.Value; Find(":"; b.Value) + 1)));
          With({esq: Lower(Trim(Left(b.Value; Find(":"; b.Value) - 1)))};
            esq in ["*"; "todos"; "tudo"] || StartsWith(esq; "todas") ||
Trim(f.Value) in esq))) = 0);
    Trim(Value); ", ");;
varCobre = IsBlank(varFrsSemCsp);;
varPedeCSP = !IsBlank(varItem.FRSPendentes) || varItem.ForaDoPrazo.Value =
"SIM";;
varPedeMotivo = varItem.Aba = "SOP" && varItem.Situacao in
["Provisionado"; "Misto"];;
varPedeJust = varItem.Situacao in ["Provisionado"; "Fora do prazo";
"Misto"; "Incompleto"];;
varPedeServico = varItem.Situacao = "Sem medição";;
varPedeInfo = varPedeCSP || varPedeMotivo || varPedeJust ||
varPedeServico;;
varFalta = Concat(
  Filter(
    Table(
      {m: If(IsBlank(varFrsSemCsp); "Falta o CSP."; "Falta o CSP destas
FRS fora do prazo: " & varFrsSemCsp & " (preencha ao lado de cada FRS e
clique em Dados OK)."); f: varPedeCSP && (IsBlank(Trim(varItem.CSP)) ||
!varCobre)};
      {m: "Falta o motivo da provisão."; f: varPedeMotivo &&
IsBlank(varItem.MotivoProvisao.Value)};
      {m: "Falta a justificativa (mínimo 15 letras)."; f: varPedeJust &&
Len(Trim(varItem.Justificativa)) < 15});
    f);
  m; " ");;
```

**Deu certo se:** sem X vermelho (aviso amarelo de delegação é normal).

**4** ☐ COLE DE NOVO 4 FÓRMULAS

Estas passaram a contar o apoio (ou a separar quem só preenche de quem valida). Em cada uma: escolha o controle e a propriedade, **Ctrl+A** na caixa fx e cole.

RecordsGallery1 → Items **formato português ( ; e ;; )**

```
With(
  {base: Filter(SOx_SCM42_Contratos;
     Competencia = ddComp.Selected.Value
     && (ddGestor.Selected.Value = "Todos os gestores" || Gestor =
ddGestor.Selected.Value))};
  SortByColumns(
    Filter(base;
      IsBlank(SearchInput1.Text)
        || SearchInput1.Text in Contrato
        || SearchInput1.Text in Fornecedor
        || SearchInput1.Text in Fiscal;
      With({meu: ";" & varEmail & ";" in ";" &
Substitute(Lower(FiscalEmail & ";" & GerenteEmail); " "; "") & ";"
                || Contrato in varApoioContratos};
        Switch(ddVisao.Selected.Value;
          "Todos"; true;
          "Meus contratos"; meu;
          "Meus pendentes"; meu && IsBlank(OkFiscal);
          "Não OK"; OkFiscal = "NÃO OK")));
    "Contrato";
    SortOrder.Ascending))
```

lblResumo → Text **formato português ( ; e ;; )**

```
With(
  {mes: Filter(SOx_SCM42_Contratos; Competencia = ddComp.Selected.Value)};
  With(
  {m: Filter(mes;
       ";" & varEmail & ";" in ";" & Substitute(Lower(FiscalEmail & ";" &
GerenteEmail); " "; "") & ";"
         || Contrato in varApoioContratos)};
  "Seus contratos em " & ddComp.Selected.Value & ": " & CountRows(m)
  & Char(10) & "No prazo " & CountRows(Filter(m; Situacao = "Medido no
prazo"))
  & " · Provisionado " & CountRows(Filter(m; Situacao = "Provisionado"))
  & " · Fora do prazo " & CountRows(Filter(m; Situacao = "Fora do prazo"))
  & " · Sem medição " & CountRows(Filter(m; Situacao = "Sem medição"))
  & " · No prazo e provisão " & CountRows(Filter(m; Situacao = "Misto"))
  & Char(10) & "Validados " & CountRows(Filter(m; !IsBlank(OkFiscal)))
  & " · Faltam " & CountRows(Filter(m; IsBlank(OkFiscal)))))
```

FormInfo → OnSuccess **formato português ( ; e ;; )**

```
With({it: FormInfo.LastSubmit; idx: FormInfo.LastSubmit.ID};
  With({cobre: IsBlank(Concat(
             Filter(Split(it.FRSPendentes; "|") As f;
               !IsBlank(Trim(f.Value));
               CountRows(
                 Filter(Split(Substitute(Substitute(it.CSP; Char(13); "");
Char(10); ";"); ";") As b;
                   !IsBlank(Find(":"; b.Value));
                   !IsBlank(Trim(Mid(b.Value; Find(":"; b.Value) + 1)));
                   With({esq: Lower(Trim(Left(b.Value; Find(":"; b.Value)
- 1)))};
                     esq in ["*"; "todos"; "tudo"] || StartsWith(esq;
"todas") || Trim(f.Value) in esq))) = 0);
             Trim(Value); ", "))};
    With({completo:
          !((!IsBlank(it.FRSPendentes) || it.ForaDoPrazo.Value = "SIM")
&& (IsBlank(Trim(it.CSP)) || !cobre))
          && !(it.Aba = "SOP" && it.Situacao in ["Provisionado"; "Misto"]
&& IsBlank(it.MotivoProvisao.Value))
          && !(it.Situacao in ["Provisionado"; "Fora do prazo"; "Misto";
"Incompleto"] && Len(Trim(it.Justificativa)) < 15);
         quando: Text(Now(); "dd/mm/yyyy hh:mm")};
      If(!IsBlank(it.FRSPendentes);
        Patch(SOx_SCM42_Contratos; LookUp(SOx_SCM42_Contratos; ID = idx);
{ForaDoPrazo: {Value: If(cobre; "NÃO"; "SIM")}}));;
      If(varNaoOk;
        If(IsBlank(Trim(it.ComentarioErro));
          Notify("Salvo. Para registrar Não OK, escreva o que estava
errado."; NotificationType.Warning);
          Patch(SOx_SCM42_Contratos; LookUp(SOx_SCM42_Contratos; ID =
idx);
            {OkFiscal: "NÃO OK"; DadosCorretos: "NÃO"; OkFiscalPor:
varNome; OkFiscalEm: quando});;
          Set(varNaoOk; false);;
          Notify("Não OK registrado. A correção vai para a planilha no
Puxar respostas."; NotificationType.Warning));
        If(!varValida;
          Notify("Salvo. Agora o fiscal ou o gerente confere e dá o OK.";
NotificationType.Success);
          completo;
          Patch(SOx_SCM42_Contratos; LookUp(SOx_SCM42_Contratos; ID =
idx);
            {OkFiscal: "OK"; DadosCorretos: "SIM"; OkFiscalPor: varNome;
OkFiscalEm: quando});;
          Notify("Dados confirmados. Obrigado!";
NotificationType.Success);
          Notify("Salvo, mas ainda falta informação obrigatória. Veja o
aviso em vermelho."; NotificationType.Warning))
      )
    )
  )
)
```

btnOk e btnNaoOk → Visible (nos 2) **formato português ( ; e ;; )**

```
varValida && IsBlank(varItem.OkFiscal) && !varNaoOk
```

**Deu certo se:** sem X vermelho.

**5** ☐ ÁRVORE → CONBOTOES → BOTÃO SALVAR

Clique em `conBotoes` → **+ Inserir** → **Botão**. Renomeie `btnSalvarApoio` (Width 120, Height 32, Size 11, como os outros). Cole:

btnSalvarApoio → Text **formato português ( ; e ;; )**

```
"Salvar"
```

btnSalvarApoio → Visible **formato português ( ; e ;; )**

```
varPode && !varValida && IsBlank(varItem.OkFiscal)
```

btnSalvarApoio → OnSelect **formato português ( ; e ;; )**

```
SubmitForm(FormInfo)
```

**Deu certo se:** nos seus contratos esse botão não aparece; quem é só apoio vê **Salvar** no lugar de Dados OK / Dados não OK.

**6** ☐ ÁRVORE → + NOVA TELA → EM BRANCO

No alto da árvore: **+ Nova tela** → **Em branco**. Renomeie a tela nova para `telaApoio`. Com ela escolhida, cole (quem não é você volta para a tela normal):

telaApoio → OnVisible **formato português ( ; e ;; )**

```
If(!varAdmin; Navigate(MainScreen1))
```

**Deu certo se:** a árvore tem MainScreen1 e telaApoio.

**7** ☐ TELAAPOIO → TÍTULO E VOLTAR

Com `telaApoio` escolhida: **+ Inserir** → **Rótulo de texto** (`lblApoioTitulo`) e **+ Inserir** → **Botão** (`btnVoltar`). Cole:

lblApoioTitulo → Text **formato português ( ; e ;; )**

```
"Apoio por contrato (só o administrador vê esta tela)"
```

lblApoioTitulo → X, Y, Width, Height, Size **formato português ( ; e ;; )**

```
20 · 16 · 900 · 40 · 18
```

btnVoltar → Text **formato português ( ; e ;; )**

```
"Voltar aos contratos"
```

btnVoltar → OnSelect **formato português ( ; e ;; )**

```
Navigate(MainScreen1)
```

btnVoltar → X, Y, Width, Height **formato português ( ; e ;; )**

```
1140 · 18 · 200 · 36
```

**Deu certo se:** o título no alto e o botão Voltar à direita.

**8** ☐ TELAAPOIO → CONTRATO, PESSOA E ADICIONAR

Mais 3 controles na `telaApoio`: **Caixa de combinação** `cbContrato`, outra **Caixa de combinação** `cbPessoa` e **Botão** `btnAddApoio`. Cole:

cbContrato → Items **formato português ( ; e ;; )**

```
Sort(Filter(SOx_SCM42_Contratos; Competencia = varComp); Contrato)
```

cbContrato → DisplayFields e SearchFields (nos 2) **formato português ( ; e ;; )**

```
["Contrato"; "Fornecedor"]
```

cbContrato → SelectMultiple **formato português ( ; e ;; )**

```
false
```

cbContrato → InputTextPlaceholder **formato português ( ; e ;; )**

```
"Contrato (digite o número ou o fornecedor)"
```

cbContrato → X, Y, Width **formato português ( ; e ;; )**

```
20 · 80 · 440
```

cbPessoa → Items **formato português ( ; e ;; )**

```
Office365Users.SearchUserV2({searchTerm: Self.SearchText; top: 10}).value
```

cbPessoa → DisplayFields e SearchFields (nos 2) **formato português ( ; e ;; )**

```
["DisplayName"; "Mail"]
```

cbPessoa → SelectMultiple **formato português ( ; e ;; )**

```
false
```

cbPessoa → InputTextPlaceholder **formato português ( ; e ;; )**

```
"Pessoa de apoio (digite o nome)"
```

cbPessoa → X, Y, Width **formato português ( ; e ;; )**

```
480 · 80 · 440
```

btnAddApoio → Text **formato português ( ; e ;; )**

```
"Adicionar apoio"
```

btnAddApoio → OnSelect **formato português ( ; e ;; )**

```
If(IsBlank(cbContrato.Selected.Contrato);
  Notify("Escolha o contrato."; NotificationType.Warning);
  IsBlank(cbPessoa.Selected.Mail);
  Notify("Escolha a pessoa (digite parte do nome).";
NotificationType.Warning);
  !IsBlank(LookUp(SOx_SCM42_Apoio; Title = cbContrato.Selected.Contrato &&
Email = Lower(cbPessoa.Selected.Mail)));
  Notify(cbPessoa.Selected.DisplayName & " já é apoio neste contrato.";
NotificationType.Information);
  Patch(SOx_SCM42_Apoio; Defaults(SOx_SCM42_Apoio);
    {Title: cbContrato.Selected.Contrato; Email:
Lower(cbPessoa.Selected.Mail);
     Nome: cbPessoa.Selected.DisplayName; IncluidoPor: varNome});;
  Reset(cbPessoa);;
  Notify("Apoio incluído. A pessoa já vê o contrato em Meus contratos
(precisa ser membro do site)."; NotificationType.Success))
```

btnAddApoio → X, Y, Width, Height **formato português ( ; e ;; )**

```
940 · 80 · 180 · 40
```

**Deu certo se:** com o contrato e a pessoa escolhidos, Adicionar apoio mostra a mensagem verde.

**9** ☐ TELAAPOIO → LISTA DE APOIOS

**+ Inserir** → **Galeria vertical em branco**, renomeie `galApoio`. Com a 1ª linha dela escolhida: **Rótulo de texto** (`lblApoioLinha`) e o ícone **Cancelar** (o ✕, `icoTirarApoio`). Cole:

galApoio → Items (com contrato escolhido mostra só ele; sem, mostra todos) **formato português ( ; e ;; )**

```
SortByColumns(
  If(IsBlank(cbContrato.Selected.Contrato);
    SOx_SCM42_Apoio;
    Filter(SOx_SCM42_Apoio; Title = cbContrato.Selected.Contrato));
  "Title")
```

galApoio → X, Y, Width, Height, TemplateSize **formato português ( ; e ;; )**

```
20 · 140 · 1320 · 600 · 44
```

lblApoioLinha → Text **formato português ( ; e ;; )**

```
ThisItem.Title & " · " & ThisItem.Nome & " · " & ThisItem.Email & "
(incluído por " & ThisItem.IncluidoPor & ")"
```

lblApoioLinha → X, Y, Width, Height **formato português ( ; e ;; )**

```
12 · 6 · 1200 · 32
```

icoTirarApoio → OnSelect **formato português ( ; e ;; )**

```
Remove(SOx_SCM42_Apoio; ThisItem);;
Notify("Apoio retirado."; NotificationType.Information)
```

icoTirarApoio → X, Y, Width, Height **formato português ( ; e ;; )**

```
1270 · 8 · 28 · 28
```

**Deu certo se:** aparece cada apoio cadastrado em uma linha, com o ✕ à direita; o ✕ tira na hora.

**10** ☐ APP → STARTSCREEN

Para o link abrir direto na tela de apoio: clique em `App` → propriedade `StartScreen`:

App → StartScreen **formato português ( ; e ;; )**

```
If(Param("tela") = "apoio"; telaApoio; MainScreen1)
```

**Deu certo se:** sem X vermelho. Salve e **publique** (B12.1).

**11** ☐ O SEU LINK DE APOIO

Pegue o **Link da Web** do app (make.powerapps.com → Aplicativos → três pontinhos do app → Detalhes) e acrescente no fim `&tela=apoio`. Exemplo: `https://apps.powerapps.com/play/e/…/a/…?tenantId=…&tela=apoio`. Salve nos favoritos do Edge como **SOx – Apoio**. O link normal (sem `&tela=apoio`) continua abrindo a tela dos contratos, para todos.

**Deu certo se:** pelo favorito abre a tela Apoio por contrato; pelo link normal, a tela de sempre. Um colega que abrir o seu link de apoio cai na tela normal.

**12** ☐ ACESSO DE QUEM É APOIO

Como os fiscais, quem é apoio precisa: (1) ser **membro do site** ControleSOxSCM42 (engrenagem ⚙ → Permissões do site → Adicionar membros) e (2) receber o **compartilhamento do app** (B12). Dica: compartilhando o site e o app com o **grupo de e-mail da gerência**, não precisa fazer pessoa por pessoa.

**Deu certo se:** a pessoa de apoio abre o app, vê o contrato em Meus contratos e consegue salvar o CSP.

# B11. Testar você mesmo

15 min · B · Montar o app

**Objetivo:** ver tudo funcionando antes de mostrar a alguém. Você vira fiscal de 2 contratos por alguns minutos.

**1** ☐ LISTA NO NAVEGADOR

Escolha 2 contratos do mês e anote: **A** (de preferência Fora do prazo ou Provisionado) e **B** (qualquer outro).

**Deu certo se:** 2 números anotados.

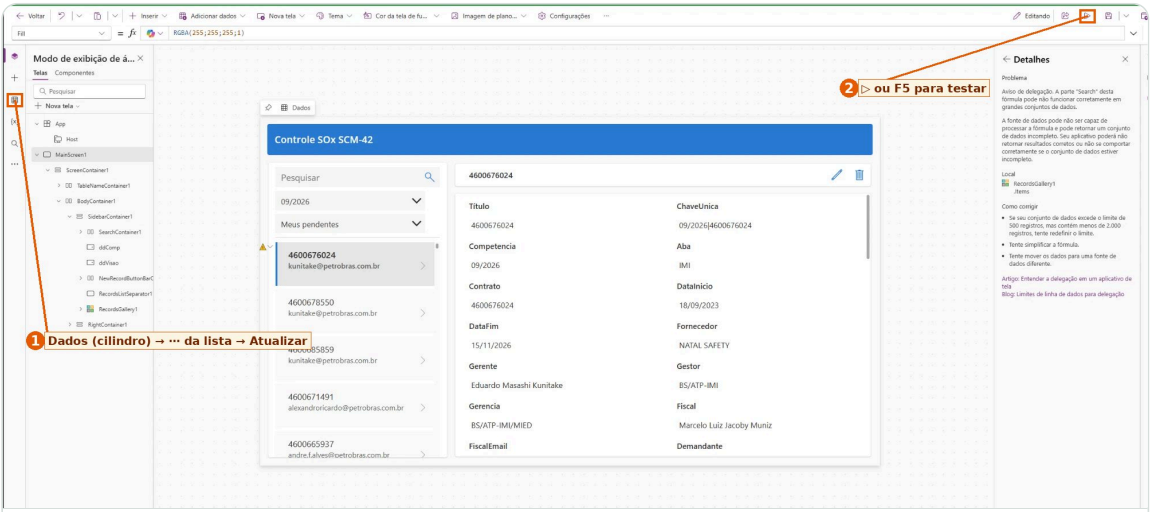
**2** ☐ LISTA → EDITAR NO MODO DE GRADE

Clique em **Editar no modo de exibição de grade**. Em A e B, troque FiscalEmail pelo seu e-mail ( `levi.vasconcelos@petrobras.com.br` , sem ponto e vírgula). **Sair do modo de exibição de grade** .

**Deu certo se:** as 2 linhas mostram o seu e-mail.

**3** ☐ STUDIO → DADOS

No painel **Dados** (ícone de cilindro à esquerda), três pontinhos na lista → **Atualizar**. Aperte F5.




PRINT DO SEU APP 1 ícone Dados (cilindro) na faixa da esquerda → ··· ao lado de SOx_SCM42_Contratos → Atualizar · 2 ▷ (ou F5) para testar; Esc volta. Clique para ampliar.

**Deu certo se:** em Meus pendentes aparecem A e B, com faixa cinza; clicando em A, o ✎ aparece no alto da direita.

**4** ☐ APP EM TESTE

Clique em **A** na lista e depois em **Dados OK**.

**Deu certo se:** mensagem verde, os botões somem, aparece **✔ Dados OK · seu nome · data**, a faixa fica verde e A sai de Meus pendentes.

**5** ☐ APP EM TESTE

Se algum dos seus contratos de teste for **Fora do prazo** ou **Provisionado**: clique nele. Aparecem o aviso vermelho e os campos do FormInfo. Clique em **Dados OK** sem preencher.

**Deu certo se:** mensagem amarela "ainda falta algo" e nada de OK. Preencha (CSP `todas: CON001`, justificativa com mais de 15 letras, motivo) e clique de novo: aí vem o OK verde.

▼ **Não deu certo?**

Nenhum dos dois é assim? Escolha na B11.1 um contrato Fora do prazo ou Provisionado como A.

**6** ☐ APP EM TESTE

Clique em **B** e em **Dados não OK**. Troque um dos campos SIM/NÃO (por exemplo Será substituído), escreva em **O que estava errado** e clique em **Salvar não OK**.

**Deu certo se:** aparece ✖ **Dados não OK** com o que estava errado; a faixa fica vermelha; B aparece no filtro **Não OK**; depois do Puxar respostas, a célula trocada fica vermelha na planilha.

**7** ☐ LISTA NO NAVEGADOR

Atualize a lista e olhe A e B no modo de grade.

**Deu certo se:** A tem OkFiscal = OK e DadosCorretos = SIM; B tem OkFiscal = NÃO OK, DadosCorretos = NÃO, ComentarioErro e o campo trocado; os dois com OkFiscalPor e OkFiscalEm.

**8** ☐ LIMPAR O TESTE

No modo de grade, apague OkFiscal, OkFiscalPor, OkFiscalEm, DadosCorretos, ComentarioErro, Observação Mês, CSP, Justificativa e MotivoProvisao de A e B e volte o campo SIM/NÃO trocado. Depois rode **Publicar online agora** no programa.

**Deu certo se:** FiscalEmail volta ao normal.

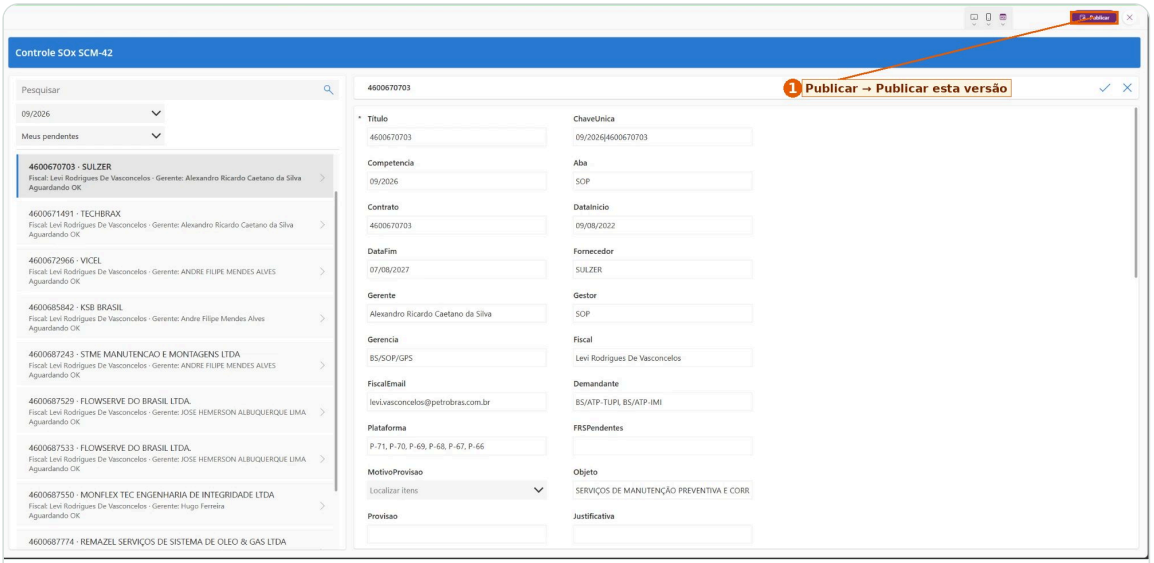
# B12. Publicar e liberar

15 min · B · Montar o app

**Objetivo:** colocar o app no ar para o pessoal.

**1** ☐ STUDIO → TOPO À DIREITA

Clique em **Publicar** → **Publicar esta versão**.

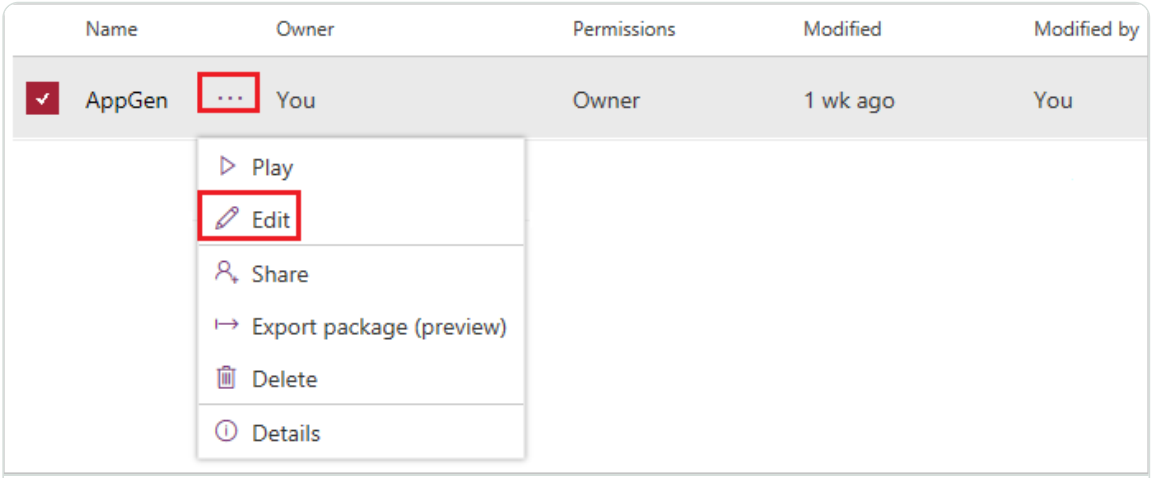



**PRINT DO SEU APP** Com o app em teste (▷), o botão roxo Publicar fica no canto de cima à direita → Publicar esta versão. Clique para ampliar.

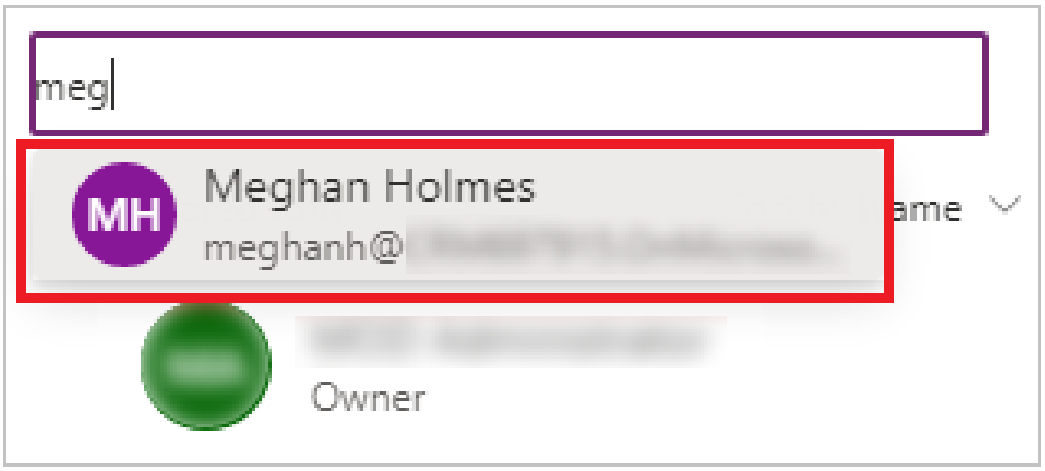
**Deu certo se:** aparece que a versão foi publicada. Repita sempre que mudar o app.

**2** ☐ MAKE.POWERAPPS.COM → APLICATIVOS

Em make.powerapps.com → **Aplicativos**, três pontinhos no Controle SOx SCM-42 → **Compartilhar**. Digite o e-mail de **um colega de teste**, deixe Coproprietário **desmarcado**, **Compartilhar**.




**PRINT REAL · MICROSOFT** Na lista de Aplicativos, três pontinhos do app abrem Editar, Compartilhar e Detalhes.
No seu Power Apps está em português: Edit = Editar · Share = Compartilhar · Details = Detalhes

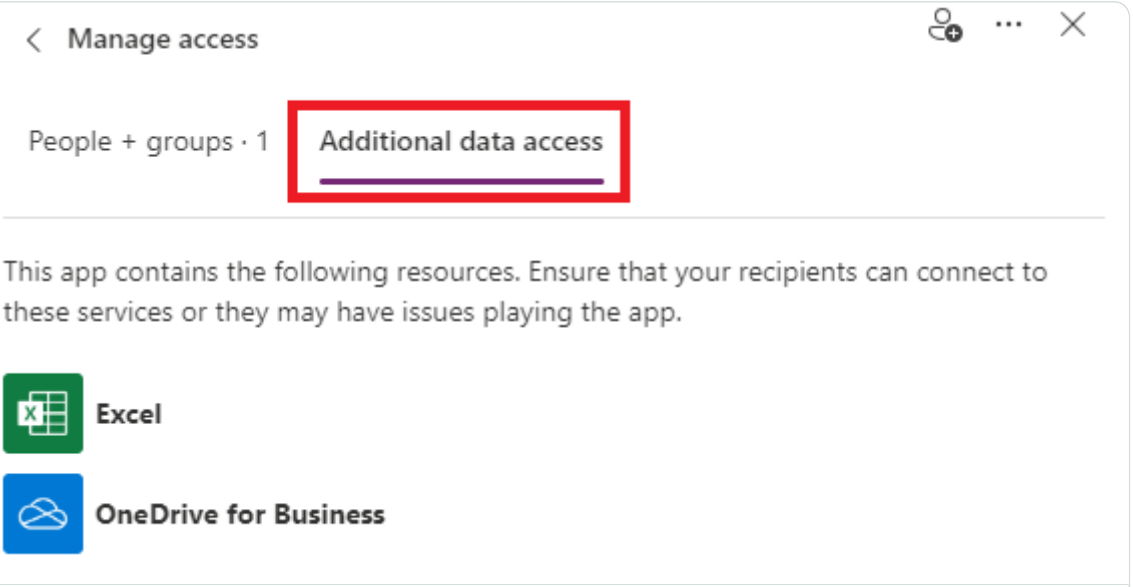



PRINT REAL · MICROSOFT Na janela de compartilhar, digite o nome e escolha a pessoa na lista.

**Deu certo se:** o colega aparece na lista de acesso.

**3** ☐ SITE → ENGRENAGEM ⚙

No site, engrenagem ⚙ → **Permissões do site** → **Adicionar membros**: inclua o colega (ou o grupo da gerência).




PRINT REAL · MICROSOFT Aba de dados do compartilhamento: lembra que a pessoa também precisa de acesso à lista (por isso a etapa no site).

No seu Power Apps está em português: Additional data access = Acesso a dados adicionais

**Deu certo se:** o colega é membro do site.

**4** ☐ APLICATIVOS → DETALHES

Três pontinhos do app → **Detalhes**. Copie o **Link da Web** e mande ao colega.

**Deu certo se:** o colega abre o app, vê todos os contratos e só age nos dele.

**5** ☐ DEPOIS DO TESTE

Compartilhe com todos os fiscais e gerentes (pode ser o grupo de e-mail da gerência). Cole o Link da Web em `url_app` no arquivo `sox_nuvem\sox_nuvem_config.json`.

**Deu certo se:** todos receberam acesso. Guarde o link: ele vai no alerta do Teams e no e-mail mensal.

**Até o dia 15:** o app só deixa cada um mexer no que é seu. Quem abrir a lista direto no SharePoint ainda consegue editar, por isso divulgue só o link do app. **Depois do Fechar competência:** a lista fica trancada de verdade, e só você edita.

↗ Microsoft: compartilhar um app (fotos reais)

# C1. Criar o fluxo, o horário e a janela de aviso

10 min · C · Alertas no Teams

**Objetivo:** um fluxo do Power Automate que roda sozinho todo dia às 8h, na nuvem, e só cobra quando faltam **X dias ou menos** para o fechamento (dia 15). A tela está em inglês: os nomes dos botões estão como aparecem para você.

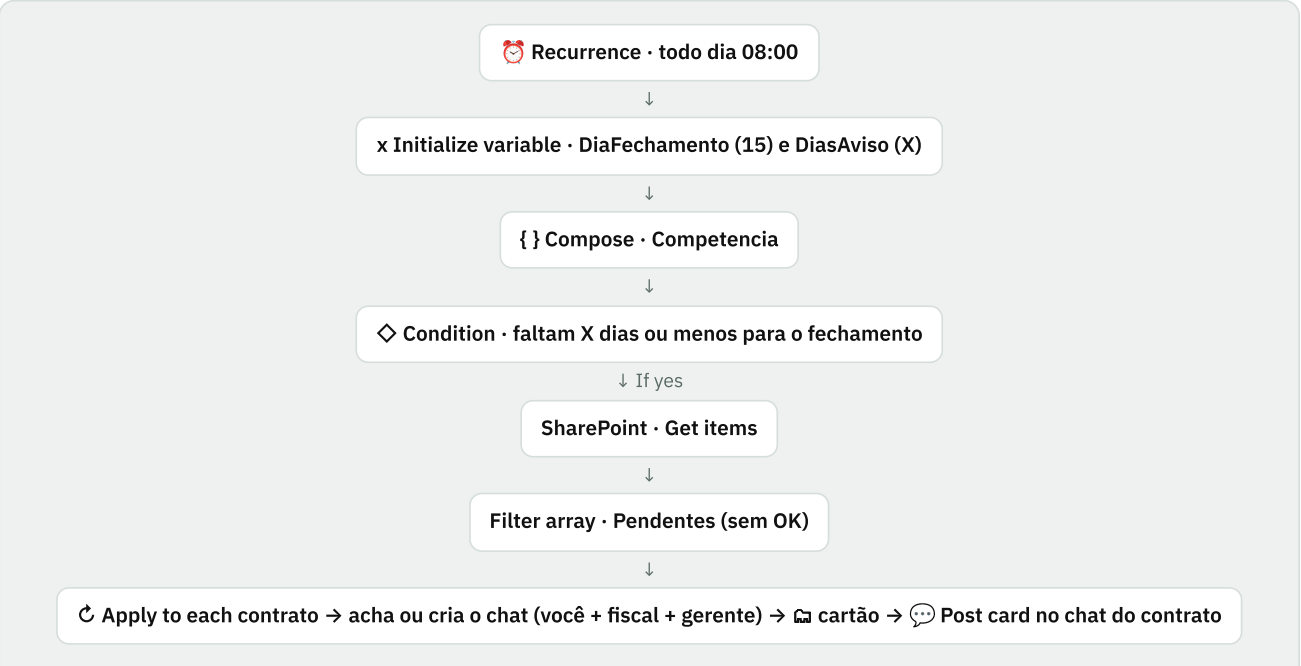



Ilustração do fluxo pronto, de cima para baixo. Cada caixa é uma "ação".

**1** ☐ NAVEGADOR

Abra make.powerautomate.com com a conta da empresa.

**Deu certo se:** aparece a página inicial do Power Automate.

**2** ☐ COLUNA DA ESQUERDA

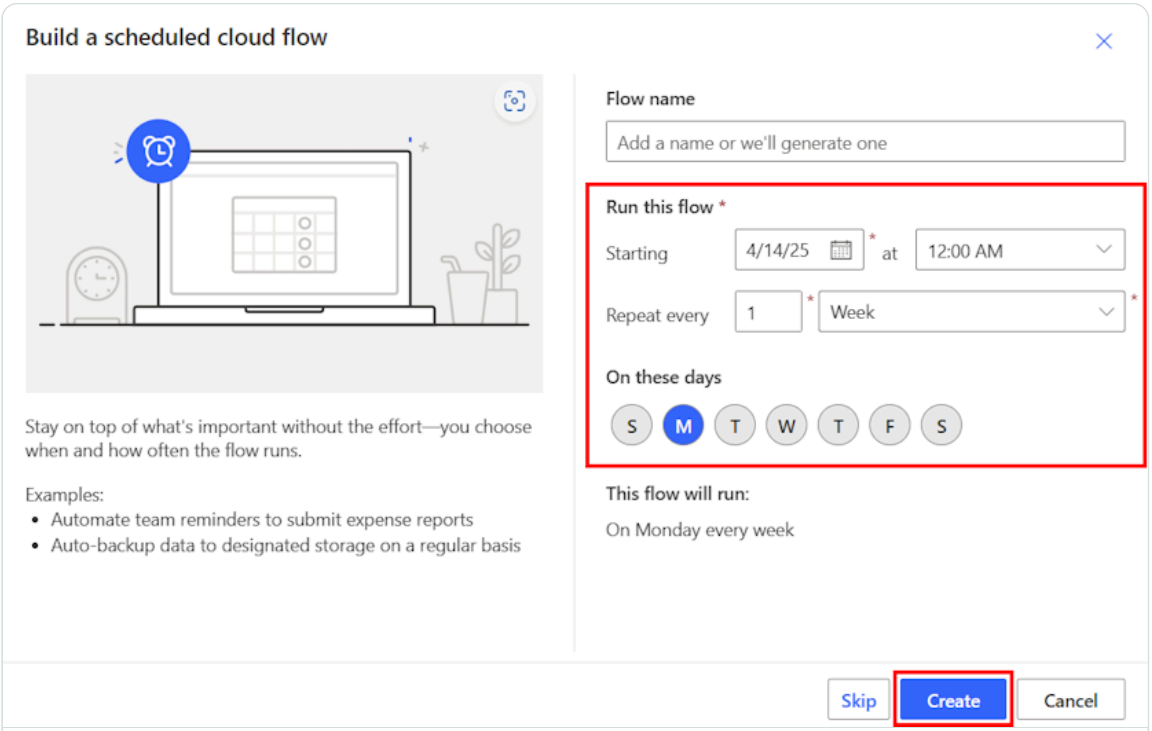
Clique em **+ Create** → **Scheduled cloud flow**.

**Deu certo se:** abre uma janela pedindo nome e horário.

**3** ☐ JANELA

**Flow name**: `SOx SCM-42 Lembretes`. **Starting**: amanhã, **8:00 AM**. **Repeat every**: **1 Day**.

Clique em **Create**.

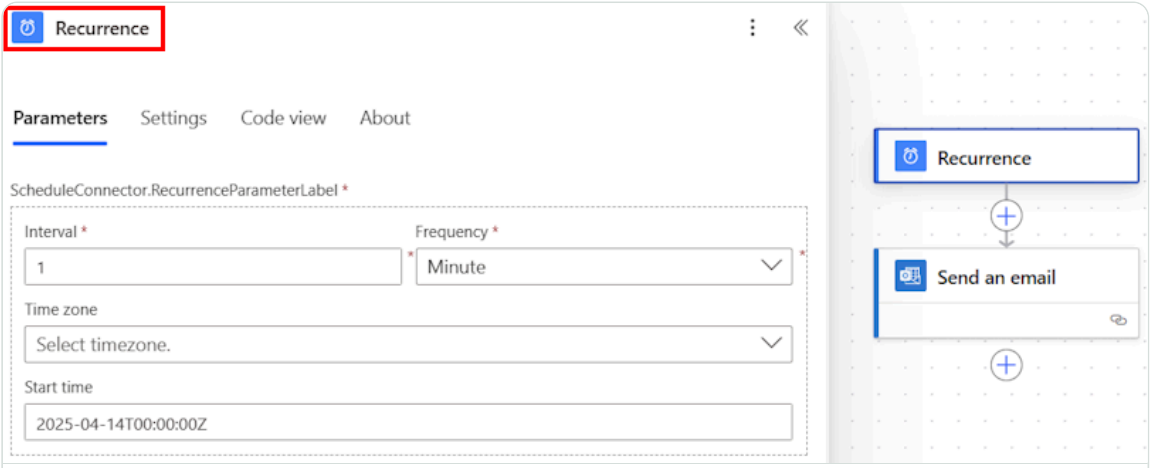



PRINT REAL · MICROSOFT Janela do fluxo agendado: nome, início, repetir a cada e o botão Criar.

**Deu certo se:** abre o editor com a caixa Recurrence.

**4** ☐ CAIXA RECURRENCE

Clique nela → **Show advanced options** . **Time zone** : **(UTC-03:00) Brasilia**. **At these hours** : **8**.

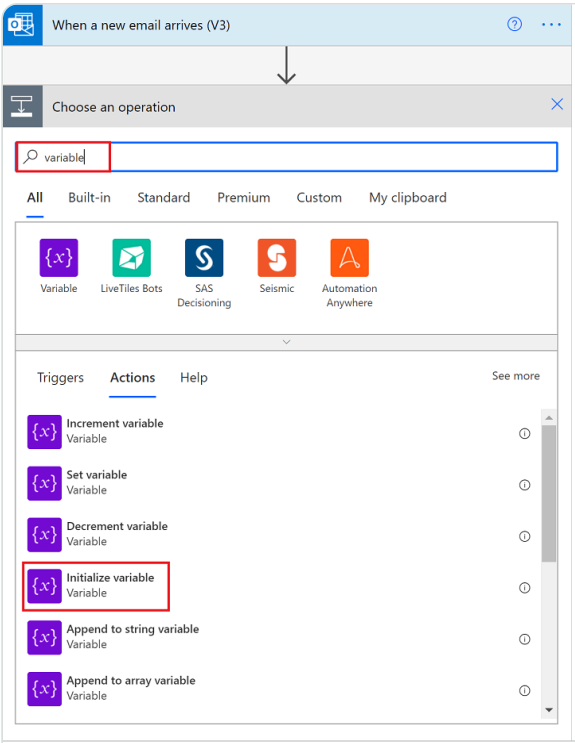



PRINT REAL · MICROSOFT Caixa Recorrência aberta: Intervalo, Frequência, Fuso horário e Hora de início.

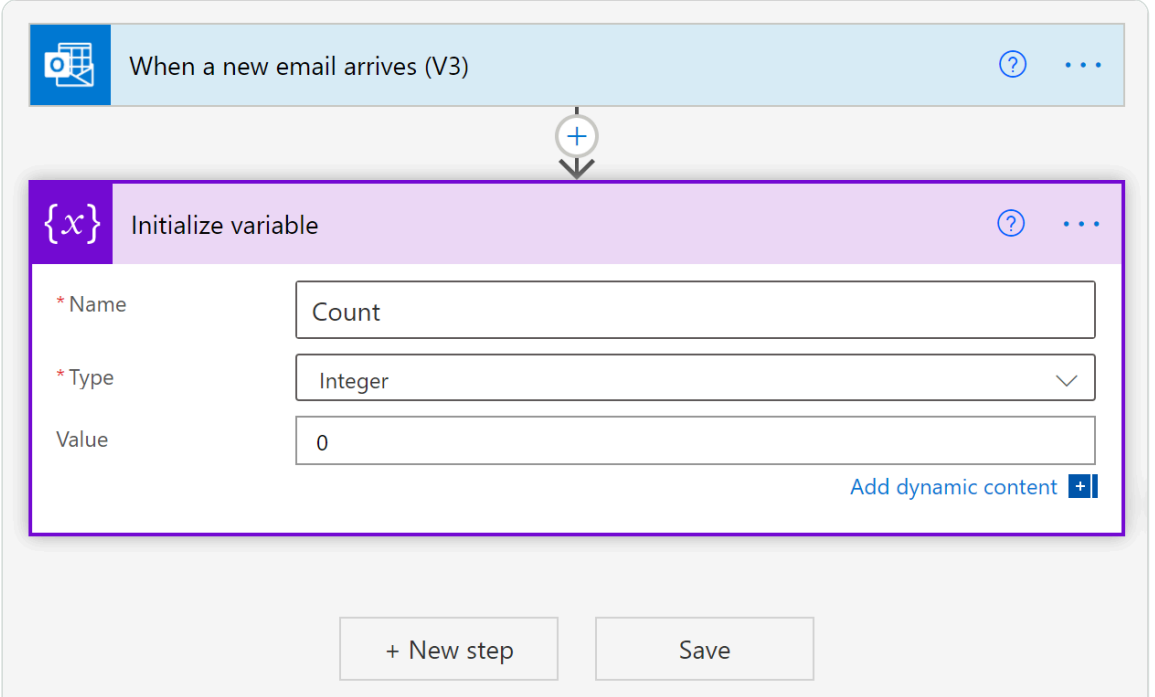
**Deu certo se:** a caixa mostra fuso Brasília e 8h.

**5** ☐ EMBAIXO DA RECURRENCE

**+ New step** → `Initialize variable` . **Name** : `DiaFechamento` . **Type** : **Integer**. **Value** : `15` .




PRINT REAL · MICROSOFT Ao clicar em + aparece a busca de ações. Digite o nome e clique na ação.




PRINT REAL · MICROSOFT Inicializar variável preenchida (no exemplo, Count/Integer/0; no seu, DiaFechamento/Inteiro/15 e DiasAviso/Inteiro/X).

**Deu certo se:** a caixa mostra DiaFechamento = 15 (o dia em que o período fecha).

**6** ☐ EMBAIXO

Outro `Initialize variable`. **Name**: `DiasAviso`. **Type**: **Integer**. **Value**: quantos dias antes do fechamento começar a cobrar, por exemplo `5`.

**Deu certo se:** DiasAviso = o seu X. Com 5, as cobranças vão do dia 10 ao dia 15. É aqui que você muda o X depois.

**7** ☐ EMBAIXO

Outro `Initialize variable`. **Name**: `ChatId`. **Type**: **String**. **Value**: deixe vazio.

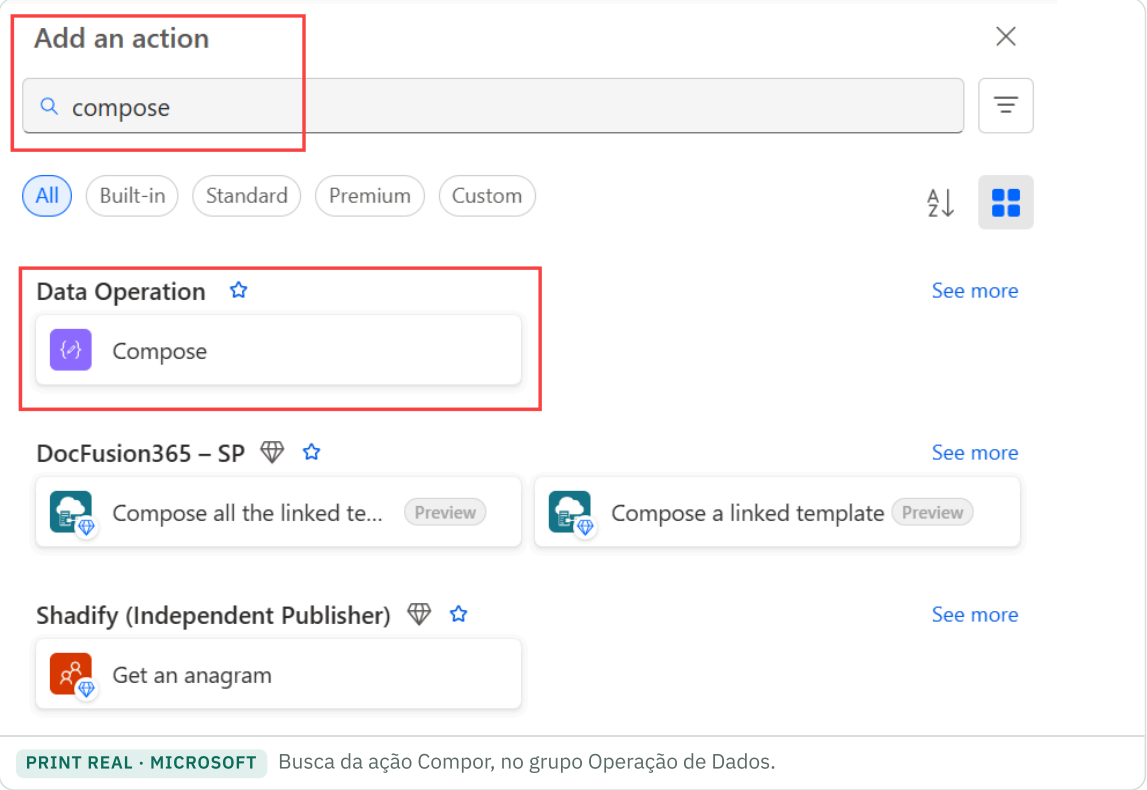
**Deu certo se:** ChatId pronta. Ela guarda, a cada contrato, qual é o chat dele.

**8** ☐ EMBAIXO

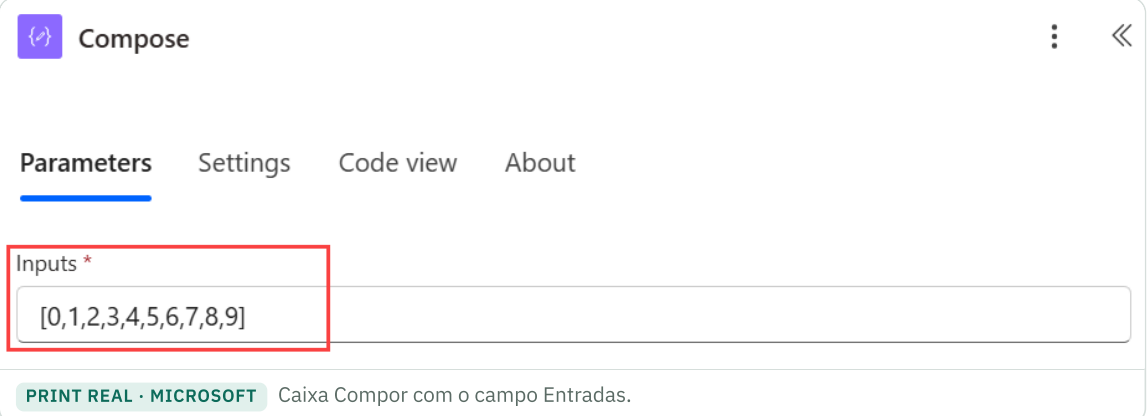
**+ New step** → `Compose`. Renomeie (··· → **Rename**) para `Competencia`. **Inputs**: clique no campo → aba **Expression** → cole → **OK**:

Competencia → Inputs (aba Expression)

```
formatDateTime(addToTime(convertFromUtc(utcNow(), 'E. South America
Standard Time'), -1, 'Month'), 'MM/yyyy')
```




PRINT REAL · MICROSOFT Busca da ação Compor, no grupo Operação de Dados.




PRINT REAL · MICROSOFT Caixa Compor com o campo Entradas.

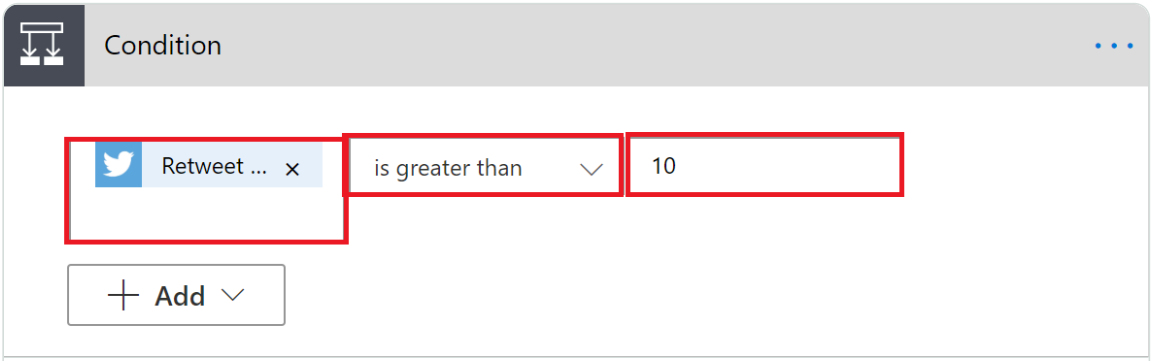
**Deu certo se:** a caixa Competencia mostra um quadrinho roxo.

**9** ☐ EMBAIXO

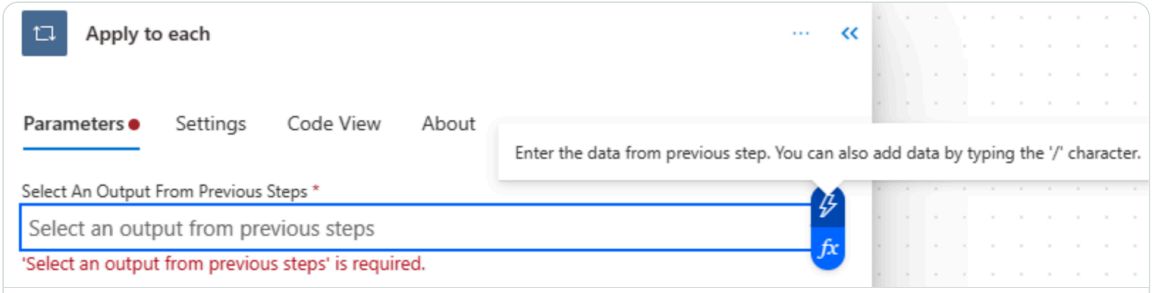
**+ New step** → **Condition**. Primeiro valor: clique no campo → aba **Expression** → cole → **OK** com o quadro. Operador: **is equal to**. Segundo valor: clique no campo → aba **Expression** → cole → **OK** com `true`.

Condition → primeiro valor (aba Expression): hoje está entre o dia 15 – X e o dia 15

```
and(greaterOrEquals(int(formatDateTime(convertFromUtc(utcNow(), 'E. South
America Standard Time'), 'dd')), sub(variables('DiaFechamento'),
variables('DiasAviso'))),
lessOrEquals(int(formatDateTime(convertFromUtc(utcNow(), 'E. South America
Standard Time'), 'dd')), variables('DiaFechamento')))
```




PRINT REAL · MICROSOFT Caixa Condição: primeiro campo, operador no meio, segundo campo.




PRINT REAL · MICROSOFT Ao clicar num campo aparecem dois ícones: ⚡ (dados de passos anteriores) e **fx** (expressão).

**Deu certo se:** a condição tem **If yes** e **If no**. Tudo o que vem a seguir vai dentro do **If yes**. Clique em **Save** .

## C2. Lista que guarda o chat de cada contrato

5 min · C · Alertas no Teams

**Objetivo:** igual ao GPS+ Contratos: cada contrato tem **um** chat no Teams com você, o fiscal e o gerente, que fica sempre aberto. Esta listinha guarda qual chat é de qual contrato, para o fluxo voltar sempre no mesmo chat (inclusive nos meses seguintes).

**1** ☐ SITE CONTROLESOXSCM42

Abra o site → **+ Novo** → **Lista** → **Lista em branco**. Nome: `SOx_SCM42_Chats`. **Criar**.

**Deu certo se:** abre uma lista vazia com a coluna Título.

**2** ☐ LISTA SOX_SCM42_CHATS

**+ Adicionar coluna** → **Uma linha de texto** → nome `ChatId` → **Salvar**.

**Deu certo se:** a lista tem as colunas Título (vai receber o número do contrato) e ChatId. Não precisa preencher nada: o fluxo preenche.

## C3. Buscar os contratos sem resposta

10 min · C · Alertas no Teams

**Objetivo:** pegar da lista os contratos do mês em que nem o fiscal nem o gerente responderam ainda.

**1** ☐ DENTRO DO IF YES → ADD AN ACTION

Busque `Get items` → escolha a do **SharePoint**. Renomeie `ItensDoMes`. **Site Address**: **ControleSOxSCM42**. **List Name**: **SOx_SCM42_Contratos**.

**Deu certo se:** a caixa mostra o site e a lista.

**2** ☐ CAIXA ITENSDOMES

Clique em **Show advanced options**. Em **Filter Query** digite `Competencia eq '`, depois aba **Dynamic content** → **Outputs** (de Competencia), e termine digitando `' and MesFechado ne 'SIM'`. Em **Top Count** coloque `5000`.

**Deu certo se:** a consulta fica: Competencia eq '[Outputs]' and MesFechado ne 'SIM'.

**3** ☐ ITENSDOMES → ···

··· → **Settings** → **Pagination** **On**, **Threshold** `5000` → **Done**.

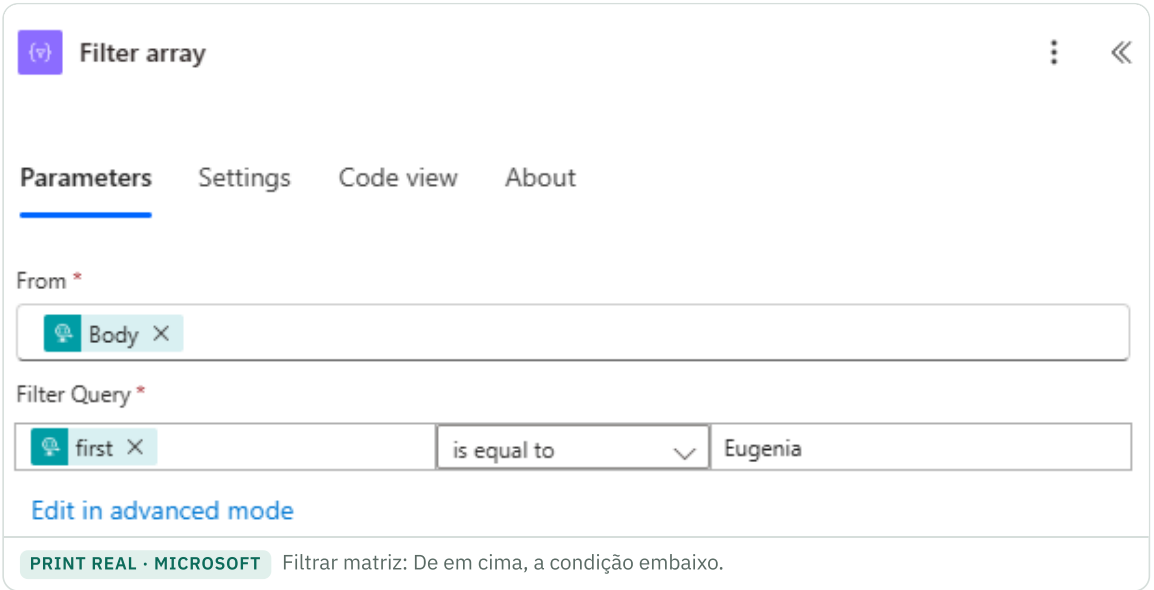
**Deu certo se:** paginação ligada.

**4** ☐ EMBAIXO → ADD AN ACTION

`Filter array`. Renomeie `Pendentes`. **From**: aba **Dynamic content** → **value** (de ItensDoMes). Condição: primeiro valor clique no campo → aba **Expression** → cole → **OK** com o quadro; **is equal to**; segundo valor clique no campo → aba **Expression** → cole → **OK** com `true`.

Pendentes → primeiro valor (aba Expression)

```
empty(item()?['OkFiscal'])
```




PRINT REAL · MICROSOFT Filtrar matriz: De em cima, a condição embaixo.

**Deu certo se:** caixa Pendentes pronta. Ela fica só com os contratos sem OK nem não OK.

5 ☐ TOPO

Save .

**Deu certo se:** salvo sem erro.

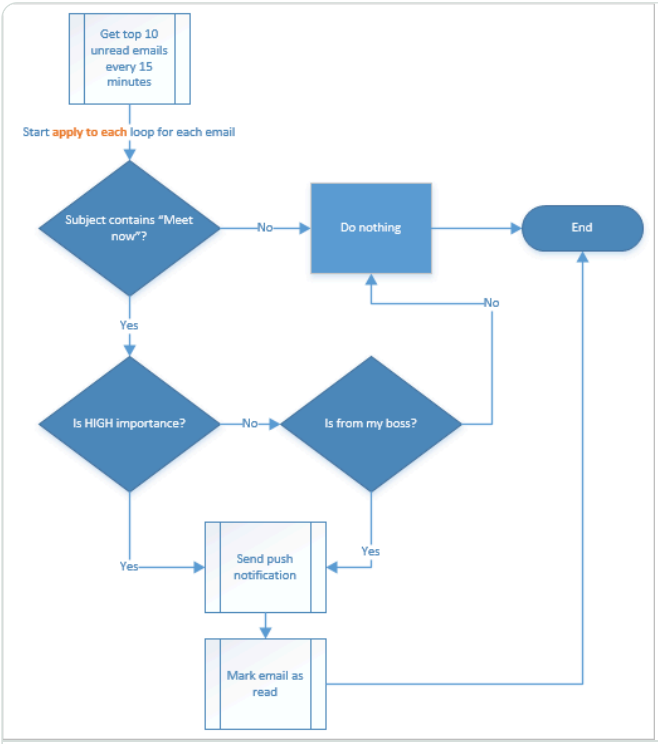
# C4. Achar ou criar o chat do contrato

20 min · C · Alertas no Teams

**Objetivo:** para cada contrato pendente: se ele já tem chat, usa o mesmo; se não, cria o chat **SOx SCM-42 - Fornecedor - Contrato** com você, o fiscal e o gerente e anota na lista SOx_SCM42_Chats.

**1** ☐ EMBAIXO DE PENDENTES → ADD AN ACTION

`Apply to each`. Renomeie `PorContrato`. **Select an output from previous steps**: clique no campo → aba **Expression** → cole → **OK** com `body('Pendentes')`. Deixe **Concurrency Control** **desligado** (um contrato por vez).




PRINT REAL · MICROSOFT Desenho do que o Aplicar a cada faz: repete as ações de dentro uma vez para cada item (aqui, cada pessoa).

**Deu certo se:** caixa PorContrato pronta, vazia por dentro.

**2** ☐ DENTRO DE PORCONTRATO → ADD AN ACTION

`Get items` (SharePoint). Renomeie `ChatDoContrato`. Site **ControleSOxSCM42**, lista **SOx_SCM42_Chats**. **Show advanced options** → **Filter Query**: clique no campo → aba **Expression** → cole → **OK** com o quadro. **Top Count**: `1`.

ChatDoContrato → Filter Query (aba Expression)

```
concat('Title eq ''', items('PorContrato')?['Contrato'], '''')
```

**Deu certo se:** pronta.

**3** ☐ EMBAIXO

**Condition**. Primeiro valor clique no campo → aba **Expression** → cole → **OK** com o quadro; **is equal to**; segundo clique no campo → aba **Expression** → cole → **OK** com `true`.

Condition → primeiro valor (aba Expression): o contrato ainda não tem chat

```
empty(body('ChatDoContrato')?['value'])
```

**Deu certo se:** a condição tem If yes (criar) e If no (já existe).

**4** ☐ IF YES → ADD AN ACTION

`Compose` . Renomeie `Membros` . **Inputs** : clique no campo → aba **Expression** → cole → **OK** :

Membros → Inputs (aba Expression): fiscal e gerente, sem repetir

```
union(split(replace(toLower(concat(coalesce(items('PorContrato')?
['FiscalEmail'], ''), ';', coalesce(items('PorContrato')?['GerenteEmail'],
'')))), ' ', ''), ';'), createArray())
```

**Deu certo se:** pronta.

**5** ☐ EMBAIXO (IF YES)

`Filter array` . Renomeie `MembrosOk` . **From** : clique no campo → aba **Expression** → cole → **OK** com `outputs('Membros')` . Clique em **Edit in advanced mode** e cole:

MembrosOk → modo avançado (colar direto): tira vazio e tira você, que entra sozinho como criador

```
@and(not(empty(item())), not(equals(item(),
'levi.vasconcelos@petrobras.com.br')))
```

**Deu certo se:** pronta.

▼ **Não deu certo?**

Se o seu e-mail não for levi.vasconcelos@petrobras.com.br, troque no texto.

**6** ☐ EMBAIXO (IF YES)

Busque `Create a chat` (Microsoft Teams). Renomeie `CriarChat` . **Members to add** : clique no campo → aba **Expression** → cole → **OK** com o quadro 1. **Title** : clique no campo → aba **Expression** → cole → **OK** com o quadro 2.

1 · CriarChat → Members to add (aba Expression)

```
join(body('MembrosOk'), ';')
```

2 · CriarChat → Title (aba Expression)

```
concat('SOx SCM-42 - ', coalesce(items('PorContrato')?['Fornecedor'], ''),
' - ', items('PorContrato')?['Contrato'])
```

**Deu certo se:** pronta.

▼ **Não deu certo?**

Se o fiscal e o gerente forem a mesma pessoa, o chat fica só com vocês dois: tudo bem.

**7** ☐ EMBAIXO (IF YES)

`Create item` (SharePoint). Site **ControleSOxSCM42**, lista **SOx_SCM42_Chats**. **Title** : clique no campo → aba **Expression** → cole → **OK** com o quadro 1. **ChatId** : clique no campo → aba **Expression** → cole → **OK** com o quadro 2.

1 · Create item → Title (aba Expression)

```
items('PorContrato')?['Contrato']
```

2 · Create item → ChatId (aba Expression)

```
body('CriarChat')?['id']
```

**Deu certo se:** pronta. Assim o chat fica anotado para os próximos dias e meses.

**8** ☐ EMBAIXO (IF YES)

`Set variable` . **Name** : **ChatId**. **Value** : clique no campo → aba **Expression** → cole → **OK** com `body('CriarChat')?['id']` .

**Deu certo se:** pronta.

**9** ☐ IF NO → ADD AN ACTION

`Set variable` . **Name** : **ChatId**. **Value** : clique no campo → aba **Expression** → cole → **OK** :

Set variable (If no) → Value (aba Expression): o chat que já existe

```
first(body('ChatDoContrato')?['value'])?['ChatId']
```

**Deu certo se:** pronta. **Save** .

# C5. O cartão no chat do contrato

15 min · C · Alertas no Teams

**Objetivo:** postar no chat do contrato um cartão igual ao do GPS+: o que está pendente, os dados do contrato, o que falta e o botão para abrir o app.

**É assim que aparece no chat do contrato** (prévia com dados de exemplo), no mesmo jeito do GPS+ Contratos:

LV **Levi Rodrigues De Vasconcelos** 08:00

⚠️ **Pendente: validação SOx SCM-42 · 09/2026**

Os dados da medição deste contrato ainda **não foram confirmados**. Basta o fiscal **ou** o gerente responder no app até o dia 15 (faltam 4 dia(s)).

**Contrato** 4600685842 · **Empresa** KSB BRASIL
**Situação** Fora do prazo · **Fiscal · Gerente** Levi R. de Vasconcelos · Andre Filipe Mendes Alves

**O que fazer**
Informar CSP das FRS fora do prazo; Preencher justificativa

Acesse o app **Controle SOx SCM-42** para responder.

Abrir no app

O chat se chama **SOx SCM-42 - KSB BRASIL - 4600685842** e tem você, o fiscal e o gerente.
É criado na primeira cobrança e reaproveitado sempre.

1 ☐ DENTRO DE PORCONTRATO, EMBAIXO DA CONDITION (FORA DO IF YES/IF NO)

`Compose` . Renomeie `Cartao` . Em **Inputs** **cole o texto abaixo direto** (sem aba Expression). Depois troque `COLE_AQUI_O_LINK_DO_APP` pelo Link da Web do app (B12).

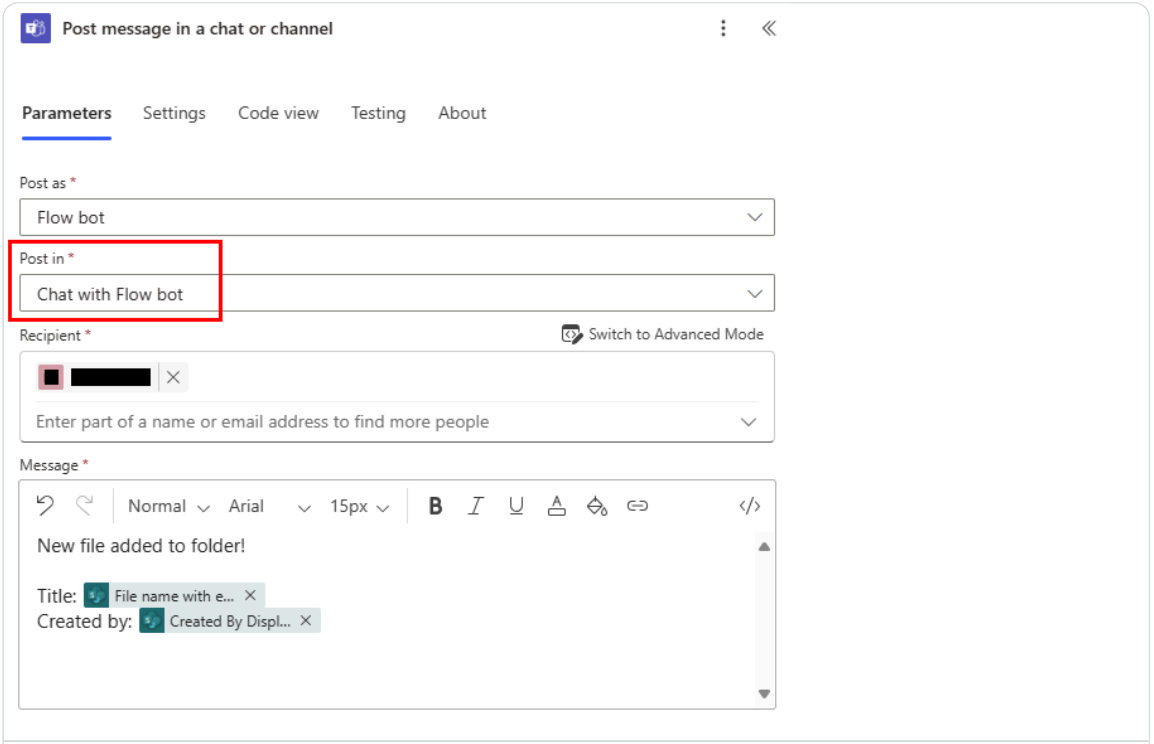
Cartao → Inputs (o cartão inteiro, colar direto)

```
{
  "type": "AdaptiveCard",
  "$schema": "http://adaptivecards.io/schemas/adaptive-card.json",
  "version": "1.4",
  "msteams": { "width": "Full" },
  "body": [
    {
      "type": "Container",
      "style": "emphasis",
      "bleed": true,
      "items": [
        { "type": "TextBlock", "text": "⚠️ Pendente: validação SOx SCM-42
· @{outputs('Competencia')}", "size": "Medium", "weight": "Bolder",
"color": "Attention", "wrap": true }
      ]
    },
    { "type": "TextBlock", "text": "Os dados da medição deste contrato
ainda **não foram confirmados**. Basta o fiscal **ou** o gerente responder
no app até o dia @{variables('DiaFechamento')} (faltam
@{sub(variables('DiaFechamento'),
int(formatDateTime(convertFromUtc(utcNow(), 'E. South America Standard
Time'), 'dd')))} dia(s)).", "wrap": true },
    {
      "type": "Container",
      "style": "emphasis",
      "items": [
        {
          "type": "ColumnSet",
          "columns": [
            { "type": "Column", "width": "stretch", "items": [
              { "type": "TextBlock", "text": "Contrato", "weight":
"Bolder" },
              { "type": "TextBlock", "text": "@{items('PorContrato')?
['Contrato']}", "spacing": "None" },
              { "type": "TextBlock", "text": "Empresa", "weight": "Bolder"
},
              { "type": "TextBlock", "text":
"@{coalesce(items('PorContrato')?['Fornecedor'], '')}", "spacing": "None",
"wrap": true }
            ] },
            { "type": "Column", "width": "stretch", "items": [
              { "type": "TextBlock", "text": "Situação", "weight":
"Bolder" },
              { "type": "TextBlock", "text":
"@{coalesce(items('PorContrato')?['Situacao'], '')}", "spacing": "None" },
              { "type": "TextBlock", "text": "Fiscal · Gerente", "weight":
"Bolder" },
              { "type": "TextBlock", "text":
"@{coalesce(items('PorContrato')?['Fiscal'], '')} ·
@{coalesce(items('PorContrato')?['Gerente'], '')}", "spacing": "None",
"wrap": true }
            ] }
          ]
        }
      ]
    },
    {
      "type": "Container",
      "style": "emphasis",
      "items": [
        { "type": "TextBlock", "text": "O que fazer", "weight": "Bolder"
```

```
},
        { "type": "TextBlock", "text": "@{coalesce(items('PorContrato')?
['OQueFazer'], 'Conferir os dados e clicar em Dados OK (ou Dados não OK,
se algo estiver errado).')}", "color": "@{if(empty(items('PorContrato')?
['OQueFazer'])), 'Default', 'Attention')}", "wrap": true }
      ]
    },
    { "type": "TextBlock", "text": "Acesse o app **Controle SOx SCM-42**
para responder.", "wrap": true }
  ],
  "actions": [
    { "type": "Action.OpenUrl", "title": "Abrir no app", "url":
"COLE_AQUI_O_LINK_DO_APP" }
  ]
}
```

2 ☐

**Dynamic content** → **Outputs** de **Cartao**.




PRINT REAL · MICROSOFT Ação do Teams com Postar como = Bot de fluxo e Postar em = Chat com o bot de Fluxo. Na sua, escolha a versão **Postar cartão**.

**Deu certo se:** pronta. **Save**.

▼ **Não deu certo?**

- **Não acho:** a antiga *Post adaptive card in a chat or channel* também serve.
- **Prefere que saia como bot:** Post as = Flow bot também posta em Group chat.

# C6. Testar com um contrato e ligar

15 min · C · Alertas no Teams

**Objetivo:** ver o chat e o cartão funcionando num contrato só, antes de soltar para todos.

**1** ☐ LISTA SOX_SCM42_CONTRATOS

Escolha um contrato do mês ainda sem resposta. No modo de grade, ponha em GerenteEmail o e-mail de **um colega que topou testar** e em FiscalEmail o seu. Anote o número.

**Deu certo se:** o contrato tem você e o colega.

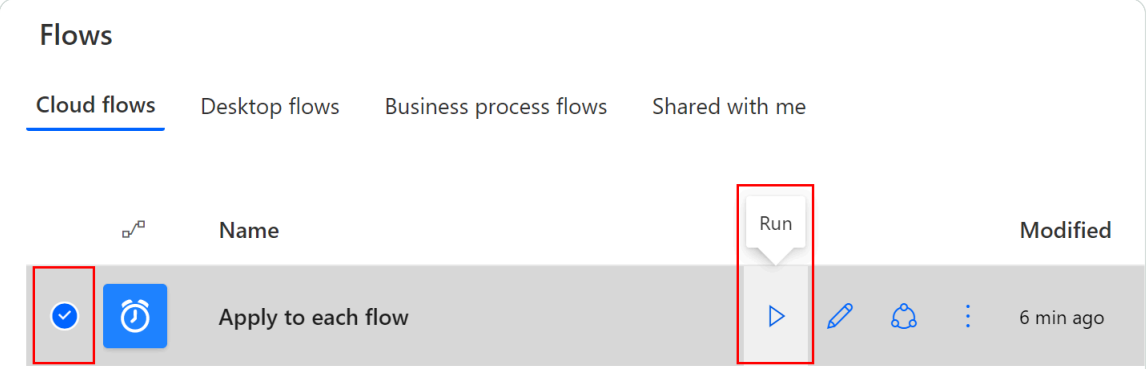
**2** ☐ FLUXO → ITENSDOMES

No fim da **Filter Query**, acrescente `and Title eq '` número `'`. Em **DiaFechamento** e **DiasAviso** coloque `31` (assim o teste vale em qualquer dia).

**Deu certo se:** o fluxo só vai olhar esse contrato.

**3** ☐ TOPO

**Save** → **Test** → **Manually** → **Test** → **Run flow** → **Done**.




PRINT REAL · MICROSOFT Também dá para rodar pela lista Meus fluxos: botão ▷ Executar na linha do fluxo.

**Deu certo se:** no Teams aparece o chat **SOx SCM-42 - Fornecedor - número** com você e o colega, e nele o cartão. Na lista SOx_SCM42_Chats aparece uma linha com o número e o ChatId. Rodando de novo, o cartão cai no **mesmo** chat.

▼ **Não deu certo?**

- **Caixa com X vermelho:** clique nela, leia o erro e mande print ao Claude.
- **"o cartão não pôde ser exibido":** algum pedaço do texto foi cortado ao colar. Copie de novo com o botão Copiar.
- **Nada aconteceu:** o contrato já tem resposta (OkFiscal preenchido) ou a Competencia não bate (ela usa o mês anterior).
- **Teams bloqueia:** peça à TI para liberar o conector do Teams no Power Automate.

**4** ☐ VOLTAR AO NORMAL

Tire o `and Title eq '...'` da Filter Query. DiaFechamento = `15`, DiasAviso = o seu X. Devolva o GerenteEmail e o FiscalEmail do contrato de teste (ou rode Publicar online agora). **Save**.

**Deu certo se:** o fluxo roda sozinho todo dia às 8h e só cobra do dia 15 − X ao dia 15.

**Como fica:** cada contrato pendente tem seu chat **SOx SCM-42 - Fornecedor - Contrato** com você, o fiscal e o gerente, sempre o mesmo. Nos últimos X dias antes do fechamento, às 8h, cai nele o cartão de pendência.

Quando o fiscal **ou** o gerente responde no app, o contrato para de ser cobrado. O chat continua lá para conversar sobre o contrato.

# D1. Dia 1: gerar, publicar e mandar o e-mail

20 min · D · Todo mês

**Objetivo:** abrir o mês: lista publicada e todo mundo avisado por e-mail.

1 ☐ WINDOWS

Abra o SAP (logado) e o Outlook clássico.

**Deu certo se:** os dois abertos.

2 ☐ SEU PROGRAMA

Escolha a competência (mês anterior) e clique em **Gerar planilha do mês**. Espere o painel chegar a 100%.

**Deu certo se:** mensagem final com o resumo e "Online: N novos". A lista e o app já mostram o mês novo.

3 ☐ ESTA PÁGINA

Preencha os campos abaixo (competência, prazo, fechamento, link do app). O e-mail se monta sozinho.

**Deu certo se:** assunto e corpo com as suas datas.

4 ☐ OUTLOOK

**Novo Email**. Em **Para**: você mesmo. Em **Cc**: todos os fiscais e gerentes (ou o grupo da gerência; a aba **Equipe** da planilha tem os nomes). Copie o assunto e o corpo daqui e cole.

**Deu certo se:** e-mail pronto com todos em cópia.

5 ☐ OUTLOOK

Confira as datas e o link e clique em **Enviar**.

**Deu certo se:** e-mail enviado. Os lembretes do Teams (parte C) começam sozinhos X dias antes do fechamento, às 8h.

**Competência**: 09/2026

**Prazo (data)**: 10/10/2026

**Fechamento**: 15/10/2026

**Link do app**: COLE_AQUI_O_LINK_DO_APP

**Assinatura**: Levi Rodrigues de Vasconcelos · Controle SOx SCM-42

Assunto

```
[SOx SCM-42] Conferência das medições de 09/2026 · responder até 10/10/2026
```

Corpo do e-mail

```
Prezados(as) fiscais e gerentes,

Já está disponível no app Controle SOx SCM-42 a conferência das medições da
competência 09/2026. Pedimos que cada um confira os contratos em que é fiscal ou
gerente até 10/10/2026.

Como fazer (leva poucos minutos por contrato):
1. Abra o app: COLE_AQUI_O_LINK_DO_APP
2. No filtro "Meus pendentes" aparecem só os contratos que esperam por você.
3. Abra cada contrato e confira as informações.
   • Tudo certo: clique em "Dados OK". Nenhum dado é alterado; fica registrado que
você conferiu.
   • Quando o contrato pedir (aviso em vermelho), preencha antes o que só você sabe:
       - Fora do prazo: o CSP de cada FRS, uma por linha (ex.: 1020291116: CON001), ou
todas: CON001
       - Provisionado: a justificativa e, nos contratos da SOP, o motivo da provisão.
     O botão Dados OK só registra o OK com o obrigatório preenchido.
   • Algo errado: clique em "Dados não OK", corrija o campo (Houve serviço, Foi
medido, Fora do prazo, Será substituído ou Demanda continuada), escreva o que estava
errado e clique em Salvar não OK. Use também o campo Observação Mês para qualquer
comentário sobre a medição.
4. Basta o fiscal OU o gerente responder: quando um responde, o contrato sai da lista
dos dois.

Prazos:
   • Até 10/10/2026: conferência. Nos últimos dias antes do fechamento, quem tiver
contrato sem resposta recebe lembrete diário no Teams.
   • 15/10/2026: o período é fechado e o app passa a ficar disponível só para
consulta.

Este controle atende ao requisito SOx SCM-42 (controle de medição). A resposta de cada
um fica registrada com nome, data e hora e compõe a evidência do mês.

Em caso de dúvida, falar comigo.

Obrigado,
Levi Rodrigues de Vasconcelos · Controle SOx SCM-42
```

## D2. Do dia 2 ao dia X: acompanhar

5 min por semana · D · Todo mês

**Objetivo:** ver quem já respondeu e corrigir os Não OK. Nos últimos X dias o Teams cobra sozinho quem não respondeu.

1 ☐ APP

Abra o app, filtro **Todos**. A última linha de cada contrato diz Aguardando OK, ✔ OK por... ou ✖ Não OK por.... O filtro **Não OK** mostra só os que você precisa corrigir (campo, valor certo e comentário aparecem embaixo do formulário).

**Deu certo se:** você vê o andamento.

2 ☐ SEU PROGRAMA

Para ter em planilha: **Puxar respostas do pessoal** (feche a planilha do mês no Excel antes). O programa pede a lista ao fluxo SOx Ponte e espera a resposta, normalmente de 1 a 5 minutos.

**Deu certo se:** abre a pasta com o controle_online do mês.

▼ **Não deu certo?**

Se passar de 20 minutos aparece um aviso: confira se o OneDrive está sincronizando e o histórico do fluxo (A7, passo 6).

3 ☐ SE PRECISAR

Republicar ( **Publicar online agora** ) no meio do mês pode: o que o pessoal mudou no app é mantido.

**Deu certo se:** —

# D3. Dia 16: fechar e guardar

15 min · D · Todo mês

**Objetivo:** trancar o mês (só você edita) e guardar a cópia de controle para o SOx.

**1** ☐ AUTOMÁTICO

À meia-noite do dia 15 o app trava sozinho: todos só visualizam, menos você.

**Deu certo se:** —

**2** ☐ EXCEL

Feche a planilha do mês, se estiver aberta.

**Deu certo se:** Excel fechado.

**3** ☐ SEU PROGRAMA

Escolha a competência e clique em **Fechar competência (dia X)**. Confirme. O fluxo SOx Ponte tranca os 230 contratos; isso leva de 5 a 15 minutos, deixe o programa aberto.

**Deu certo se:** a janela final mostra quantos contratos foram trancados e a pasta `controles\fechamento_AAAA_MM`. Ele marcou o mês, trancou a lista, puxou as respostas para a planilha e guardou a cópia somente leitura com o manifesto SHA-256.

**4** ☐ EXPLORADOR DE ARQUIVOS

Leve a pasta `fechamento_AAAA_MM` inteira para o repositório de controle (biblioteca do SharePoint com retenção).

**Deu certo se:** a evidência do mês está guardada.

**5** ☐ SÓ SE PRECISAR CORRIGIR

Você (admin) ainda edita no app e na lista. Depois de corrigir, rode **Fechar competência** de novo para refazer a cópia. **Reabrir competência** destranca para todos (o app continua travado pela data depois do dia 15).

**Deu certo se:** —

# Referência: o que muda no código

Consulta · À mão só se a A2 der PROBLEMA

**Objetivo:** mostrar cada uma das 28 mudanças que o script da A2 faz: onde fica, por quê, como estava e como fica. Se precisar fazer à mão (etapa A2b), use em cada uma: **1** copie o **Procure** e ache com `Ctrl` + `F`; **2** selecione o trecho igual ao **Antes**; **3** cole o **Depois** por cima.

### Como a planilha e o app se alimentam, mês a mês

| Quando | Planilha (Python) | Lista e app |
|---|---|---|
| Dia 1: Gerar | A planilha nasce com as fórmulas de sempre. A memória do banco preenche Será substituído, Demanda, Justificativa, CSP e Motivo. | Publicar envia uma linha por contrato. Fora do Prazo e Provisão vão com o **resultado** das fórmulas. |
| Dias 1 a 15 | Nada muda. | O pessoal muda só os 8 campos destravados. Com FRS pendente, Fora do Prazo é recalculado pelo CSP. |
| Puxar ou Fechar | Os valores voltam para a planilha. Fora do Prazo `="SIM"` / `="NÃO"` recebe o valor do app; a fórmula do CSP fica e se recalcula. Situação e Resumos recalculam ao abrir. Células mudadas ficam vermelhas (regra Origem). | Respostas vão também para o banco (memória) e para o controle do mês, com OK e aprovação. |
| Fechar (dia 16) | Cópia somente leitura com SHA-256 em controles\fechamento_AAAA_MM. | Cada contrato do mês fica só leitura no SharePoint, menos para admin_emails. |
| Mês seguinte | A memória entra na planilha nova. | Linhas novas, competência nova. |

**Ponto de atenção para o SOx (não mudei):** a memória traz a Justificativa e o CSP do mês anterior já preenchidos. O mais seguro é trazer só Será substituído e Demanda continuada. Peça ao Claude se quiser esse ajuste.

---

### 1. Corrige o import do replace (leitura da ME33K)

**Onde:** Início do arquivo, nos imports.
**Por quê:** A função enriquecer_documentos usa replace(), que não estava importado. Todo contrato com pedido fora do RM caía em erro na ME33K.
**O que muda:** Troca o trecho Antes pelo Depois.

1 · Procure (Ctrl + F)

```
from dataclasses import asdict, dataclass, field
```

2 · Antes (selecione isto no seu arquivo)

```
from dataclasses import asdict, dataclass, field
```

3 · Depois (cole por cima)

```
from dataclasses import asdict, dataclass, field, replace
```

### 2. Coluna GerenteEmail na publicação

**Onde:** Lista CABECALHO_STAGING_NUVEM.
**Por quê:** O app precisa do e-mail do gerente para saber quem pode aprovar.
**O que muda:** Troca o trecho Antes pelo Depois.

1 · Procure (Ctrl + F)

```
"FiscalEmail",
```

2 · Antes (selecione isto no seu arquivo)

```
    "FiscalEmail",
    "Demandante",
```

3 · Depois (cole por cima)

```
    "FiscalEmail",
    "GerenteEmail",
    "Demandante",
```

### 3. Função que monta ;email; de fiscal e gerente

**Onde:** Nova função, logo antes de coletar_linhas_nuvem.
**Por quê:** Transforma o nome do fiscal ou gerente (um ou vários, separados por vírgula) em ;email1;email2;. O ; nas pontas evita que ana@ seja achada dentro de joana@. Usa o cadastro do banco e, se faltar, o Outlook, como o programa já faz.
**O que muda:** Acrescenta código novo logo acima deste trecho (o trecho continua igual).

1 · Procure (Ctrl + F)

```
def coletar_linhas_nuvem(
```

2 · Antes (selecione isto no seu arquivo)

```
def coletar_linhas_nuvem(
```

3 · Depois (cole por cima)

```
def _emails_lista(nomes: str, cache: dict[str, str]) -> str:
    """Nomes separados por vírgula viram ;email1;email2; para o app comparar com o
login."""
    achados: list[str] = []
    for nome in re.split(r"[,;/]", str(nomes or "")):
        nome = nome.strip()
        if not nome:
            continue
        email = cache.get(nome.lower()) or resolver_email_fiscal(nome)
        email = str(email or "").strip().lower()
        if "@" in email and email not in achados:
            achados.append(email)
    return ";" + ";".join(achados) + ";" if achados else ""


def coletar_linhas_nuvem(
```

### 4. FiscalEmail e GerenteEmail no formato ;email;

**Onde:** Função coletar_linhas_nuvem, onde monta cada linha da lista.
**Por quê:** Antes só havia um e-mail de fiscal, e só quando o nome inteiro estava no cadastro.
**O que muda:** Troca o trecho Antes pelo Depois.

1 · Procure (Ctrl + F)

```
"FiscalEmail": emails.get(fiscal.strip().lower(), ""),
```

2 · Antes (selecione isto no seu arquivo)

```
                    "FiscalEmail": emails.get(fiscal.strip().lower(), ""),
```

3 · Depois (cole por cima)

```
                    "FiscalEmail": _emails_lista(fiscal, emails),
                    "GerenteEmail": _emails_lista(_celula_texto(ws, mapa, linha,
"gerente"), emails),
```

---

### 5. Provisão das linhas com fórmula vai preenchida para a lista

**Onde:** Função coletar_linhas_nuvem, campo Provisao.
**Por quê:** Quando a coluna Provisão é fórmula (contratos com CSP), o texto ia vazio. Agora usa a coluna escondida Texto provisão, que guarda o mesmo texto sem fórmula. O app usa a Provisão para saber quando a Justificativa pode ser editada.
**O que muda:** Troca o trecho Antes pelo Depois.

1 · Procure (Ctrl + F)

```
"Provisao": _celula_texto(ws, mapa, linha, "provisao", "texto provisao"),
```

2 · Antes (selecione isto no seu arquivo)

```
                    "Provisao": _celula_texto(ws, mapa, linha, "provisao", "texto
provisao"),
```

3 · Depois (cole por cima)

```
                    "Provisao": _celula_texto(ws, mapa, linha, "provisao")
                    or _celula_texto(ws, mapa, linha, "texto provisao"),
```

---

### 6. Fora do Prazo lido certo das fórmulas

**Onde:** Função coletar_linhas_nuvem, leitura de Fora do Prazo.
**Por quê:** Na planilha, Fora do Prazo é sempre fórmula: ="SIM"/="NÃO" ou a fórmula do CSP. Antes, qualquer fórmula virava SIM só se houvesse FRS pendente. Agora ="SIM" vira SIM, e a fórmula do CSP é calculada como no Excel (csp_cobre_frs para cada FRS pendente).
**O que muda:** Troca o trecho Antes pelo Depois.

1 · Procure (Ctrl + F)

```
fora = "SIM" if _celula_texto(ws, mapa, linha, "frs pendentes") else "NÃO"
```

2 · Antes (selecione isto no seu arquivo)

```
                if not (isinstance(fora_bruto, str) and fora_bruto.startswith("=")):
                    fora = _sim_nao_valor(fora_bruto) or _texto(fora_bruto)
                if not fora:
                    # fórmula: se há FRS pendentes, trata como SIM até ter CSP
                    fora = "SIM" if _celula_texto(ws, mapa, linha, "frs pendentes")
else "NÃO"
```

3 · Depois (cole por cima)

```
                if isinstance(fora_bruto, str) and fora_bruto.startswith('="') and
fora_bruto.endswith('"'):
                    fora = _sim_nao_valor(fora_bruto[2:-1])
                elif not (isinstance(fora_bruto, str) and fora_bruto.startswith("=")):
                    fora = _sim_nao_valor(fora_bruto) or _texto(fora_bruto)
                if not fora:
                    # fórmula do CSP: SIM enquanto alguma FRS pendente não tiver CSP
                    _frs = [n for n in _celula_texto(ws, mapa, linha, "frs
pendentes").split("|") if n]
                    _csp = _celula_texto(ws, mapa, linha, "csp")
                    fora = "SIM" if any(not csp_cobre_frs(_csp, n) for n in _frs) else
"NÃO"
```

---

### 7. Republicar não apaga Houve serviço, Foi medido e Fora do prazo corrigidos no app

**Onde:** Lista _CAMPOS_MANUAIS_NUVEM.
**Por quê:** Na planilha esses três campos são editáveis. Ficando nesta lista, o que o pessoal mudar no app não é sobrescrito se você publicar de novo.
**O que muda:** Troca o trecho Antes pelo Depois.

1 · Procure (Ctrl + F)

```
_CAMPOS_MANUAIS_NUVEM = (
```

2 · Antes (selecione isto no seu arquivo)

```
_CAMPOS_MANUAIS_NUVEM = (
    "Justificativa",
```

3 · Depois (cole por cima)

```
_CAMPOS_MANUAIS_NUVEM = (
    "HouveServico",
    "FoiMedido",
    "ForaDoPrazo",
    "Justificativa",
```

---

### 8. Republicar não apaga o que o pessoal respondeu no app

**Onde:** Função _sp_payload_item.
**Por quê:** Antes, se a planilha tivesse um valor, ele substituía o que estava na lista. Agora, para os campos editáveis, o que já está na lista (a resposta do pessoal) vale mais.
**O que muda:** Troca o trecho Antes pelo Depois.

1 · Procure (Ctrl + F)

```
if antigo and not str(valor).strip():
```

2 · Antes (selecione isto no seu arquivo)

```
            if antigo and not str(valor).strip():
                valor = antigo
```

3 · Depois (cole por cima)

```
            if antigo:  # a resposta dada no app vale mais que a da planilha
                valor = antigo
```

---

### 9. Cria as colunas do OK e da aprovação na lista

**Onde:** Função garantir_estrutura_nuvem, lista texto.
**Por quê:** O app grava quem deu OK, quem aprovou e quando. O Python cria essas colunas sozinho na próxima publicação.
**O que muda:** Troca o trecho Antes pelo Depois.

1 · Procure (Ctrl + F)

```
"AprovacaoGerente", "MesFechado",
```

2 · Antes (selecione isto no seu arquivo)

```
        "AprovacaoGerente", "MesFechado",
    ]
```

3 · Depois (cole por cima)

```
        "AprovacaoGerente", "MesFechado",
        "GerenteEmail", "OkFiscal", "OkFiscalPor", "OkFiscalEm",
        "AprovacaoPor", "AprovacaoEm",
    ]
```

---

### 10. Puxar respostas também lê o OK e a aprovação

**Onde:** Função _sp_listar_itens_competencia, select.
**Por quê:** Sem isso, o Puxar respostas não trazia o OK e a aprovação.
**O que muda:** Troca o trecho Antes pelo Depois.

1 · Procure (Ctrl + F)

```
"ComentarioErro,AprovacaoGerente,ComentarioGerente,MesFechado"
```

2 · Antes (selecione isto no seu arquivo)

```
"ComentarioErro,AprovacaoGerente,ComentarioGerente,MesFechado"
```

3 · Depois (cole por cima)

```
"ComentarioErro,AprovacaoGerente,ComentarioGerente,MesFechado,"
        "GerenteEmail,OkFiscal,OkFiscalPor,OkFiscalEm,AprovacaoPor,AprovacaoEm"
```

---

### 11. Planilha de controle sai com o OK e a aprovação

**Onde:** Função _arquivar_controle_mes.
**Por quê:** A planilha final do mês (controle_online_AAAA_MM.xlsx) passa a mostrar quem

conferiu e quem aprovou cada contrato, e quando. É a evidência SOx.
**O que muda:** Troca o trecho Antes pelo Depois.

1 · Procure (Ctrl + F)

```
campos = list(CABECALHO_STAGING_NUVEM)
```

2 · Antes (selecione isto no seu arquivo)

```
    campos = list(CABECALHO_STAGING_NUVEM)
```

3 · Depois (cole por cima)

```
    campos = list(CABECALHO_STAGING_NUVEM) + [
        "OkFiscal", "OkFiscalPor", "OkFiscalEm", "AprovacaoPor", "AprovacaoEm",
    ]
```

---

### 12. Fora do Prazo corrigido no app volta para a planilha

**Onde:** Função gravar_itens_na_planilha (usada pelo Puxar e pelo Fechar).
**Por quê:** Antes, como Fora do Prazo é sempre fórmula, a correção do app nunca voltava. Agora o ="SIM"/="NÃO" simples recebe o valor do app, ainda como fórmula. A fórmula do CSP fica como está e se recalcula sozinha com o CSP que volta do app.
**O que muda:** Troca o trecho Antes pelo Depois.

1 · Procure (Ctrl + F)

```
if pular_formula and isinstance(atual, str) and atual.startswith("="):
```

2 · Antes (selecione isto no seu arquivo)

```
                    if pular_formula and isinstance(atual, str) and
atual.startswith("="):
                        continue
```

3 · Depois (cole por cima)

```
                    if pular_formula and isinstance(atual, str) and
atual.startswith("="):
                        # SIM/NÃO simples recebe o valor do app; a fórmula do CSP se
recalcula sozinha
                        valor_app = _sim_nao_valor(_texto(it.get(campo_sp)))
                        if atual.startswith('="') and valor_app in {"SIM", "NÃO"} and
atual != formula_sim_nao(valor_app):
                            celula.value = formula_sim_nao(valor_app)
                            mudou = True
                        continue
```

---

### 13. Rascunhos do Teams continuam funcionando com ;email;

**Onde:** Função gerar_rascunhos_teams_nuvem.
**Por quê:** O link do Teams não aceita ;email;. Aqui ele volta a ser email1,email2.
**O que muda:** Troca o trecho Antes pelo Depois.

1 · Procure (Ctrl + F)

```
email = (it.get("FiscalEmail") or "").strip()
```

2 · Antes (selecione isto no seu arquivo)

```
        email = (it.get("FiscalEmail") or "").strip()
```

3 · Depois (cole por cima)

```
        email = ",".join(e for e in (it.get("FiscalEmail") or "").split(";") if
e.strip())
```

---

### 14. Funções que travam o mês e guardam a cópia de controle

**Onde:** Novas funções, logo antes de fechar_competencia_nuvem.
**Por quê:** _sp_travar_itens deixa cada contrato do mês só leitura para todos no SharePoint, menos os e-mails de admin_emails, que ficam com controle total. _sp_destravar_itens desfaz isso. _guardar_copia_fechamento copia a planilha do mês e o controle final para controles/fechamento_AAAA_MM, marca como somente leitura e grava o SHA-256 de cada um.
**O que muda:** Acrescenta código novo logo acima deste trecho (o trecho continua igual).

1 · Procure (Ctrl + F)

```
def fechar_competencia_nuvem(
```

2 · Antes (selecione isto no seu arquivo)

```
def fechar_competencia_nuvem(
```

3 · Depois (cole por cima)

```
def _sp_ids_papeis(site: str, token: str) -> dict[str, int]:
    """Ids dos níveis Leitura, Controle Total e Acesso Limitado do site."""
    import requests as req

    ids: dict[str, int] = {}
    for nome, tipo in (("leitura", 2), ("total", 5), ("limitado", 1)):
        r = req.get(
            f"{site}/_api/web/roledefinitions/getbytype({tipo})?$select=Id",
            headers=_sp_headers(token),
            timeout=60,
        )
        if r.status_code < 400:
            ids[nome] = int(r.json()["d"]["Id"])
    return ids


def _sp_id_usuario(site: str, token: str, digest: str, email: str) -> int | None:
    import requests as req

    r = req.post(
        f"{site}/_api/web/ensureuser",
        headers=_sp_headers(token, digest),
        data=json.dumps({"logonName": f"i:0#.f|membership|{email}"}),
        timeout=60,
    )
    if r.status_code >= 400:
        print(f"  Não achei o usuário {email} no site: {r.status_code}", flush=True)
        return None
    return int(r.json()["d"]["Id"])


def _sp_travar_itens(site: str, lista: str, token: str, competencia: date) ->
dict[str, int]:
    """Mês fechado: cada contrato vira só leitura para todos; os admins ficam com
controle total."""
    import requests as req

    digest = _sp_digest(site, token)
    papeis = _sp_ids_papeis(site, token)
    if "leitura" not in papeis or "total" not in papeis:
        raise RuntimeError("Não li os níveis de permissão do site (Leitura / Controle
Total).")
    admins = [i for i in (_sp_id_usuario(site, token, digest, e) for e in
sorted(_admin_emails())) if i]
    if not admins:
        raise RuntimeError("Nenhum admin de admin_emails existe no site. Nada foi
travado.")
    itens = _sp_listar_itens_competencia(site, lista, token, competencia)
    base = f"{site}/_api/web/lists/getbytitle('{lista}')/items"
    ok = falha = 0
    for indice, it in enumerate(itens, 1):
        iid = it.get("Id")
        try:
            req.post(
                f"{base}
({iid})/breakroleinheritance(copyRoleAssignments=true,clearSubscopes=true)",
                headers=_sp_headers(token, digest), timeout=60,
            ).raise_for_status()
            # admin primeiro, para quem está rodando nunca perder o acesso no meio
            for admin in admins:
                req.post(
```

```
                    f"{base}({iid})/roleassignments/addroleassignment(principalid=
{admin},roledefid={papeis['total']})",
                    headers=_sp_headers(token, digest), timeout=60,
                ).raise_for_status()
            r = req.get(
                f"{base}({iid})/roleassignments?$expand=RoleDefinitionBindings",
                headers=_sp_headers(token), timeout=60,
            )
            r.raise_for_status()
            for atribuicao in r.json()["d"]["results"]:
                principal = int(atribuicao["PrincipalId"])
                if principal in admins:
                    continue
                atuais = [int(x["Id"]) for x in atribuicao["RoleDefinitionBindings"]
["results"]]
                for papel in atuais:
                    if papel in (papeis["leitura"], papeis.get("limitado")):
                        continue
                    req.post(
                        f"{base}
({iid})/roleassignments/removeroleassignment(principalid={principal},roledefid=
{papel})",
                        headers=_sp_headers(token, digest), timeout=60,
                    ).raise_for_status()
                if papeis["leitura"] not in atuais and atuais !=
[papeis.get("limitado")]:
                    req.post(
                        f"{base}({iid})/roleassignments/addroleassignment(principalid=
{principal},roledefid={papeis['leitura']})",
                        headers=_sp_headers(token, digest), timeout=60,
                    ).raise_for_status()
            ok += 1
        except Exception as exc:
            falha += 1
            print(f"  Não travei o item {iid}: {exc}", flush=True)
        if indice % 25 == 0:
            digest = _sp_digest(site, token)
            print(f"  Travados {indice}/{len(itens)}...", flush=True)
    print(f"Trava: {ok} contrato(s) só leitura; {falha} falha(s).", flush=True)
    return {"ok": ok, "falha": falha, "total": len(itens)}


def _sp_destravar_itens(site: str, lista: str, token: str, competencia: date) ->
dict[str, int]:
    """Reabrir: cada contrato volta a seguir a permissão normal da lista."""
    import requests as req

    digest = _sp_digest(site, token)
    itens = _sp_listar_itens_competencia(site, lista, token, competencia)
    base = f"{site}/_api/web/lists/getbytitle('{lista}')/items"
    ok = falha = 0
    for indice, it in enumerate(itens, 1):
        r = req.post(
            f"{base}({it.get('Id')})/resetroleinheritance",
            headers=_sp_headers(token, digest), timeout=60,
        )
        if r.status_code < 400:
            ok += 1
        else:
            falha += 1
        if indice % 40 == 0:
            digest = _sp_digest(site, token)
    print(f"Destrava: {ok} contrato(s) voltaram ao normal; {falha} falha(s).",
```

```
flush=True)
    return {"ok": ok, "falha": falha, "total": len(itens)}


def _guardar_copia_fechamento(competencia: date, arquivos: list[Path]) -> Path:
    """Copia os arquivos finais para controles/fechamento_AAAA_MM, somente leitura,
com hash."""
    import stat

    pasta = _pasta_controles() / f"fechamento_{competencia:%Y_%m}"
    pasta.mkdir(parents=True, exist_ok=True)
    linhas = [
        f"Controle SOx SCM-42 - competência {competencia:%m/%Y}",
        f"Fechado em {datetime.now():%d/%m/%Y %H:%M}",
        "SHA-256 de cada arquivo (se o arquivo mudar, o código muda):",
        "",
    ]
    for origem in arquivos:
        origem = Path(origem)
        if not origem.is_file():
            continue
        destino = pasta / origem.name
        if destino.exists():
            os.chmod(destino, stat.S_IWRITE)
        shutil.copy2(origem, destino)
        linhas.append(f"{hashlib.sha256(destino.read_bytes()).hexdigest()}
{destino.name}")
        os.chmod(destino, stat.S_IREAD)
    manifesto = pasta / "manifesto_sha256.txt"
    if manifesto.exists():
        os.chmod(manifesto, stat.S_IWRITE)
    manifesto.write_text("\n".join(linhas) + "\n", encoding="utf-8")
    os.chmod(manifesto, stat.S_IREAD)
    return pasta


def fechar_competencia_nuvem(
```

---

### 15. Fechar competência trava a lista e guarda a cópia

**Onde:** Função fechar_competencia_nuvem.
**Por quê:** Depois de marcar MesFechado, trava os contratos do mês. Depois de puxar, guarda a cópia somente leitura com hash. Ordem: marca, trava, puxa, guarda.
**O que muda:** Troca o trecho Antes pelo Depois.

1 · Procure (Ctrl + F)

```
print("Puxando cópia final para seus controles...", flush=True)
```

2 · Antes (selecione isto no seu arquivo)

```
    print("Puxando cópia final para seus controles...", flush=True)
    puxado = puxar_lista_sharepoint(competencia)
```

3 · Depois (cole por cima)

```
    print("Travando os contratos do mês: só o administrador continua editando...",
flush=True)
    trava = _sp_travar_itens(site, lista, token, competencia)
    print("Puxando cópia final para seus controles...", flush=True)
    puxado = puxar_lista_sharepoint(competencia)
    copia = _guardar_copia_fechamento(
        competencia, [Path(puxado.get("arquivo_controle") or ""), RAIZ /
nome_arquivo(competencia)]
    )
    print(f"Cópia de controle (somente leitura): {copia}", flush=True)
```

---

### 16. Resumo do fechamento mostra a trava e a pasta da cópia

**Onde:** Função fechar_competencia_nuvem, dicionário resumo.
**Por quê:** A janela de fim do fechamento passa a mostrar quantos contratos foram travados e onde está a cópia.
**O que muda:** Troca o trecho Antes pelo Depois.

1 · Procure (Ctrl + F)

```
"arquivo_controle": puxado.get("arquivo_controle"),
```

2 · Antes (selecione isto no seu arquivo)

```
        "arquivo_controle": puxado.get("arquivo_controle"),
```

3 · Depois (cole por cima)

```
        "arquivo_controle": puxado.get("arquivo_controle"),
        "trava": trava,
        "copia_controle": str(copia),
```

---

### 17. Reabrir competência destrava a lista

**Onde:** Função reabrir_competencia_nuvem.
**Por quê:** Reabrir devolve a permissão normal a cada contrato do mês, antes de desmarcar MesFechado.
**O que muda:** Acrescenta código novo logo acima deste trecho (o trecho continua igual).

1 · Procure (Ctrl + F)

```
marca = _sp_marcar_mes_fechado_itens(site, lista, token, competencia, fechada=False)
```

2 · Antes (selecione isto no seu arquivo)

```
    marca = _sp_marcar_mes_fechado_itens(site, lista, token, competencia,
fechada=False)
```

3 · Depois (cole por cima)

```
    _sp_destravar_itens(site, lista, token, competencia)
    marca = _sp_marcar_mes_fechado_itens(site, lista, token, competencia,
fechada=False)
```

### 18. Janela do fechamento mostra a cópia

**Onde:** Função _abrir_interface, botão Fechar competência.
**Por quê:** Mostra a pasta da cópia de controle ao final do fechamento.
**O que muda:** Troca o trecho Antes pelo Depois.

1 · Procure (Ctrl + F)

```
f"Controle:\n{resumo.get('arquivo_controle')}",
```

2 · Antes (selecione isto no seu arquivo)

```
                    f"Controle:\n{resumo.get('arquivo_controle')}",
```

3 · Depois (cole por cima)

```
                    f"Controle:\n{resumo.get('arquivo_controle')}\n\n"
                    f"Cópia somente leitura:\n{resumo.get('copia_controle')}",
```

---

### 19. Publicar cria a lista online se ela não existir

**Onde:** Nova função, logo antes de garantir_estrutura_nuvem.
**Por quê:** Se a lista SOx_SCM42_Contratos ainda não existir no site, o Python cria a lista e todas as colunas na primeira publicação. Se já existir, só confere as colunas. Também indexa Competencia e ChaveUnica, para o app e o Python continuarem rápidos quando a lista passar de 5.000 linhas (cerca de 2 anos).
**O que muda:** Acrescenta código novo logo acima deste trecho (o trecho continua igual).

1 · Procure (Ctrl + F)

```
def garantir_estrutura_nuvem(
```

2 · Antes (selecione isto no seu arquivo)

```
def garantir_estrutura_nuvem(
```

3 · Depois (cole por cima)

```
_CAMPOS_LONGOS_NUVEM = {
    "Objeto", "Provisao", "Justificativa", "Observacao", "CSP", "FRSPendentes",
    "MotivoIntervencao", "ComentarioErro", "ComentarioGerente",
}


def _sp_garantir_lista_contratos(site: str, lista: str, token: str, digest: str) ->
bool:
    """Cria a lista de contratos no SharePoint se ainda não existir. True se criou
agora."""
    import requests as req

    r = req.get(
        f"{site}/_api/web/lists/getbytitle('{lista}')?$select=Id",
        headers=_sp_headers(token),
        timeout=60,
    )
    criou = False
    if r.status_code == 404:
        body = {
            "__metadata": {"type": "SP.List"},
            "AllowContentTypes": True,
            "BaseTemplate": 100,
            "ContentTypesEnabled": True,
            "Description": "Controle SOx SCM-42: um contrato por linha e por
competência. Preenchida pelo Python.",
            "Title": lista,
        }
        r2 = req.post(
            f"{site}/_api/web/lists",
            headers=_sp_headers(token, digest),
            data=json.dumps(body),
            timeout=90,
        )
        if r2.status_code not in {200, 201}:
            raise RuntimeError(
                f"Não consegui criar a lista {lista} em {site}: {r2.status_code}
{r2.text[:200]}. "
                "Confira se o site existe e se você é proprietário dele."
            )
        print(f"Lista criada: {lista}", flush=True)
        criou = True
    elif r.status_code >= 400:
        raise RuntimeError(f"Não consegui abrir a lista {lista}: {r.status_code}
{r.text[:200]}")
    for nome in CABECALHO_STAGING_NUVEM:
        _sp_criar_campo_texto(site, lista, token, digest, nome, multilinha=nome in
_CAMPOS_LONGOS_NUVEM)
    for nome in ("Competencia", "ChaveUnica"):
        try:
            req.post(
                f"
{site}/_api/web/lists/getbytitle('{lista}')/fields/getbyinternalnameortitle('{nome}')"
,
                headers=_sp_headers(token, digest, merge=True),
                data=json.dumps({"__metadata": {"type": "SP.Field"}, "Indexed":
True}),
                timeout=60,
            )
        except Exception:
            pass
```

```
    return criou


def garantir_estrutura_nuvem(
```

---

### 20. Conferir a estrutura começa pela lista de contratos

**Onde:** Função garantir_estrutura_nuvem, logo depois do login.
**Por quê:** Chama a função acima antes de criar as colunas novas. Como Publicar online agora sempre chama garantir_estrutura_nuvem, basta publicar.
**O que muda:** Troca o trecho Antes pelo Depois.

1 · Procure (Ctrl + F)

```
token = _token_sharepoint(interativo=interativo)
```

2 · Antes (selecione isto no seu arquivo)

```
    token = _token_sharepoint(interativo=interativo)
    digest = _sp_digest(site, token)

    texto = [
```

3 · Depois (cole por cima)

```
    token = _token_sharepoint(interativo=interativo)
    digest = _sp_digest(site, token)
    if _sp_garantir_lista_contratos(site, lista, token, digest):
        digest = _sp_digest(site, token)

    texto = [
```

---

### 21. Ponte: Publicar espera o fluxo e mostra o andamento

**Onde:** Bloco da ponte (atualiza a versão anterior, se você já tinha).
**Por quê:** Publicar agora espera o fluxo responder, mostra o tempo a cada 30 segundos e diz no fim quantos contratos entraram na lista. Só vale para quem já rodou a versão anterior do ajuste.
**O que muda:** Troca o trecho Antes pelo Depois.

1 · Procure (Ctrl + F)

```
xml = f"<Field Type='Text' DisplayName='{nome}' Name='{nome}' StaticName='{nome}'
MaxLength='255' />"
```

2 · Antes (selecione isto no seu arquivo)

```
# ------------------------------------------------------------- ponte Power
Automate
# A Microsoft bloqueou o login direto do Python no SharePoint (erro AADSTS65002).
# O Python agora grava pedidos numa pasta do OneDrive; o fluxo "SOx Ponte" faz o
# trabalho no SharePoint com o login do próprio usuário e devolve a lista num arquivo.
_PONTE_CACHE: dict[str, tuple[float, list[dict[str, Any]]]] = {}
_PONTE_ESCOLHAS = {
    "HouveServico": ["SIM", "NÃO"],
    "FoiMedido": ["SIM", "NÃO"],
    "ForaDoPrazo": ["SIM", "NÃO"],
    "SeraSubstituido": ["SIM", "NÃO"],
    "DemandaContinuada": ["SIM", "NÃO"],
    "MotivoProvisao": [
        "Atraso do Fornecedor", "Atraso Fiscalização de Campo", "Problema Interno
GPS",
        "Contrato Bloqueado", "Outros",
    ],
}
_PONTE_LONGOS = {
    "Objeto", "Provisao", "Justificativa", "Observacao", "CSP", "FRSPendentes",
    "MotivoIntervencao", "ComentarioErro", "ComentarioGerente",
}
_PONTE_EXTRAS = [
    "OQueFazer", "DadosCorretos", "CampoErro", "ValorCorreto", "AprovacaoGerente",
    "MesFechado", "ComentarioErro", "ComentarioGerente", "OkFiscal", "OkFiscalPor",
    "OkFiscalEm", "AprovacaoPor", "AprovacaoEm",
]
# Campos que o pessoal preenche no app: republicar não sobrescreve.
_PONTE_DO_APP = set(_CAMPOS_MANUAIS_NUVEM) | {
    "Observacao", "OkFiscal", "OkFiscalPor", "OkFiscalEm", "AprovacaoGerente",
    "AprovacaoPor", "AprovacaoEm", "ComentarioGerente", "MesFechado",
}


def _usar_ponte() -> bool:
    return str(garantir_config_nuvem().get("modo_online") or "ponte").lower() ==
"ponte"


def _pasta_ponte() -> Path:
    cfg = garantir_config_nuvem()
    base = str(cfg.get("pasta_ponte") or "").strip()
    if not base:
        onedrive = os.environ.get("OneDriveCommercial") or os.environ.get("OneDrive")
or ""
        if not onedrive:
            raise RuntimeError(
                "Não achei a pasta do OneDrive neste computador. Abra o OneDrive,
entre com a "
                "conta da Petrobras, ou escreva o caminho em pasta_ponte no
sox_nuvem_config.json."
            )
        base = str(Path(onedrive) / "SOx_SCM42_ponte")
    pasta = Path(base)
    for sub in ("pedidos", "da_lista"):
        (pasta / sub).mkdir(parents=True, exist_ok=True)
    return pasta


def _ponte_colunas(lista: str) -> list[dict[str, Any]]:
    nomes = list(dict.fromkeys(list(CABECALHO_STAGING_NUVEM) + _PONTE_EXTRAS))
```

```
    base = f"_api/web/lists/getbytitle('{lista}')"
    saida = []
    for nome in nomes:
        if nome in _PONTE_ESCOLHAS:
            opcoes = "".join(f"<CHOICE>{o}</CHOICE>" for o in _PONTE_ESCOLHAS[nome])
            xml = (
                f"<Field Type='Choice' DisplayName='{nome}' Name='{nome}'
StaticName='{nome}' "
                f"FillInChoice='TRUE' Format='Dropdown'><CHOICES>{opcoes}</CHOICES>
</Field>"
            )
        elif nome in _PONTE_LONGOS:
            xml = (
                f"<Field Type='Note' DisplayName='{nome}' Name='{nome}'
StaticName='{nome}' "
                f"RichText='FALSE' NumLines='6' />"
            )
        else:
            xml = f"<Field Type='Text' DisplayName='{nome}' Name='{nome}'
StaticName='{nome}' MaxLength='255' />"
        saida.append({
            "nome": nome,
            "uri_existe": f"{base}/fields?$select=Id&$filter=" + quote(f"InternalName
eq '{nome}'"),
            "corpo": {"parameters": {"SchemaXml": xml, "Options": 24}},
        })
    return saida


def _ponte_pedido(acao: str, competencia: date, **dados: Any) -> str:
    import uuid

    pid = datetime.now().strftime("%Y%m%d%H%M%S") + uuid.uuid4().hex[:6]
    cfg = garantir_config_nuvem()
    lista = str(cfg.get("lista_contratos") or "SOx_SCM42_Contratos")
    comp = competencia.strftime("%m/%Y")
    base = f"_api/web/lists/getbytitle('{lista}')"
    filtro = quote(f"Competencia eq '{comp}'")
    # Endereços prontos: no fluxo cada caixa só lê outputs('Pedido')?['...'].
    pedido = {
        "acao": acao,
        "id": pid,
        "competencia": comp,
        "arquivo_comp": competencia.strftime("%Y_%m"),
        "lista": lista,
        "uri_lista_existe": "_api/web/lists?$select=Id&$filter=" + quote(f"Title eq
'{lista}'"),
        "uri_criar_lista": "_api/web/lists",
        "corpo_criar_lista": {
            "Title": lista,
            "BaseTemplate": 100,
            "Description": "Controle SOx SCM-42: um contrato por linha e por
competência.",
        },
        "uri_criar_coluna": f"{base}/fields/createfieldasxml",
        "uri_indexar": f"{base}/fields/getbyinternalnameortitle('Competencia')",
        "uri_itens": f"{base}/items",
        "uri_existentes": f"{base}/items?$select=Id,ChaveUnica&$top=5000&$filter=
{filtro}",
        "uri_ids": f"{base}/items?$select=Id&$top=5000&$filter={filtro}",
        "uri_lista": f"{base}/items?$top=5000&$filter={filtro}",
        "uri_grupos": (
            "_api/web?$select=AssociatedOwnerGroup/Id,AssociatedMemberGroup/Id,"
```

```
"AssociatedVisitorGroup/Id&$expand=AssociatedOwnerGroup,AssociatedMemberGroup,"
            "AssociatedVisitorGroup"
        ),
        **dados,
    }
    destino = _pasta_ponte() / "pedidos" / f"
{acao}__{competencia.strftime('%Y_%m')}__{pid}.json"
    destino.write_text(json.dumps(pedido, ensure_ascii=True), encoding="utf-8")
    print(f"Pedido enviado para o fluxo SOx Ponte: {destino.name}", flush=True)
    return pid


def _ponte_valor(v: Any) -> Any:
    if isinstance(v, dict):
        return v.get("Value") if "Value" in v else ""
    return "" if v is None else v


def _ponte_esperar_lista(competencia: date, pid: str, minutos: int = 20) ->
list[dict[str, Any]]:
    import time

    arquivo = _pasta_ponte() / "da_lista" /
f"lista_{competencia.strftime('%Y_%m')}__{pid}.json"
    print(f"Aguardando o fluxo SOx Ponte devolver {arquivo.name} (até {minutos}
min)...", flush=True)
    limite = time.time() + minutos * 60
    while time.time() < limite:
        if arquivo.is_file():
            time.sleep(3)
            try:
                bruto = json.loads(arquivo.read_text(encoding="utf-8-sig"))
            except Exception:
                time.sleep(5)
                continue
            if isinstance(bruto, dict):
                bruto = bruto.get("value") or []
            itens = [{k: _ponte_valor(v) for k, v in it.items()} for it in bruto]
            _PONTE_CACHE[competencia.strftime("%m/%Y")] = (time.time(), itens)
            print(f"Fluxo respondeu: {len(itens)} contrato(s).", flush=True)
            return itens
        time.sleep(10)
    raise TimeoutError(
        "O fluxo SOx Ponte não respondeu a tempo. Confira se o OneDrive está
sincronizando "
        "(nuvem azul na barra do Windows) e se o fluxo está Ativado no Power Automate
"
        "(Meus fluxos → SOx Ponte → Histórico de execuções)."
    )


def _ponte_puxar(competencia: date) -> list[dict[str, Any]]:
    import time

    cache = _PONTE_CACHE.get(competencia.strftime("%m/%Y"))
    if cache and time.time() - cache[0] < 180:
        return cache[1]
    pid = _ponte_pedido("puxar", competencia)
    return _ponte_esperar_lista(competencia, pid)


def publicar_pela_ponte(caminho_planilha: Path, competencia: date) -> dict[str, Any]:
```

```
    cfg = garantir_config_nuvem()
    site, lista = _site_e_lista(cfg)
    exportar_staging_nuvem(caminho_planilha, competencia)
    linhas = coletar_linhas_nuvem(caminho_planilha, competencia)
    itens = []
    for linha in linhas:
        chave = linha.get("ChaveUnica") or ""
        if not chave:
            continue
        if len(linha.get("Observacao", "")) > 4000:
            linha["Observacao"] = linha["Observacao"][:4000] + "..."
        if len(linha.get("Objeto", "")) > 2000:
            linha["Objeto"] = linha["Objeto"][:2000]
        novo = _sp_payload_item("SP.Data", linha, None)
        novo.pop("__metadata", None)
        atualiza = {k: v for k, v in novo.items() if k not in _PONTE_DO_APP}
        itens.append({"chave": chave, "novo": novo, "atualiza": atualiza})
    _ponte_pedido("publicar", competencia, colunas=_ponte_colunas(lista), itens=itens)
    url = cfg.get("lista_url") or f"{site}/Lists/{lista}"
    return {
        "criados": len(itens),
        "atualizados": 0,
        "falhas": 0,
        "lista_url": url,
        "mensagem": (
            f"Enviado para a lista online: {len(itens)} contrato(s).\n\n"
            "O fluxo SOx Ponte grava na lista em até 5 minutos.\n"
            "Depois atualize a lista no navegador (F5) para conferir.\n\n"
            "O que o pessoal já respondeu no app não é apagado."
        ),
    }


def _fechar_pela_ponte(competencia: date, por: str) -> dict[str, Any]:
    chave = competencia.strftime("%m/%Y")
    ciclos = _carregar_ciclos_local()
    ciclos[chave] = {
        "fechada": "SIM",
        "fechado_em": datetime.now().strftime("%d/%m/%Y %H:%M"),
        "fechado_por": por,
    }
    _salvar_ciclos_local(ciclos)
    print(f"Fechando {chave}: o fluxo marca MesFechado=SIM e deixa só leitura...",
flush=True)
    pid = _ponte_pedido("fechar", competencia)
    itens = _ponte_esperar_lista(competencia, pid, minutos=30)
    puxado = puxar_lista_sharepoint(competencia)
    copia = _guardar_copia_fechamento(
        competencia, [Path(puxado.get("arquivo_controle") or ""), RAIZ /
nome_arquivo(competencia)]
    )
    print(f"Cópia de controle (somente leitura): {copia}", flush=True)
    resumo = {
        "acao": "fechar",
        "competencia": chave,
        "fechado_por": por,
        "itens": {"total": len(itens)},
        "arquivo_controle": puxado.get("arquivo_controle"),
        "trava": {"modo": "fluxo SOx Ponte", "itens": len(itens)},
        "copia_controle": str(copia),
        "regra": "Depois do fechamento, só o grupo de Proprietários do site edita a
lista.",
    }
```

```
    (PASTA_NUVEM / f"SOx_fechamento_{competencia.strftime('%Y_%m')}.json").write_text(
        json.dumps(resumo, ensure_ascii=False, indent=2), encoding="utf-8"
    )
    print(f"Competência {chave} FECHADA.", flush=True)
    return resumo


def _reabrir_pela_ponte(competencia: date, por: str) -> dict[str, Any]:
    chave = competencia.strftime("%m/%Y")
    ciclos = _carregar_ciclos_local()
    ciclos[chave] = {
        "fechada": "NÃO",
        "reaberto_em": datetime.now().strftime("%d/%m/%Y %H:%M"),
        "reaberto_por": por,
    }
    _salvar_ciclos_local(ciclos)
    pid = _ponte_pedido("reabrir", competencia)
    itens = _ponte_esperar_lista(competencia, pid, minutos=30)
    print(f"Competência {chave} REABERTA.", flush=True)
    return {"acao": "reabrir", "competencia": chave, "reaberto_por": por, "itens":
{"total": len(itens)}}


def publicar_lista_sharepoint(
```

3 · Depois (cole por cima)

```
# ---------------------------------------------------------------- ponte Power
Automate
# A Microsoft bloqueou o login direto do Python no SharePoint (erro AADSTS65002).
# O Python agora grava pedidos numa pasta do OneDrive; o fluxo "SOx Ponte" faz o
# trabalho no SharePoint com o login do próprio usuário e devolve a lista num arquivo.
# O pedido vai direto ao OneDrive online (cookies SSO, como no Sigma-C); sem cookies,
# usa a pasta sincronizada deste computador.
_PONTE_CACHE: dict[str, tuple[float, list[dict[str, Any]]]] = {}
_PONTE_ONLINE: dict[str, Any] = {}
_PONTE_SITE = "https://petrobrasbr-
my.sharepoint.com/personal/levi_vasconcelos_petrobras_com_br"
_PONTE_PASTA = "/personal/levi_vasconcelos_petrobras_com_br/Documents/SOx_SCM42_ponte"
_PONTE_ESCOLHAS = {
    "HouveServico": ["SIM", "NÃO"],
    "FoiMedido": ["SIM", "NÃO"],
    "ForaDoPrazo": ["SIM", "NÃO"],
    "SeraSubstituido": ["SIM", "NÃO"],
    "DemandaContinuada": ["SIM", "NÃO"],
    "MotivoProvisao": [
        "Atraso do Fornecedor", "Atraso Fiscalização de Campo", "Problema Interno
GPS",
        "Contrato Bloqueado", "Outros",
    ],
}
_PONTE_LONGOS = {
    "Objeto", "Provisao", "Justificativa", "Observacao", "CSP", "FRSPendentes",
    "MotivoIntervencao", "ComentarioErro", "ComentarioGerente",
}
_PONTE_EXTRAS = [
    "OQueFazer", "DadosCorretos", "CampoErro", "ValorCorreto", "AprovacaoGerente",
    "MesFechado", "ComentarioErro", "ComentarioGerente", "OkFiscal", "OkFiscalPor",
    "OkFiscalEm", "AprovacaoPor", "AprovacaoEm",
]
# Campos que o pessoal preenche no app: republicar não sobrescreve.
_PONTE_DO_APP = set(_CAMPOS_MANUAIS_NUVEM) | {
    "Observacao", "OkFiscal", "OkFiscalPor", "OkFiscalEm", "AprovacaoGerente",
    "AprovacaoPor", "AprovacaoEm", "ComentarioGerente", "MesFechado",
}


def _usar_ponte() -> bool:
    return str(garantir_config_nuvem().get("modo_online") or "ponte").lower() ==
"ponte"


def _pasta_ponte() -> Path:
    cfg = garantir_config_nuvem()
    base = str(cfg.get("pasta_ponte") or "").strip()
    if not base:
        onedrive = os.environ.get("OneDriveCommercial") or os.environ.get("OneDrive")
or ""
        if not onedrive:
            raise RuntimeError(
                "Não achei a pasta do OneDrive neste computador. Abra o OneDrive,
entre com a "
                "conta da Petrobras, ou escreva o caminho em pasta_ponte no
sox_nuvem_config.json."
            )
        base = str(Path(onedrive) / "SOx_SCM42_ponte")
    pasta = Path(base)
    for sub in ("pedidos", "da_lista"):
```

```
        (pasta / sub).mkdir(parents=True, exist_ok=True)
    return pasta


def _ponte_cookies(host: str) -> str:
    # Mesmos cookies SSO que o Sigma-C e a automação de certidões já guardam.
    cfg = garantir_config_nuvem()
    pasta_cert = Path.home() / "Downloads" / "automatizacao certidao"
    candidatos = [Path(str(cfg.get("ponte_cookies")))] if cfg.get("ponte_cookies")
else []
    candidatos += [
        RAIZ / ".sp_cookies.json",
        pasta_cert / ".sp_cookies.json",
        pasta_cert / "automatizacao_certidoes.db",
        pasta_cert / "certidoes.db",
    ]
    for caminho in candidatos:
        try:
            if not caminho.is_file():
                continue
            if caminho.suffix.lower() == ".json":
                dados = json.loads(caminho.read_text(encoding="utf-8",
errors="replace"))
            else:
                import sqlite3

                con = sqlite3.connect(str(caminho))
                try:
                    linha = con.execute("SELECT cookies FROM sessao WHERE
id=1").fetchone()
                finally:
                    con.close()
                dados = json.loads(linha[0]) if linha and linha[0] else None
        except Exception:
            continue
        if isinstance(dados, str) and dados.strip():
            return dados.strip()
        if isinstance(dados, dict):
            dados = dados.get("cookies") or [{"name": k, "value": v} for k, v in
dados.items()]
        if not isinstance(dados, list):
            continue
        todos = [c for c in dados if isinstance(c, dict) and c.get("name")]

        def do_host(c: dict[str, Any]) -> bool:
            dominio = str(c.get("domain") or "").lstrip(".").lower()
            return not dominio or host == dominio or host.endswith("." + dominio)

        escolhidos = [c for c in todos if do_host(c)] or todos
        vistos: dict[str, str] = {}
        for c in escolhidos:
            vistos.setdefault(str(c["name"]), str(c.get("value", "")))
        if vistos:
            print(f"Cookies SSO: {caminho}", flush=True)
            return "; ".join(f"{k}={v}" for k, v in vistos.items())
    return ""


def _ponte_http(metodo: str, url: str, cookies: str, corpo: bytes | None = None,
                digest: str = "", tipo: str = "application/octet-stream") ->
tuple[int, bytes]:
    import ssl
    import urllib.error
```

```
    import urllib.request
    from urllib.parse import urlparse

    cab = {"Accept": "application/json;odata=verbose", "Cookie": cookies, "User-
Agent": "Mozilla/5.0"}
    if digest:
        cab["X-RequestDigest"] = digest
    if corpo is not None:
        cab["Content-Type"] = tipo
    req = urllib.request.Request(url, data=corpo, headers=cab, method=metodo)
    try:
        with urllib.request.urlopen(req, context=ssl.create_default_context(),
timeout=60) as resp:
            # Cookie vencido: o SharePoint manda para a tela de login da Microsoft.
            if urlparse(resp.geturl()).netloc.lower() != urlparse(url).netloc.lower():
                return 401, b"login expirado"
            return resp.status, resp.read()
    except urllib.error.HTTPError as exc:
        return exc.code, exc.read()


def _ponte_online() -> dict[str, str] | None:
    import time
    from urllib.parse import urlparse

    if time.time() - _PONTE_ONLINE.get("em", 0) < 1200:
        return _PONTE_ONLINE.get("dados")
    cfg = garantir_config_nuvem()
    dados = None
    if str(cfg.get("ponte_envio") or "online").lower() != "pasta":
        site = str(cfg.get("ponte_site") or _PONTE_SITE).rstrip("/")
        cookies = _ponte_cookies((urlparse(site).hostname or "").lower())
        if not cookies:
            print("Sem cookies SSO salvos (Sigma-C/certidões): uso a pasta
sincronizada do OneDrive.", flush=True)
        else:
            try:
                status, corpo = _ponte_http(
                    "POST", f"{site}/_api/contextinfo", cookies, b"",
tipo="application/json;odata=verbose"
                )
                info = json.loads(corpo.decode("utf-8")) if status == 200 else {}
                digest = (info.get("d") or {}).get("GetContextWebInformation",
{}).get("FormDigestValue", "")
            except Exception as exc:
                status, digest = str(exc), ""
            if digest:
                dados = {
                    "site": site,
                    "pasta": str(cfg.get("ponte_pasta_servidor") or
_PONTE_PASTA).rstrip("/"),
                    "cookies": cookies,
                    "digest": digest,
                }
            else:
                print(
                    f"OneDrive online não aceitou os cookies ({status}). Abra o Sigma-
C ou as certidões "
                    "para renovar o login. Por agora uso a pasta sincronizada.",
                    flush=True,
                )
    _PONTE_ONLINE.update(em=time.time(), dados=dados)
    return dados
```

```
def _ponte_subir_online(sub: str, nome: str, corpo: bytes) -> bool:
    from urllib.parse import quote as q, urlencode

    on = _ponte_online()
    if not on:
        return False
    pasta = f"{on['pasta']}/{sub}"
    url = (
        f"
{on['site']}/_api/web/GetFolderByServerRelativeUrl(@p)/Files/add(url=@n,overwrite=true
)?"
        + urlencode({"@p": f"'{pasta}'", "@n": f"'{nome}'"})
    )
    status, resposta = _ponte_http("POST", url, on["cookies"], corpo, on["digest"])
    if status == 404:  # pasta ainda não existe no OneDrive online
        _ponte_http("POST", f"{on['site']}/_api/web/folders/add('{q(pasta)}')",
on["cookies"], b"", on["digest"])
        status, resposta = _ponte_http("POST", url, on["cookies"], corpo,
on["digest"])
    if status in (200, 201):
        return True
    trecho = resposta.decode("utf-8", errors="replace")[:200]
    print(f"Envio online falhou (HTTP {status}): {trecho}", flush=True)
    _PONTE_ONLINE.clear()
    return False


def _ponte_ler_online(sub: str, nome: str) -> bytes | None:
    from urllib.parse import urlencode

    on = _ponte_online()
    if not on:
        return None
    url = (
        f"{on['site']}/_api/web/GetFileByServerRelativeUrl(@f)/$value?"
        + urlencode({"@f": f"'{on['pasta']}/{sub}/{nome}'"})
    )
    try:
        status, corpo = _ponte_http("GET", url, on["cookies"])
    except Exception:
        return None
    return corpo if status == 200 else None


def _ponte_colunas(lista: str) -> list[dict[str, Any]]:
    nomes = list(dict.fromkeys(list(CABECALHO_STAGING_NUVEM) + _PONTE_EXTRAS))
    base = f"_api/web/lists/getbytitle('{lista}')"
    saida = []
    for nome in nomes:
        if nome in _PONTE_ESCOLHAS:
            opcoes = "".join(f"<CHOICE>{o}</CHOICE>" for o in _PONTE_ESCOLHAS[nome])
            xml = (
                f"<Field Type='Choice' DisplayName='{nome}' Name='{nome}'
StaticName='{nome}' "
                f"FillInChoice='TRUE' Format='Dropdown'><CHOICES>{opcoes}</CHOICES>
</Field>"
            )
        elif nome in _PONTE_LONGOS:
            xml = (
                f"<Field Type='Note' DisplayName='{nome}' Name='{nome}'
StaticName='{nome}' "
```

```
                f"RichText='FALSE' NumLines='6' />"
            )
        else:
            xml = f"<Field Type='Text' DisplayName='{nome}' Name='{nome}'
StaticName='{nome}' MaxLength='255' />"
        saida.append({
            "nome": nome,
            "uri_existe": f"{base}/fields?$select=Id&$filter=" + quote(f"InternalName
eq '{nome}'"),
            "corpo": {"parameters": {"SchemaXml": xml, "Options": 24}},
        })
    return saida


def _ponte_pedido(acao: str, competencia: date, **dados: Any) -> str:
    import uuid

    pid = datetime.now().strftime("%Y%m%d%H%M%S") + uuid.uuid4().hex[:6]
    cfg = garantir_config_nuvem()
    lista = str(cfg.get("lista_contratos") or "SOx_SCM42_Contratos")
    comp = competencia.strftime("%m/%Y")
    base = f"_api/web/lists/getbytitle('{lista}')"
    filtro = quote(f"Competencia eq '{comp}'")
    # Endereços prontos: no fluxo cada caixa só lê outputs('Pedido')?['...'].
    pedido = {
        "acao": acao,
        "id": pid,
        "competencia": comp,
        "arquivo_comp": competencia.strftime("%Y_%m"),
        "lista": lista,
        "uri_lista_existe": "_api/web/lists?$select=Id&$filter=" + quote(f"Title eq
'{lista}'"),
        "uri_criar_lista": "_api/web/lists",
        "corpo_criar_lista": {
            "Title": lista,
            "BaseTemplate": 100,
            "Description": "Controle SOx SCM-42: um contrato por linha e por
competência.",
        },
        "uri_criar_coluna": f"{base}/fields/createfieldasxml",
        "uri_indexar": f"{base}/fields/getbyinternalnameortitle('Competencia')",
        "uri_itens": f"{base}/items",
        "uri_existentes": f"{base}/items?$select=Id,ChaveUnica&$top=5000&$filter=
{filtro}",
        "uri_ids": f"{base}/items?$select=Id&$top=5000&$filter={filtro}",
        "uri_lista": f"{base}/items?$top=5000&$filter={filtro}",
        "uri_grupos": (
            "_api/web?$select=AssociatedOwnerGroup/Id,AssociatedMemberGroup/Id,"

"AssociatedVisitorGroup/Id&$expand=AssociatedOwnerGroup,AssociatedMemberGroup,"
            "AssociatedVisitorGroup"
        ),
        **dados,
    }
    nome = f"{acao}__{competencia.strftime('%Y_%m')}__{pid}.json"
    corpo = json.dumps(pedido, ensure_ascii=True).encode("utf-8")
    if _ponte_subir_online("pedidos", nome, corpo):
        print(f"Pedido enviado direto ao OneDrive online (não depende da
sincronização): {nome}", flush=True)
        return pid
    (_pasta_ponte() / "pedidos" / nome).write_bytes(corpo)
    print(f"Pedido gravado na pasta sincronizada do OneDrive: {nome}", flush=True)
    return pid
```

```
def _ponte_valor(v: Any) -> Any:
    if isinstance(v, dict):
        return v.get("Value") if "Value" in v else ""
    return "" if v is None else v


def _ponte_esperar_lista(competencia: date, pid: str, minutos: int = 20) ->
list[dict[str, Any]]:
    import time

    nome = f"lista_{competencia.strftime('%Y_%m')}__{pid}.json"
    try:
        arquivo: Path | None = _pasta_ponte() / "da_lista" / nome
    except RuntimeError:
        arquivo = None
    online = _ponte_online() is not None
    print(f"Aguardando o fluxo SOx Ponte responder (normal: 1 a 5 min, máximo
{minutos} min)...", flush=True)
    inicio = time.time()
    limite = inicio + minutos * 60
    avisos = {
        120: (
            "Ainda aguardando. Confira se o fluxo SOx Ponte está On no Power
Automate."
            if online else
            "Ainda aguardando. Confira se o OneDrive está sincronizando (nuvem azul
perto do relógio)."
        ),
        300: "Demorando mais que o normal. Veja no Power Automate: SOx Ponte → 28-day
run history.",
    }
    proximo = 30
    while time.time() < limite:
        decorrido = int(time.time() - inicio)
        if decorrido >= proximo:
            print(f"  ... {decorrido // 60} min {decorrido % 60:02d} s esperando o
fluxo", flush=True)
            for seg, msg in avisos.items():
                if proximo - 30 < seg <= proximo:
                    print(f"  {msg}", flush=True)
            proximo += 30
        texto = None
        if arquivo is not None and arquivo.is_file():
            time.sleep(3)
            try:
                texto = arquivo.read_text(encoding="utf-8-sig")
            except OSError:
                texto = ""
        elif online:
            baixado = _ponte_ler_online("da_lista", nome)
            if baixado is not None:
                texto = baixado.decode("utf-8-sig")
        if texto is not None:
            try:
                bruto = json.loads(texto)
            except Exception:
                time.sleep(5)
                continue
            if isinstance(bruto, dict):
                bruto = bruto.get("value") or []
            itens = [{k: _ponte_valor(v) for k, v in it.items()} for it in bruto]
```

```
            _PONTE_CACHE[competencia.strftime("%m/%Y")] = (time.time(), itens)
            print(f"Fluxo respondeu: {len(itens)} contrato(s).", flush=True)
            return itens
        time.sleep(10)
    raise TimeoutError(
        "O fluxo SOx Ponte não respondeu a tempo. Confira se o OneDrive está
sincronizando "
        "(nuvem azul na barra do Windows) e se o fluxo está Ativado no Power Automate
"
        "(Meus fluxos → SOx Ponte → Histórico de execuções)."
    )


def _ponte_puxar(competencia: date) -> list[dict[str, Any]]:
    import time

    cache = _PONTE_CACHE.get(competencia.strftime("%m/%Y"))
    if cache and time.time() - cache[0] < 180:
        return cache[1]
    pid = _ponte_pedido("puxar", competencia)
    return _ponte_esperar_lista(competencia, pid)


def publicar_pela_ponte(caminho_planilha: Path, competencia: date) -> dict[str, Any]:
    cfg = garantir_config_nuvem()
    site, lista = _site_e_lista(cfg)
    exportar_staging_nuvem(caminho_planilha, competencia)
    linhas = coletar_linhas_nuvem(caminho_planilha, competencia)
    itens = []
    for linha in linhas:
        chave = linha.get("ChaveUnica") or ""
        if not chave:
            continue
        if len(linha.get("Observacao", "")) > 4000:
            linha["Observacao"] = linha["Observacao"][:4000] + "..."
        if len(linha.get("Objeto", "")) > 2000:
            linha["Objeto"] = linha["Objeto"][:2000]
        novo = _sp_payload_item("SP.Data", linha, None)
        novo.pop("__metadata", None)
        atualiza = {k: v for k, v in novo.items() if k not in _PONTE_DO_APP}
        itens.append({"chave": chave, "novo": novo, "atualiza": atualiza})
    pid = _ponte_pedido("publicar", competencia, colunas=_ponte_colunas(lista),
itens=itens)
    url = cfg.get("lista_url") or f"{site}/Lists/{lista}"
    try:
        na_lista = _ponte_esperar_lista(competencia, pid)
    except TimeoutError as exc:
        print(f"ATENÇÃO: {exc}", flush=True)
        return {
            "criados": 0, "atualizados": 0, "falhas": len(itens), "lista_url": url,
            "mensagem": (
                f"O pedido foi enviado ({len(itens)} contratos), mas o fluxo SOx Ponte
não respondeu em 20 minutos.\n\n"
                "Confira:\n1. O OneDrive está sincronizando (nuvem azul perto do
relógio).\n"
                "2. O fluxo SOx Ponte está ligado (On).\n"
                "3. Power Automate → SOx Ponte → 28-day run history: a execução
falhou?\n\n"
                "Se o fluxo terminar depois, os contratos aparecem na lista sozinhos."
            ),
        }
    chaves = {str(it.get("ChaveUnica") or "") for it in na_lista}
    faltando = [it["chave"] for it in itens if it["chave"] not in chaves]
```

```
    print(f"Pronto: {len(itens) - len(faltando)} de {len(itens)} contratos estão na
lista online.", flush=True)
    if faltando:
        print("Não entraram: " + ", ".join(faltando[:10]), flush=True)
    apagados, nao_apagados = _ponte_apagar_sobras(competencia, itens, na_lista)
    return {
        "criados": len(itens) - len(faltando),
        "atualizados": 0,
        "falhas": len(faltando),
        "lista_url": url,
        "mensagem": (
            ("Publicado na lista online.\n\n" if not faltando else "Publicado, mas com
falhas.\n\n")
            + f"Na lista: {len(itens) - len(faltando)} de {len(itens)} contratos de
{competencia.strftime('%m/%Y')}.\n"
            + (f"Não entraram: {len(faltando)} (veja a lista na janela do programa e o
histórico do fluxo).\n" if faltando else "")
            + (f"Apagados da lista (não estão mais na planilha): {apagados}.\n" if
apagados else "")
            + (f"Não consegui apagar {nao_apagados} contrato(s) que saíram da
planilha. Veja a janela do programa.\n" if nao_apagados else "")
            + "\nO que o pessoal já respondeu no app não é apagado."
        ),
    }


def _ponte_apagar_sobras(competencia: date, itens: list[dict[str, Any]], na_lista:
list[dict[str, Any]]) -> tuple[int, int]:
    # A lista do mês fica igual à planilha: some o contrato que saiu dela.
    novas = {it["chave"] for it in itens}
    sobras = [it for it in na_lista if str(it.get("ChaveUnica") or "") not in novas]
    ids = [int(it["Id"]) for it in sobras if str(it.get("Id") or "").isdigit()]
    if not ids or not itens:
        return 0, 0
    if len(ids) > max(10, len(na_lista) // 2):
        print(
            f"ATENÇÃO: {len(ids)} contratos da lista de
{competencia.strftime('%m/%Y')} não estão na planilha. "
            "São muitos; não apaguei nada. Confira se a competência está certa.",
            flush=True,
        )
        return 0, len(ids)
    nomes = ", ".join(str(it.get("Title") or it.get("Contrato") or it.get("Id")) for
it in sobras[:10])
    print(f"Apagando da lista {len(ids)} contrato(s) que não estão mais na planilha:
{nomes}", flush=True)
    pid = _ponte_pedido("apagar", competencia, ids=ids)
    try:
        depois = _ponte_esperar_lista(competencia, pid)
    except TimeoutError as exc:
        print(f"ATENÇÃO: {exc}", flush=True)
        return 0, len(ids)
    restam = {str(it.get("Id")) for it in depois} & {str(i) for i in ids}
    if restam:
        print(
            f"Não consegui apagar {len(restam)}. Confira se o fluxo SOx Ponte tem o
Caso 4 (apagar), etapa A10 do roteiro.",
            flush=True,
        )
    else:
        print(f"Apagados: {len(ids)}.", flush=True)
    return len(ids) - len(restam), len(restam)
```

```
def _fechar_pela_ponte(competencia: date, por: str) -> dict[str, Any]:
    chave = competencia.strftime("%m/%Y")
    ciclos = _carregar_ciclos_local()
    ciclos[chave] = {
        "fechada": "SIM",
        "fechado_em": datetime.now().strftime("%d/%m/%Y %H:%M"),
        "fechado_por": por,
    }
    _salvar_ciclos_local(ciclos)
    print(f"Fechando {chave}: o fluxo marca MesFechado=SIM e deixa só leitura...",
flush=True)
    pid = _ponte_pedido("fechar", competencia)
    itens = _ponte_esperar_lista(competencia, pid, minutos=30)
    puxado = puxar_lista_sharepoint(competencia)
    copia = _guardar_copia_fechamento(
        competencia, [Path(puxado.get("arquivo_controle") or ""), RAIZ /
nome_arquivo(competencia)]
    )
    print(f"Cópia de controle (somente leitura): {copia}", flush=True)
    resumo = {
        "acao": "fechar",
        "competencia": chave,
        "fechado_por": por,
        "itens": {"total": len(itens)},
        "arquivo_controle": puxado.get("arquivo_controle"),
        "trava": {"modo": "fluxo SOx Ponte", "itens": len(itens)},
        "copia_controle": str(copia),
        "regra": "Depois do fechamento, só o grupo de Proprietários do site edita a
lista.",
    }
    (PASTA_NUVEM / f"SOx_fechamento_{competencia.strftime('%Y_%m')}.json").write_text(
        json.dumps(resumo, ensure_ascii=False, indent=2), encoding="utf-8"
    )
    print(f"Competência {chave} FECHADA.", flush=True)
    return resumo


def _reabrir_pela_ponte(competencia: date, por: str) -> dict[str, Any]:
    chave = competencia.strftime("%m/%Y")
    ciclos = _carregar_ciclos_local()
    ciclos[chave] = {
        "fechada": "NÃO",
        "reaberto_em": datetime.now().strftime("%d/%m/%Y %H:%M"),
        "reaberto_por": por,
    }
    _salvar_ciclos_local(ciclos)
    pid = _ponte_pedido("reabrir", competencia)
    itens = _ponte_esperar_lista(competencia, pid, minutos=30)
    print(f"Competência {chave} REABERTA.", flush=True)
    return {"acao": "reabrir", "competencia": chave, "reaberto_por": por, "itens":
{"total": len(itens)}}


def publicar_lista_sharepoint(
```

---

#### 22. Ponte: pedido vai direto ao OneDrive online

**Onde:** Bloco da ponte (atualiza a versão anterior, se você já tinha).
**Por quê:** O pedido sobe direto para SOx_SCM42_ponte no OneDrive online, com os cookies SSO do Sigma-C/certidões, e a resposta é lida online. Não depende mais da sincronização do

OneDrive. Sem cookies, usa a pasta sincronizada como antes.
**O que muda:** Troca o trecho Antes pelo Depois.

1 · Procure (Ctrl + F)

```
+ (f"Não entraram: {len(faltando)} (veja a lista na janela do programa e o histórico
do fluxo).\n" if faltando else "")
```

2 · Antes (selecione isto no seu arquivo)

```
# ------------------------------------------------------------- ponte Power
Automate
# A Microsoft bloqueou o login direto do Python no SharePoint (erro AADSTS65002).
# O Python agora grava pedidos numa pasta do OneDrive; o fluxo "SOx Ponte" faz o
# trabalho no SharePoint com o login do próprio usuário e devolve a lista num arquivo.
_PONTE_CACHE: dict[str, tuple[float, list[dict[str, Any]]]] = {}
_PONTE_ESCOLHAS = {
    "HouveServico": ["SIM", "NÃO"],
    "FoiMedido": ["SIM", "NÃO"],
    "ForaDoPrazo": ["SIM", "NÃO"],
    "SeraSubstituido": ["SIM", "NÃO"],
    "DemandaContinuada": ["SIM", "NÃO"],
    "MotivoProvisao": [
        "Atraso do Fornecedor", "Atraso Fiscalização de Campo", "Problema Interno
GPS",
        "Contrato Bloqueado", "Outros",
    ],
}
_PONTE_LONGOS = {
    "Objeto", "Provisao", "Justificativa", "Observacao", "CSP", "FRSPendentes",
    "MotivoIntervencao", "ComentarioErro", "ComentarioGerente",
}
_PONTE_EXTRAS = [
    "OQueFazer", "DadosCorretos", "CampoErro", "ValorCorreto", "AprovacaoGerente",
    "MesFechado", "ComentarioErro", "ComentarioGerente", "OkFiscal", "OkFiscalPor",
    "OkFiscalEm", "AprovacaoPor", "AprovacaoEm",
]
# Campos que o pessoal preenche no app: republicar não sobrescreve.
_PONTE_DO_APP = set(_CAMPOS_MANUAIS_NUVEM) | {
    "Observacao", "OkFiscal", "OkFiscalPor", "OkFiscalEm", "AprovacaoGerente",
    "AprovacaoPor", "AprovacaoEm", "ComentarioGerente", "MesFechado",
}


def _usar_ponte() -> bool:
    return str(garantir_config_nuvem().get("modo_online") or "ponte").lower() ==
"ponte"


def _pasta_ponte() -> Path:
    cfg = garantir_config_nuvem()
    base = str(cfg.get("pasta_ponte") or "").strip()
    if not base:
        onedrive = os.environ.get("OneDriveCommercial") or os.environ.get("OneDrive")
or ""
        if not onedrive:
            raise RuntimeError(
                "Não achei a pasta do OneDrive neste computador. Abra o OneDrive,
entre com a "
                "conta da Petrobras, ou escreva o caminho em pasta_ponte no
sox_nuvem_config.json."
            )
        base = str(Path(onedrive) / "SOx_SCM42_ponte")
    pasta = Path(base)
    for sub in ("pedidos", "da_lista"):
        (pasta / sub).mkdir(parents=True, exist_ok=True)
    return pasta


def _ponte_colunas(lista: str) -> list[dict[str, Any]]:
    nomes = list(dict.fromkeys(list(CABECALHO_STAGING_NUVEM) + _PONTE_EXTRAS))
```

```
    base = f"_api/web/lists/getbytitle('{lista}')"
    saida = []
    for nome in nomes:
        if nome in _PONTE_ESCOLHAS:
            opcoes = "".join(f"<CHOICE>{o}</CHOICE>" for o in _PONTE_ESCOLHAS[nome])
            xml = (
                f"<Field Type='Choice' DisplayName='{nome}' Name='{nome}'
StaticName='{nome}' "
                f"FillInChoice='TRUE' Format='Dropdown'><CHOICES>{opcoes}</CHOICES>
</Field>"
            )
        elif nome in _PONTE_LONGOS:
            xml = (
                f"<Field Type='Note' DisplayName='{nome}' Name='{nome}'
StaticName='{nome}' "
                f"RichText='FALSE' NumLines='6' />"
            )
        else:
            xml = f"<Field Type='Text' DisplayName='{nome}' Name='{nome}'
StaticName='{nome}' MaxLength='255' />"
        saida.append({
            "nome": nome,
            "uri_existe": f"{base}/fields?$select=Id&$filter=" + quote(f"InternalName
eq '{nome}'"),
            "corpo": {"parameters": {"SchemaXml": xml, "Options": 24}},
        })
    return saida


def _ponte_pedido(acao: str, competencia: date, **dados: Any) -> str:
    import uuid

    pid = datetime.now().strftime("%Y%m%d%H%M%S") + uuid.uuid4().hex[:6]
    cfg = garantir_config_nuvem()
    lista = str(cfg.get("lista_contratos") or "SOx_SCM42_Contratos")
    comp = competencia.strftime("%m/%Y")
    base = f"_api/web/lists/getbytitle('{lista}')"
    filtro = quote(f"Competencia eq '{comp}'")
    # Endereços prontos: no fluxo cada caixa só lê outputs('Pedido')?['...'].
    pedido = {
        "acao": acao,
        "id": pid,
        "competencia": comp,
        "arquivo_comp": competencia.strftime("%Y_%m"),
        "lista": lista,
        "uri_lista_existe": "_api/web/lists?$select=Id&$filter=" + quote(f"Title eq
'{lista}'"),
        "uri_criar_lista": "_api/web/lists",
        "corpo_criar_lista": {
            "Title": lista,
            "BaseTemplate": 100,
            "Description": "Controle SOx SCM-42: um contrato por linha e por
competência.",
        },
        "uri_criar_coluna": f"{base}/fields/createfieldasxml",
        "uri_indexar": f"{base}/fields/getbyinternalnameortitle('Competencia')",
        "uri_itens": f"{base}/items",
        "uri_existentes": f"{base}/items?$select=Id,ChaveUnica&$top=5000&$filter=
{filtro}",
        "uri_ids": f"{base}/items?$select=Id&$top=5000&$filter={filtro}",
        "uri_lista": f"{base}/items?$top=5000&$filter={filtro}",
        "uri_grupos": (
            "_api/web?$select=AssociatedOwnerGroup/Id,AssociatedMemberGroup/Id,"
```

```
"AssociatedVisitorGroup/Id&$expand=AssociatedOwnerGroup,AssociatedMemberGroup,"
            "AssociatedVisitorGroup"
        ),
        **dados,
    }
    destino = _pasta_ponte() / "pedidos" / f"
{acao}__{competencia.strftime('%Y_%m')}__{pid}.json"
    destino.write_text(json.dumps(pedido, ensure_ascii=True), encoding="utf-8")
    print(f"Pedido enviado para o fluxo SOx Ponte: {destino.name}", flush=True)
    return pid


def _ponte_valor(v: Any) -> Any:
    if isinstance(v, dict):
        return v.get("Value") if "Value" in v else ""
    return "" if v is None else v


def _ponte_esperar_lista(competencia: date, pid: str, minutos: int = 20) ->
list[dict[str, Any]]:
    import time

    arquivo = _pasta_ponte() / "da_lista" /
f"lista_{competencia.strftime('%Y_%m')}__{pid}.json"
    print(f"Aguardando o fluxo SOx Ponte responder (normal: 1 a 5 min, máximo
{minutos} min)...", flush=True)
    inicio = time.time()
    limite = inicio + minutos * 60
    avisos = {
        120: "Ainda aguardando. Confira se o OneDrive está sincronizando (nuvem azul
perto do relógio).",
        300: "Demorando mais que o normal. Veja no Power Automate: SOx Ponte → 28-day
run history.",
    }
    proximo = 30
    while time.time() < limite:
        decorrido = int(time.time() - inicio)
        if decorrido >= proximo:
            print(f"  ... {decorrido // 60} min {decorrido % 60:02d} s esperando o
fluxo", flush=True)
            for seg, msg in avisos.items():
                if proximo - 30 < seg <= proximo:
                    print(f"  {msg}", flush=True)
            proximo += 30
        if arquivo.is_file():
            time.sleep(3)
            try:
                bruto = json.loads(arquivo.read_text(encoding="utf-8-sig"))
            except Exception:
                time.sleep(5)
                continue
            if isinstance(bruto, dict):
                bruto = bruto.get("value") or []
            itens = [{k: _ponte_valor(v) for k, v in it.items()} for it in bruto]
            _PONTE_CACHE[competencia.strftime("%m/%Y")] = (time.time(), itens)
            print(f"Fluxo respondeu: {len(itens)} contrato(s).", flush=True)
            return itens
        time.sleep(10)
    raise TimeoutError(
        "O fluxo SOx Ponte não respondeu a tempo. Confira se o OneDrive está
sincronizando "
        "(nuvem azul na barra do Windows) e se o fluxo está Ativado no Power Automate
```

```
"
        "(Meus fluxos → SOx Ponte → Histórico de execuções)."
    )


def _ponte_puxar(competencia: date) -> list[dict[str, Any]]:
    import time

    cache = _PONTE_CACHE.get(competencia.strftime("%m/%Y"))
    if cache and time.time() - cache[0] < 180:
        return cache[1]
    pid = _ponte_pedido("puxar", competencia)
    return _ponte_esperar_lista(competencia, pid)


def publicar_pela_ponte(caminho_planilha: Path, competencia: date) -> dict[str, Any]:
    cfg = garantir_config_nuvem()
    site, lista = _site_e_lista(cfg)
    exportar_staging_nuvem(caminho_planilha, competencia)
    linhas = coletar_linhas_nuvem(caminho_planilha, competencia)
    itens = []
    for linha in linhas:
        chave = linha.get("ChaveUnica") or ""
        if not chave:
            continue
        if len(linha.get("Observacao", "")) > 4000:
            linha["Observacao"] = linha["Observacao"][:4000] + "..."
        if len(linha.get("Objeto", "")) > 2000:
            linha["Objeto"] = linha["Objeto"][:2000]
        novo = _sp_payload_item("SP.Data", linha, None)
        novo.pop("__metadata", None)
        atualiza = {k: v for k, v in novo.items() if k not in _PONTE_DO_APP}
        itens.append({"chave": chave, "novo": novo, "atualiza": atualiza})
    pid = _ponte_pedido("publicar", competencia, colunas=_ponte_colunas(lista),
itens=itens)
    url = cfg.get("lista_url") or f"{site}/Lists/{lista}"
    try:
        na_lista = _ponte_esperar_lista(competencia, pid)
    except TimeoutError as exc:
        print(f"ATENÇÃO: {exc}", flush=True)
        return {
            "criados": 0, "atualizados": 0, "falhas": len(itens), "lista_url": url,
            "mensagem": (
                f"O pedido foi enviado ({len(itens)} contratos), mas o fluxo SOx Ponte
não respondeu em 20 minutos.\n\n"
                "Confira:\n1. O OneDrive está sincronizando (nuvem azul perto do
relógio).\n"
                "2. O fluxo SOx Ponte está ligado (On).\n"
                "3. Power Automate → SOx Ponte → 28-day run history: a execução
falhou?\n\n"
                "Se o fluxo terminar depois, os contratos aparecem na lista sozinhos."
            ),
        }
    chaves = {str(it.get("ChaveUnica") or "") for it in na_lista}
    faltando = [it["chave"] for it in itens if it["chave"] not in chaves]
    print(f"Pronto: {len(itens) - len(faltando)} de {len(itens)} contratos estão na
lista online.", flush=True)
    if faltando:
        print("Não entraram: " + ", ".join(faltando[:10]), flush=True)
    return {
        "criados": len(itens) - len(faltando),
        "atualizados": 0,
        "falhas": len(faltando),
```

```
        "lista_url": url,
        "mensagem": (
            ("Publicado na lista online.\n\n" if not faltando else "Publicado, mas com
falhas.\n\n")
            + f"Na lista: {len(itens) - len(faltando)} de {len(itens)} contratos de
{competencia.strftime('%m/%Y')}.\n"
            + (f"Não entraram: {len(faltando)} (veja a lista na janela do programa e o
histórico do fluxo).\n" if faltando else "")
            + "\nO que o pessoal já respondeu no app não é apagado."
        ),
    }


def _fechar_pela_ponte(competencia: date, por: str) -> dict[str, Any]:
    chave = competencia.strftime("%m/%Y")
    ciclos = _carregar_ciclos_local()
    ciclos[chave] = {
        "fechada": "SIM",
        "fechado_em": datetime.now().strftime("%d/%m/%Y %H:%M"),
        "fechado_por": por,
    }
    _salvar_ciclos_local(ciclos)
    print(f"Fechando {chave}: o fluxo marca MesFechado=SIM e deixa só leitura...",
flush=True)
    pid = _ponte_pedido("fechar", competencia)
    itens = _ponte_esperar_lista(competencia, pid, minutos=30)
    puxado = puxar_lista_sharepoint(competencia)
    copia = _guardar_copia_fechamento(
        competencia, [Path(puxado.get("arquivo_controle") or ""), RAIZ /
nome_arquivo(competencia)]
    )
    print(f"Cópia de controle (somente leitura): {copia}", flush=True)
    resumo = {
        "acao": "fechar",
        "competencia": chave,
        "fechado_por": por,
        "itens": {"total": len(itens)},
        "arquivo_controle": puxado.get("arquivo_controle"),
        "trava": {"modo": "fluxo SOx Ponte", "itens": len(itens)},
        "copia_controle": str(copia),
        "regra": "Depois do fechamento, só o grupo de Proprietários do site edita a
lista.",
    }
    (PASTA_NUVEM / f"SOx_fechamento_{competencia.strftime('%Y_%m')}.json").write_text(
        json.dumps(resumo, ensure_ascii=False, indent=2), encoding="utf-8"
    )
    print(f"Competência {chave} FECHADA.", flush=True)
    return resumo


def _reabrir_pela_ponte(competencia: date, por: str) -> dict[str, Any]:
    chave = competencia.strftime("%m/%Y")
    ciclos = _carregar_ciclos_local()
    ciclos[chave] = {
        "fechada": "NÃO",
        "reaberto_em": datetime.now().strftime("%d/%m/%Y %H:%M"),
        "reaberto_por": por,
    }
    _salvar_ciclos_local(ciclos)
    pid = _ponte_pedido("reabrir", competencia)
    itens = _ponte_esperar_lista(competencia, pid, minutos=30)
    print(f"Competência {chave} REABERTA.", flush=True)
    return {"acao": "reabrir", "competencia": chave, "reaberto_por": por, "itens":
```

```
{"total": len(itens)}


def publicar_lista_sharepoint(
```

3 · Depois (cole por cima)

```
# ---------------------------------------------------------------- ponte Power
Automate
# A Microsoft bloqueou o login direto do Python no SharePoint (erro AADSTS65002).
# O Python agora grava pedidos numa pasta do OneDrive; o fluxo "SOx Ponte" faz o
# trabalho no SharePoint com o login do próprio usuário e devolve a lista num arquivo.
# O pedido vai direto ao OneDrive online (cookies SSO, como no Sigma-C); sem cookies,
# usa a pasta sincronizada deste computador.
_PONTE_CACHE: dict[str, tuple[float, list[dict[str, Any]]]] = {}
_PONTE_ONLINE: dict[str, Any] = {}
_PONTE_SITE = "https://petrobrasbr-
my.sharepoint.com/personal/levi_vasconcelos_petrobras_com_br"
_PONTE_PASTA = "/personal/levi_vasconcelos_petrobras_com_br/Documents/SOx_SCM42_ponte"
_PONTE_ESCOLHAS = {
    "HouveServico": ["SIM", "NÃO"],
    "FoiMedido": ["SIM", "NÃO"],
    "ForaDoPrazo": ["SIM", "NÃO"],
    "SeraSubstituido": ["SIM", "NÃO"],
    "DemandaContinuada": ["SIM", "NÃO"],
    "MotivoProvisao": [
        "Atraso do Fornecedor", "Atraso Fiscalização de Campo", "Problema Interno
GPS",
        "Contrato Bloqueado", "Outros",
    ],
}
_PONTE_LONGOS = {
    "Objeto", "Provisao", "Justificativa", "Observacao", "CSP", "FRSPendentes",
    "MotivoIntervencao", "ComentarioErro", "ComentarioGerente",
}
_PONTE_EXTRAS = [
    "OQueFazer", "DadosCorretos", "CampoErro", "ValorCorreto", "AprovacaoGerente",
    "MesFechado", "ComentarioErro", "ComentarioGerente", "OkFiscal", "OkFiscalPor",
    "OkFiscalEm", "AprovacaoPor", "AprovacaoEm",
]
# Campos que o pessoal preenche no app: republicar não sobrescreve.
_PONTE_DO_APP = set(_CAMPOS_MANUAIS_NUVEM) | {
    "Observacao", "OkFiscal", "OkFiscalPor", "OkFiscalEm", "AprovacaoGerente",
    "AprovacaoPor", "AprovacaoEm", "ComentarioGerente", "MesFechado",
}


def _usar_ponte() -> bool:
    return str(garantir_config_nuvem().get("modo_online") or "ponte").lower() ==
"ponte"


def _pasta_ponte() -> Path:
    cfg = garantir_config_nuvem()
    base = str(cfg.get("pasta_ponte") or "").strip()
    if not base:
        onedrive = os.environ.get("OneDriveCommercial") or os.environ.get("OneDrive")
or ""
        if not onedrive:
            raise RuntimeError(
                "Não achei a pasta do OneDrive neste computador. Abra o OneDrive,
entre com a "
                "conta da Petrobras, ou escreva o caminho em pasta_ponte no
sox_nuvem_config.json."
            )
        base = str(Path(onedrive) / "SOx_SCM42_ponte")
    pasta = Path(base)
    for sub in ("pedidos", "da_lista"):
```

```
        (pasta / sub).mkdir(parents=True, exist_ok=True)
    return pasta


def _ponte_cookies(host: str) -> str:
    # Mesmos cookies SSO que o Sigma-C e a automação de certidões já guardam.
    cfg = garantir_config_nuvem()
    pasta_cert = Path.home() / "Downloads" / "automatizacao certidao"
    candidatos = [Path(str(cfg.get("ponte_cookies")))] if cfg.get("ponte_cookies")
else []
    candidatos += [
        RAIZ / ".sp_cookies.json",
        pasta_cert / ".sp_cookies.json",
        pasta_cert / "automatizacao_certidoes.db",
        pasta_cert / "certidoes.db",
    ]
    for caminho in candidatos:
        try:
            if not caminho.is_file():
                continue
            if caminho.suffix.lower() == ".json":
                dados = json.loads(caminho.read_text(encoding="utf-8",
errors="replace"))
            else:
                import sqlite3

                con = sqlite3.connect(str(caminho))
                try:
                    linha = con.execute("SELECT cookies FROM sessao WHERE
id=1").fetchone()
                finally:
                    con.close()
                dados = json.loads(linha[0]) if linha and linha[0] else None
        except Exception:
            continue
        if isinstance(dados, str) and dados.strip():
            return dados.strip()
        if isinstance(dados, dict):
            dados = dados.get("cookies") or [{"name": k, "value": v} for k, v in
dados.items()]
        if not isinstance(dados, list):
            continue
        todos = [c for c in dados if isinstance(c, dict) and c.get("name")]

        def do_host(c: dict[str, Any]) -> bool:
            dominio = str(c.get("domain") or "").lstrip(".").lower()
            return not dominio or host == dominio or host.endswith("." + dominio)

        escolhidos = [c for c in todos if do_host(c)] or todos
        vistos: dict[str, str] = {}
        for c in escolhidos:
            vistos.setdefault(str(c["name"]), str(c.get("value", "")))
        if vistos:
            print(f"Cookies SSO: {caminho}", flush=True)
            return "; ".join(f"{k}={v}" for k, v in vistos.items())
    return ""


def _ponte_http(metodo: str, url: str, cookies: str, corpo: bytes | None = None,
                digest: str = "", tipo: str = "application/octet-stream") ->
tuple[int, bytes]:
    import ssl
    import urllib.error
```

```
    import urllib.request
    from urllib.parse import urlparse

    cab = {"Accept": "application/json;odata=verbose", "Cookie": cookies, "User-
Agent": "Mozilla/5.0"}
    if digest:
        cab["X-RequestDigest"] = digest
    if corpo is not None:
        cab["Content-Type"] = tipo
    req = urllib.request.Request(url, data=corpo, headers=cab, method=metodo)
    try:
        with urllib.request.urlopen(req, context=ssl.create_default_context(),
timeout=60) as resp:
            # Cookie vencido: o SharePoint manda para a tela de login da Microsoft.
            if urlparse(resp.geturl()).netloc.lower() != urlparse(url).netloc.lower():
                return 401, b"login expirado"
            return resp.status, resp.read()
    except urllib.error.HTTPError as exc:
        return exc.code, exc.read()


def _ponte_online() -> dict[str, str] | None:
    import time
    from urllib.parse import urlparse

    if time.time() - _PONTE_ONLINE.get("em", 0) < 1200:
        return _PONTE_ONLINE.get("dados")
    cfg = garantir_config_nuvem()
    dados = None
    if str(cfg.get("ponte_envio") or "online").lower() != "pasta":
        site = str(cfg.get("ponte_site") or _PONTE_SITE).rstrip("/")
        cookies = _ponte_cookies((urlparse(site).hostname or "").lower())
        if not cookies:
            print("Sem cookies SSO salvos (Sigma-C/certidões): uso a pasta
sincronizada do OneDrive.", flush=True)
        else:
            try:
                status, corpo = _ponte_http(
                    "POST", f"{site}/_api/contextinfo", cookies, b"",
tipo="application/json;odata=verbose"
                )
                info = json.loads(corpo.decode("utf-8")) if status == 200 else {}
                digest = (info.get("d") or {}).get("GetContextWebInformation",
{}).get("FormDigestValue", "")
            except Exception as exc:
                status, digest = str(exc), ""
            if digest:
                dados = {
                    "site": site,
                    "pasta": str(cfg.get("ponte_pasta_servidor") or
_PONTE_PASTA).rstrip("/"),
                    "cookies": cookies,
                    "digest": digest,
                }
            else:
                print(
                    f"OneDrive online não aceitou os cookies ({status}). Abra o Sigma-
C ou as certidões "
                    "para renovar o login. Por agora uso a pasta sincronizada.",
                    flush=True,
                )
    _PONTE_ONLINE.update(em=time.time(), dados=dados)
    return dados
```

```
def _ponte_subir_online(sub: str, nome: str, corpo: bytes) -> bool:
    from urllib.parse import quote as q, urlencode

    on = _ponte_online()
    if not on:
        return False
    pasta = f"{on['pasta']}/{sub}"
    url = (
        f"
{on['site']}/_api/web/GetFolderByServerRelativeUrl(@p)/Files/add(url=@n,overwrite=true
)?"
        + urlencode({"@p": f"'{pasta}'", "@n": f"'{nome}'"})
    )
    status, resposta = _ponte_http("POST", url, on["cookies"], corpo, on["digest"])
    if status == 404:  # pasta ainda não existe no OneDrive online
        _ponte_http("POST", f"{on['site']}/_api/web/folders/add('{q(pasta)}')",
on["cookies"], b"", on["digest"])
        status, resposta = _ponte_http("POST", url, on["cookies"], corpo,
on["digest"])
    if status in (200, 201):
        return True
    trecho = resposta.decode("utf-8", errors="replace")[:200]
    print(f"Envio online falhou (HTTP {status}): {trecho}", flush=True)
    _PONTE_ONLINE.clear()
    return False


def _ponte_ler_online(sub: str, nome: str) -> bytes | None:
    from urllib.parse import urlencode

    on = _ponte_online()
    if not on:
        return None
    url = (
        f"{on['site']}/_api/web/GetFileByServerRelativeUrl(@f)/$value?"
        + urlencode({"@f": f"'{on['pasta']}/{sub}/{nome}'"})
    )
    try:
        status, corpo = _ponte_http("GET", url, on["cookies"])
    except Exception:
        return None
    return corpo if status == 200 else None


def _ponte_colunas(lista: str) -> list[dict[str, Any]]:
    nomes = list(dict.fromkeys(list(CABECALHO_STAGING_NUVEM) + _PONTE_EXTRAS))
    base = f"_api/web/lists/getbytitle('{lista}')"
    saida = []
    for nome in nomes:
        if nome in _PONTE_ESCOLHAS:
            opcoes = "".join(f"<CHOICE>{o}</CHOICE>" for o in _PONTE_ESCOLHAS[nome])
            xml = (
                f"<Field Type='Choice' DisplayName='{nome}' Name='{nome}'
StaticName='{nome}' "
                f"FillInChoice='TRUE' Format='Dropdown'><CHOICES>{opcoes}</CHOICES>
</Field>"
            )
        elif nome in _PONTE_LONGOS:
            xml = (
                f"<Field Type='Note' DisplayName='{nome}' Name='{nome}'
StaticName='{nome}' "
```

```
                f"RichText='FALSE' NumLines='6' />"
            )
        else:
            xml = f"<Field Type='Text' DisplayName='{nome}' Name='{nome}'
StaticName='{nome}' MaxLength='255' />"
        saida.append({
            "nome": nome,
            "uri_existe": f"{base}/fields?$select=Id&$filter=" + quote(f"InternalName
eq '{nome}'"),
            "corpo": {"parameters": {"SchemaXml": xml, "Options": 24}},
        })
    return saida


def _ponte_pedido(acao: str, competencia: date, **dados: Any) -> str:
    import uuid

    pid = datetime.now().strftime("%Y%m%d%H%M%S") + uuid.uuid4().hex[:6]
    cfg = garantir_config_nuvem()
    lista = str(cfg.get("lista_contratos") or "SOx_SCM42_Contratos")
    comp = competencia.strftime("%m/%Y")
    base = f"_api/web/lists/getbytitle('{lista}')"
    filtro = quote(f"Competencia eq '{comp}'")
    # Endereços prontos: no fluxo cada caixa só lê outputs('Pedido')?['...'].
    pedido = {
        "acao": acao,
        "id": pid,
        "competencia": comp,
        "arquivo_comp": competencia.strftime("%Y_%m"),
        "lista": lista,
        "uri_lista_existe": "_api/web/lists?$select=Id&$filter=" + quote(f"Title eq
'{lista}'"),
        "uri_criar_lista": "_api/web/lists",
        "corpo_criar_lista": {
            "Title": lista,
            "BaseTemplate": 100,
            "Description": "Controle SOx SCM-42: um contrato por linha e por
competência.",
        },
        "uri_criar_coluna": f"{base}/fields/createfieldasxml",
        "uri_indexar": f"{base}/fields/getbyinternalnameortitle('Competencia')",
        "uri_itens": f"{base}/items",
        "uri_existentes": f"{base}/items?$select=Id,ChaveUnica&$top=5000&$filter=
{filtro}",
        "uri_ids": f"{base}/items?$select=Id&$top=5000&$filter={filtro}",
        "uri_lista": f"{base}/items?$top=5000&$filter={filtro}",
        "uri_grupos": (
            "_api/web?$select=AssociatedOwnerGroup/Id,AssociatedMemberGroup/Id,"

"AssociatedVisitorGroup/Id&$expand=AssociatedOwnerGroup,AssociatedMemberGroup,"
            "AssociatedVisitorGroup"
        ),
        **dados,
    }
    nome = f"{acao}__{competencia.strftime('%Y_%m')}__{pid}.json"
    corpo = json.dumps(pedido, ensure_ascii=True).encode("utf-8")
    if _ponte_subir_online("pedidos", nome, corpo):
        print(f"Pedido enviado direto ao OneDrive online (não depende da
sincronização): {nome}", flush=True)
        return pid
    (_pasta_ponte() / "pedidos" / nome).write_bytes(corpo)
    print(f"Pedido gravado na pasta sincronizada do OneDrive: {nome}", flush=True)
    return pid
```

```
def _ponte_valor(v: Any) -> Any:
    if isinstance(v, dict):
        return v.get("Value") if "Value" in v else ""
    return "" if v is None else v


def _ponte_esperar_lista(competencia: date, pid: str, minutos: int = 20) ->
list[dict[str, Any]]:
    import time

    nome = f"lista_{competencia.strftime('%Y_%m')}__{pid}.json"
    try:
        arquivo: Path | None = _pasta_ponte() / "da_lista" / nome
    except RuntimeError:
        arquivo = None
    online = _ponte_online() is not None
    print(f"Aguardando o fluxo SOx Ponte responder (normal: 1 a 5 min, máximo
{minutos} min)...", flush=True)
    inicio = time.time()
    limite = inicio + minutos * 60
    avisos = {
        120: (
            "Ainda aguardando. Confira se o fluxo SOx Ponte está On no Power
Automate."
            if online else
            "Ainda aguardando. Confira se o OneDrive está sincronizando (nuvem azul
perto do relógio)."
        ),
        300: "Demorando mais que o normal. Veja no Power Automate: SOx Ponte → 28-day
run history.",
    }
    proximo = 30
    while time.time() < limite:
        decorrido = int(time.time() - inicio)
        if decorrido >= proximo:
            print(f"  ... {decorrido // 60} min {decorrido % 60:02d} s esperando o
fluxo", flush=True)
            for seg, msg in avisos.items():
                if proximo - 30 < seg <= proximo:
                    print(f"  {msg}", flush=True)
            proximo += 30
        texto = None
        if arquivo is not None and arquivo.is_file():
            time.sleep(3)
            try:
                texto = arquivo.read_text(encoding="utf-8-sig")
            except OSError:
                texto = ""
        elif online:
            baixado = _ponte_ler_online("da_lista", nome)
            if baixado is not None:
                texto = baixado.decode("utf-8-sig")
        if texto is not None:
            try:
                bruto = json.loads(texto)
            except Exception:
                time.sleep(5)
                continue
            if isinstance(bruto, dict):
                bruto = bruto.get("value") or []
            itens = [{k: _ponte_valor(v) for k, v in it.items()} for it in bruto]
```

```
            _PONTE_CACHE[competencia.strftime("%m/%Y")] = (time.time(), itens)
            print(f"Fluxo respondeu: {len(itens)} contrato(s).", flush=True)
            return itens
        time.sleep(10)
    raise TimeoutError(
        "O fluxo SOx Ponte não respondeu a tempo. Confira se o OneDrive está
sincronizando "
        "(nuvem azul na barra do Windows) e se o fluxo está Ativado no Power Automate
"
        "(Meus fluxos → SOx Ponte → Histórico de execuções)."
    )


def _ponte_puxar(competencia: date) -> list[dict[str, Any]]:
    import time

    cache = _PONTE_CACHE.get(competencia.strftime("%m/%Y"))
    if cache and time.time() - cache[0] < 180:
        return cache[1]
    pid = _ponte_pedido("puxar", competencia)
    return _ponte_esperar_lista(competencia, pid)


def publicar_pela_ponte(caminho_planilha: Path, competencia: date) -> dict[str, Any]:
    cfg = garantir_config_nuvem()
    site, lista = _site_e_lista(cfg)
    exportar_staging_nuvem(caminho_planilha, competencia)
    linhas = coletar_linhas_nuvem(caminho_planilha, competencia)
    itens = []
    for linha in linhas:
        chave = linha.get("ChaveUnica") or ""
        if not chave:
            continue
        if len(linha.get("Observacao", "")) > 4000:
            linha["Observacao"] = linha["Observacao"][:4000] + "..."
        if len(linha.get("Objeto", "")) > 2000:
            linha["Objeto"] = linha["Objeto"][:2000]
        novo = _sp_payload_item("SP.Data", linha, None)
        novo.pop("__metadata", None)
        atualiza = {k: v for k, v in novo.items() if k not in _PONTE_DO_APP}
        itens.append({"chave": chave, "novo": novo, "atualiza": atualiza})
    pid = _ponte_pedido("publicar", competencia, colunas=_ponte_colunas(lista),
itens=itens)
    url = cfg.get("lista_url") or f"{site}/Lists/{lista}"
    try:
        na_lista = _ponte_esperar_lista(competencia, pid)
    except TimeoutError as exc:
        print(f"ATENÇÃO: {exc}", flush=True)
        return {
            "criados": 0, "atualizados": 0, "falhas": len(itens), "lista_url": url,
            "mensagem": (
                f"O pedido foi enviado ({len(itens)} contratos), mas o fluxo SOx Ponte
não respondeu em 20 minutos.\n\n"
                "Confira:\n1. O OneDrive está sincronizando (nuvem azul perto do
relógio).\n"
                "2. O fluxo SOx Ponte está ligado (On).\n"
                "3. Power Automate → SOx Ponte → 28-day run history: a execução
falhou?\n\n"
                "Se o fluxo terminar depois, os contratos aparecem na lista sozinhos."
            ),
        }
    chaves = {str(it.get("ChaveUnica") or "") for it in na_lista}
    faltando = [it["chave"] for it in itens if it["chave"] not in chaves]
```

```
    print(f"Pronto: {len(itens) - len(faltando)} de {len(itens)} contratos estão na
lista online.", flush=True)
    if faltando:
        print("Não entraram: " + ", ".join(faltando[:10]), flush=True)
    apagados, nao_apagados = _ponte_apagar_sobras(competencia, itens, na_lista)
    return {
        "criados": len(itens) - len(faltando),
        "atualizados": 0,
        "falhas": len(faltando),
        "lista_url": url,
        "mensagem": (
            ("Publicado na lista online.\n\n" if not faltando else "Publicado, mas com
falhas.\n\n")
            + f"Na lista: {len(itens) - len(faltando)} de {len(itens)} contratos de
{competencia.strftime('%m/%Y')}.\n"
            + (f"Não entraram: {len(faltando)} (veja a lista na janela do programa e o
histórico do fluxo).\n" if faltando else "")
            + (f"Apagados da lista (não estão mais na planilha): {apagados}.\n" if
apagados else "")
            + (f"Não consegui apagar {nao_apagados} contrato(s) que saíram da
planilha. Veja a janela do programa.\n" if nao_apagados else "")
            + "\nO que o pessoal já respondeu no app não é apagado."
        ),
    }


def _ponte_apagar_sobras(competencia: date, itens: list[dict[str, Any]], na_lista:
list[dict[str, Any]]) -> tuple[int, int]:
    # A lista do mês fica igual à planilha: some o contrato que saiu dela.
    novas = {it["chave"] for it in itens}
    sobras = [it for it in na_lista if str(it.get("ChaveUnica") or "") not in novas]
    ids = [int(it["Id"]) for it in sobras if str(it.get("Id") or "").isdigit()]
    if not ids or not itens:
        return 0, 0
    if len(ids) > max(10, len(na_lista) // 2):
        print(
            f"ATENÇÃO: {len(ids)} contratos da lista de
{competencia.strftime('%m/%Y')} não estão na planilha. "
            "São muitos; não apaguei nada. Confira se a competência está certa.",
            flush=True,
        )
        return 0, len(ids)
    nomes = ", ".join(str(it.get("Title") or it.get("Contrato") or it.get("Id")) for
it in sobras[:10])
    print(f"Apagando da lista {len(ids)} contrato(s) que não estão mais na planilha:
{nomes}", flush=True)
    pid = _ponte_pedido("apagar", competencia, ids=ids)
    try:
        depois = _ponte_esperar_lista(competencia, pid)
    except TimeoutError as exc:
        print(f"ATENÇÃO: {exc}", flush=True)
        return 0, len(ids)
    restam = {str(it.get("Id")) for it in depois} & {str(i) for i in ids}
    if restam:
        print(
            f"Não consegui apagar {len(restam)}. Confira se o fluxo SOx Ponte tem o
Caso 4 (apagar), etapa A10 do roteiro.",
            flush=True,
        )
    else:
        print(f"Apagados: {len(ids)}.", flush=True)
    return len(ids) - len(restam), len(restam)
```

```
def _fechar_pela_ponte(competencia: date, por: str) -> dict[str, Any]:
    chave = competencia.strftime("%m/%Y")
    ciclos = _carregar_ciclos_local()
    ciclos[chave] = {
        "fechada": "SIM",
        "fechado_em": datetime.now().strftime("%d/%m/%Y %H:%M"),
        "fechado_por": por,
    }
    _salvar_ciclos_local(ciclos)
    print(f"Fechando {chave}: o fluxo marca MesFechado=SIM e deixa só leitura...",
flush=True)
    pid = _ponte_pedido("fechar", competencia)
    itens = _ponte_esperar_lista(competencia, pid, minutos=30)
    puxado = puxar_lista_sharepoint(competencia)
    copia = _guardar_copia_fechamento(
        competencia, [Path(puxado.get("arquivo_controle") or ""), RAIZ /
nome_arquivo(competencia)]
    )
    print(f"Cópia de controle (somente leitura): {copia}", flush=True)
    resumo = {
        "acao": "fechar",
        "competencia": chave,
        "fechado_por": por,
        "itens": {"total": len(itens)},
        "arquivo_controle": puxado.get("arquivo_controle"),
        "trava": {"modo": "fluxo SOx Ponte", "itens": len(itens)},
        "copia_controle": str(copia),
        "regra": "Depois do fechamento, só o grupo de Proprietários do site edita a
lista.",
    }
    (PASTA_NUVEM / f"SOx_fechamento_{competencia.strftime('%Y_%m')}.json").write_text(
        json.dumps(resumo, ensure_ascii=False, indent=2), encoding="utf-8"
    )
    print(f"Competência {chave} FECHADA.", flush=True)
    return resumo


def _reabrir_pela_ponte(competencia: date, por: str) -> dict[str, Any]:
    chave = competencia.strftime("%m/%Y")
    ciclos = _carregar_ciclos_local()
    ciclos[chave] = {
        "fechada": "NÃO",
        "reaberto_em": datetime.now().strftime("%d/%m/%Y %H:%M"),
        "reaberto_por": por,
    }
    _salvar_ciclos_local(ciclos)
    pid = _ponte_pedido("reabrir", competencia)
    itens = _ponte_esperar_lista(competencia, pid, minutos=30)
    print(f"Competência {chave} REABERTA.", flush=True)
    return {"acao": "reabrir", "competencia": chave, "reaberto_por": por, "itens":
{"total": len(itens)}}


def publicar_lista_sharepoint(
```

---

**23. Ponte: Publicar apaga da lista o que saiu da planilha**

**Onde:** Bloco da ponte (atualiza a versão anterior, se você já tinha).
**Por quê:** Depois de publicar, os contratos da competência que não estão mais na planilha são

apagados da lista (pedido apagar, Caso 4 do fluxo). Os que continuam mantêm as respostas.
**O que muda:** Troca o trecho Antes pelo Depois.

1 · Procure (Ctrl + F)

```
+ (f"Não entraram: {len(faltando)} (veja a lista na janela do programa e o histórico
do fluxo).\n" if faltando else "")
```

2 · Antes (selecione isto no seu arquivo)

```
# ---------------------------------------------------------------- ponte Power
Automate
# A Microsoft bloqueou o login direto do Python no SharePoint (erro AADSTS65002).
# O Python agora grava pedidos numa pasta do OneDrive; o fluxo "SOx Ponte" faz o
# trabalho no SharePoint com o login do próprio usuário e devolve a lista num arquivo.
# O pedido vai direto ao OneDrive online (cookies SSO, como no Sigma-C); sem cookies,
# usa a pasta sincronizada deste computador.
_PONTE_CACHE: dict[str, tuple[float, list[dict[str, Any]]]] = {}
_PONTE_ONLINE: dict[str, Any] = {}
_PONTE_SITE = "https://petrobrasbr-
my.sharepoint.com/personal/levi_vasconcelos_petrobras_com_br"
_PONTE_PASTA = "/personal/levi_vasconcelos_petrobras_com_br/Documents/SOx_SCM42_ponte"
_PONTE_ESCOLHAS = {
    "HouveServico": ["SIM", "NÃO"],
    "FoiMedido": ["SIM", "NÃO"],
    "ForaDoPrazo": ["SIM", "NÃO"],
    "SeraSubstituido": ["SIM", "NÃO"],
    "DemandaContinuada": ["SIM", "NÃO"],
    "MotivoProvisao": [
        "Atraso do Fornecedor", "Atraso Fiscalização de Campo", "Problema Interno
GPS",
        "Contrato Bloqueado", "Outros",
    ],
}
_PONTE_LONGOS = {
    "Objeto", "Provisao", "Justificativa", "Observacao", "CSP", "FRSPendentes",
    "MotivoIntervencao", "ComentarioErro", "ComentarioGerente",
}
_PONTE_EXTRAS = [
    "OQueFazer", "DadosCorretos", "CampoErro", "ValorCorreto", "AprovacaoGerente",
    "MesFechado", "ComentarioErro", "ComentarioGerente", "OkFiscal", "OkFiscalPor",
    "OkFiscalEm", "AprovacaoPor", "AprovacaoEm",
]
# Campos que o pessoal preenche no app: republicar não sobrescreve.
_PONTE_DO_APP = set(_CAMPOS_MANUAIS_NUVEM) | {
    "Observacao", "OkFiscal", "OkFiscalPor", "OkFiscalEm", "AprovacaoGerente",
    "AprovacaoPor", "AprovacaoEm", "ComentarioGerente", "MesFechado",
}


def _usar_ponte() -> bool:
    return str(garantir_config_nuvem().get("modo_online") or "ponte").lower() ==
"ponte"


def _pasta_ponte() -> Path:
    cfg = garantir_config_nuvem()
    base = str(cfg.get("pasta_ponte") or "").strip()
    if not base:
        onedrive = os.environ.get("OneDriveCommercial") or os.environ.get("OneDrive")
or ""
        if not onedrive:
            raise RuntimeError(
                "Não achei a pasta do OneDrive neste computador. Abra o OneDrive,
entre com a "
                "conta da Petrobras, ou escreva o caminho em pasta_ponte no
sox_nuvem_config.json."
            )
        base = str(Path(onedrive) / "SOx_SCM42_ponte")
    pasta = Path(base)
    for sub in ("pedidos", "da_lista"):
```

```
        (pasta / sub).mkdir(parents=True, exist_ok=True)
    return pasta


def _ponte_cookies(host: str) -> str:
    # Mesmos cookies SSO que o Sigma-C e a automação de certidões já guardam.
    cfg = garantir_config_nuvem()
    pasta_cert = Path.home() / "Downloads" / "automatizacao certidao"
    candidatos = [Path(str(cfg.get("ponte_cookies")))] if cfg.get("ponte_cookies")
else []
    candidatos += [
        RAIZ / ".sp_cookies.json",
        pasta_cert / ".sp_cookies.json",
        pasta_cert / "automatizacao_certidoes.db",
        pasta_cert / "certidoes.db",
    ]
    for caminho in candidatos:
        try:
            if not caminho.is_file():
                continue
            if caminho.suffix.lower() == ".json":
                dados = json.loads(caminho.read_text(encoding="utf-8",
errors="replace"))
            else:
                import sqlite3

                con = sqlite3.connect(str(caminho))
                try:
                    linha = con.execute("SELECT cookies FROM sessao WHERE
id=1").fetchone()
                finally:
                    con.close()
                dados = json.loads(linha[0]) if linha and linha[0] else None
        except Exception:
            continue
        if isinstance(dados, str) and dados.strip():
            return dados.strip()
        if isinstance(dados, dict):
            dados = dados.get("cookies") or [{"name": k, "value": v} for k, v in
dados.items()]
        if not isinstance(dados, list):
            continue
        todos = [c for c in dados if isinstance(c, dict) and c.get("name")]

        def do_host(c: dict[str, Any]) -> bool:
            dominio = str(c.get("domain") or "").lstrip(".").lower()
            return not dominio or host == dominio or host.endswith("." + dominio)

        escolhidos = [c for c in todos if do_host(c)] or todos
        vistos: dict[str, str] = {}
        for c in escolhidos:
            vistos.setdefault(str(c["name"]), str(c.get("value", "")))
        if vistos:
            print(f"Cookies SSO: {caminho}", flush=True)
            return "; ".join(f"{k}={v}" for k, v in vistos.items())
    return ""


def _ponte_http(metodo: str, url: str, cookies: str, corpo: bytes | None = None,
                digest: str = "", tipo: str = "application/octet-stream") ->
tuple[int, bytes]:
    import ssl
    import urllib.error
```

```
    import urllib.request
    from urllib.parse import urlparse

    cab = {"Accept": "application/json;odata=verbose", "Cookie": cookies, "User-
Agent": "Mozilla/5.0"}
    if digest:
        cab["X-RequestDigest"] = digest
    if corpo is not None:
        cab["Content-Type"] = tipo
    req = urllib.request.Request(url, data=corpo, headers=cab, method=metodo)
    try:
        with urllib.request.urlopen(req, context=ssl.create_default_context(),
timeout=60) as resp:
            # Cookie vencido: o SharePoint manda para a tela de login da Microsoft.
            if urlparse(resp.geturl()).netloc.lower() != urlparse(url).netloc.lower():
                return 401, b"login expirado"
            return resp.status, resp.read()
    except urllib.error.HTTPError as exc:
        return exc.code, exc.read()


def _ponte_online() -> dict[str, str] | None:
    import time
    from urllib.parse import urlparse

    if time.time() - _PONTE_ONLINE.get("em", 0) < 1200:
        return _PONTE_ONLINE.get("dados")
    cfg = garantir_config_nuvem()
    dados = None
    if str(cfg.get("ponte_envio") or "online").lower() != "pasta":
        site = str(cfg.get("ponte_site") or _PONTE_SITE).rstrip("/")
        cookies = _ponte_cookies((urlparse(site).hostname or "").lower())
        if not cookies:
            print("Sem cookies SSO salvos (Sigma-C/certidões): uso a pasta
sincronizada do OneDrive.", flush=True)
        else:
            try:
                status, corpo = _ponte_http(
                    "POST", f"{site}/_api/contextinfo", cookies, b"",
tipo="application/json;odata=verbose"
                )
                info = json.loads(corpo.decode("utf-8")) if status == 200 else {}
                digest = (info.get("d") or {}).get("GetContextWebInformation",
{}).get("FormDigestValue", "")
            except Exception as exc:
                status, digest = str(exc), ""
            if digest:
                dados = {
                    "site": site,
                    "pasta": str(cfg.get("ponte_pasta_servidor") or
_PONTE_PASTA).rstrip("/"),
                    "cookies": cookies,
                    "digest": digest,
                }
            else:
                print(
                    f"OneDrive online não aceitou os cookies ({status}). Abra o Sigma-
C ou as certidões "
                    "para renovar o login. Por agora uso a pasta sincronizada.",
                    flush=True,
                )
    _PONTE_ONLINE.update(em=time.time(), dados=dados)
    return dados
```

```
def _ponte_subir_online(sub: str, nome: str, corpo: bytes) -> bool:
    from urllib.parse import quote as q, urlencode

    on = _ponte_online()
    if not on:
        return False
    pasta = f"{on['pasta']}/{sub}"
    url = (
        f"
{on['site']}/_api/web/GetFolderByServerRelativeUrl(@p)/Files/add(url=@n,overwrite=true
)?"
        + urlencode({"@p": f"'{pasta}'", "@n": f"'{nome}'"})
    )
    status, resposta = _ponte_http("POST", url, on["cookies"], corpo, on["digest"])
    if status == 404:  # pasta ainda não existe no OneDrive online
        _ponte_http("POST", f"{on['site']}/_api/web/folders/add('{q(pasta)}')",
on["cookies"], b"", on["digest"])
        status, resposta = _ponte_http("POST", url, on["cookies"], corpo,
on["digest"])
    if status in (200, 201):
        return True
    trecho = resposta.decode("utf-8", errors="replace")[:200]
    print(f"Envio online falhou (HTTP {status}): {trecho}", flush=True)
    _PONTE_ONLINE.clear()
    return False


def _ponte_ler_online(sub: str, nome: str) -> bytes | None:
    from urllib.parse import urlencode

    on = _ponte_online()
    if not on:
        return None
    url = (
        f"{on['site']}/_api/web/GetFileByServerRelativeUrl(@f)/$value?"
        + urlencode({"@f": f"'{on['pasta']}/{sub}/{nome}'"})
    )
    try:
        status, corpo = _ponte_http("GET", url, on["cookies"])
    except Exception:
        return None
    return corpo if status == 200 else None


def _ponte_colunas(lista: str) -> list[dict[str, Any]]:
    nomes = list(dict.fromkeys(list(CABECALHO_STAGING_NUVEM) + _PONTE_EXTRAS))
    base = f"_api/web/lists/getbytitle('{lista}')"
    saida = []
    for nome in nomes:
        if nome in _PONTE_ESCOLHAS:
            opcoes = "".join(f"<CHOICE>{o}</CHOICE>" for o in _PONTE_ESCOLHAS[nome])
            xml = (
                f"<Field Type='Choice' DisplayName='{nome}' Name='{nome}'
StaticName='{nome}' "
                f"FillInChoice='TRUE' Format='Dropdown'><CHOICES>{opcoes}</CHOICES>
</Field>"
            )
        elif nome in _PONTE_LONGOS:
            xml = (
                f"<Field Type='Note' DisplayName='{nome}' Name='{nome}'
StaticName='{nome}' "
```

```
                f"RichText='FALSE' NumLines='6' />"
            )
        else:
            xml = f"<Field Type='Text' DisplayName='{nome}' Name='{nome}'
StaticName='{nome}' MaxLength='255' />"
        saida.append({
            "nome": nome,
            "uri_existe": f"{base}/fields?$select=Id&$filter=" + quote(f"InternalName
eq '{nome}'"),
            "corpo": {"parameters": {"SchemaXml": xml, "Options": 24}},
        })
    return saida


def _ponte_pedido(acao: str, competencia: date, **dados: Any) -> str:
    import uuid

    pid = datetime.now().strftime("%Y%m%d%H%M%S") + uuid.uuid4().hex[:6]
    cfg = garantir_config_nuvem()
    lista = str(cfg.get("lista_contratos") or "SOx_SCM42_Contratos")
    comp = competencia.strftime("%m/%Y")
    base = f"_api/web/lists/getbytitle('{lista}')"
    filtro = quote(f"Competencia eq '{comp}'")
    # Endereços prontos: no fluxo cada caixa só lê outputs('Pedido')?['...'].
    pedido = {
        "acao": acao,
        "id": pid,
        "competencia": comp,
        "arquivo_comp": competencia.strftime("%Y_%m"),
        "lista": lista,
        "uri_lista_existe": "_api/web/lists?$select=Id&$filter=" + quote(f"Title eq
'{lista}'"),
        "uri_criar_lista": "_api/web/lists",
        "corpo_criar_lista": {
            "Title": lista,
            "BaseTemplate": 100,
            "Description": "Controle SOx SCM-42: um contrato por linha e por
competência.",
        },
        "uri_criar_coluna": f"{base}/fields/createfieldasxml",
        "uri_indexar": f"{base}/fields/getbyinternalnameortitle('Competencia')",
        "uri_itens": f"{base}/items",
        "uri_existentes": f"{base}/items?$select=Id,ChaveUnica&$top=5000&$filter=
{filtro}",
        "uri_ids": f"{base}/items?$select=Id&$top=5000&$filter={filtro}",
        "uri_lista": f"{base}/items?$top=5000&$filter={filtro}",
        "uri_grupos": (
            "_api/web?$select=AssociatedOwnerGroup/Id,AssociatedMemberGroup/Id,"

"AssociatedVisitorGroup/Id&$expand=AssociatedOwnerGroup,AssociatedMemberGroup,"
            "AssociatedVisitorGroup"
        ),
        **dados,
    }
    nome = f"{acao}__{competencia.strftime('%Y_%m')}__{pid}.json"
    corpo = json.dumps(pedido, ensure_ascii=True).encode("utf-8")
    if _ponte_subir_online("pedidos", nome, corpo):
        print(f"Pedido enviado direto ao OneDrive online (não depende da
sincronização): {nome}", flush=True)
        return pid
    (_pasta_ponte() / "pedidos" / nome).write_bytes(corpo)
    print(f"Pedido gravado na pasta sincronizada do OneDrive: {nome}", flush=True)
    return pid
```

```
def _ponte_valor(v: Any) -> Any:
    if isinstance(v, dict):
        return v.get("Value") if "Value" in v else ""
    return "" if v is None else v


def _ponte_esperar_lista(competencia: date, pid: str, minutos: int = 20) ->
list[dict[str, Any]]:
    import time

    nome = f"lista_{competencia.strftime('%Y_%m')}__{pid}.json"
    try:
        arquivo: Path | None = _pasta_ponte() / "da_lista" / nome
    except RuntimeError:
        arquivo = None
    online = _ponte_online() is not None
    print(f"Aguardando o fluxo SOx Ponte responder (normal: 1 a 5 min, máximo
{minutos} min)...", flush=True)
    inicio = time.time()
    limite = inicio + minutos * 60
    avisos = {
        120: (
            "Ainda aguardando. Confira se o fluxo SOx Ponte está On no Power
Automate."
            if online else
            "Ainda aguardando. Confira se o OneDrive está sincronizando (nuvem azul
perto do relógio)."
        ),
        300: "Demorando mais que o normal. Veja no Power Automate: SOx Ponte → 28-day
run history.",
    }
    proximo = 30
    while time.time() < limite:
        decorrido = int(time.time() - inicio)
        if decorrido >= proximo:
            print(f"  ... {decorrido // 60} min {decorrido % 60:02d} s esperando o
fluxo", flush=True)
            for seg, msg in avisos.items():
                if proximo - 30 < seg <= proximo:
                    print(f"  {msg}", flush=True)
            proximo += 30
        texto = None
        if arquivo is not None and arquivo.is_file():
            time.sleep(3)
            try:
                texto = arquivo.read_text(encoding="utf-8-sig")
            except OSError:
                texto = ""
        elif online:
            baixado = _ponte_ler_online("da_lista", nome)
            if baixado is not None:
                texto = baixado.decode("utf-8-sig")
        if texto is not None:
            try:
                bruto = json.loads(texto)
            except Exception:
                time.sleep(5)
                continue
            if isinstance(bruto, dict):
                bruto = bruto.get("value") or []
            itens = [{k: _ponte_valor(v) for k, v in it.items()} for it in bruto]
```

```
            _PONTE_CACHE[competencia.strftime("%m/%Y")] = (time.time(), itens)
            print(f"Fluxo respondeu: {len(itens)} contrato(s).", flush=True)
            return itens
        time.sleep(10)
    raise TimeoutError(
        "O fluxo SOx Ponte não respondeu a tempo. Confira se o OneDrive está
sincronizando "
        "(nuvem azul na barra do Windows) e se o fluxo está Ativado no Power Automate
"
        "(Meus fluxos → SOx Ponte → Histórico de execuções)."
    )


def _ponte_puxar(competencia: date) -> list[dict[str, Any]]:
    import time

    cache = _PONTE_CACHE.get(competencia.strftime("%m/%Y"))
    if cache and time.time() - cache[0] < 180:
        return cache[1]
    pid = _ponte_pedido("puxar", competencia)
    return _ponte_esperar_lista(competencia, pid)


def publicar_pela_ponte(caminho_planilha: Path, competencia: date) -> dict[str, Any]:
    cfg = garantir_config_nuvem()
    site, lista = _site_e_lista(cfg)
    exportar_staging_nuvem(caminho_planilha, competencia)
    linhas = coletar_linhas_nuvem(caminho_planilha, competencia)
    itens = []
    for linha in linhas:
        chave = linha.get("ChaveUnica") or ""
        if not chave:
            continue
        if len(linha.get("Observacao", "")) > 4000:
            linha["Observacao"] = linha["Observacao"][:4000] + "..."
        if len(linha.get("Objeto", "")) > 2000:
            linha["Objeto"] = linha["Objeto"][:2000]
        novo = _sp_payload_item("SP.Data", linha, None)
        novo.pop("__metadata", None)
        atualiza = {k: v for k, v in novo.items() if k not in _PONTE_DO_APP}
        itens.append({"chave": chave, "novo": novo, "atualiza": atualiza})
    pid = _ponte_pedido("publicar", competencia, colunas=_ponte_colunas(lista),
itens=itens)
    url = cfg.get("lista_url") or f"{site}/Lists/{lista}"
    try:
        na_lista = _ponte_esperar_lista(competencia, pid)
    except TimeoutError as exc:
        print(f"ATENÇÃO: {exc}", flush=True)
        return {
            "criados": 0, "atualizados": 0, "falhas": len(itens), "lista_url": url,
            "mensagem": (
                f"O pedido foi enviado ({len(itens)} contratos), mas o fluxo SOx Ponte
não respondeu em 20 minutos.\n\n"
                "Confira:\n1. O OneDrive está sincronizando (nuvem azul perto do
relógio).\n"
                "2. O fluxo SOx Ponte está ligado (On).\n"
                "3. Power Automate → SOx Ponte → 28-day run history: a execução
falhou?\n\n"
                "Se o fluxo terminar depois, os contratos aparecem na lista sozinhos."
            ),
        }
    chaves = {str(it.get("ChaveUnica") or "") for it in na_lista}
    faltando = [it["chave"] for it in itens if it["chave"] not in chaves]
```

```
    print(f"Pronto: {len(itens) - len(faltando)} de {len(itens)} contratos estão na
lista online.", flush=True)
    if faltando:
        print("Não entraram: " + ", ".join(faltando[:10]), flush=True)
    return {
        "criados": len(itens) - len(faltando),
        "atualizados": 0,
        "falhas": len(faltando),
        "lista_url": url,
        "mensagem": (
            ("Publicado na lista online.\n\n" if not faltando else "Publicado, mas com
falhas.\n\n")
            + f"Na lista: {len(itens) - len(faltando)} de {len(itens)} contratos de
{competencia.strftime('%m/%Y')}.\n"
            + (f"Não entraram: {len(faltando)} (veja a lista na janela do programa e o
histórico do fluxo).\n" if faltando else "")
            + "\nO que o pessoal já respondeu no app não é apagado."
        ),
    }


def _fechar_pela_ponte(competencia: date, por: str) -> dict[str, Any]:
    chave = competencia.strftime("%m/%Y")
    ciclos = _carregar_ciclos_local()
    ciclos[chave] = {
        "fechada": "SIM",
        "fechado_em": datetime.now().strftime("%d/%m/%Y %H:%M"),
        "fechado_por": por,
    }
    _salvar_ciclos_local(ciclos)
    print(f"Fechando {chave}: o fluxo marca MesFechado=SIM e deixa só leitura...",
flush=True)
    pid = _ponte_pedido("fechar", competencia)
    itens = _ponte_esperar_lista(competencia, pid, minutos=30)
    puxado = puxar_lista_sharepoint(competencia)
    copia = _guardar_copia_fechamento(
        competencia, [Path(puxado.get("arquivo_controle") or ""), RAIZ /
nome_arquivo(competencia)]
    )
    print(f"Cópia de controle (somente leitura): {copia}", flush=True)
    resumo = {
        "acao": "fechar",
        "competencia": chave,
        "fechado_por": por,
        "itens": {"total": len(itens)},
        "arquivo_controle": puxado.get("arquivo_controle"),
        "trava": {"modo": "fluxo SOx Ponte", "itens": len(itens)},
        "copia_controle": str(copia),
        "regra": "Depois do fechamento, só o grupo de Proprietários do site edita a
lista.",
    }
    (PASTA_NUVEM / f"SOx_fechamento_{competencia.strftime('%Y_%m')}.json").write_text(
        json.dumps(resumo, ensure_ascii=False, indent=2), encoding="utf-8"
    )
    print(f"Competência {chave} FECHADA.", flush=True)
    return resumo


def _reabrir_pela_ponte(competencia: date, por: str) -> dict[str, Any]:
    chave = competencia.strftime("%m/%Y")
    ciclos = _carregar_ciclos_local()
    ciclos[chave] = {
        "fechada": "NÃO",
```

```
        "reaberto_em": datetime.now().strftime("%d/%m/%Y %H:%M"),
        "reaberto_por": por,
    }
    _salvar_ciclos_local(ciclos)
    pid = _ponte_pedido("reabrir", competencia)
    itens = _ponte_esperar_lista(competencia, pid, minutos=30)
    print(f"Competência {chave} REABERTA.", flush=True)
    return {"acao": "reabrir", "competencia": chave, "reaberto_por": por, "itens":
{"total": len(itens)}}


def publicar_lista_sharepoint(
```

3 · Depois (cole por cima)

```
# ---------------------------------------------------------------- ponte Power
Automate
# A Microsoft bloqueou o login direto do Python no SharePoint (erro AADSTS65002).
# O Python agora grava pedidos numa pasta do OneDrive; o fluxo "SOx Ponte" faz o
# trabalho no SharePoint com o login do próprio usuário e devolve a lista num arquivo.
# O pedido vai direto ao OneDrive online (cookies SSO, como no Sigma-C); sem cookies,
# usa a pasta sincronizada deste computador.
_PONTE_CACHE: dict[str, tuple[float, list[dict[str, Any]]]] = {}
_PONTE_ONLINE: dict[str, Any] = {}
_PONTE_SITE = "https://petrobrasbr-
my.sharepoint.com/personal/levi_vasconcelos_petrobras_com_br"
_PONTE_PASTA = "/personal/levi_vasconcelos_petrobras_com_br/Documents/SOx_SCM42_ponte"
_PONTE_ESCOLHAS = {
    "HouveServico": ["SIM", "NÃO"],
    "FoiMedido": ["SIM", "NÃO"],
    "ForaDoPrazo": ["SIM", "NÃO"],
    "SeraSubstituido": ["SIM", "NÃO"],
    "DemandaContinuada": ["SIM", "NÃO"],
    "MotivoProvisao": [
        "Atraso do Fornecedor", "Atraso Fiscalização de Campo", "Problema Interno
GPS",
        "Contrato Bloqueado", "Outros",
    ],
}
_PONTE_LONGOS = {
    "Objeto", "Provisao", "Justificativa", "Observacao", "CSP", "FRSPendentes",
    "MotivoIntervencao", "ComentarioErro", "ComentarioGerente",
}
_PONTE_EXTRAS = [
    "OQueFazer", "DadosCorretos", "CampoErro", "ValorCorreto", "AprovacaoGerente",
    "MesFechado", "ComentarioErro", "ComentarioGerente", "OkFiscal", "OkFiscalPor",
    "OkFiscalEm", "AprovacaoPor", "AprovacaoEm",
]
# Campos que o pessoal preenche no app: republicar não sobrescreve.
_PONTE_DO_APP = set(_CAMPOS_MANUAIS_NUVEM) | {
    "Observacao", "OkFiscal", "OkFiscalPor", "OkFiscalEm", "AprovacaoGerente",
    "AprovacaoPor", "AprovacaoEm", "ComentarioGerente", "MesFechado",
}


def _usar_ponte() -> bool:
    return str(garantir_config_nuvem().get("modo_online") or "ponte").lower() ==
"ponte"


def _pasta_ponte() -> Path:
    cfg = garantir_config_nuvem()
    base = str(cfg.get("pasta_ponte") or "").strip()
    if not base:
        onedrive = os.environ.get("OneDriveCommercial") or os.environ.get("OneDrive")
or ""
        if not onedrive:
            raise RuntimeError(
                "Não achei a pasta do OneDrive neste computador. Abra o OneDrive,
entre com a "
                "conta da Petrobras, ou escreva o caminho em pasta_ponte no
sox_nuvem_config.json."
            )
        base = str(Path(onedrive) / "SOx_SCM42_ponte")
    pasta = Path(base)
    for sub in ("pedidos", "da_lista"):
```

```
        (pasta / sub).mkdir(parents=True, exist_ok=True)
    return pasta


def _ponte_cookies(host: str) -> str:
    # Mesmos cookies SSO que o Sigma-C e a automação de certidões já guardam.
    cfg = garantir_config_nuvem()
    pasta_cert = Path.home() / "Downloads" / "automatizacao certidao"
    candidatos = [Path(str(cfg.get("ponte_cookies")))] if cfg.get("ponte_cookies")
else []
    candidatos += [
        RAIZ / ".sp_cookies.json",
        pasta_cert / ".sp_cookies.json",
        pasta_cert / "automatizacao_certidoes.db",
        pasta_cert / "certidoes.db",
    ]
    for caminho in candidatos:
        try:
            if not caminho.is_file():
                continue
            if caminho.suffix.lower() == ".json":
                dados = json.loads(caminho.read_text(encoding="utf-8",
errors="replace"))
            else:
                import sqlite3

                con = sqlite3.connect(str(caminho))
                try:
                    linha = con.execute("SELECT cookies FROM sessao WHERE
id=1").fetchone()
                finally:
                    con.close()
                dados = json.loads(linha[0]) if linha and linha[0] else None
        except Exception:
            continue
        if isinstance(dados, str) and dados.strip():
            return dados.strip()
        if isinstance(dados, dict):
            dados = dados.get("cookies") or [{"name": k, "value": v} for k, v in
dados.items()]
        if not isinstance(dados, list):
            continue
        todos = [c for c in dados if isinstance(c, dict) and c.get("name")]

        def do_host(c: dict[str, Any]) -> bool:
            dominio = str(c.get("domain") or "").lstrip(".").lower()
            return not dominio or host == dominio or host.endswith("." + dominio)

        escolhidos = [c for c in todos if do_host(c)] or todos
        vistos: dict[str, str] = {}
        for c in escolhidos:
            vistos.setdefault(str(c["name"]), str(c.get("value", "")))
        if vistos:
            print(f"Cookies SSO: {caminho}", flush=True)
            return "; ".join(f"{k}={v}" for k, v in vistos.items())
    return ""


def _ponte_http(metodo: str, url: str, cookies: str, corpo: bytes | None = None,
                digest: str = "", tipo: str = "application/octet-stream") ->
tuple[int, bytes]:
    import ssl
    import urllib.error
```

```
    import urllib.request
    from urllib.parse import urlparse

    cab = {"Accept": "application/json;odata=verbose", "Cookie": cookies, "User-
Agent": "Mozilla/5.0"}
    if digest:
        cab["X-RequestDigest"] = digest
    if corpo is not None:
        cab["Content-Type"] = tipo
    req = urllib.request.Request(url, data=corpo, headers=cab, method=metodo)
    try:
        with urllib.request.urlopen(req, context=ssl.create_default_context(),
timeout=60) as resp:
            # Cookie vencido: o SharePoint manda para a tela de login da Microsoft.
            if urlparse(resp.geturl()).netloc.lower() != urlparse(url).netloc.lower():
                return 401, b"login expirado"
            return resp.status, resp.read()
    except urllib.error.HTTPError as exc:
        return exc.code, exc.read()


def _ponte_online() -> dict[str, str] | None:
    import time
    from urllib.parse import urlparse

    if time.time() - _PONTE_ONLINE.get("em", 0) < 1200:
        return _PONTE_ONLINE.get("dados")
    cfg = garantir_config_nuvem()
    dados = None
    if str(cfg.get("ponte_envio") or "online").lower() != "pasta":
        site = str(cfg.get("ponte_site") or _PONTE_SITE).rstrip("/")
        cookies = _ponte_cookies((urlparse(site).hostname or "").lower())
        if not cookies:
            print("Sem cookies SSO salvos (Sigma-C/certidões): uso a pasta
sincronizada do OneDrive.", flush=True)
        else:
            try:
                status, corpo = _ponte_http(
                    "POST", f"{site}/_api/contextinfo", cookies, b"",
tipo="application/json;odata=verbose"
                )
                info = json.loads(corpo.decode("utf-8")) if status == 200 else {}
                digest = (info.get("d") or {}).get("GetContextWebInformation",
{}).get("FormDigestValue", "")
            except Exception as exc:
                status, digest = str(exc), ""
            if digest:
                dados = {
                    "site": site,
                    "pasta": str(cfg.get("ponte_pasta_servidor") or
_PONTE_PASTA).rstrip("/"),
                    "cookies": cookies,
                    "digest": digest,
                }
            else:
                print(
                    f"OneDrive online não aceitou os cookies ({status}). Abra o Sigma-
C ou as certidões "
                    "para renovar o login. Por agora uso a pasta sincronizada.",
                    flush=True,
                )
    _PONTE_ONLINE.update(em=time.time(), dados=dados)
    return dados
```

```
def _ponte_subir_online(sub: str, nome: str, corpo: bytes) -> bool:
    from urllib.parse import quote as q, urlencode

    on = _ponte_online()
    if not on:
        return False
    pasta = f"{on['pasta']}/{sub}"
    url = (
        f"
{on['site']}/_api/web/GetFolderByServerRelativeUrl(@p)/Files/add(url=@n,overwrite=true
)?"
        + urlencode({"@p": f"'{pasta}'", "@n": f"'{nome}'"})
    )
    status, resposta = _ponte_http("POST", url, on["cookies"], corpo, on["digest"])
    if status == 404:  # pasta ainda não existe no OneDrive online
        _ponte_http("POST", f"{on['site']}/_api/web/folders/add('{q(pasta)}')",
on["cookies"], b"", on["digest"])
        status, resposta = _ponte_http("POST", url, on["cookies"], corpo,
on["digest"])
    if status in (200, 201):
        return True
    trecho = resposta.decode("utf-8", errors="replace")[:200]
    print(f"Envio online falhou (HTTP {status}): {trecho}", flush=True)
    _PONTE_ONLINE.clear()
    return False


def _ponte_ler_online(sub: str, nome: str) -> bytes | None:
    from urllib.parse import urlencode

    on = _ponte_online()
    if not on:
        return None
    url = (
        f"{on['site']}/_api/web/GetFileByServerRelativeUrl(@f)/$value?"
        + urlencode({"@f": f"'{on['pasta']}/{sub}/{nome}'"})
    )
    try:
        status, corpo = _ponte_http("GET", url, on["cookies"])
    except Exception:
        return None
    return corpo if status == 200 else None


def _ponte_colunas(lista: str) -> list[dict[str, Any]]:
    nomes = list(dict.fromkeys(list(CABECALHO_STAGING_NUVEM) + _PONTE_EXTRAS))
    base = f"_api/web/lists/getbytitle('{lista}')"
    saida = []
    for nome in nomes:
        if nome in _PONTE_ESCOLHAS:
            opcoes = "".join(f"<CHOICE>{o}</CHOICE>" for o in _PONTE_ESCOLHAS[nome])
            xml = (
                f"<Field Type='Choice' DisplayName='{nome}' Name='{nome}'
StaticName='{nome}' "
                f"FillInChoice='TRUE' Format='Dropdown'><CHOICES>{opcoes}</CHOICES>
</Field>"
            )
        elif nome in _PONTE_LONGOS:
            xml = (
                f"<Field Type='Note' DisplayName='{nome}' Name='{nome}'
StaticName='{nome}' "
```

```
                f"RichText='FALSE' NumLines='6' />"
            )
        else:
            xml = f"<Field Type='Text' DisplayName='{nome}' Name='{nome}'
StaticName='{nome}' MaxLength='255' />"
        saida.append({
            "nome": nome,
            "uri_existe": f"{base}/fields?$select=Id&$filter=" + quote(f"InternalName
eq '{nome}'"),
            "corpo": {"parameters": {"SchemaXml": xml, "Options": 24}},
        })
    return saida


def _ponte_pedido(acao: str, competencia: date, **dados: Any) -> str:
    import uuid

    pid = datetime.now().strftime("%Y%m%d%H%M%S") + uuid.uuid4().hex[:6]
    cfg = garantir_config_nuvem()
    lista = str(cfg.get("lista_contratos") or "SOx_SCM42_Contratos")
    comp = competencia.strftime("%m/%Y")
    base = f"_api/web/lists/getbytitle('{lista}')"
    filtro = quote(f"Competencia eq '{comp}'")
    # Endereços prontos: no fluxo cada caixa só lê outputs('Pedido')?['...'].
    pedido = {
        "acao": acao,
        "id": pid,
        "competencia": comp,
        "arquivo_comp": competencia.strftime("%Y_%m"),
        "lista": lista,
        "uri_lista_existe": "_api/web/lists?$select=Id&$filter=" + quote(f"Title eq
'{lista}'"),
        "uri_criar_lista": "_api/web/lists",
        "corpo_criar_lista": {
            "Title": lista,
            "BaseTemplate": 100,
            "Description": "Controle SOx SCM-42: um contrato por linha e por
competência.",
        },
        "uri_criar_coluna": f"{base}/fields/createfieldasxml",
        "uri_indexar": f"{base}/fields/getbyinternalnameortitle('Competencia')",
        "uri_itens": f"{base}/items",
        "uri_existentes": f"{base}/items?$select=Id,ChaveUnica&$top=5000&$filter=
{filtro}",
        "uri_ids": f"{base}/items?$select=Id&$top=5000&$filter={filtro}",
        "uri_lista": f"{base}/items?$top=5000&$filter={filtro}",
        "uri_grupos": (
            "_api/web?$select=AssociatedOwnerGroup/Id,AssociatedMemberGroup/Id,"

"AssociatedVisitorGroup/Id&$expand=AssociatedOwnerGroup,AssociatedMemberGroup,"
            "AssociatedVisitorGroup"
        ),
        **dados,
    }
    nome = f"{acao}__{competencia.strftime('%Y_%m')}__{pid}.json"
    corpo = json.dumps(pedido, ensure_ascii=True).encode("utf-8")
    if _ponte_subir_online("pedidos", nome, corpo):
        print(f"Pedido enviado direto ao OneDrive online (não depende da
sincronização): {nome}", flush=True)
        return pid
    (_pasta_ponte() / "pedidos" / nome).write_bytes(corpo)
    print(f"Pedido gravado na pasta sincronizada do OneDrive: {nome}", flush=True)
    return pid
```

```
def _ponte_valor(v: Any) -> Any:
    if isinstance(v, dict):
        return v.get("Value") if "Value" in v else ""
    return "" if v is None else v


def _ponte_esperar_lista(competencia: date, pid: str, minutos: int = 20) ->
list[dict[str, Any]]:
    import time

    nome = f"lista_{competencia.strftime('%Y_%m')}__{pid}.json"
    try:
        arquivo: Path | None = _pasta_ponte() / "da_lista" / nome
    except RuntimeError:
        arquivo = None
    online = _ponte_online() is not None
    print(f"Aguardando o fluxo SOx Ponte responder (normal: 1 a 5 min, máximo
{minutos} min)...", flush=True)
    inicio = time.time()
    limite = inicio + minutos * 60
    avisos = {
        120: (
            "Ainda aguardando. Confira se o fluxo SOx Ponte está On no Power
Automate."
            if online else
            "Ainda aguardando. Confira se o OneDrive está sincronizando (nuvem azul
perto do relógio)."
        ),
        300: "Demorando mais que o normal. Veja no Power Automate: SOx Ponte → 28-day
run history.",
    }
    proximo = 30
    while time.time() < limite:
        decorrido = int(time.time() - inicio)
        if decorrido >= proximo:
            print(f"  ... {decorrido // 60} min {decorrido % 60:02d} s esperando o
fluxo", flush=True)
            for seg, msg in avisos.items():
                if proximo - 30 < seg <= proximo:
                    print(f"  {msg}", flush=True)
            proximo += 30
        texto = None
        if arquivo is not None and arquivo.is_file():
            time.sleep(3)
            try:
                texto = arquivo.read_text(encoding="utf-8-sig")
            except OSError:
                texto = ""
        elif online:
            baixado = _ponte_ler_online("da_lista", nome)
            if baixado is not None:
                texto = baixado.decode("utf-8-sig")
        if texto is not None:
            try:
                bruto = json.loads(texto)
            except Exception:
                time.sleep(5)
                continue
            if isinstance(bruto, dict):
                bruto = bruto.get("value") or []
            itens = [{k: _ponte_valor(v) for k, v in it.items()} for it in bruto]
```

```
            _PONTE_CACHE[competencia.strftime("%m/%Y")] = (time.time(), itens)
            print(f"Fluxo respondeu: {len(itens)} contrato(s).", flush=True)
            return itens
        time.sleep(10)
    raise TimeoutError(
        "O fluxo SOx Ponte não respondeu a tempo. Confira se o OneDrive está
sincronizando "
        "(nuvem azul na barra do Windows) e se o fluxo está Ativado no Power Automate
"
        "(Meus fluxos → SOx Ponte → Histórico de execuções)."
    )


def _ponte_puxar(competencia: date) -> list[dict[str, Any]]:
    import time

    cache = _PONTE_CACHE.get(competencia.strftime("%m/%Y"))
    if cache and time.time() - cache[0] < 180:
        return cache[1]
    pid = _ponte_pedido("puxar", competencia)
    return _ponte_esperar_lista(competencia, pid)


def publicar_pela_ponte(caminho_planilha: Path, competencia: date) -> dict[str, Any]:
    cfg = garantir_config_nuvem()
    site, lista = _site_e_lista(cfg)
    exportar_staging_nuvem(caminho_planilha, competencia)
    linhas = coletar_linhas_nuvem(caminho_planilha, competencia)
    itens = []
    for linha in linhas:
        chave = linha.get("ChaveUnica") or ""
        if not chave:
            continue
        if len(linha.get("Observacao", "")) > 4000:
            linha["Observacao"] = linha["Observacao"][:4000] + "..."
        if len(linha.get("Objeto", "")) > 2000:
            linha["Objeto"] = linha["Objeto"][:2000]
        novo = _sp_payload_item("SP.Data", linha, None)
        novo.pop("__metadata", None)
        atualiza = {k: v for k, v in novo.items() if k not in _PONTE_DO_APP}
        itens.append({"chave": chave, "novo": novo, "atualiza": atualiza})
    pid = _ponte_pedido("publicar", competencia, colunas=_ponte_colunas(lista),
itens=itens)
    url = cfg.get("lista_url") or f"{site}/Lists/{lista}"
    try:
        na_lista = _ponte_esperar_lista(competencia, pid)
    except TimeoutError as exc:
        print(f"ATENÇÃO: {exc}", flush=True)
        return {
            "criados": 0, "atualizados": 0, "falhas": len(itens), "lista_url": url,
            "mensagem": (
                f"O pedido foi enviado ({len(itens)} contratos), mas o fluxo SOx Ponte
não respondeu em 20 minutos.\n\n"
                "Confira:\n1. O OneDrive está sincronizando (nuvem azul perto do
relógio).\n"
                "2. O fluxo SOx Ponte está ligado (On).\n"
                "3. Power Automate → SOx Ponte → 28-day run history: a execução
falhou?\n\n"
                "Se o fluxo terminar depois, os contratos aparecem na lista sozinhos."
            ),
        }
    chaves = {str(it.get("ChaveUnica") or "") for it in na_lista}
    faltando = [it["chave"] for it in itens if it["chave"] not in chaves]
```

```
    print(f"Pronto: {len(itens) - len(faltando)} de {len(itens)} contratos estão na
lista online.", flush=True)
    if faltando:
        print("Não entraram: " + ", ".join(faltando[:10]), flush=True)
    apagados, nao_apagados = _ponte_apagar_sobras(competencia, itens, na_lista)
    return {
        "criados": len(itens) - len(faltando),
        "atualizados": 0,
        "falhas": len(faltando),
        "lista_url": url,
        "mensagem": (
            ("Publicado na lista online.\n\n" if not faltando else "Publicado, mas com
falhas.\n\n")
            + f"Na lista: {len(itens) - len(faltando)} de {len(itens)} contratos de
{competencia.strftime('%m/%Y')}.\n"
            + (f"Não entraram: {len(faltando)} (veja a lista na janela do programa e o
histórico do fluxo).\n" if faltando else "")
            + (f"Apagados da lista (não estão mais na planilha): {apagados}.\n" if
apagados else "")
            + (f"Não consegui apagar {nao_apagados} contrato(s) que saíram da
planilha. Veja a janela do programa.\n" if nao_apagados else "")
            + "\nO que o pessoal já respondeu no app não é apagado."
        ),
    }


def _ponte_apagar_sobras(competencia: date, itens: list[dict[str, Any]], na_lista:
list[dict[str, Any]]) -> tuple[int, int]:
    # A lista do mês fica igual à planilha: some o contrato que saiu dela.
    novas = {it["chave"] for it in itens}
    sobras = [it for it in na_lista if str(it.get("ChaveUnica") or "") not in novas]
    ids = [int(it["Id"]) for it in sobras if str(it.get("Id") or "").isdigit()]
    if not ids or not itens:
        return 0, 0
    if len(ids) > max(10, len(na_lista) // 2):
        print(
            f"ATENÇÃO: {len(ids)} contratos da lista de
{competencia.strftime('%m/%Y')} não estão na planilha. "
            "São muitos; não apaguei nada. Confira se a competência está certa.",
            flush=True,
        )
        return 0, len(ids)
    nomes = ", ".join(str(it.get("Title") or it.get("Contrato") or it.get("Id")) for
it in sobras[:10])
    print(f"Apagando da lista {len(ids)} contrato(s) que não estão mais na planilha:
{nomes}", flush=True)
    pid = _ponte_pedido("apagar", competencia, ids=ids)
    try:
        depois = _ponte_esperar_lista(competencia, pid)
    except TimeoutError as exc:
        print(f"ATENÇÃO: {exc}", flush=True)
        return 0, len(ids)
    restam = {str(it.get("Id")) for it in depois} & {str(i) for i in ids}
    if restam:
        print(
            f"Não consegui apagar {len(restam)}. Confira se o fluxo SOx Ponte tem o
Caso 4 (apagar), etapa A10 do roteiro.",
            flush=True,
        )
    else:
        print(f"Apagados: {len(ids)}.", flush=True)
    return len(ids) - len(restam), len(restam)
```

```
def _fechar_pela_ponte(competencia: date, por: str) -> dict[str, Any]:
    chave = competencia.strftime("%m/%Y")
    ciclos = _carregar_ciclos_local()
    ciclos[chave] = {
        "fechada": "SIM",
        "fechado_em": datetime.now().strftime("%d/%m/%Y %H:%M"),
        "fechado_por": por,
    }
    _salvar_ciclos_local(ciclos)
    print(f"Fechando {chave}: o fluxo marca MesFechado=SIM e deixa só leitura...",
flush=True)
    pid = _ponte_pedido("fechar", competencia)
    itens = _ponte_esperar_lista(competencia, pid, minutos=30)
    puxado = puxar_lista_sharepoint(competencia)
    copia = _guardar_copia_fechamento(
        competencia, [Path(puxado.get("arquivo_controle") or ""), RAIZ /
nome_arquivo(competencia)]
    )
    print(f"Cópia de controle (somente leitura): {copia}", flush=True)
    resumo = {
        "acao": "fechar",
        "competencia": chave,
        "fechado_por": por,
        "itens": {"total": len(itens)},
        "arquivo_controle": puxado.get("arquivo_controle"),
        "trava": {"modo": "fluxo SOx Ponte", "itens": len(itens)},
        "copia_controle": str(copia),
        "regra": "Depois do fechamento, só o grupo de Proprietários do site edita a
lista.",
    }
    (PASTA_NUVEM / f"SOx_fechamento_{competencia.strftime('%Y_%m')}.json").write_text(
        json.dumps(resumo, ensure_ascii=False, indent=2), encoding="utf-8"
    )
    print(f"Competência {chave} FECHADA.", flush=True)
    return resumo


def _reabrir_pela_ponte(competencia: date, por: str) -> dict[str, Any]:
    chave = competencia.strftime("%m/%Y")
    ciclos = _carregar_ciclos_local()
    ciclos[chave] = {
        "fechada": "NÃO",
        "reaberto_em": datetime.now().strftime("%d/%m/%Y %H:%M"),
        "reaberto_por": por,
    }
    _salvar_ciclos_local(ciclos)
    pid = _ponte_pedido("reabrir", competencia)
    itens = _ponte_esperar_lista(competencia, pid, minutos=30)
    print(f"Competência {chave} REABERTA.", flush=True)
    return {"acao": "reabrir", "competencia": chave, "reaberto_por": por, "itens":
{"total": len(itens)}}


def publicar_lista_sharepoint(
```

---

#### 24. E-mails sem ponto e vírgula nas pontas

**Onde:** Função _emails_lista.
**Por quê:** FiscalEmail e GerenteEmail passam a ser gravados limpos (nome@petrobras.com.br; com dois nomes: a@petrobras.com.br;b@petrobras.com.br). O

app e os alertas do Teams já comparam assim.
**O que muda:** Troca o trecho Antes pelo Depois.

1 · Procure (Ctrl + F)

```
"""Nomes separados por vírgula viram ;email1;email2; para o app comparar com o
login."""
```

2 · Antes (selecione isto no seu arquivo)

```
    """Nomes separados por vírgula viram ;email1;email2; para o app comparar com o
login."""
```

3 · Depois (cole por cima)

```
    """Nomes separados por vírgula viram email1;email2 (sem ; nas pontas)."""
```

---

### 25. E-mails sem ponto e vírgula nas pontas (retorno)

**Onde:** Função _emails_lista, última linha.
**Por quê:** Mesma mudança da linha acima.
**O que muda:** Troca o trecho Antes pelo Depois.

1 · Procure (Ctrl + F)

```
return ";" + ";".join(achados) + ";" if achados else ""
```

2 · Antes (selecione isto no seu arquivo)

```
    return ";" + ";".join(achados) + ";" if achados else ""
```

3 · Depois (cole por cima)

```
    return ";".join(achados)
```

---

### 26. Ponte Power Automate: funções novas

**Onde:** Novo bloco, logo antes de publicar_lista_sharepoint.
**Por quê:** A Microsoft bloqueou o login direto do Python no SharePoint (erro AADSTS65002). Estas funções gravam pedidos numa pasta do OneDrive (SOx_SCM42_ponte) e esperam o fluxo SOx Ponte devolver a lista. O fluxo usa o seu próprio login, então não precisa da TI.
**O que muda:** Acrescenta código novo logo acima deste trecho (o trecho continua igual).

1 · Procure (Ctrl + F)

```
def publicar_lista_sharepoint(
```

2 · Antes (selecione isto no seu arquivo)

```
def publicar_lista_sharepoint(
```

3 · Depois (cole por cima)

```
# ---------------------------------------------------------------- ponte Power
Automate
# A Microsoft bloqueou o login direto do Python no SharePoint (erro AADSTS65002).
# O Python agora grava pedidos numa pasta do OneDrive; o fluxo "SOx Ponte" faz o
# trabalho no SharePoint com o login do próprio usuário e devolve a lista num arquivo.
# O pedido vai direto ao OneDrive online (cookies SSO, como no Sigma-C); sem cookies,
# usa a pasta sincronizada deste computador.
_PONTE_CACHE: dict[str, tuple[float, list[dict[str, Any]]]] = {}
_PONTE_ONLINE: dict[str, Any] = {}
_PONTE_SITE = "https://petrobrasbr-
my.sharepoint.com/personal/levi_vasconcelos_petrobras_com_br"
_PONTE_PASTA = "/personal/levi_vasconcelos_petrobras_com_br/Documents/SOx_SCM42_ponte"
_PONTE_ESCOLHAS = {
    "HouveServico": ["SIM", "NÃO"],
    "FoiMedido": ["SIM", "NÃO"],
    "ForaDoPrazo": ["SIM", "NÃO"],
    "SeraSubstituido": ["SIM", "NÃO"],
    "DemandaContinuada": ["SIM", "NÃO"],
    "MotivoProvisao": [
        "Atraso do Fornecedor", "Atraso Fiscalização de Campo", "Problema Interno
GPS",
        "Contrato Bloqueado", "Outros",
    ],
}
_PONTE_LONGOS = {
    "Objeto", "Provisao", "Justificativa", "Observacao", "CSP", "FRSPendentes",
    "MotivoIntervencao", "ComentarioErro", "ComentarioGerente",
}
_PONTE_EXTRAS = [
    "OQueFazer", "DadosCorretos", "CampoErro", "ValorCorreto", "AprovacaoGerente",
    "MesFechado", "ComentarioErro", "ComentarioGerente", "OkFiscal", "OkFiscalPor",
    "OkFiscalEm", "AprovacaoPor", "AprovacaoEm",
]
# Campos que o pessoal preenche no app: republicar não sobrescreve.
_PONTE_DO_APP = set(_CAMPOS_MANUAIS_NUVEM) | {
    "Observacao", "OkFiscal", "OkFiscalPor", "OkFiscalEm", "AprovacaoGerente",
    "AprovacaoPor", "AprovacaoEm", "ComentarioGerente", "MesFechado",
}


def _usar_ponte() -> bool:
    return str(garantir_config_nuvem().get("modo_online") or "ponte").lower() ==
"ponte"


def _pasta_ponte() -> Path:
    cfg = garantir_config_nuvem()
    base = str(cfg.get("pasta_ponte") or "").strip()
    if not base:
        onedrive = os.environ.get("OneDriveCommercial") or os.environ.get("OneDrive")
or ""
        if not onedrive:
            raise RuntimeError(
                "Não achei a pasta do OneDrive neste computador. Abra o OneDrive,
entre com a "
                "conta da Petrobras, ou escreva o caminho em pasta_ponte no
sox_nuvem_config.json."
            )
        base = str(Path(onedrive) / "SOx_SCM42_ponte")
    pasta = Path(base)
    for sub in ("pedidos", "da_lista"):
```

```
        (pasta / sub).mkdir(parents=True, exist_ok=True)
    return pasta


def _ponte_cookies(host: str) -> str:
    # Mesmos cookies SSO que o Sigma-C e a automação de certidões já guardam.
    cfg = garantir_config_nuvem()
    pasta_cert = Path.home() / "Downloads" / "automatizacao certidao"
    candidatos = [Path(str(cfg.get("ponte_cookies")))] if cfg.get("ponte_cookies")
else []
    candidatos += [
        RAIZ / ".sp_cookies.json",
        pasta_cert / ".sp_cookies.json",
        pasta_cert / "automatizacao_certidoes.db",
        pasta_cert / "certidoes.db",
    ]
    for caminho in candidatos:
        try:
            if not caminho.is_file():
                continue
            if caminho.suffix.lower() == ".json":
                dados = json.loads(caminho.read_text(encoding="utf-8",
errors="replace"))
            else:
                import sqlite3

                con = sqlite3.connect(str(caminho))
                try:
                    linha = con.execute("SELECT cookies FROM sessao WHERE
id=1").fetchone()
                finally:
                    con.close()
                dados = json.loads(linha[0]) if linha and linha[0] else None
        except Exception:
            continue
        if isinstance(dados, str) and dados.strip():
            return dados.strip()
        if isinstance(dados, dict):
            dados = dados.get("cookies") or [{"name": k, "value": v} for k, v in
dados.items()]
        if not isinstance(dados, list):
            continue
        todos = [c for c in dados if isinstance(c, dict) and c.get("name")]

        def do_host(c: dict[str, Any]) -> bool:
            dominio = str(c.get("domain") or "").lstrip(".").lower()
            return not dominio or host == dominio or host.endswith("." + dominio)

        escolhidos = [c for c in todos if do_host(c)] or todos
        vistos: dict[str, str] = {}
        for c in escolhidos:
            vistos.setdefault(str(c["name"]), str(c.get("value", "")))
        if vistos:
            print(f"Cookies SSO: {caminho}", flush=True)
            return "; ".join(f"{k}={v}" for k, v in vistos.items())
    return ""


def _ponte_http(metodo: str, url: str, cookies: str, corpo: bytes | None = None,
                digest: str = "", tipo: str = "application/octet-stream") ->
tuple[int, bytes]:
    import ssl
    import urllib.error
```

```
    import urllib.request
    from urllib.parse import urlparse

    cab = {"Accept": "application/json;odata=verbose", "Cookie": cookies, "User-
Agent": "Mozilla/5.0"}
    if digest:
        cab["X-RequestDigest"] = digest
    if corpo is not None:
        cab["Content-Type"] = tipo
    req = urllib.request.Request(url, data=corpo, headers=cab, method=metodo)
    try:
        with urllib.request.urlopen(req, context=ssl.create_default_context(),
timeout=60) as resp:
            # Cookie vencido: o SharePoint manda para a tela de login da Microsoft.
            if urlparse(resp.geturl()).netloc.lower() != urlparse(url).netloc.lower():
                return 401, b"login expirado"
            return resp.status, resp.read()
    except urllib.error.HTTPError as exc:
        return exc.code, exc.read()


def _ponte_online() -> dict[str, str] | None:
    import time
    from urllib.parse import urlparse

    if time.time() - _PONTE_ONLINE.get("em", 0) < 1200:
        return _PONTE_ONLINE.get("dados")
    cfg = garantir_config_nuvem()
    dados = None
    if str(cfg.get("ponte_envio") or "online").lower() != "pasta":
        site = str(cfg.get("ponte_site") or _PONTE_SITE).rstrip("/")
        cookies = _ponte_cookies((urlparse(site).hostname or "").lower())
        if not cookies:
            print("Sem cookies SSO salvos (Sigma-C/certidões): uso a pasta
sincronizada do OneDrive.", flush=True)
        else:
            try:
                status, corpo = _ponte_http(
                    "POST", f"{site}/_api/contextinfo", cookies, b"",
tipo="application/json;odata=verbose"
                )
                info = json.loads(corpo.decode("utf-8")) if status == 200 else {}
                digest = (info.get("d") or {}).get("GetContextWebInformation",
{}).get("FormDigestValue", "")
            except Exception as exc:
                status, digest = str(exc), ""
            if digest:
                dados = {
                    "site": site,
                    "pasta": str(cfg.get("ponte_pasta_servidor") or
_PONTE_PASTA).rstrip("/"),
                    "cookies": cookies,
                    "digest": digest,
                }
            else:
                print(
                    f"OneDrive online não aceitou os cookies ({status}). Abra o Sigma-
C ou as certidões "
                    "para renovar o login. Por agora uso a pasta sincronizada.",
                    flush=True,
                )
    _PONTE_ONLINE.update(em=time.time(), dados=dados)
    return dados
```

```
def _ponte_subir_online(sub: str, nome: str, corpo: bytes) -> bool:
    from urllib.parse import quote as q, urlencode

    on = _ponte_online()
    if not on:
        return False
    pasta = f"{on['pasta']}/{sub}"
    url = (
        f"
{on['site']}/_api/web/GetFolderByServerRelativeUrl(@p)/Files/add(url=@n,overwrite=true
)?"
        + urlencode({"@p": f"'{pasta}'", "@n": f"'{nome}'"})
    )
    status, resposta = _ponte_http("POST", url, on["cookies"], corpo, on["digest"])
    if status == 404:  # pasta ainda não existe no OneDrive online
        _ponte_http("POST", f"{on['site']}/_api/web/folders/add('{q(pasta)}')",
on["cookies"], b"", on["digest"])
        status, resposta = _ponte_http("POST", url, on["cookies"], corpo,
on["digest"])
    if status in (200, 201):
        return True
    trecho = resposta.decode("utf-8", errors="replace")[:200]
    print(f"Envio online falhou (HTTP {status}): {trecho}", flush=True)
    _PONTE_ONLINE.clear()
    return False


def _ponte_ler_online(sub: str, nome: str) -> bytes | None:
    from urllib.parse import urlencode

    on = _ponte_online()
    if not on:
        return None
    url = (
        f"{on['site']}/_api/web/GetFileByServerRelativeUrl(@f)/$value?"
        + urlencode({"@f": f"'{on['pasta']}/{sub}/{nome}'"})
    )
    try:
        status, corpo = _ponte_http("GET", url, on["cookies"])
    except Exception:
        return None
    return corpo if status == 200 else None


def _ponte_colunas(lista: str) -> list[dict[str, Any]]:
    nomes = list(dict.fromkeys(list(CABECALHO_STAGING_NUVEM) + _PONTE_EXTRAS))
    base = f"_api/web/lists/getbytitle('{lista}')"
    saida = []
    for nome in nomes:
        if nome in _PONTE_ESCOLHAS:
            opcoes = "".join(f"<CHOICE>{o}</CHOICE>" for o in _PONTE_ESCOLHAS[nome])
            xml = (
                f"<Field Type='Choice' DisplayName='{nome}' Name='{nome}'
StaticName='{nome}' "
                f"FillInChoice='TRUE' Format='Dropdown'><CHOICES>{opcoes}</CHOICES>
</Field>"
            )
        elif nome in _PONTE_LONGOS:
            xml = (
                f"<Field Type='Note' DisplayName='{nome}' Name='{nome}'
StaticName='{nome}' "
```

```
                f"RichText='FALSE' NumLines='6' />"
            )
        else:
            xml = f"<Field Type='Text' DisplayName='{nome}' Name='{nome}'
StaticName='{nome}' MaxLength='255' />"
        saida.append({
            "nome": nome,
            "uri_existe": f"{base}/fields?$select=Id&$filter=" + quote(f"InternalName
eq '{nome}'"),
            "corpo": {"parameters": {"SchemaXml": xml, "Options": 24}},
        })
    return saida


def _ponte_pedido(acao: str, competencia: date, **dados: Any) -> str:
    import uuid

    pid = datetime.now().strftime("%Y%m%d%H%M%S") + uuid.uuid4().hex[:6]
    cfg = garantir_config_nuvem()
    lista = str(cfg.get("lista_contratos") or "SOx_SCM42_Contratos")
    comp = competencia.strftime("%m/%Y")
    base = f"_api/web/lists/getbytitle('{lista}')"
    filtro = quote(f"Competencia eq '{comp}'")
    # Endereços prontos: no fluxo cada caixa só lê outputs('Pedido')?['...'].
    pedido = {
        "acao": acao,
        "id": pid,
        "competencia": comp,
        "arquivo_comp": competencia.strftime("%Y_%m"),
        "lista": lista,
        "uri_lista_existe": "_api/web/lists?$select=Id&$filter=" + quote(f"Title eq
'{lista}'"),
        "uri_criar_lista": "_api/web/lists",
        "corpo_criar_lista": {
            "Title": lista,
            "BaseTemplate": 100,
            "Description": "Controle SOx SCM-42: um contrato por linha e por
competência.",
        },
        "uri_criar_coluna": f"{base}/fields/createfieldasxml",
        "uri_indexar": f"{base}/fields/getbyinternalnameortitle('Competencia')",
        "uri_itens": f"{base}/items",
        "uri_existentes": f"{base}/items?$select=Id,ChaveUnica&$top=5000&$filter=
{filtro}",
        "uri_ids": f"{base}/items?$select=Id&$top=5000&$filter={filtro}",
        "uri_lista": f"{base}/items?$top=5000&$filter={filtro}",
        "uri_grupos": (
            "_api/web?$select=AssociatedOwnerGroup/Id,AssociatedMemberGroup/Id,"

"AssociatedVisitorGroup/Id&$expand=AssociatedOwnerGroup,AssociatedMemberGroup,"
            "AssociatedVisitorGroup"
        ),
        **dados,
    }
    nome = f"{acao}__{competencia.strftime('%Y_%m')}__{pid}.json"
    corpo = json.dumps(pedido, ensure_ascii=True).encode("utf-8")
    if _ponte_subir_online("pedidos", nome, corpo):
        print(f"Pedido enviado direto ao OneDrive online (não depende da
sincronização): {nome}", flush=True)
        return pid
    (_pasta_ponte() / "pedidos" / nome).write_bytes(corpo)
    print(f"Pedido gravado na pasta sincronizada do OneDrive: {nome}", flush=True)
    return pid
```

```
def _ponte_valor(v: Any) -> Any:
    if isinstance(v, dict):
        return v.get("Value") if "Value" in v else ""
    return "" if v is None else v


def _ponte_esperar_lista(competencia: date, pid: str, minutos: int = 20) ->
list[dict[str, Any]]:
    import time

    nome = f"lista_{competencia.strftime('%Y_%m')}__{pid}.json"
    try:
        arquivo: Path | None = _pasta_ponte() / "da_lista" / nome
    except RuntimeError:
        arquivo = None
    online = _ponte_online() is not None
    print(f"Aguardando o fluxo SOx Ponte responder (normal: 1 a 5 min, máximo
{minutos} min)...", flush=True)
    inicio = time.time()
    limite = inicio + minutos * 60
    avisos = {
        120: (
            "Ainda aguardando. Confira se o fluxo SOx Ponte está On no Power
Automate."
            if online else
            "Ainda aguardando. Confira se o OneDrive está sincronizando (nuvem azul
perto do relógio)."
        ),
        300: "Demorando mais que o normal. Veja no Power Automate: SOx Ponte → 28-day
run history.",
    }
    proximo = 30
    while time.time() < limite:
        decorrido = int(time.time() - inicio)
        if decorrido >= proximo:
            print(f"  ... {decorrido // 60} min {decorrido % 60:02d} s esperando o
fluxo", flush=True)
            for seg, msg in avisos.items():
                if proximo - 30 < seg <= proximo:
                    print(f"  {msg}", flush=True)
            proximo += 30
        texto = None
        if arquivo is not None and arquivo.is_file():
            time.sleep(3)
            try:
                texto = arquivo.read_text(encoding="utf-8-sig")
            except OSError:
                texto = ""
        elif online:
            baixado = _ponte_ler_online("da_lista", nome)
            if baixado is not None:
                texto = baixado.decode("utf-8-sig")
        if texto is not None:
            try:
                bruto = json.loads(texto)
            except Exception:
                time.sleep(5)
                continue
            if isinstance(bruto, dict):
                bruto = bruto.get("value") or []
            itens = [{k: _ponte_valor(v) for k, v in it.items()} for it in bruto]
```

```
            _PONTE_CACHE[competencia.strftime("%m/%Y")] = (time.time(), itens)
            print(f"Fluxo respondeu: {len(itens)} contrato(s).", flush=True)
            return itens
        time.sleep(10)
    raise TimeoutError(
        "O fluxo SOx Ponte não respondeu a tempo. Confira se o OneDrive está
sincronizando "
        "(nuvem azul na barra do Windows) e se o fluxo está Ativado no Power Automate
"
        "(Meus fluxos → SOx Ponte → Histórico de execuções)."
    )


def _ponte_puxar(competencia: date) -> list[dict[str, Any]]:
    import time

    cache = _PONTE_CACHE.get(competencia.strftime("%m/%Y"))
    if cache and time.time() - cache[0] < 180:
        return cache[1]
    pid = _ponte_pedido("puxar", competencia)
    return _ponte_esperar_lista(competencia, pid)


def publicar_pela_ponte(caminho_planilha: Path, competencia: date) -> dict[str, Any]:
    cfg = garantir_config_nuvem()
    site, lista = _site_e_lista(cfg)
    exportar_staging_nuvem(caminho_planilha, competencia)
    linhas = coletar_linhas_nuvem(caminho_planilha, competencia)
    itens = []
    for linha in linhas:
        chave = linha.get("ChaveUnica") or ""
        if not chave:
            continue
        if len(linha.get("Observacao", "")) > 4000:
            linha["Observacao"] = linha["Observacao"][:4000] + "..."
        if len(linha.get("Objeto", "")) > 2000:
            linha["Objeto"] = linha["Objeto"][:2000]
        novo = _sp_payload_item("SP.Data", linha, None)
        novo.pop("__metadata", None)
        atualiza = {k: v for k, v in novo.items() if k not in _PONTE_DO_APP}
        itens.append({"chave": chave, "novo": novo, "atualiza": atualiza})
    pid = _ponte_pedido("publicar", competencia, colunas=_ponte_colunas(lista),
itens=itens)
    url = cfg.get("lista_url") or f"{site}/Lists/{lista}"
    try:
        na_lista = _ponte_esperar_lista(competencia, pid)
    except TimeoutError as exc:
        print(f"ATENÇÃO: {exc}", flush=True)
        return {
            "criados": 0, "atualizados": 0, "falhas": len(itens), "lista_url": url,
            "mensagem": (
                f"O pedido foi enviado ({len(itens)} contratos), mas o fluxo SOx Ponte
não respondeu em 20 minutos.\n\n"
                "Confira:\n1. O OneDrive está sincronizando (nuvem azul perto do
relógio).\n"
                "2. O fluxo SOx Ponte está ligado (On).\n"
                "3. Power Automate → SOx Ponte → 28-day run history: a execução
falhou?\n\n"
                "Se o fluxo terminar depois, os contratos aparecem na lista sozinhos."
            ),
        }
    chaves = {str(it.get("ChaveUnica") or "") for it in na_lista}
    faltando = [it["chave"] for it in itens if it["chave"] not in chaves]
```

```
    print(f"Pronto: {len(itens) - len(faltando)} de {len(itens)} contratos estão na
lista online.", flush=True)
    if faltando:
        print("Não entraram: " + ", ".join(faltando[:10]), flush=True)
    apagados, nao_apagados = _ponte_apagar_sobras(competencia, itens, na_lista)
    return {
        "criados": len(itens) - len(faltando),
        "atualizados": 0,
        "falhas": len(faltando),
        "lista_url": url,
        "mensagem": (
            ("Publicado na lista online.\n\n" if not faltando else "Publicado, mas com
falhas.\n\n")
            + f"Na lista: {len(itens) - len(faltando)} de {len(itens)} contratos de
{competencia.strftime('%m/%Y')}.\n"
            + (f"Não entraram: {len(faltando)} (veja a lista na janela do programa e o
histórico do fluxo).\n" if faltando else "")
            + (f"Apagados da lista (não estão mais na planilha): {apagados}.\n" if
apagados else "")
            + (f"Não consegui apagar {nao_apagados} contrato(s) que saíram da
planilha. Veja a janela do programa.\n" if nao_apagados else "")
            + "\nO que o pessoal já respondeu no app não é apagado."
        ),
    }


def _ponte_apagar_sobras(competencia: date, itens: list[dict[str, Any]], na_lista:
list[dict[str, Any]]) -> tuple[int, int]:
    # A lista do mês fica igual à planilha: some o contrato que saiu dela.
    novas = {it["chave"] for it in itens}
    sobras = [it for it in na_lista if str(it.get("ChaveUnica") or "") not in novas]
    ids = [int(it["Id"]) for it in sobras if str(it.get("Id") or "").isdigit()]
    if not ids or not itens:
        return 0, 0
    if len(ids) > max(10, len(na_lista) // 2):
        print(
            f"ATENÇÃO: {len(ids)} contratos da lista de
{competencia.strftime('%m/%Y')} não estão na planilha. "
            "São muitos; não apaguei nada. Confira se a competência está certa.",
            flush=True,
        )
        return 0, len(ids)
    nomes = ", ".join(str(it.get("Title") or it.get("Contrato") or it.get("Id")) for
it in sobras[:10])
    print(f"Apagando da lista {len(ids)} contrato(s) que não estão mais na planilha:
{nomes}", flush=True)
    pid = _ponte_pedido("apagar", competencia, ids=ids)
    try:
        depois = _ponte_esperar_lista(competencia, pid)
    except TimeoutError as exc:
        print(f"ATENÇÃO: {exc}", flush=True)
        return 0, len(ids)
    restam = {str(it.get("Id")) for it in depois} & {str(i) for i in ids}
    if restam:
        print(
            f"Não consegui apagar {len(restam)}. Confira se o fluxo SOx Ponte tem o
Caso 4 (apagar), etapa A10 do roteiro.",
            flush=True,
        )
    else:
        print(f"Apagados: {len(ids)}.", flush=True)
    return len(ids) - len(restam), len(restam)
```

```
def _fechar_pela_ponte(competencia: date, por: str) -> dict[str, Any]:
    chave = competencia.strftime("%m/%Y")
    ciclos = _carregar_ciclos_local()
    ciclos[chave] = {
        "fechada": "SIM",
        "fechado_em": datetime.now().strftime("%d/%m/%Y %H:%M"),
        "fechado_por": por,
    }
    _salvar_ciclos_local(ciclos)
    print(f"Fechando {chave}: o fluxo marca MesFechado=SIM e deixa só leitura...",
flush=True)
    pid = _ponte_pedido("fechar", competencia)
    itens = _ponte_esperar_lista(competencia, pid, minutos=30)
    puxado = puxar_lista_sharepoint(competencia)
    copia = _guardar_copia_fechamento(
        competencia, [Path(puxado.get("arquivo_controle") or ""), RAIZ /
nome_arquivo(competencia)]
    )
    print(f"Cópia de controle (somente leitura): {copia}", flush=True)
    resumo = {
        "acao": "fechar",
        "competencia": chave,
        "fechado_por": por,
        "itens": {"total": len(itens)},
        "arquivo_controle": puxado.get("arquivo_controle"),
        "trava": {"modo": "fluxo SOx Ponte", "itens": len(itens)},
        "copia_controle": str(copia),
        "regra": "Depois do fechamento, só o grupo de Proprietários do site edita a
lista.",
    }
    (PASTA_NUVEM / f"SOx_fechamento_{competencia.strftime('%Y_%m')}.json").write_text(
        json.dumps(resumo, ensure_ascii=False, indent=2), encoding="utf-8"
    )
    print(f"Competência {chave} FECHADA.", flush=True)
    return resumo


def _reabrir_pela_ponte(competencia: date, por: str) -> dict[str, Any]:
    chave = competencia.strftime("%m/%Y")
    ciclos = _carregar_ciclos_local()
    ciclos[chave] = {
        "fechada": "NÃO",
        "reaberto_em": datetime.now().strftime("%d/%m/%Y %H:%M"),
        "reaberto_por": por,
    }
    _salvar_ciclos_local(ciclos)
    pid = _ponte_pedido("reabrir", competencia)
    itens = _ponte_esperar_lista(competencia, pid, minutos=30)
    print(f"Competência {chave} REABERTA.", flush=True)
    return {"acao": "reabrir", "competencia": chave, "reaberto_por": por, "itens":
{"total": len(itens)}}


def publicar_lista_sharepoint(
```

---

**27. Ponte Power Automate: ligada na configuração**

**Onde:** Função _config_nuvem_padrao.
**Por quê:** modo_online = ponte faz publicar, puxar, fechar e reabrir passarem pelo fluxo.

pasta_ponte vazio = usa o OneDrive da Petrobras deste computador.
**O que muda:** Troca o trecho Antes pelo Depois.

1 · Procure (Ctrl + F)

```
"publicar_automatico": True,
```

2 · Antes (selecione isto no seu arquivo)

```
        "publicar_automatico": True,
```

3 · Depois (cole por cima)

```
        "publicar_automatico": True,
        "modo_online": "ponte",
        "pasta_ponte": "",
```

---

### 28. Publicar usa a ponte

**Onde:** Início de publicar_lista_sharepoint.
**Por quê:** Em vez de entrar no SharePoint (bloqueado), grava o pedido para o fluxo.
**O que muda:** Troca o trecho Antes pelo Depois.

1 · Procure (Ctrl + F)

```
def publicar_lista_sharepoint(caminho_planilha: Path, competencia: date) -> dict[str,
Any]:
```

2 · Antes (selecione isto no seu arquivo)

```
def publicar_lista_sharepoint(caminho_planilha: Path, competencia: date) -> dict[str,
Any]:
    """Gera staging e faz upsert na lista SharePoint (online, sem Power Automate)."""
```

3 · Depois (cole por cima)

```
def publicar_lista_sharepoint(caminho_planilha: Path, competencia: date) -> dict[str,
Any]:
    """Gera staging e faz upsert na lista SharePoint (online, sem Power Automate)."""
    if _usar_ponte():
        return publicar_pela_ponte(caminho_planilha, competencia)
```

---

### 29. Puxar respostas usa a ponte

**Onde:** Função puxar_lista_sharepoint, leitura dos itens.
**Por quê:** Pede ao fluxo a lista do mês e espera o arquivo voltar. O resto (planilha, banco, controle) continua igual.
**O que muda:** Troca o trecho Antes pelo Depois.

1 · Procure (Ctrl + F)

```
itens = _sp_listar_itens_competencia(site, lista, token, competencia)
```

2 · Antes (selecione isto no seu arquivo)

```
    token = _token_sharepoint(interativo=True)
    itens = _sp_listar_itens_competencia(site, lista, token, competencia)
```

3 · Depois (cole por cima)

```
    if _usar_ponte():
        itens = _ponte_puxar(competencia)
    else:
        token = _token_sharepoint(interativo=True)
        itens = _sp_listar_itens_competencia(site, lista, token, competencia)
```

---

### 30. Fechar competência usa a ponte

**Onde:** Início de fechar_competencia_nuvem.
**Por quê:** O fluxo marca MesFechado=SIM e deixa cada contrato do mês só leitura (menos para os Proprietários do site). Depois o Python puxa e guarda a cópia de controle.
**O que muda:** Troca o trecho Antes pelo Depois.

1 · Procure (Ctrl + F)

```
por = (por or (sorted(_admin_emails())[:1] or ["admin"])[0]).strip()
```

2 · Antes (selecione isto no seu arquivo)

```
    por = (por or (sorted(_admin_emails())[:1] or ["admin"])[0]).strip()
    garantir_estrutura_nuvem(interativo=True)
```

3 · Depois (cole por cima)

```
    por = (por or (sorted(_admin_emails())[:1] or ["admin"])[0]).strip()
    if _usar_ponte():
        return _fechar_pela_ponte(competencia, por)
    garantir_estrutura_nuvem(interativo=True)
```

---

### 31. Reabrir competência usa a ponte

**Onde:** Início de reabrir_competencia_nuvem.
**Por quê:** O fluxo devolve a permissão normal e marca MesFechado=NÃO.
**O que muda:** Troca o trecho Antes pelo Depois.

1 · Procure (Ctrl + F)

```
"""Admin: reabre o mês para o pessoal voltar a editar as linhas dele."""
```

2 · Antes (selecione isto no seu arquivo)

```
    """Admin: reabre o mês para o pessoal voltar a editar as linhas dele."""
    cfg = garantir_config_nuvem()
    site, lista = _site_e_lista(cfg)
    lista_ciclos = str(cfg.get("lista_ciclos") or "SOx_SCM42_Ciclos")
    por = (por or (sorted(_admin_emails())[:1] or ["admin"])[0]).strip()
```

3 · Depois (cole por cima)

```
    """Admin: reabre o mês para o pessoal voltar a editar as linhas dele."""
    cfg = garantir_config_nuvem()
    site, lista = _site_e_lista(cfg)
    lista_ciclos = str(cfg.get("lista_ciclos") or "SOx_SCM42_Ciclos")
    por = (por or (sorted(_admin_emails())[:1] or ["admin"])[0]).strip()
    if _usar_ponte():
        return _reabrir_pela_ponte(competencia, por)
```

### 32. Conferir estrutura não pede login na ponte

**Onde:** Início de garantir_estrutura_nuvem.
**Por quê:** Na ponte quem cria a lista e as colunas é o fluxo, a cada publicação.
**O que muda:** Troca o trecho Antes pelo Depois.

1 · Procure (Ctrl + F)

```
"""Garante colunas novas na lista de contratos + lista SOx_SCM42_Ciclos."""
```

2 · Antes (selecione isto no seu arquivo)

```
    """Garante colunas novas na lista de contratos + lista SOx_SCM42_Ciclos."""
    import requests as req
```

3 · Depois (cole por cima)

```
    """Garante colunas novas na lista de contratos + lista SOx_SCM42_Ciclos."""
    import requests as req

    if _usar_ponte():
        return {"modo": "ponte", "lista":
str(garantir_config_nuvem().get("lista_contratos") or "")}
```

### 33. Mensagem do Publicar explica a ponte

**Onde:** Função _abrir_interface, botão Publicar online agora.
**Por quê:** Avisa que o fluxo grava em até 5 minutos, em vez de dizer que já publicou.
**O que muda:** Troca o trecho Antes pelo Depois.

1 · Procure (Ctrl + F)

```
pub = publicar_lista_sharepoint(saida, competencia)
```

2 · Antes (selecione isto no seu arquivo)

```
            pub = publicar_lista_sharepoint(saida, competencia)
            msg = (
```

3 · Depois (cole por cima)

```
            pub = publicar_lista_sharepoint(saida, competencia)
            msg = pub.get("mensagem") or (
```

Prints marcados "Print real · Microsoft": documentação oficial da Microsoft (MicrosoftDocs/powerapps-docs e power-automate-docs), licença CC BY 4.0. Telas em inglês; a tradução de cada botão está na legenda. Marcados "Ilustração": desenhos feitos para este roteiro.